

1-后端八股整理

后端八股整理

JAVASE

集合

反射

深拷贝浅拷贝

异常机制

代理

String相关

迭代器

泛型

IO模型

JDK新特性

JUC

线程基础

volatile关键字

ThreadLocal

线程池

AQS原理

CAS与原子类

synchronized与Lock

并发经典题

线程安全与JMM

JVM

对象创建与内存布局

JVM内存区域

垃圾回收

类加载机制

JVM调优实战

【了解】执行引擎与JIT

redis

数据类型

高可用

过期策略

缓存三剑客

缓存与数据库双写一致性

看门狗机制

事务与一致性

线程模型

BigKey和HotKey问题

setnx

mysql

事务与MVCC

日志与执行流程

索引原理

锁机制

主从复制与隔离级别

操作系统

进程线程协程

内存管理

死锁

IO多路复用

CPU缓存与一致性协议

Linux常用命令

操作系统补充

计算机网络

网络分层与基础

从输入URL到页面展示

TCP

HTTP与HTTPS

DNS与CDN

网络排查与高频场景

进阶内容

分布式理论

微服务架构与拆分

DDD与领域建模

trace与可观测

高并发系统设计

服务治理与高可用

分库分表

概念与分片键

查询与常见问题

分片算法

Spring框架相关

SpringMVC

SpringIOC

SpringAOP

Spring常见注解

Spring事务

Spring自动装配

中间件

kafka核心原理

xxl-job

RocketMQ

RabbitMQ

Netty

Elasticsearch

Dubbo

消息队列选型

设计模式

策略模式

单例模式

观察者模式

数据结构与算法

数组链表与复杂度

树堆与哈希

排序算法

栈与队列

后端八股整理

JAVASE

集合

集合

对于集合的考察，面试官比较喜欢上来先问你自己在项目中会用什么集合，怎么用的

这种类型的问题其实很常见，因为可以考察你的知识广度和深度，回答的时候要从两方面回答

1. 先大而广地罗列一下，讲一下主要区别

面试官你好，Java 集合框架主要分为两条主线，**Collection** 接口和 **Map** 接口，**Collection** 下主要有三个子接口：**List**（有序、可重复）、**Set**（无序、唯一）和 **Queue**（队列），**Map** 接口则是存储键值对（Key-Value）的结构。

在具体实现上，常用的有 `ArrayList`、`LinkedList`、`HashSet`、`TreeSet`，以及最核心的 `HashMap`、`ConcurrentHashMap` 和 `TreeMap`

2. 然后有侧重地介绍自己比较熟悉的

比如我项目里面规则树的实现，为了在决策过程中实现 $O(1)$ 级别的节点跳转，我用 `HashMap` 来存储节点 ID 与其对应逻辑实体的映射关系。

3. 在这之后面试官就会顺着你讲的展开了，这样子面试的节奏就是你的了

集合的话我比较推荐看 `hashmap`，因为 `hashmap` 的八股是集合里比较有深度的，可以展示你的水平

类似的问题也有会出现在 `redis` 八股的考察，会问你 `redis` 有哪些数据结构，你在项目中会用什么，怎么用。

ArrayList与LinkedList的区别

• 底层结构：

- `ArrayList` 基于动态数组。它在内存中是连续存储的。
- `LinkedList` 基于双向链表（JDK1.7 之后）。它的节点在内存中是分散存储的，通过指针连接。

• 性能差异：

- **查询**：`ArrayList` 支持随机访问，通过下标能直接算出地址，复杂度是 $O(1)$ 。`LinkedList` 需要从头遍历，复杂度是 $O(n)$ 。

- 增删：
 - LinkedList 在已知节点的情况下，增删只需改动指针，是 $O(1)$ 。
 - ArrayList 如果在中间增删，需要挪动后续所有元素，是 $O(n)$ 。

追问1：你在项目中怎么选这两种数据结构

- 绝大多数场景（90%）我都会选 ArrayList。
 - 原因：除了支持快速查询，ArrayList 的连续内存对 CPU 缓存（Cache Line）非常友好，遍历速度极快。而 LinkedList 内存分散，容易造成缓存未命中。
 - 只有在‘需要频繁在头部或中间插入删除，且几乎不随机访问’的极少数特殊场景下，我才会考虑 LinkedList。”

追问2：你说 LinkedList 插入更快，那我在 List 中间插入 1000 万条数据，谁更快？

这要看情况。如果是用 `for` 循环根据下标 `i` 去插入，ArrayList 反而更快。因为 LinkedList 每次都要 `get(i)` 遍历找到位置，这一步就是 $O(n)$ ，总复杂度变成 $O(n^2)$ 。只有用 `Iterator` 遍历并插入，LinkedList 才能发挥 $O(1)$ 插入的优势。

追问3：ArrayList 是线程安全的吗？如果不安全怎么办？

不安全。多线程下建议使用 `CopyOnWriteArrayList`（适合读多写少）或者用 `Collections.synchronizedList` 包装一下

ArrayList的扩容机制

ArrayList 的扩容主要发生在 `add` 操作时。

1. 初始化：在 JDK 1.8 中，如果使用无参构造器，它初始化的是一个空数组（懒加载）。只有当第一次 `add` 添加元素时，才会初始化为默认容量 10。
2. 触发条件：当当前元素个数等于数组容量时，就会触发扩容。
3. 增长策略：新容量通常是旧容量的 1.5 倍。
 - 源码细节：`int newCapacity = oldCapacity + (oldCapacity >> 1);`（右移一位相当于除以 2）。
4. 数据迁移：计算好新容量后，底层会调用 `Arrays.copyOf`（本质是 `System.arraycopy`），在堆中创建一个新数组，把老数据复制过去，然后丢弃老数组。

追问1: 既然扩容需要复制数组, 效率很低, 怎么优化

最直接的优化就是**预估容量**。如果我能预知列表大概要存 1000 个对象, 我就直接 `new ArrayList<>(1000)`。这样就避免了中间多次的扩容 (10 -> 15 -> 22...), 既省了时间也减少了数组碎片的产生。

HashMap的底层原理 (重要)

HashMap 的底层结构在JDK 1.7 和 1.8 中是有区别的, 我分开来讲, JDK 1.7采用的是**数组+链表**的结构, 而 JDK 1.8 做了优化, 采用了**数组+链表+红黑树**的结构。

具体的插入流程 (put) 是这样的

1. 首先, 根据 Key 计算 Hash 值, 经过扰动函数处理后, 通过(数组长度-1) & 键的哈希值 公式确定它在数组中的下标。
2. 如果该位置为空, 直接插入。
3. 如果该位置已经有数据了 (发生Hash冲突), 会遍历链表检查Key是否存在:
 - 如果Key相同, 就覆盖Value。
 - 如果Key不同, JDK 1.8 会判断当前节点是链表还是红黑树。如果是链表, 采用**尾插法**追加到末尾; 如果是红黑树, 则调用树的插入方法。
4. **重点是**, 在 JDK 1.8 中, 当链表长度超过 8 且数组长度大于等于 64 时, 链表会转换为红黑树, 这样能将查找复杂度从 $O(n)$ 降低到 $O(\log n)$, 解决链表过长导致的性能下降问题。

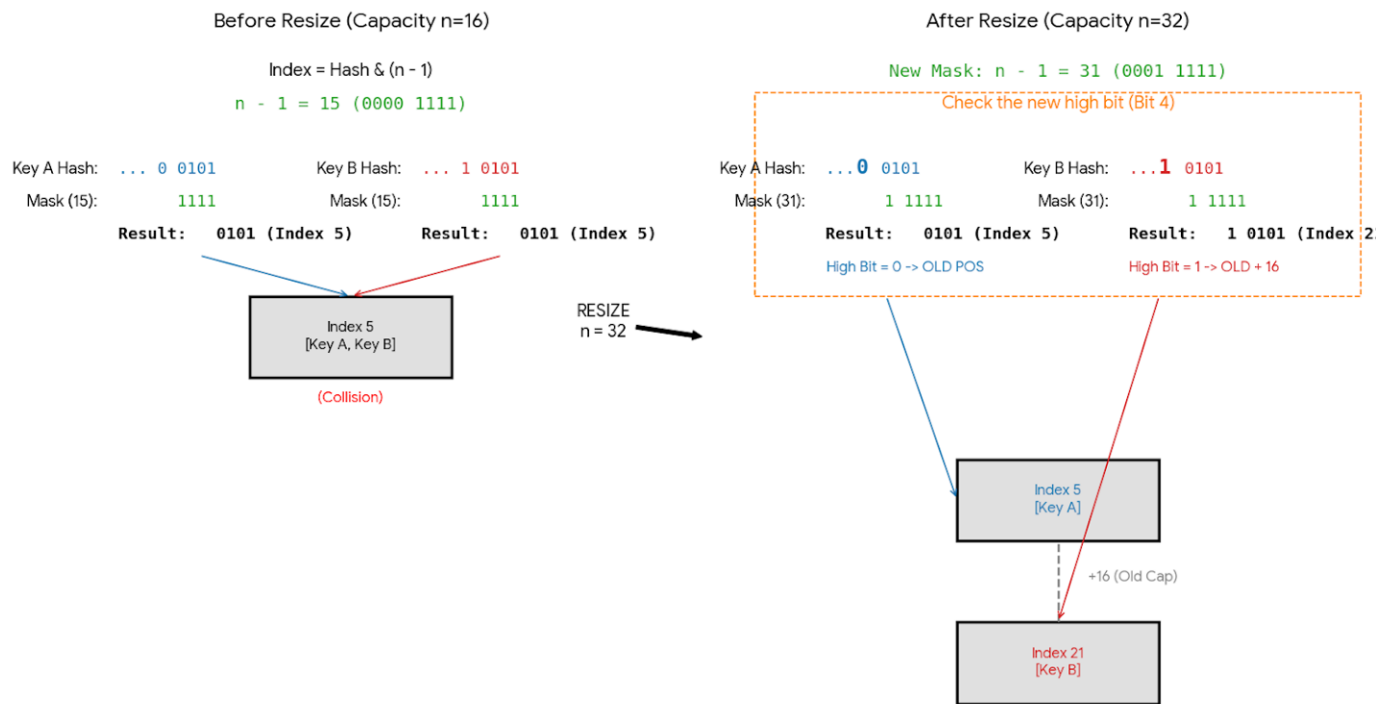
追问1: 为什么 JDK 1.8 要用尾插法? 1.7 的头插法有什么问题

JDK 1.7 用头插法是因为作者认为最新插入的数据被访问的概率更大。但在**多线程并发扩容**时, 头插法会改变链表的顺序, 容易导致**环形链表**, 从而引发死循环 (CPU 100%)。1.8 改用尾插法, 扩容后链表顺序保持不变, 就解决了这个问题。

追问2: HashMap的扩容机制是怎么样的 (重要)

HashMap 默认初始容量是 16, 加载因子是 0.75。当元素数量超过 `容量 * 0.75` 时, 就会触发扩容, 容量变为原来的**2倍**。扩容时需要重新计算每个元素在数组中的位置。不过 JDK 1.8 做了优化, 不需要重新 Hash, 只需要看 hash 值新增的那一位比特是 0 还是 1。如果是 0, 位置不变; 如果是 1, 位置变为 `原索引 + 原容量`。这样效率更高。

帮助理解的例子



追问3：为什么 HashMap 的容量必须是 2 的幂

为了支持两个优化：

1. **极速寻址**：只有当容量是 2 的幂时，`hash % 容量` 才能等价于位运算 `hash & (容量 - 1)`，位运算比取模快得多。
2. **扩容优化**：就是刚才讲的，利用高位比特判断位置，避免重新 Hash。

追问4：HashMap 是线程安全的吗

HashMap **不是**线程安全的。在多线程环境下使用它，主要会产生两个严重问题：

1. **数据丢失（覆盖）**：这是最常见的问题。
 - 在 JDK 1.7 和 1.8 中都存在。当两个线程同时进行 `put` 操作，并且 Key 的 Hash 值冲突时，它们可能会同时判断“当前位置为空”。
 - 于是，线程 A 刚准备插入，线程 B 抢先插入了。等线程 A 恢复执行时，它不知道该位置已经被占了，就会直接写入，把线程 B 的数据覆盖掉。
2. **死循环（CPU 100%）**：这是 JDK 1.7 的经典 Bug。
 - JDK 1.7 在扩容时采用的是**头插法**。
 - 在并发扩容时，如果链表结构被破坏，可能会导致链表成环（A指向B，B又指向A）。

- 当下次 `get` 操作遍历这个链表时，程序就会进入死循环，导致服务器 CPU 飙升。
- 补充：虽然 JDK 1.8 改用了尾插法解决了死循环问题，但数据覆盖的问题依然存在，所以它依然是不安全的。

补充，生产环境要使用线程安全的map有三种方案

1. **Hashtable**：虽然安全，但它给所有方法都加了 `synchronized`，锁粒度太粗，全表只有一把锁，效率极低，现在基本没人用了。
2. **Collections.synchronizedMap**：它是对 `HashMap` 做了一层包装，本质上也是用 `synchronized` 锁住整个 Map，并发性能依然不高。
3. **ConcurrentHashMap**：这是最佳选择。它通过降低锁粒度（分段锁或节点锁）实现了高效的并发。

ConcurrentHashMap的实现原理

`ConcurrentHashMap` 的核心思想是分而治之，尽量缩小锁的范围。JDK 1.7 和 1.8 的实现差异很大：

1. JDK 1.7：分段锁

- **结构**：它由一个 `Segment` 数组组成。每个 `Segment` 本质上继承了 `ReentrantLock`。
- **原理**：它把一个大的 `HashMap` 切分成了 16 个小段（`Segment`）。
- **并发度**：理论上支持 16 个线程并发写入，只要它们访问的是不同的段，就不会互斥。
- **缺点**：并发度受限于 `Segment` 的个数，且 `Segment` 对象本身占用不少内存。

2. JDK 1.8：CAS + Synchronized

- **结构**：摒弃了 `Segment`，结构和 `HashMap 1.8` 一样（数组+链表+红黑树）。
- **原理**：
 - 它直接锁链表/红黑树的头节点。
 - 如果是第一次插入（桶为空）：利用 **CAS** (`Compare And Swap`) 指令进行无锁插入，速度极快。
 - 如果发生冲突（桶不为空）：使用 `synchronized` 锁住当前桶的头节点。
- **优势**：锁粒度细化到了每一个 `Node`（桶）。只要 Hash 不冲突，几千个线程都能并发写，性能比 1.7 提升巨大。”

追问1：为什么 JDK 1.8 要把 `ReentrantLock` 换成 `synchronized`? `synchronized` 不是性能很差吗

这其实是一个误区。

1. **JDK 优化**：从 JDK 1.6 开始，JVM 对 `synchronized` 做了大量优化（偏向锁、轻量级锁、锁消除

等），在低竞争下，它的性能已经不输 ReentrantLock 了。

2. **节省内存**：ReentrantLock 需要创建额外的 Segment 对象，而 synchronized 直接锁住 Node 对象头，不需要额外内存。
3. **自动释放**：synchronized 自动释放锁，而 Lock 必须手动 unlock，代码更简洁安全。

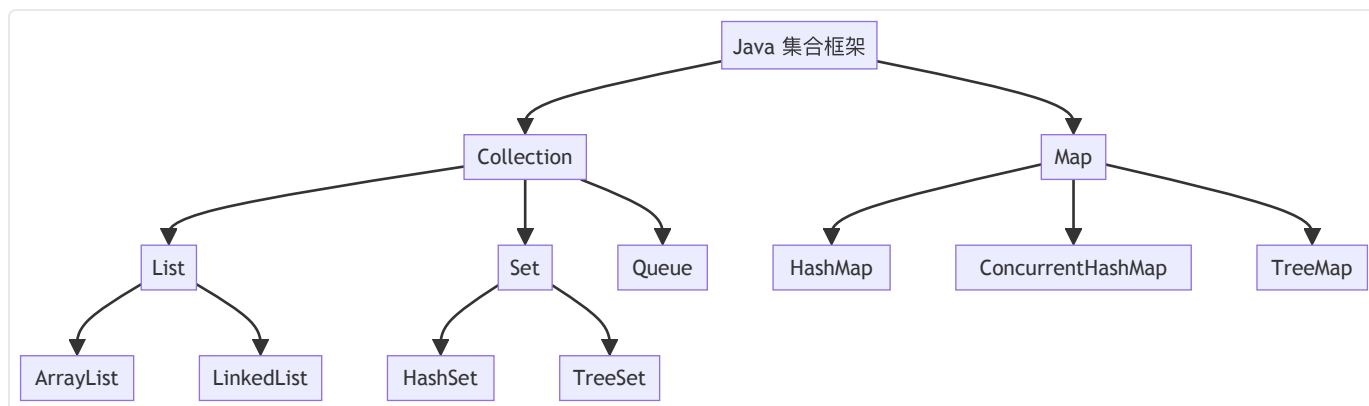
追问2：ConcurrentHashMap 的 Key 和 Value 能为 null 吗？为什么？

坚决不能（HashMap 可以）。

原因：主要是为了避免二义性（Ambiguity）。假设我 `map.get(key)` 返回了 null。

- 在单线程（HashMap）中，我可以再调用 `map.containsKey(key)` 来确认是‘没这个键’还是‘值为 null’。
- 但在多线程（ConcurrentHashMap）中，这两次调用之间，可能有另一个线程把数据改了。所以我没法判断最初那个 null 到底是什么意思。为了系统健壮性，并发集合直接禁止了 null。

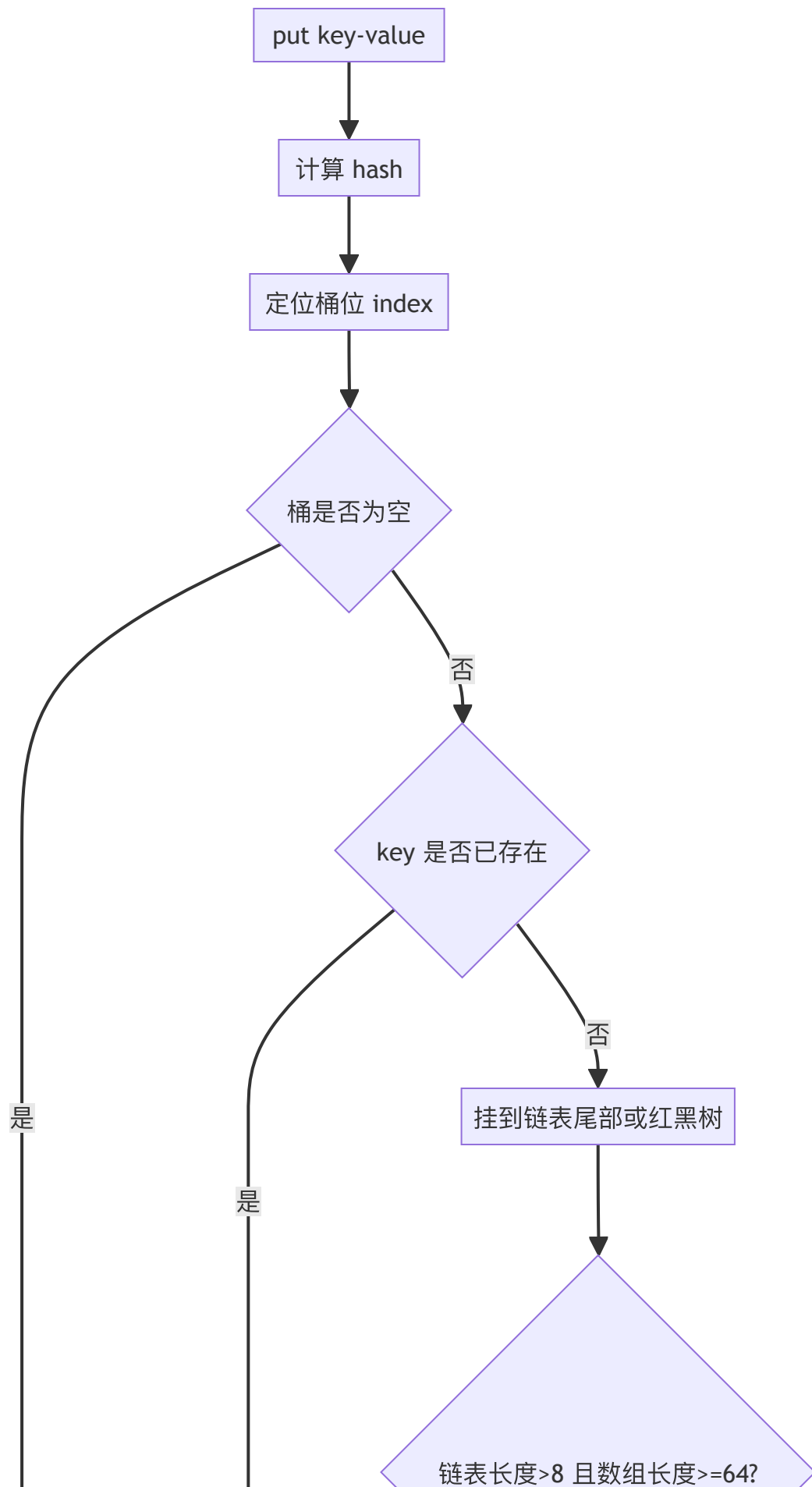
图解补充：集合体系与选型

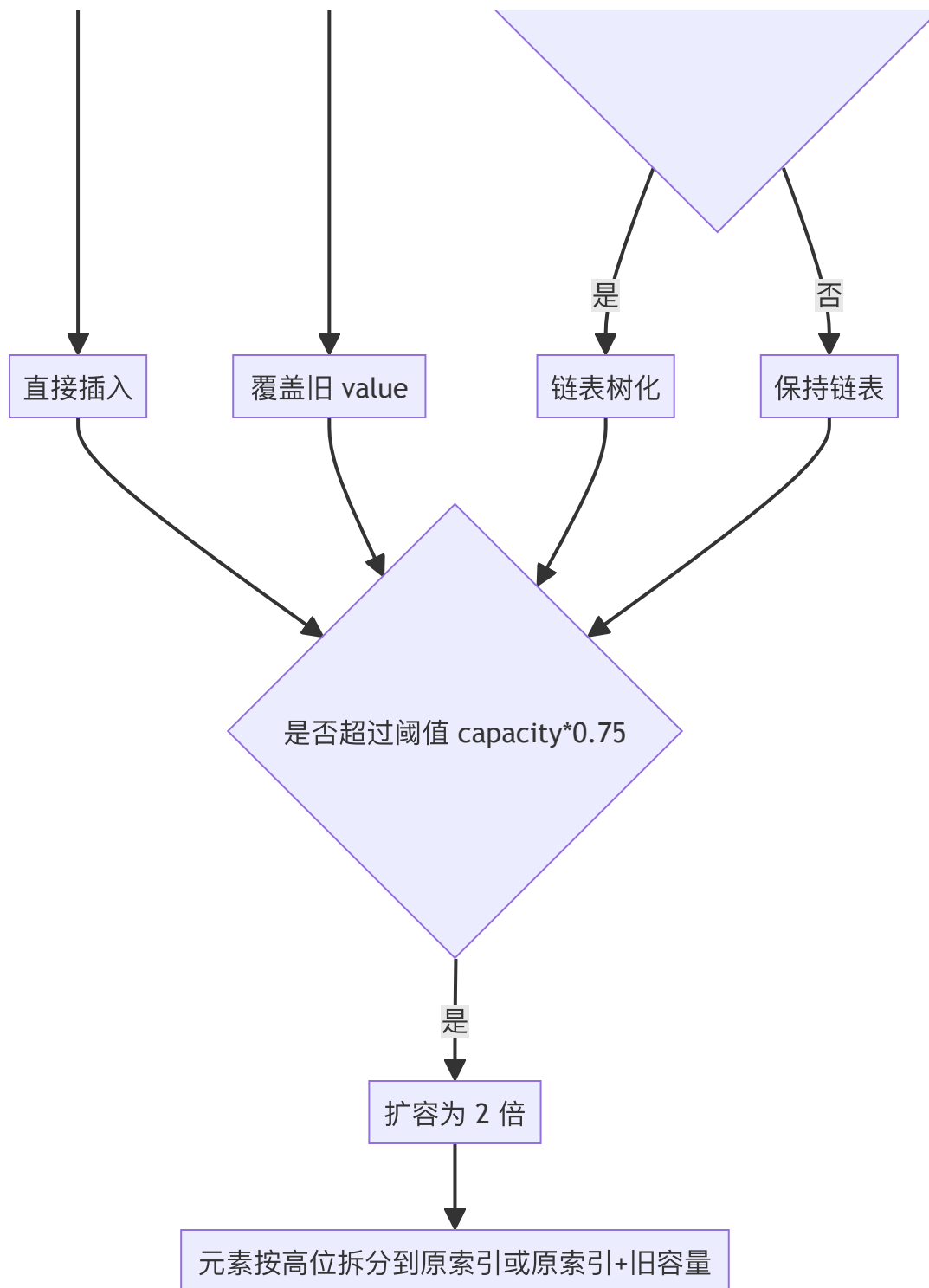


结构	底层	查询	中间插入/删除	线程安全	典型场景
<code>ArrayList</code>	动态数组	快, <code>O(1)</code>	慢, 可能搬迁元素	否	绝大多数业务列表
<code>LinkedList</code>	双向链表	慢, <code>O(n)</code>	已知节点时快	否	极少数频繁头尾插入
<code>HashMap</code>	数组+链表/红黑树	平均快, <code>O(1)</code>	插入快	否	普通 KV 映射

ConcurrentHashMap	数组+链表/红黑树	平均快, $O(1)$	并发写更稳	是	并发缓存、共享状态
TreeMap	红黑树	$O(\log n)$	$O(\log n)$	否	需要有序键

图解补充：HashMap put 与扩容





高频追问补充

追问补充1：为什么实际开发里 LinkedList 用得很少？

主要是因为它理论上的插入优势，往往会被遍历定位成本抵消。再加上链表节点分散，CPU 缓存命中率差，真实业务里通常 `ArrayList` 更快也更省内存。

追问补充2: HashMap 为什么链表转红黑树的阈值是 8?

这个值本质上是时间复杂度和空间开销之间的折中。链表太短时直接转树不划算，链表太长又会拉低查询性能，所以 JDK 选择在冲突较严重时再树化，同时要求数组长度至少 64，优先扩容而不是立刻树化。

追问补充3: 面试里怎么回答集合选型更像工程实践?

可以按这个顺序回答：

1. 先讲业务形态，是列表、去重、队列还是 KV 映射。
2. 再讲读多写多、是否需要排序、是否并发访问。
3. 最后给出具体实现和原因，比如 `ArrayList + HashMap`、`ConcurrentHashMap`、`TreeMap`。

速记版

集合面试可以按这条线背：

- 先分 `Collection` 和 `Map`
- `List` 常问 `ArrayList` vs `LinkedList`
- `Map` 常问 `HashMap`、扩容、线程安全
- 并发场景收口到 `ConcurrentHashMap`

一句话记忆：

`ArrayList` 看查询和遍历，`LinkedList` 看节点插删，`HashMap` 看寻址和扩容，`ConcurrentHashMap` 看并发控制。

反射

反射

反射是 Java 能够成为动态语言的关键，也是所有主流框架（Spring、MyBatis）的基石

所以如果考察反射，很多时候会跟spring/mybatis一起考察

什么是反射？它有什么用？

Q：什么是 Java 反射机制？你在哪里用过？

A：

反射的本质是：在运行状态中，对于任意一个类，都能够知道这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意方法和属性。简单来说，就是把 Java 类中的各种成分（成员变量、方法、构造器等）映射成一个个 Java 对象（Field、Method、Constructor）。

至于用途，虽然我在写业务代码时很少直接写反射，但我用的所有框架底层都在用反射。比如 Spring 的 IOC（控制反转），它读取 XML 或注解配置，通过反射实例化 Bean 并注入依赖；还有 MyBatis，它也是通过反射将数据库结果集映射成 Java 对象的。

反射的原理：

其实比较好理解，java中有一个核心类，`java.lang.Class`

当 JVM 加载一个 `.class` 文件时，会在堆内存中产生一个代表这个类的 `Class` 对象。这个对象包含了该类的完整结构信息（类名、包名、父类、接口、方法、属性等）。程序员可以通过操作这个 `Class` 对象，动态地获取信息或执行操作。

追问1：反射机制有什么优缺点

优点是灵活性极高。它允许程序在运行时动态加载类，这使得我们可以编写通用的框架代码，不需要在编译期就将被调用的类写死。

缺点主要有两个：

1. 性能开销：反射涉及动态解析，JVM 无法对这些代码进行极致的 JIT 优化，所以比直接调用要慢

（虽然现在 JDK 优化得已经很快了，但在高频调用场景下仍需注意）。

2. **安全隐患**：反射可以绕过泛型检查，甚至通过 `setAccessible(true)` 访问私有成员，可能会破坏类的封装性。

获取 Class 对象的几种方式

Q：获取 Class 对象有哪几种方式？它们有什么区别？

A：

主要有三种方式：

1. **`Class.forName("全类名")`**：这是最常用的方式，通常用于读取配置文件加载类（比如 JDBC 加载驱动）。它需要捕获异常。
2. **`类名.class`**：这种方式最简单，但需要编译期就导入该类，一般用于参数传递（比如 `new ArrayList<>(String.class)`）。
3. **`对象.getClass()`**：必须要有实例化后的对象才能用。

区别在于：`Class.forName()` 是动态加载，最灵活；`类名.class` 是静态加载；`getClass()` 依赖实例。在编写框架或通用工具时，第一种用得最多。

深拷贝浅拷贝

反射

反射是 Java 能够成为动态语言的关键，也是所有主流框架（Spring、MyBatis）的基石

所以如果考察反射，很多时候会跟spring/mybatis一起考察

什么是反射？它有什么用？

Q：什么是 Java 反射机制？你在哪里用过？

A：

反射的本质是：在运行状态中，对于任意一个类，都能够知道这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意方法和属性。简单来说，就是把 Java 类中的各种成分（成员变量、方法、构造器等）映射成一个个 Java 对象（Field、Method、Constructor）。

至于用途，虽然我在写业务代码时很少直接写反射，但我用的所有框架底层都在用反射。比如 Spring 的 IOC（控制反转），它读取 XML 或注解配置，通过反射实例化 Bean 并注入依赖；还有 MyBatis，它也是通过反射将数据库结果集映射成 Java 对象的。

反射的原理：

其实比较好理解，java中有一个核心类，`java.lang.Class`

当 JVM 加载一个 `.class` 文件时，会在堆内存中产生一个代表这个类的 `Class` 对象。这个对象包含了该类的完整结构信息（类名、包名、父类、接口、方法、属性等）。程序员可以通过操作这个 `Class` 对象，动态地获取信息或执行操作。

追问1：反射机制有什么优缺点

优点是灵活性极高。它允许程序在运行时动态加载类，这使得我们可以编写通用的框架代码，不需要在编译期就将被调用的类写死。

缺点主要有两个：

1. 性能开销：反射涉及动态解析，JVM 无法对这些代码进行极致的 JIT 优化，所以比直接调用要慢

（虽然现在 JDK 优化得已经很快了，但在高频调用场景下仍需注意）。

2. **安全隐患**：反射可以绕过泛型检查，甚至通过 `setAccessible(true)` 访问私有成员，可能会破坏类的封装性。

获取 Class 对象的几种方式

Q：获取 Class 对象有哪几种方式？它们有什么区别？

A：

主要有三种方式：

1. `Class.forName("全类名")`：这是最常用的方式，通常用于读取配置文件加载类（比如 JDBC 加载驱动）。它需要捕获异常。
2. `类名.class`：这种方式最简单，但需要编译期就导入该类，一般用于参数传递（比如 `new ArrayList<>(String.class)`）。
3. `对象.getClass()`：必须要有实例化后的对象才能用。

区别在于：`Class.forName()` 是动态加载，最灵活；`类名.class` 是静态加载；`getClass()` 依赖实例。在编写框架或通用工具时，第一种用得最多。

深拷贝浅拷贝

这部分内容其实是考察 Java 内存模型（堆/栈）的理解。

面试官想听到的关键词是：引用地址 vs 实际对象。

深拷贝和浅拷贝的核心区别

Q：什么是深拷贝和浅拷贝？它们的区别是什么？

A:

面试官你好。这两者的核心区别在于对‘引用类型’成员变量的处理方式不同。

1. 浅拷贝:

- 它只复制对象本身，对于对象内部的引用类型变量（比如对象里有个 `List`），它只复制了引用的地址，没有复制引用的对象。
- **后果**：原对象和新对象指向堆内存中的**同一个实体**。修改新对象的 `List`，原对象的 `List` 也会跟着变。
- Java 中默认的 `Object.clone()` 方法就是浅拷贝。

2. 深拷贝:

- 它不仅复制对象本身，还会**递归地复制**对象内部所有引用指向的对象。
- **后果**：原对象和新对象在内存中是**完全独立**的。修改其中一个，互不影响。

如何实现深拷贝

Q：在项目中，如果让你实现一个对象的深拷贝，你会怎么做？

A:

虽然理论上可以通过重写 `clone()` 方法，在里面手动对每个引用对象再次调用 `clone` 来实现，但这在实际开发中**非常不推荐**。因为代码太繁琐，容易漏写，且维护成本高。

在工程实践中，我通常使用**序列化**来实现深拷贝。具体有两种方式：

1. JSON 序列化（推荐）：

- 这是最常用的方法。利用 Jackson 或 FastJson 把对象转成 JSON 字符串，再转回 Java 对象。
- 比如：`User copy = JSON.parseObject(JSON.toJSONString(original), User.class);`
- 优点是代码简单，通用性强。

2. Java 原生序列化:

- 让对象实现 `Serializable` 接口，然后通过**字节流**写出去再读回来。
- 这种方式比较重，但在处理一些不支持 JSON 转换的复杂对象时（比如对象里有循环引用）会用到。

String与浅拷贝

Q: String 也是引用类型，那为什么我在浅拷贝一个对象后，修改 String 字段，原对象没变呢？

A:

String 是引用类型，浅拷贝时复制的也是地址。但是，**String 是不可变类**。当我修改副本的 String 属性时（比如 `user.name = "NewName"`），并非修改了堆内存中原来的那个字符串对象，而是指向了一个新的字符串对象。原对象的引用依然指向老的字符串，所以看起来互不影响。

异常机制

异常机制

异常机制这块很适合面试官连续追问，通常会从 `Exception` 和 `Error` 区别 开始，一路问到受检异常、运行时异常、`throw/throws`、`finally`。

Error 和 Exception 的区别

Q: Error 和 Exception 有什么区别?

它们都继承自 `Throwable`，但语义完全不同。

`Error` 通常表示系统级、JVM 级的问题，比如：

- `OutOfMemoryError`
- `StackOverflowError`

这类问题一般不是业务代码能优雅恢复的，所以通常不建议捕获后继续运行。

`Exception` 则更多表示程序运行过程中的异常情况，往往是可以通过代码处理的，比如：

- 文件不存在
- 参数非法
- 空指针
- 数组越界

所以一句话总结就是：

`Error` 更偏“系统已经出大问题了”，`Exception` 更偏“程序要显式处理异常场景”。

受检异常和运行时异常

Q: 受检异常和运行时异常的区别是什么?

Java 的异常大致可以分为两类：

第一类是受检异常（Checked Exception）。

这类异常在编译期就会被检查，方法如果可能抛出它，要么 `try-catch`，要么在方法签名里 `throws`。

常见的比如：

- `IOException`
- `SQLException`
- `ClassNotFoundException`

第二类是运行时异常（RuntimeException）。

这类异常不强制在编译期处理，通常代表代码逻辑本身有问题。

常见的比如：

- `NullPointerException`
- `ArrayIndexOutOfBoundsException`
- `ClassCastException`

所以可以把它理解为：

受检异常强调“外部环境可能出错”，运行时异常更强调“代码本身写得有问题”。

throw 和 throws 的区别

Q: throw 和 throws 有什么区别？

`throw` 和 `throws` 都和异常相关，但位置和作用完全不同。

`throw` 用在方法体内部，表示主动抛出一个具体异常对象。

`throws` 用在方法签名上，表示这个方法可能抛出哪些异常。

所以一句话记忆：

- `throw` 是“真的抛”
- `throws` 是“先声明可能会抛”

finally 一定会执行吗

Q: finally 中的代码一定会执行吗?

大多数正常场景下, `finally` 都会执行, 所以它很适合写资源释放逻辑。

但它不是绝对一定执行, 典型例外包括:

- 在 `try` 中调用了 `System.exit()`
- JVM 直接崩溃
- 程序死机、断电
- 线程陷入死循环, 根本没有机会走到 `finally`

所以更准确的说法是:

在 JVM 还能正常继续执行控制流的前提下, `finally` 基本都会执行。

追问1: finally 里写 return 有什么问题?

不推荐。

因为它会覆盖 `try` 或 `catch` 里的返回值, 代码可读性很差, 也容易埋坑。

面试里最好直接表态: `finally` 应该只做清理动作, 不要写 `return`。

异常处理的最佳实践

Q: 项目里你是怎么设计异常体系的?

如果是在项目里, 我一般会这样设计:

- 业务异常和系统异常分开
- 自定义业务异常, 承载错误码和错误信息
- Controller 层统一做全局异常处理
- 日志里保留上下文信息, 但不给前端暴露内部堆栈

也就是说, 异常处理要做到两件事:

- 对用户或调用方来说, 返回的信息清晰、稳定
- 对开发排障来说, 日志信息充分、可追踪

代理

代理

JDK 动态代理& CGLIB

Q：了解动态代理吗？JDK 动态代理和 CGLIB 有什么区别？

A：

面试官你好，动态代理主要用于在**不修改源码**的情况下，对方法进行功能增强（比如加日志、事务控制），它是 Spring AOP 的底层基石。在 Java 中主要有两种实现方式：

1. **JDK 动态代理**：这是 Java 原生提供的。
 - 它的核心是 `Proxy` 类和 `InvocationHandler` 接口。
 - **特点是**：它要求被代理的类**必须实现接口**。因为它生成的代理对象默认继承了 `Proxy` 类，利用反射机制调用目标方法。
2. **CGLIB 动态代理**：这是一个第三方高性能代码生成包。
 - 它在内存中动态生成一个被代理类的子类。
 - **特点是**：它**不要求实现接口**，是通过**继承**的方式来重写父类方法实现增强的。

为什么 JDK 代理必须基于接口

Q：为什么 JDK 动态代理强制要求目标类必须实现接口？不能像 CGLIB 那样直接代理类吗？

A:

这主要受限于 Java 的单继承机制。因为 JDK 动态代理生成的那个 Proxy 代理类，它在底层代码中已经显式继承了 `java.lang.reflect.Proxy` 这个父类。Java 语法规则规定，一个类只能有一个父类。既然它已经继承了 `Proxy` 类，就无法再继承业务目标类了。所以，它只能通过实现接口的方式来与目标类建立联系，这就是为什么 JDK 代理必须基于接口的原因。

追问1: 那 CGLIB 基于继承机制，有什么缺点或者限制吗

的。因为 CGLIB 是通过生成子类并重写方法来实现代理的。在 Java 中，`final` 修饰的方法和类是无法被继承或重写的。所以，如果目标类被标记为 `final`，或者目标方法被标记为 `final`，CGLIB 就无法对它进行代理（增强会失效），这是使用继承机制天然的局限性。

静态代理 vs 动态代理

Q: 静态代理和动态代理有什么区别？

A:

静态代理是指在编译期就已经写好了代理类的 `.java` 文件。比如我写一个 `UserProxy` 类，手动实现 `UserService` 接口，在内部调用 `Target` 的方法前后打印日志。它的缺点很明显：代码冗余。如果我有 100 个 Service 需要加日志，我就得写 100 个代理类，维护成本极高。

动态代理则是把这个过程推迟到了运行期。我们不需要手动写代理类，而是让 JVM（JDK）或者字节码框架（CGLIB）在内存中自动生成代理类的字节码。这样我们只需要写一套通用的增强逻辑，就能应用到成千上万个不同的类上，实现了代码的解耦和复用。

String相关

String相关

String 是 Java 面试里非常高频的基础题，常见考法一般是：底层结构、为什么不可变、字符串常量池、以及和 `StringBuffer`、`StringBuilder` 的区别。

String 的底层实现

Q: String 底层是怎么实现的? JDK1.8 和 JDK1.9 之后有什么区别?

在 JDK1.8 之前，String 底层主要是 `char[]`。

在 JDK1.9 之后，底层改成了 `byte[] + coder` 的方式，也就是所谓的紧凑字符串（Compact Strings）。

这样做的原因是：很多字符串其实只需要 Latin-1 编码，一个字节就够了。

如果继续统一使用 `char[]`，每个字符两个字节，会造成额外的内存浪费。

所以 JDK1.9 之后，String 更关注 节省内存占用。

String 为什么不可变

Q: 为什么 String 被设计成不可变的?

String 设计成不可变，核心原因主要有三个：

第一是安全性。

字符串常用于类加载、数据库连接、网络地址、用户名密码等敏感场景。

如果 String 可变，那么一个方法拿到字符串引用后，可能在外部被偷偷改掉，这会带来很大的安全风险。

第二是线程安全。

String 不可变后，多线程之间共享同一个字符串对象时，不需要额外同步。

第三是便于缓存和复用。

比如字符串常量池、字符串哈希值缓存，都依赖于“内容一旦创建就不变”这个前提。

追问1: String 不可变是怎么做到的？

主要靠两点：

- 底层存储数组是私有的，外部拿不到可直接修改的入口
- String 类本身是 `final`，不能被继承后破坏语义

所以可以理解为：既不让你改内部状态，也不让子类改规则。

字符串常量池

Q：什么是字符串常量池？有什么好处？

字符串常量池是堆中的一块特殊区域，用于复用内容相同的字符串常量。

比如：

这里 `s1` 和 `s2` 指向的是常量池中的同一个对象。

这样做的好处主要是：

- 节省内存
- 避免重复创建对象
- 提升字符串比较和使用效率

本质上，它是 JVM 基于“String 不可变”做的一种缓存优化。

追问1: `new String("hello")` 会创建几个对象?

这取决于常量池里原来有没有 `"hello"`。

- 如果常量池没有, 那么会先在常量池中创建一个, 再在堆中创建一个 `new String` 对象, 一共两个。
- 如果常量池已经有了, 那么只会在堆里再创建一个对象, 一共一个新对象。

intern 方法

Q: `intern()` 方法是做什么的?

`intern()` 的作用是尝试把字符串对象和常量池中的字符串关联起来。

如果常量池中已经有相同内容的字符串, 就返回池中的引用;

如果没有, 就把该字符串放入常量池 (或让池中记录这个引用) 并返回池中的引用。

因此它常用于:

- 大量重复字符串的去重
- 需要统一引用语义的场景

但也不能滥用, 因为如果字符串非常多且重复率不高, 反而会增加常量池压力。

String、StringBuffer、StringBuilder 的区别

Q: String、StringBuffer、StringBuilder 有什么区别?

三者最核心的区别在于 **可变性** 和 **线程安全**。

String:

- 不可变
- 因为不可变，所以天然线程安全
- 适合字符串内容基本不变的场景

StringBuffer:

- 可变
- 方法大多带同步，线程安全
- 适合多线程下频繁修改字符串，但性能相对一般

StringBuilder:

- 可变
- 不保证线程安全
- 单线程下拼接性能最好，也是开发中最常用的可变字符串类

如果面试官问项目中怎么选，我的回答会很直接：

单线程拼接优先用 **StringBuilder**；多线程且共享时再考虑 **StringBuffer**；普通常量或少量拼接直接用 **String**。

String 的长度限制

Q: String 有长度限制吗?

有，但编译期和运行期要分开看。

编译期，字符串字面量会进入 class 文件常量池，对应 `CONSTANT_Utf8_info` 结构，这一层有长度限制。

运行期，String 的底层长度本质上受数组长度和 JVM 可用内存限制，不是说能无限大。

面试里这里不用展开得太底层，知道“存在限制，而且编译期和运行期的限制来源不同”就够了。

迭代器

迭代器

为什么要有 Iterator? for-each 循环的本质

Q: Java 既然已经有 for 循环了, 为什么还要设计 Iterator? 平时用的增强 for 循环 (for-each) 和它有什么关系?

A:

Iterator 的主要作用是‘统一’和‘解耦’。不同的集合底层结构不同, 比如 ArrayList 是数组, LinkedList 是链表, HashSet 是哈希表。如果没有 Iterator, 我就需要了解每种集合的内部结构才能遍历它 (比如数组用下标, 链表用指针)。Iterator 提供了一套统一的 API (hasNext、next), 让我不需要关心容器的底层实现就能遍历所有集合。

至于增强 for 循环 (for-each), 它其实就是 Java 提供的一个语法糖。在编译成字节码后, 它本质上还是被转成了 Iterator 的调用。所以用 for-each 遍历 List, 本质上就是在用 Iterator。

原理:

- **设计模式:** 这是典型的**迭代器模式**。提供一种方法顺序访问一个聚合对象中的各个元素, 而又不暴露其内部的表示。
- **语法糖验证:** 可以查看 `.class` 反编译后的代码, 会发现 `for(String s : list)` 变成了 `while(iterator.hasNext()) { ... }`。

追问1: 那既然 for-each 底层就是 Iterator, 为什么我在 for-each 里面删除元素会报错

虽然 for-each 底层是 Iterator, 但它隐藏了**迭代器对象**。当我们调用 `list.remove()` 删除元素时, 集合的结构改变了, 但背后的那个隐形迭代器并不知道, 它的预期状态和实际状态不一致了, 所以下一次循环时就会触发 **Fail-Fast** 机制, 抛出异常。如果要在遍历中删除, 必须显式获取 Iterator 对象, 并调用 `iterator.remove()`。

Fail-Fast机制

Q: 什么是 Fail-Fast 机制? 它是怎么检测到并发修改的?

Fail-Fast (快速失败) 是 Java 集合的一种错误检测机制。它的初衷是: 一旦发现数据可能不一致, 立马抛异常, 宁可停止程序也不给出错误结果。

实现原理是基于一个变量 `modCount` :

1. 集合类 (如 `ArrayList`) 内部有一个 `modCount` 变量, 用来记录集合被修改的次数 (add/remove 都会让它 +1) 。
2. 当我们创建迭代器时, 迭代器内部会复制一份 `modCount` , 记为 `expectedModCount` 。
3. 此后每次调用 `iterator.next()` , 迭代器都会检查: `modCount` 是否等于 `expectedModCount` ?
4. 如果不相等, 说明在迭代期间, 集合被其他代码 (或者多线程) 修改了, 于是直接抛出 `ConcurrentModificationException` 。

追问1: Fail-Fast 是线程安全的吗? 有没有不抛异常的集合

Fail-Fast 不是线程安全的, 它只是尽力检测错误, 不能作为并发控制的手段。如果需要 Fail-Safe (安全失败) 的集合, 可以使用 `java.util.concurrent` 包下的类, 比如 `CopyOnWriteArrayList`。它的原理是写时复制, 遍历的是原数组的快照, 修改的是新数组, 所以永远不会抛出并发修改异常, 但可能读不到最新的数据 (弱一致性) 。

泛型

泛型

什么是泛型？什么是类型擦除？

Q：谈谈你对 Java 泛型的理解？什么是类型擦除？

A：

泛型的主要目的是为了增强代码的类型安全和重用性。在没有泛型之前，我们从集合中取对象必须强制转换，容易抛出 `ClassCastException`。有了泛型，编译器就能帮我们做类型检查。

Java 的泛型其实是伪泛型，这是通过类型擦除实现的。简单来说，Java 的泛型只存在于编译阶段。当编译器把 `.java` 翻译成 `.class` 文件时，它会把所有的泛型参数（如 `<T>`，`<String>`）都擦除掉

【拓展】通配符与 PECS 原则

Q： `List`、`List` 和 `List` 有什么区别？平时怎么用？

A：这三个都是泛型的通配符，区别主要在于读写权限不同，使用时遵循 PECS 原则（Producer Extends, Consumer Super）：

1. **? extends T**（上界通配符）：

- 表示集合里装的是 T 的子类。
- **特点**：只能读（取出来一定是 T），不能写（因为不知道具体是哪个子类，怕类型不匹配）。
- **适用**：它适合作为生产者（Producer），也就是只读数据的场景。

2. **? super T**（下界通配符）：

- 表示集合里装的是 T 的父类。
- **特点**：可以写（可以安全地放入 T 及其子类），但读出来只能是 Object（类型丢失）。
- **适用**：它适合作为消费者（Consumer），也就是写入数据的场景。

3. **?**（无界通配符）：

- 等价于 **? extends Object**，通常只用于只读且不关心具体类型的场景。

IO模型

IO模型

IO 模型这块在 Java 八股里，最常见的就是 `BIO / NIO / AIO`，以及“同步异步、阻塞非阻塞”的区别。

BIO、NIO、AIO 的区别

Q: BIO、NIO、AIO 分别是什么？有什么区别？

如果只抓面试最核心的区别，我会这样总结：

BIO 是同步阻塞 IO。

线程发起读写后，会一直阻塞等待结果返回。模型简单，但高并发下一个连接往往就要占一个线程，线程资源很容易成为瓶颈。

NIO 是同步非阻塞 IO。

它引入了 `Channel`、`Buffer`、`Selector` 等概念，可以基于 IO 多路复用，让一个线程同时管理多个连接。

AIO 是异步非阻塞 IO。

应用发起 IO 请求后，不需要主动轮询，等底层完成后通过回调通知应用继续处理。

所以三者的主线可以理解为：

- BIO：简单，但扩展性差
- NIO：工程里更常见，适合高并发网络编程
- AIO：理论上更先进，但在 Java 生态里使用没有 NIO 普遍

同步和异步

Q: 同步和异步怎么理解？

同步和异步，强调的是 **任务结果的获取方式**。

同步 是我发起一个调用后，要等这个调用完成，才能继续往下走。

异步 是我发起调用后，不必原地等待，后续通过通知、回调、Future 等方式拿结果。

所以同步异步的核心不是“线程会不会卡住”，而是：

调用方是不是必须自己等结果。

阻塞和非阻塞

Q：阻塞和非阻塞怎么理解？

阻塞和非阻塞，强调的是 **线程在等待结果时的状态**。

阻塞：线程会挂起等待，期间不能干别的事。

非阻塞：线程发起请求后，如果条件不满足，会立刻返回，线程可以继续做别的事。

所以阻塞非阻塞的核心是：

等待期间线程是不是被卡住。

追问1：同步一定阻塞吗？异步一定非阻塞吗？

不一定。

这是面试里特别爱考的点。

- 同步不一定阻塞，比如同步轮询也可以是非阻塞的
- 异步也不一定完全非阻塞，某些框架内部仍然可能用阻塞线程池来承接任务

所以这两组概念不是同一维度。

Java 工程里怎么回答这类题

Q：项目里如果面试官问你 NIO 的实际价值，你会怎么答？

我会从“为什么需要它”来回答。

在高并发网络场景下，如果继续沿用 BIO，就会出现“一连接一线程”的问题。

当连接数很大、请求里还夹杂大量阻塞等待时，线程数量会迅速膨胀，带来上下文切换和内存开销。

NIO 通过 Selector 把多个连接复用到少量线程上处理，因此更适合：

- Netty 这类网络框架
- 网关、IM、RPC 通信
- 高并发长连接场景

所以 NIO 的实际价值，本质上是：

用更少的线程支撑更多连接。

JDK新特性

JDK新特性

JDK 新特性的考察，一般不是让你背版本号，而是看你能不能说出“这个特性解决了什么问题、适合什么场景、项目里是否真用过”。

JDK5 到 JDK8 的核心变化

Q: JDK5、JDK8 分别有哪些你觉得最重要的新特性？

面试官你好，如果只抓面试里最常见、最有代表性的特性，我会这样回答：

JDK5 重点是引入了 泛型、枚举、注解、自动装箱拆箱。

其中泛型和注解影响最大，前者提升了类型安全，后者为 Spring 这种框架式开发打下了基础。

JDK8 重点是 Lambda 表达式、Stream 流、Optional、接口默认方法、时间 API。

这一版最核心的变化，是 Java 开始明显吸收函数式编程思想，让集合处理、空值处理和时间处理更现代化。

如果从“项目里最常用”的角度看，JDK8 的几个特性几乎是现在 Java 开发的标配。

Lambda 表达式

Q: 什么是 Lambda 表达式？它解决了什么问题？

Lambda 表达式本质上是 **对匿名内部类的简化**，它的核心价值是：**把函数当成参数传递**。

以前如果我想给集合排序、给线程池提交任务、给 Stream 做过滤，往往需要写很多模板化代码；有了 Lambda 之后，可以把关注点放在“做什么”，而不是“怎么写一个匿名类”。

比如：

相比传统匿名内部类，代码更短，可读性也更强。

追问1: Lambda 的底层是匿名内部类吗?

不是简单地等价于匿名内部类。

JVM 层面主要借助 `invokedynamic` 指令和 `LambdaMetafactory` 来实现，因此它不是每次都真的生成一个新的匿名类文件。

你可以把它理解为：语法层面像匿名类，运行时实现更轻量、更灵活。

Stream 流

Q: Stream 流的核心价值是什么?

Stream 流的核心价值，是把集合处理从“命令式写法”变成“声明式写法”。

传统写法里，我们往往要自己写 `for` 循环、判断条件、临时变量；而 Stream 更强调“我要过滤什么、映射什么、聚合什么”。

常见操作包括：

- `filter` : 过滤
- `map` : 转换
- `sorted` : 排序
- `collect` : 收集
- `count` : 统计

例如：

它的优势是代码表达力强，尤其适合做列表筛选、转换、统计类场景。

追问1: Stream 一定比 for 循环快吗?

不一定。

如果只是简单遍历，`for` 循环往往更直接，性能未必差。

Stream 的核心优势主要是 可读性、链式表达、便于组合操作。

如果是非常高频、极度关注性能的热点路径，还是要以压测结果为准，不能机械地说 Stream 一定更好。

Optional

Q: Optional 是干什么的？为什么它能减少空指针问题？

Optional 是一个“可能有值，也可能没值”的容器类。

它的意义不是彻底消灭 `null`，而是 强迫开发者显式处理空值。

例如以前我们可能这样写：

而使用 Optional 后，可以写成更清晰的链式处理。

它比较适合：

- 返回值可能为空的方法
- 多层对象取值时的空值兜底
- 需要默认值、异常抛出、条件分支时

追问1: Optional 适合当实体类字段吗？

一般不推荐。

Optional 更适合作为方法返回值，而不是实体类字段或 RPC 传输字段。

因为这样会增加序列化、框架兼容、代码理解上的复杂度。

JDK21 的虚拟线程

Q: 你了解 JDK21 的虚拟线程吗？

虚拟线程可以理解成由 JVM 管理的轻量级线程。

传统平台线程是一个 **Java 线程** 对应一个操作系统线程，创建和切换成本都比较高；

而虚拟线程是 **大量用户态线程复用少量平台线程**，因此更适合高并发、阻塞等待多的场景。

它特别适合：

- 大量 IO 阻塞任务
- 一请求一线程模型
- 需要提升并发量，但不想手动把代码改造成异步回调风格

它的优势主要是：

- 创建成本低
- 切换成本低
- 编程模型仍然接近同步代码，心智负担比手写异步低

追问1：虚拟线程会完全替代线程池吗？

不会这么绝对。

虚拟线程更适合“高并发 IO 阻塞”场景，但 CPU 密集型任务的瓶颈仍然是 CPU 本身。

如果是计算型任务，线程数量仍然不能无限增加。

所以更准确的说法是：**虚拟线程降低了阻塞型并发编程的成本，但没有消灭线程模型和资源调度的基本约束。**

线程基础

线程基础

线程基础题通常是 JUC 面试的开场题，面试官会先确认你对线程状态、创建方式、常见方法这些最基础的概念是否清楚。

线程的创建方式

Q: Java 创建线程有哪些方式?

常见的创建线程方式主要有四种:

第一种，继承 Thread 类。

直接重写 `run()` 方法，简单直观，但耦合度高，不够灵活。

第二种，实现 Runnable 接口。

这是更常见的方式，因为它把“任务”和“线程”解耦了。

第三种，实现 Callable 接口配合 FutureTask。

它适合有返回值、还能抛异常的异步任务。

第四种，通过线程池创建线程。

这是生产环境最推荐的方式，因为线程池可以复用线程、统一管理资源。

如果面试官问项目里怎么用，我会直接回答：**生产环境优先线程池，不会手动频繁 new Thread。**

线程的状态流转

Q: Java 线程有哪些状态?

Java 在线程状态上，`Thread.State` 枚举里定义了六种：

- `NEW`
- `RUNNABLE`
- `BLOCKED`
- `WAITING`
- `TIMED_WAITING`
- `TERMINATED`

比较容易混淆的是：

`RUNNABLE` 不等于“此刻一定正在 CPU 上运行”，它表示线程已经具备运行资格，可能在运行，也可能在等待 CPU 调度。

`BLOCKED` 一般是抢锁失败。

`WAITING` 一般是调用了 `wait()`、`join()` 等无限等待方法。

`TIMED_WAITING` 一般是 `sleep()`、带超时的 `wait()`、带超时的 `join()`。

start 和 run 的区别

Q: `start()` 和 `run()` 有什么区别？

`run()` 只是普通方法调用，不会启动新线程。

`start()` 才会真正创建新的执行路径，让 JVM 去调度这个线程，然后由新线程去执行 `run()` 方法。

所以如果直接调用 `run()`，本质上还是主线程自己在执行。

sleep 和 wait 的区别

Q: `sleep` 和 `wait` 有什么区别？

这是一个特别经典的对比题。

`sleep` 是 `Thread` 类的方法。

它表示让当前线程休眠一段时间，但 **不会释放锁**。

`wait` 是 `Object` 类的方法。

它必须在同步代码块里调用，调用后线程会进入等待状态，并且 **会释放当前持有的对象锁**。

所以一句话记：

- `sleep` ：线程休眠，不释放锁
- `wait` ：线程等待，释放锁

volatile关键字

volatile关键字

volatile 是并发编程中的高频考点，面试官通常会从"volatile 有什么作用"切入，然后深挖可见性、重排序等底层原理。

volatile的作用

Q: volatile 关键字有什么作用？

面试官你好，volatile 主要有两个作用：

第一个是**保证可见性**。当一个线程修改了 volatile 变量的值，这个新值会立即刷新到主内存，其他线程读取这个变量时，会从主内存中读取最新值，而不是使用自己工作内存中的旧值。

第二个是**禁止指令重排序**。编译器和处理器为了优化性能，可能会对指令进行重排序，但 volatile 通过内存屏障，禁止特定类型的重排序，保证代码按照我们预期的顺序执行。

需要注意的是，volatile **不能保证原子性**，这是它和 synchronized 的一个重要区别。

追问1: 什么是可见性问题？为什么会出现？

可见性问题是指出一个线程修改了共享变量的值，其他线程无法立即看到这个修改。

出现这个问题的原因是，现代计算机为了提高性能，每个 CPU 核心都有自己的缓存。线程在读取变量时，会先从自己的工作内存（缓存）中读取，而不是直接从主内存读取。当一个线程修改了变量，可能只是修改了自己缓存中的值，还没有刷新到主内存，其他线程的缓存中还是旧值，就会出现可见性问题。

举个例子，比如我有一个 `boolean flag = false`，线程 A 把它改成 true，但线程 B 可能一直读到 false，导致程序出现意外的行为。给 flag 加上 volatile 关键字就能解决这个问题。

volatile的底层原理

Q: volatile 是如何保证可见性和禁止重排序的?

面试官你好, volatile 的底层实现依赖于**内存屏障** (Memory Barrier) 。

对于可见性, 当写一个 volatile 变量时, JMM (Java 内存模型) 会在写操作后插入一个 Store 屏障, 强制将缓存中的数据刷新到主内存。当读一个 volatile 变量时, JMM 会在读操作前插入一个 Load 屏障, 强制从主内存中读取最新值, 使工作内存中的缓存失效。

对于禁止重排序, volatile 通过内存屏障, 禁止了几种重排序: 写 volatile 变量之前的操作不能重排序到写操作之后, 读 volatile 变量之后的操作不能重排序到读操作之前。这样就保证了代码的执行顺序符合我们的预期。

追问1: 什么是指令重排序? 为什么要重排序?

指令重排序是指编译器和处理器为了优化性能, 可能会改变指令的执行顺序, 但会保证单线程下的执行结果不变。

重排序的目的是提高 CPU 的执行效率, 减少流水线停顿。但在多线程环境下, 重排序可能导致程序出现意外的结果, 所以在某些关键位置, 我们需要用 volatile 来禁止重排序。

volatile不能保证原子性

Q: volatile 能保证原子性吗? 为什么?

面试官你好, volatile **不能保证原子性**。

我举个例子, 比如 `count++` 这个操作, 看起来是一条语句, 但实际上包含三个步骤: 读取 count 的值、将 count 的值加 1、将新值写回 count。

在多线程环境下, 可能出现这样的情况: 线程 A 和线程 B 同时读取 `count = 0`, 然后都计算出 1, 最后都写回 1。这样执行了两次 `count++`, 但最终 count 只等于 1, 而不是期望的 2。

如果要保证原子性, 我们需要使用 `synchronized` 或者 `Lock`, 或者使用 `AtomicInteger` 这样的原子类。

追问1: 什么情况下 volatile 可以保证线程安全?

当满足以下两个条件时，volatile 可以保证线程安全：

第一，对变量的写操作不依赖当前值，比如直接赋值 `flag = true`，而不是 `count++` 这种依赖当前值的操作。

第二，该变量不需要与其他变量共同参与不变约束。

典型的场景就是用 volatile 修饰一个布尔变量作为状态标志，或者在双重检查锁定（DCL）单例模式中使用。

volatile与synchronized的区别

Q: volatile 和 synchronized 有什么区别？

面试官你好，volatile 和 synchronized 的区别主要有以下几点：

首先，**作用范围不同**。volatile 只能修饰变量，而 synchronized 可以修饰方法和代码块。

其次，**原子性不同**。volatile 不能保证原子性，synchronized 可以保证原子性。但它们都能保证可见性和有序性。

第三，**性能不同**。volatile 是轻量级的同步机制，性能更好，不会阻塞线程。synchronized 是重量级的同步机制，可能会阻塞线程，但 JDK 1.6 之后做了很多优化，性能也提升了不少。

最后，**使用场景不同**。volatile 适用于一写多读的场景，比如状态标志。synchronized 适用于需要原子性的场景，比如 count++ 这种操作。

总结一下，能用 synchronized 就用 synchronized，除非只需要保证可见性，不需要原子性，那用 volatile 就够了。

追问1: 既然 synchronized 功能更强大，为什么还要用 volatile？

因为 volatile 的性能更好，开销更小。如果我只需要保证可见性，不需要原子性，用 volatile 就够了，没必要用 synchronized 增加额外的开销。

volatile的典型应用场景

Q: volatile 有哪些典型的应用场景？

面试官你好，volatile 的典型应用场景主要有两个：

第一个是**状态标志**。最常见的就是用 volatile 修饰一个布尔变量，作为线程的停止标志。比如我有一个 `volatile boolean stop = false`，在线程的 run 方法里用 `while (!stop)` 循环执行任务，当其他线程调用 `stop = true` 时，执行任务的线程能立即看到这个修改，退出循环。

第二个是**双重检查锁定（DCL）单例模式**。在单例模式中，我们会用 volatile 修饰 instance 变量。这是因为 `instance = new Singleton()` 这行代码实际上包含三个步骤：分配内存空间、初始化对象、将 instance 指向分配的内存地址。由于指令重排序，可能变成先分配内存、再指向内存、最后初始化对象。如果线程 A 执行到指向内存这一步时，线程 B 进入第一次检查，发现 instance 不为 null，直接返回，但此时对象还没有初始化完成，就会出现空指针异常。volatile 的作用就是禁止这种重排序，保证对象初始化完成后，才将引用赋值给 instance。

追问1：DCL 单例模式中，为什么要双重检查？

第一次检查是在 synchronized 外面，目的是避免每次调用 getInstance() 都进入同步块，提高性能。如果 instance 已经初始化了，直接返回，不需要加锁。

第二次检查是在 synchronized 里面，目的是防止多个线程同时通过第一次检查，进入同步块后，重复创建实例。

比如线程 A 和 B 同时调用 getInstance()，都通过了第一次检查，然后线程 A 先获取锁，创建了实例。线程 B 获取锁后，如果不进行第二次检查，就会再创建一个实例，这就违背了单例模式的初衷。

ThreadLocal

ThreadLocal

ThreadLocal 是 JUC 中的高频考点，面试官通常会问"ThreadLocal 是什么"，然后深挖内存泄漏问题。

ThreadLocal是什么

Q: ThreadLocal 是什么？有什么作用？

面试官你好，ThreadLocal 是 Java 提供的一个**线程局部变量**工具类。它的作用是让每个线程都拥有自己独立的变量副本，线程之间互不干扰。

简单来说，就是我在 ThreadLocal 里存一个值，线程 A 存的是 A 的值，线程 B 存的是 B 的值，它们互相看不到对方的值，这样就避免了线程安全问题。

典型的使用场景有这么几个：第一个是**数据库连接管理**，每个线程持有自己的数据库连接。第二个是**用户信息传递**，在 Web 应用中，把用户信息存在 ThreadLocal 里，整个请求链路都能访问。第三个是**SimpleDateFormat**，因为 SimpleDateFormat 不是线程安全的，可以用 ThreadLocal 让每个线程持有自己的实例。

追问1: 为什么不直接用局部变量，要用 ThreadLocal?

局部变量确实是线程安全的，但它的作用域只在方法内部。如果我想在整个线程的多个方法之间共享数据，又不想通过参数传递，就可以用 ThreadLocal。

比如在 Web 应用中，我在 Filter 里把用户信息存到 ThreadLocal，然后在 Service、Dao 等各层都能直接获取，不需要层层传递参数。

ThreadLocal的底层原理

Q: ThreadLocal 的底层实现原理是什么？

面试官你好，ThreadLocal 的底层实现其实挺巧妙的。

每个 **Thread** 对象内部都有一个 **ThreadLocalMap** 属性，这个 Map 的 Key 是 ThreadLocal 对象，Value 是我们存储的值。

当我们调用 `threadLocal.set(value)` 时，实际上是：

1. 获取当前线程的 ThreadLocalMap
2. 把 ThreadLocal 对象作为 Key，value 作为 Value，存到这个 Map 里

当我们调用 `threadLocal.get()` 时，实际上是：

1. 获取当前线程的 ThreadLocalMap
2. 用 ThreadLocal 对象作为 Key，从 Map 里取出 Value

所以，数据实际上是存在**线程对象**里的，而不是 ThreadLocal 对象里。这样每个线程都有自己的 Map，自然就隔离了。

追问1: ThreadLocalMap 和 HashMap 有什么区别？

ThreadLocalMap 和 HashMap 虽然都是 Map，但实现上有很大区别：

第一个是**解决冲突的方式不同**。HashMap 用的是链表法（或红黑树），而 ThreadLocalMap 用的是**开放地址法**（线性探测）。当发生 Hash 冲突时，ThreadLocalMap 会往后找下一个空位置。

第二个是**Key 的类型不同**。ThreadLocalMap 的 Key 是 **WeakReference**，也就是弱引用，这是为了防止内存泄漏。

第三个是**扩容机制不同**。HashMap 的扩容阈值是 0.75，而 ThreadLocalMap 的扩容阈值是 2/3。

ThreadLocal的内存泄漏问题

Q: ThreadLocal 会导致内存泄漏吗？为什么？

面试官你好，ThreadLocal 可能会导致内存泄漏，这是一个很经典的问题。

我先说一下原因：ThreadLocalMap 的 Key 是 **WeakReference**（弱引用），Value 是**强引用**。

当 ThreadLocal 对象没有外部强引用时，在 GC 时，Key 会被回收，变成 null。但是 Value 还是强引用，不会被回收。这样就会出现 Key 为 null，但 Value 还在的情况，这个 Value 就永远访问不到了，但又占着内存，这就是内存泄漏。

而且，如果使用的是线程池，线程会被复用，线程对象不会被回收，ThreadLocalMap 也不会被回收，这些泄漏的 Value 就会一直存在，越积越多。

追问1：为什么 Key 要设计成弱引用？

这是为了降低内存泄漏的风险。

如果 Key 是强引用，那么即使外部没有对 ThreadLocal 的引用了，ThreadLocalMap 里还持有强引用，ThreadLocal 对象就永远不会被回收。这样不仅 ThreadLocal 对象泄漏了，Value 也泄漏了。

用弱引用的话，至少 ThreadLocal 对象可以被回收，只是 Value 可能泄漏。而且 ThreadLocalMap 在 set、get、remove 时，会主动清理 Key 为 null 的 Entry，这样可以一定程度上缓解内存泄漏。

追问2：如何避免 ThreadLocal 的内存泄漏？

最佳实践是：用完之后一定要调用 remove() 方法。

```
1  ThreadLocal<User> userThreadLocal = new ThreadLocal<>();
2
3  try {
4      userThreadLocal.set(user);
5      // 业务逻辑
6  } finally {
7      userThreadLocal.remove(); // 必须调用 remove
8  }
```

在 Web 应用中，通常在 Filter 或拦截器的 finally 块中调用 remove()，确保请求结束后清理 ThreadLocal。

另外，如果可以的话，尽量把 ThreadLocal 定义成 **static final** 的，这样 ThreadLocal 对象本身不会被回收，就不会出现 Key 为 null 的情况。

ThreadLocal的使用场景

Q: ThreadLocal 有哪些典型的使用场景？

面试官你好，ThreadLocal 的典型使用场景主要有这么几个：

第一个是数据库连接管理。在传统的 JDBC 编程中，每个线程需要持有自己的数据库连接，可以用 ThreadLocal 来存储。这样在事务管理时，同一个线程的多个 DAO 操作可以共享同一个连接。

第二个是用户信息传递。在 Web 应用中，我们可以在 Filter 或拦截器里把用户信息（如用户 ID、用户名）存到 ThreadLocal，然后在 Controller、Service、Dao 等各层都能直接获取，不需要层层传递参数。Spring Security 就是这么做的。

第三个是 SimpleDateFormat。SimpleDateFormat 不是线程安全的，如果多个线程共享一个实例，会出现问题。可以用 ThreadLocal 让每个线程持有自己的 SimpleDateFormat 实例。

第四个是分布式追踪的 Traceld。在微服务架构中，我们会给每个请求分配一个 Traceld，用于追踪整个调用链路。这个 Traceld 可以存在 ThreadLocal 里，方便在日志中打印。

追问1: 你在项目中是怎么用 ThreadLocal 的？

在我的项目中，主要用 ThreadLocal 来传递用户信息。

我们在网关层会解析 JWT Token，获取用户 ID 和用户名，然后通过 HTTP Header 传递给下游服务。下游服务在拦截器里把用户信息存到 ThreadLocal，这样在整个请求处理过程中，任何地方都能通过 `UserContext.getUserId()` 获取当前用户信息，不需要在每个方法里传递 `userId` 参数。

在拦截器的 finally 块中，我们会调用 `UserContext.clear()` 来清理 ThreadLocal，避免内存泄漏。

ThreadLocal的注意事项

Q: 使用 ThreadLocal 有哪些注意事项？

面试官你好，使用 ThreadLocal 主要有这么几个注意事项：

第一个是一定要记得 remove。这是最重要的，尤其是在使用线程池的场景下，线程会被复用，如果不 remove，上一个请求的数据可能会影响下一个请求。

第二个是不要存储大对象。因为 ThreadLocal 的数据是存在线程对象里的，如果存储的对象很大，会占用大量内存。而且在线程池场景下，线程不会被回收，这些大对象会一直占着内存。

第三个是注意父子线程的数据传递。ThreadLocal 的数据是线程隔离的，子线程无法访问父线程的 ThreadLocal 数据。如果需要传递，可以使用 **InheritableThreadLocal**。

第四个是避免在异步场景下使用。比如在一个线程里设置了 ThreadLocal，然后提交了一个异步任务到线程池，异步任务是在另一个线程执行的，就拿不到 ThreadLocal 的值了。如果需要传递，可以使用阿里的 **TransmittableThreadLocal**。

追问1: InheritableThreadLocal 是什么？

InheritableThreadLocal 是 ThreadLocal 的子类，它可以让**子线程继承父线程的 ThreadLocal 数据**。

实现原理是：在创建子线程时，会把父线程的 InheritableThreadLocal 数据复制一份给子线程。

但是要注意，InheritableThreadLocal 只在**创建子线程时**复制数据。如果使用线程池，线程是复用的，不会重新创建，就无法传递数据了。这种情况需要用阿里的 TransmittableThreadLocal。

线程池

线程池

线程池是 JUC 中面试出现频率最高的知识点，几乎是必问题。

面试官通常会从"你项目中怎么用线程池"切入，然后深挖底层原理、参数配置、拒绝策略等。

为什么要使用线程池

Q：为什么要使用线程池？有什么好处？

面试官你好，使用线程池主要有三大好处：

第一个是**降低资源消耗**。因为线程的创建和销毁是有开销的，需要调用操作系统内核。线程池通过复用已创建的线程，避免了频繁创建销毁的开销。

第二个是**提高响应速度**。任务到达时，不需要等待线程创建，可以立即执行。

第三个是**提高线程的可管理性**。线程是稀缺资源，如果无限制创建，会消耗系统资源，甚至导致系统崩溃。线程池可以统一分配、调优和监控线程。

ThreadPoolExecutor的七大参数

Q：ThreadPoolExecutor 有哪些核心参数？分别是什么作用？

面试官你好，ThreadPoolExecutor 有七个核心参数：

我详细说一下这七个参数：

第一个是 **corePoolSize（核心线程数）**，它表示线程池中常驻的线程数量。即使这些线程处于空闲状态，也不会被回收，除非设置了 `allowCoreThreadTimeOut`。

第二个是 **maximumPoolSize（最大线程数）**，它表示线程池允许创建的最大线程数量。当任务队列满了，且当前线程数小于最大线程数时，会创建新线程处理任务。

第三个是 **keepAliveTime（空闲线程存活时间）**，当线程数超过核心线程数时，多余的空闲线程在被回收前的最大等待时间。

第四个是 **unit（时间单位）**，就是 `keepAliveTime` 的时间单位，比如 `TimeUnit.SECONDS`。

第五个是 **workQueue（任务队列）**，用于存放等待执行的任务。常见的有 `ArrayBlockingQueue`（有界队列）、`LinkedBlockingQueue`（无界队列，默认容量是 `Integer.MAX_VALUE`）、还有 `SynchronousQueue`（不存储元素的队列，每个插入操作必须等待一个移除操作）。

第六个是 **threadFactory（线程工厂）**，用于创建新线程。我们可以自定义线程名称、优先级等。

第七个是 **handler（拒绝策略）**，当任务队列满了，且线程数达到最大值时，如何处理新提交的任务。

线程池的工作流程

Q：线程池的执行流程是怎样的？

面试官你好，线程池的执行流程可以总结为以下几步：

首先，我们调用 `execute()` 或 `submit()` 提交任务。

然后，线程池会判断当前运行的线程数是不是小于核心线程数。如果小于，就直接创建新线程执行任务，即使有空闲线程也会创建新的。

如果当前运行的线程数大于等于核心线程数，任务会被放入任务队列等待。

如果任务队列已满，且当前线程数小于最大线程数，就会创建新的非核心线程执行任务。

最后，如果任务队列满了，且线程数已达到最大值，就会执行拒绝策略。

我画个流程图可能更清楚：

追问1: 为什么线程数小于核心线程数时, 即使有空闲线程也要创建新线程?

这是线程池的设计策略。核心线程是"常驻"的, 线程池会尽快达到核心线程数, 以保证有足够的线程处理任务。这样可以避免任务在队列中等待, 提高响应速度。

四种拒绝策略

Q: 线程池有哪些拒绝策略? 分别适用什么场景?

面试官你好, JDK 提供了**四种**内置的拒绝策略:

第一个是 **AbortPolicy** (默认), 它会直接抛出异常 `RejectedExecutionException`。这个适用于需要感知任务被拒绝的情况, 及时发现问题。

第二个是 **CallerRunsPolicy**, 它会由调用线程 (提交任务的线程) 自己执行任务。这个适用于任务不能丢失, 且可以接受降低提交速度的场景。这种策略会降低任务提交的速度, 起到一定的限流作用。

第三个是 **DiscardPolicy**, 它会直接丢弃任务, 不抛异常。这个适用于允许任务丢失的场景, 比如日志记录。

第四个是 **DiscardOldestPolicy**, 它会丢弃队列中最老的任务, 然后尝试重新提交当前任务。这个适用于优先处理新任务的场景。

追问1: 生产环境中你会选择哪种拒绝策略?

这要看具体业务场景:

如果是**核心业务**, 比如订单、支付这种, 我会使用 **AbortPolicy**, 抛异常后可以记录日志、告警, 不能让任务悄悄丢失。

如果是**非核心业务**, 比如日志、埋点这种, 可以用 **DiscardPolicy**, 允许部分数据丢失。

如果是**需要限流**的场景, 我会用 **CallerRunsPolicy**, 让调用线程自己执行, 降低提交速度。

常见的线程池类型

Q: Executors 提供了哪些常见的线程池? 有什么问题?

面试官你好，Executors 工具类提供了几种快速创建线程池的方法：

第一个是 `newFixedThreadPool(int n)`，它是固定大小的线程池，核心线程数等于最大线程数等于 `n`。它使用 `LinkedBlockingQueue`（无界队列）。问题是队列无界，可能导致内存溢出（OOM）。

第二个是 `newSingleThreadExecutor()`，它只有一个线程的线程池，也使用 `LinkedBlockingQueue`。同样存在 OOM 风险。

第三个是 `newCachedThreadPool()`，它的核心线程数为 0，最大线程数为 `Integer.MAX_VALUE`，使用 `SynchronousQueue`（不存储任务）。问题是可能创建大量线程，导致系统资源耗尽。

第四个是 `newScheduledThreadPool(int n)`，它支持定时和周期性任务执行。同样使用无界队列，存在 OOM 风险。

追问1：为什么阿里巴巴开发手册不推荐使用 Executors 创建线程池？

主要有两个原因：

第一个是 `FixedThreadPool` 和 `SingleThreadExecutor` 使用的是 `LinkedBlockingQueue`，默认容量是 `Integer.MAX_VALUE`（约 21 亿）。如果任务提交速度大于处理速度，队列会无限增长，导致内存溢出。

第二个是 `CachedThreadPool` 的最大线程数是 `Integer.MAX_VALUE`，可能创建大量线程，导致 CPU 和内存资源耗尽。

所以推荐的做法是使用 `ThreadPoolExecutor` 手动创建线程池，明确指定核心参数，避免资源失控。比如这样：

```
1  ThreadPoolExecutor executor = new ThreadPoolExecutor(  
2      10,                // 核心线程数  
3      20,                // 最大线程数  
4      60L,               // 空闲线程存活时间  
5      TimeUnit.SECONDS,  
6      new ArrayBlockingQueue<>(100), // 有界队列  
7      new ThreadPoolExecutor.CallerRunsPolicy() // 拒绝策略  
8  );
```

如何合理配置线程池参数

Q：如何合理设置线程池的核心线程数和最大线程数？

面试官你好，线程池参数的配置需要根据**任务类型**来决定：

第一种是 CPU 密集型任务，任务主要消耗 CPU 资源，比如计算、加密这种。推荐配置是核心线程数等于 CPU 核心数加 1。原因是 CPU 密集型任务，线程数过多会导致频繁的上下文切换，反而降低性能。加 1 是为了在某个线程因为偶尔的内存页失效或其他原因暂停时，额外的线程可以顶上。

第二种是 IO 密集型任务，任务主要等待 IO 操作，比如网络请求、数据库查询、文件读写这种。推荐配置是核心线程数等于 CPU 核心数乘以 2，或者 CPU 核心数除以（1 减去阻塞系数）。阻塞系数通常在 0.8 到 0.9 之间，也就是线程有 80% 到 90% 的时间在等待 IO。原因是 IO 密集型任务，线程大部分时间在等待，可以创建更多线程来提高 CPU 利用率。

第三种是混合型任务，建议拆分成 CPU 密集型和 IO 密集型任务，分别用不同的线程池处理。

追问1：你在项目中是怎么配置线程池的？

在我的项目中，我会根据实际业务场景来配置：

比如**异步任务处理**，像发送短信、邮件这种，这是典型的 IO 密集型任务。我会设置核心线程数为 20，最大线程数为 50，使用有界队列（比如 `ArrayBlockingQueue(200)`），拒绝策略用 `CallerRunsPolicy`。

如果是**数据计算任务**，这是 CPU 密集型任务。我会设置核心线程数等于 CPU 核心数加 1，最大线程数与核心线程数相同，使用较小的队列。

重点是，生产环境中，我会通过压测和监控（比如线程池的活跃线程数、队列大小、拒绝次数）来动态调整参数，找到最优配置。

线程池的关闭

Q：如何正确关闭线程池？shutdown 和 shutdownNow 有什么区别？

面试官你好，线程池提供了两种关闭方法：

第一个是 `shutdown()`，这是温柔关闭。它不再接受新任务，但会执行完队列中的所有任务。已提交的任务会继续执行，线程池状态变为 `SHUTDOWN`。

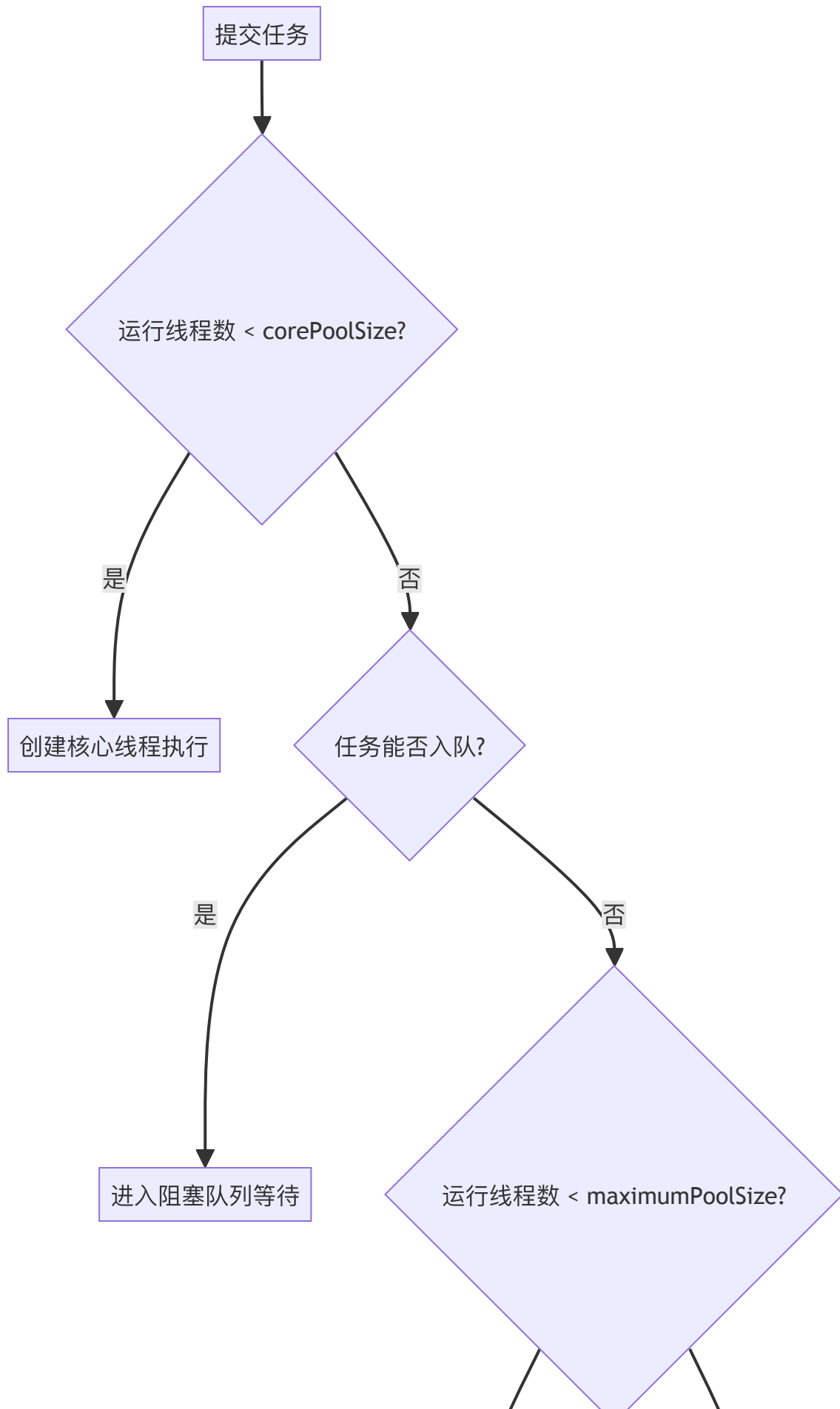
第二个是 `shutdownNow()`，这是强制关闭。它不再接受新任务，尝试停止正在执行的任务，并返回队列中未执行的任务列表。它通过调用线程的 `interrupt()` 方法来中断线程，线程池状态变为 `STOP`。

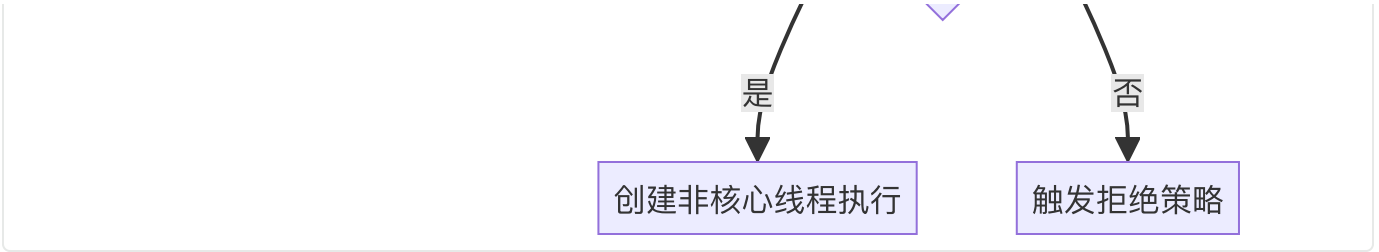
推荐的做法是这样：

追问1：为什么不能直接用 `shutdownNow()`？

因为 `shutdownNow()` 只是尝试中断线程，但如果任务没有正确处理中断信号（比如没有检查 `Thread.interrupted()`），任务可能无法停止。而且，正在执行的任务可能处理到一半，导致数据不一致。所以生产环境中，优先使用 `shutdown()` 优雅关闭。

图解补充：线程池任务流转





图解补充：参数和队列怎么记

维度	核心结论	面试表达
<code>corePoolSize</code>	常驻线程数	先保响应，再谈扩容
<code>maximumPoolSize</code>	极限兜底线程数	队列扛不住时再扩线程
<code>workQueue</code>	决定是“排队”还是“扩线程”	队列类型直接影响系统行为
<code>handler</code>	最后的降级策略	核心业务不能悄悄丢任务
<code>keepAliveTime</code>	非核心线程回收时间	高峰过后及时缩容

队列	特点	常见风险	适用场景
<code>ArrayBlockingQueue</code>	有界、数组实现	容量太小易拒绝	明确限流的业务
<code>LinkedBlockingQueue</code>	默认近似无界	任务堆积可能 OOM	不建议直接无脑用
<code>SynchronousQueue</code>	不存元素，直接交接	容易快速拉高线程数	短任务、高吞吐

高频追问补充

追问补充1: `execute()` 和 `submit()` 有什么区别?

`execute()` 只提交任务，没有返回值；`submit()` 会返回 `Future`，可以拿结果，也会把异常包装到 `Future` 里。面试里一般会补一句：如果要感知异步任务异常，不能只调 `submit()` 不看 `Future`。

追问补充2：生产里线程池参数怎么定，不要只背公式，怎么回答更靠谱？

更好的答法是：

1. 先按 CPU 密集型、IO 密集型做初始估算。
2. 队列一定设成有界，先把系统上限卡住。
3. 再结合压测看活跃线程数、队列长度、拒绝次数、任务平均耗时。
4. 最终根据业务 SLA 调整，而不是死背“CPU 核数乘 2”。

追问补充3：核心业务为什么常选 `AbortPolicy` 或 `CallerRunsPolicy` ？

因为核心业务最怕“悄悄丢任务”。`AbortPolicy` 适合快速失败并告警，`CallerRunsPolicy` 适合反压，把压力回传给调用方。真正线上系统一般不会选静默丢弃。

速记版

线程池可以背成一句话：

先开核心线程 -> 再进队列 -> 队列满了再扩线程 -> 还扛不住就拒绝

面试里最容易被追问的四个点：

- 七大参数
- 执行流程
- 拒绝策略
- 为什么不用 Executors

AQS原理

AQS原理

AQS 是 JUC 中最核心的底层框架，面试官通常用它来考察候选人对并发编程的理解深度。

AQS是什么

Q: 什么是 AQS? 它的作用是什么?

面试官你好，AQS 全称是 **AbstractQueuedSynchronizer**（抽象队列同步器），它是 JUC 包中最核心的基础框架。

AQS 的作用是为实现锁和同步器提供一个**通用的框架**。像 **ReentrantLock**、**Semaphore**、**CountDownLatch**、**ReentrantReadWriteLock** 这些同步工具，底层都是基于 AQS 实现的。

AQS 的核心思想是：如果请求的共享资源空闲，就把当前请求资源的线程设置为有效的工作线程，并将共享资源设置为锁定状态。如果请求的共享资源被占用，就需要一套**线程阻塞等待以及被唤醒时锁分配的机制**。AQS 用的是 **CLH 队列锁**（一个虚拟的双向队列）来实现这个机制，将暂时获取不到锁的线程加入到队列中。

追问1: 为什么要有 AQS? 直接用 **synchronized** 不行吗?

synchronized 是 JVM 层面的，功能比较固定，不够灵活。而 AQS 提供了一个可扩展的框架，让我们可以根据不同的需求实现各种同步器。

比如 **ReentrantLock** 可以实现公平锁和非公平锁，**Semaphore** 可以控制并发数量，**CountDownLatch** 可以实现等待多个线程完成。这些功能用 **synchronized** 都很难实现，但基于 AQS 就很容易。

AQS的核心原理

Q: AQS 的核心原理是什么?

面试官你好，AQS 的核心原理主要包括三个部分：

第一个是 **state 状态**。AQS 内部维护了一个 **volatile int state** 变量，用来表示同步状态。比如在 ReentrantLock 中，state = 0 表示锁空闲，state > 0 表示锁被占用，state 的值表示重入次数。

第二个是 **CLH 队列**。AQS 内部维护了一个 **FIFO 的双向队列**，用来存放等待获取锁的线程。当线程获取锁失败时，会被封装成一个 Node 节点，加入到队列尾部。当持有锁的线程释放锁时，会唤醒队列中的第一个线程。

第三个是**独占模式和共享模式**。AQS 支持两种模式：

- **独占模式 (Exclusive)**：同一时刻只有一个线程能获取资源，比如 ReentrantLock。
- **共享模式 (Shared)**：多个线程可以同时获取资源，比如 Semaphore、CountDownLatch。

追问1：CLH 队列是什么？

CLH 队列是一种**基于链表的 FIFO 队列**，全称是 Craig, Landin, and Hagersten 队列（三个人名的缩写）。

在 AQS 中，CLH 队列的每个节点 (Node) 包含：

- **thread**：等待的线程
- **waitStatus**：节点状态（如 CANCELLED、SIGNAL 等）
- **prev**：前驱节点
- **next**：后继节点

当线程获取锁失败时，会被封装成 Node 加入队列尾部。当锁被释放时，会唤醒队列头部的下一个节点。

AQS的工作流程

Q：AQS 的加锁和释放锁的流程是怎样的？

面试官你好，我以独占模式（比如 ReentrantLock）为例，说一下 AQS 的工作流程：

加锁流程（acquire）：

1. 首先，线程调用 `tryAcquire()` 尝试获取锁（这个方法由子类实现）。
2. 如果获取成功（state 从 0 变成 1），直接返回，线程获得锁。
3. 如果获取失败（state 已经是 1），说明锁被占用，线程会被封装成 Node 节点，加入到 CLH 队列的尾部。
4. 加入队列后，线程会进入自旋，再次尝试获取锁。如果前驱节点是头节点，就尝试获取锁。
5. 如果还是获取失败，线程会被 `LockSupport.park()` 阻塞，进入等待状态。

释放锁流程（release）：

1. 线程调用 `tryRelease()` 释放锁（这个方法由子类实现）。
2. 如果释放成功（state 从 1 变成 0），会唤醒 CLH 队列中的第一个等待线程。
3. 被唤醒的线程会再次尝试获取锁，如果成功，就从队列中移除，成为新的持有锁的线程。

流程图可能更清楚：

线程 A 获取锁成功（state: 0 → 1）

↓

线程 B 尝试获取锁失败

↓

线程 B 加入 CLH 队列尾部

↓

线程 B 被 park 阻塞

↓

线程 A 释放锁（state: 1 → 0）

↓

唤醒队列头部的线程 B

↓

线程 B 获取锁成功（state: 0 → 1）

追问1：为什么加入队列后还要自旋？

这是一种优化策略。因为如果前驱节点就是头节点，说明当前线程是队列中的第一个等待线程，很可能马上就能获取到锁。这时候自旋一下，可能就获取到了，避免了线程阻塞和唤醒的开销。

如果自旋几次还是获取不到，说明锁的持有时间比较长，这时候再阻塞，避免浪费 CPU 资源。

ReentrantLock如何基于AQS实现

Q: ReentrantLock 是如何基于 AQS 实现的?

面试官你好，ReentrantLock 内部有一个 **Sync** 类，它继承了 AQS。Sync 又有两个子类：**FairSync**（公平锁）和 **NonfairSync**（非公平锁）。

非公平锁的实现：

当调用 `lock()` 时：

1. 首先用 CAS 尝试将 state 从 0 改成 1，如果成功，直接获取锁。
2. 如果失败，调用 AQS 的 `acquire()` 方法，进入队列等待。

在 `tryAcquire()` 方法中：

1. 如果 state = 0，再次用 CAS 尝试获取锁（插队）。
2. 如果当前线程已经持有锁（state > 0），state 加 1（可重入）。
3. 否则返回 false，线程进入队列。

公平锁的实现：

公平锁和非公平锁的区别在于：公平锁在 `tryAcquire()` 时，会先检查队列中是否有等待的线程。如果有，当前线程必须排队，不能插队。

这就是为什么公平锁性能比非公平锁差，因为多了一次队列检查。

追问1: state 是如何保证线程安全的?

state 是用 **volatile** 修饰的，保证了可见性。

修改 state 时，AQS 使用 **CAS (Compare And Swap)** 操作，保证了原子性。CAS 是一种无锁的原子操作，通过硬件指令实现，性能很高。

比如在获取锁时，会用 `compareAndSetState(0, 1)` 来尝试将 state 从 0 改成 1。如果成功，说明获取锁成功；如果失败，说明有其他线程抢先了。

AQS的共享模式

Q: AQS 的共享模式是怎么实现的?

面试官你好, AQS 的共享模式和独占模式的主要区别在于: **共享模式下, 多个线程可以同时获取资源。**

典型的例子是 **Semaphore (信号量)**。Semaphore 可以控制同时访问某个资源的线程数量。

实现原理:

在共享模式下, state 表示**可用的资源数量**。比如 `new Semaphore(3)` 表示最多允许 3 个线程同时访问。

当线程调用 `acquire()` 时:

1. 调用 `tryAcquireShared()`, 尝试将 state 减 1。
2. 如果 `state >= 0`, 获取成功, 线程继续执行。
3. 如果 `state < 0`, 获取失败, 线程加入队列等待。

当线程调用 `release()` 时:

1. 调用 `tryReleaseShared()`, 将 state 加 1。
2. 如果队列中有等待的线程, 唤醒它们。

共享模式的特点是: 当一个线程释放资源后, 可能会唤醒**多个**等待的线程, 而不是只唤醒一个。

追问1: CountdownLatch 是如何基于 AQS 实现的?

CountDownLatch 也是基于 AQS 的共享模式实现的。

在 CountDownLatch 中, state 表示**计数器的值**。比如 `new CountDownLatch(3)` 表示需要等待 3 个线程完成。

当线程调用 `await()` 时:

- 如果 `state > 0`, 线程加入队列等待。
- 如果 `state = 0`, 线程直接返回。

当线程调用 `countDown()` 时:

- 将 state 减 1。
- 如果 state 变成 0, 唤醒所有等待的线程。

这样就实现了"等待多个线程完成"的功能。

AQS的优势

Q: AQS 相比 `synchronized` 有什么优势?

面试官你好, AQS 相比 `synchronized` 的优势主要有这么几点:

第一个是灵活性更高。AQS 提供了一个可扩展的框架, 我们可以根据不同的需求实现各种同步器。比如公平锁、非公平锁、读写锁、信号量等, 这些用 `synchronized` 都很难实现。

第二个是功能更强大。AQS 支持可中断的锁获取 (`lockInterruptibly`)、超时获取 (`tryLock(timeout)`)、非阻塞获取 (`tryLock`), 而 `synchronized` 只能阻塞等待。

第三个是性能更好。在高并发场景下, AQS 的性能通常比 `synchronized` 好。虽然 JDK 1.6 后 `synchronized` 做了很多优化, 但 AQS 的灵活性让它在某些场景下性能更优。

第四个是支持条件变量。AQS 支持 `Condition`, 可以实现更复杂的线程协作, 比如生产者-消费者模式。`synchronized` 只有一个等待队列, 而 AQS 可以有多个 `Condition` 队列。

追问1: 什么时候用 AQS, 什么时候用 `synchronized`?

这要看具体场景:

如果是简单的同步需求, 比如保护一个临界区, 用 `synchronized` 就够了, 代码简单, 性能也不差。

如果需要更高级的功能, 比如可中断、超时、公平锁、读写锁, 就用 **AQS 相关的类** (如 `ReentrantLock`、`ReadWriteLock`)。

如果可以实现自定义的同步器, 就可以继承 AQS, 实现自己的 `tryAcquire` 和 `tryRelease` 方法。

更新: 2026-01-30 20:59:34

原文: <https://www.yuque.com/u63312805/wnyutc/erxkffw0ggr6bq6c>

CAS与原子类

CAS与原子类

CAS、原子类、乐观锁悲观锁通常是一套组合题，面试官很喜欢顺着这一条线往下挖。

CAS 是什么

Q: 什么是 CAS?

CAS 的全称是 **Compare And Swap**，也叫比较并交换。

它包含三个值：

- 内存中的旧值 V
- 预期值 A
- 要修改的新值 B

执行时会先比较：如果 $V == A$ ，就把值更新成 B；如果不相等，说明这段时间被其他线程改过了，本次更新失败。

所以它本质上是一种 **无锁的乐观并发控制思想**。

CAS 的优缺点

Q: CAS 有什么优缺点?

优点：

- 不需要阻塞线程
- 在低竞争场景下性能很好
- 避免了传统锁的上下文切换开销

缺点：

- 高竞争下可能一直自旋，浪费 CPU
- 只能保证单个共享变量更新的原子性
- 会有 ABA 问题

所以 CAS 并不是万能替代锁，而是更适合“短时间、冲突不高”的更新场景。

ABA 问题

Q：什么是 ABA 问题？怎么解决？

ABA 问题的意思是：

一个线程看到某个值从 A 变成 B，又变回 A。

它会误以为这个值从来没被改过，于是 CAS 成功，但实际上共享变量已经经历过一次变化了。

解决方案通常是给值加一个 版本号 或 时间戳，也就是“值 + 版本”一起比较。

JUC 里常见的实现是 `AtomicStampedReference`。

原子类

Q：Java 里有哪些常见原子类？

JUC 里常见的原子类可以分成几类：

基础类型原子类：

- `AtomicInteger`
- `AtomicLong`
- `AtomicBoolean`

数组原子类：

- `AtomicIntegerArray`
- `AtomicLongArray`

引用原子类：

- `AtomicReference`
- `AtomicStampedReference`
- `AtomicMarkableReference`

字段更新器：

- `AtomicIntegerFieldUpdater`
- `AtomicLongFieldUpdater`
- `AtomicReferenceFieldUpdater`

它们底层大量依赖的就是 CAS。

乐观锁和悲观锁

Q：乐观锁和悲观锁怎么理解？

悲观锁 的思想是：先假设冲突一定会发生，所以先加锁再操作。

典型代表就是 `synchronized`、`ReentrantLock`、数据库行锁。

乐观锁 的思想是：先假设冲突不太会发生，不先加锁，提交更新时再检查有没有被别人改过。

典型代表就是 CAS、数据库版本号机制。

如果冲突频繁，悲观锁往往更稳；

如果冲突不高、操作很短，乐观锁往往效率更高。

synchronized与Lock

synchronized与Lock

synchronized 和 Lock 是 Java 并发编程中最常用的两种锁机制，面试中经常会被拿来对比。

面试官通常会问"synchronized 和 Lock 有什么区别"，然后深挖锁升级、公平锁、死锁等问题。

synchronized与Lock的区别

Q: synchronized 和 Lock 有什么区别?

面试官你好，synchronized 和 Lock 的主要区别可以从以下几个维度来看：

我详细说一下：

第一个是**使用方式**，synchronized 是关键字，使用简单，自动加锁解锁。而 Lock 是接口，需要手动 lock 和 unlock，通常在 finally 块中释放锁。

第二个是**锁的获取**，synchronized 只能阻塞等待，无法中断。而 Lock 提供了 `tryLock()`（尝试获取锁）、`tryLock(timeout)`（超时获取）、`lockInterruptibly()`（可中断获取）等方法。

第三个是**公平性**，synchronized 是非公平锁，线程获取锁的顺序不确定。而 ReentrantLock 可以在构造时指定是否为公平锁（`new ReentrantLock(true)`）。

追问1: 什么时候用 synchronized，什么时候用 Lock?

这要看具体场景：

推荐使用 synchronized 的场景有这么几种：第一个是锁竞争不激烈的场景。第二个是代码简单，不需要高级功能（如超时、中断、公平锁）。第三个是 JDK 1.6 后，synchronized 性能已经很好，且代码更简洁。

推荐使用 Lock 的场景也有几种：第一个是需要**尝试获取锁**（tryLock），避免死锁。第二个是需要**可中断的锁获取**（lockInterruptibly）。第三个是需要**公平锁**，保证线程按顺序获取锁。第四个是需要**读写锁**（ReadWriteLock），读多写少的场景。

总结一下就是：能用 `synchronized` 就用 `synchronized`，除非需要 `Lock` 的高级特性。

synchronized 的底层原理

Q: `synchronized` 的底层实现原理是什么？

面试官你好，`synchronized` 的底层实现依赖于 JVM 的 Monitor（监视器锁）。

我分两种情况来说：

第一种是同步代码块，编译后会在代码块前后插入 `monitorenter` 和 `monitorexit` 指令。`monitorenter` 会尝试获取对象的 Monitor，如果获取成功，锁计数器 +1；如果失败，线程阻塞等待。`monitorexit` 会释放 Monitor，锁计数器 -1，当计数器为 0 时，锁被释放。

第二种是同步方法，方法上加 `synchronized` 关键字，编译后会在方法的访问标志中添加 `ACC_SYNCHRONIZED` 标记。JVM 在调用方法时，会检查这个标记，自动获取和释放 Monitor。

Monitor 的本质是这样的：每个 Java 对象都有一个 Monitor 对象（对象头中的 Mark Word 指向 Monitor）。Monitor 内部有一个计数器、一个等待队列和一个同步队列。

追问1: `synchronized` 锁的是什么？

`synchronized` 锁的是**对象**，具体来说有三种情况：

第一种是**修饰实例方法**，锁的是**当前实例对象 (this)**。

第二种是**修饰静态方法**，锁的是**当前类的 Class 对象**。

第三种是**修饰代码块**，锁的是**括号里指定的对象**。

```
1  // 锁的是 this
2  public synchronized void method1() { }
3
4  // 锁的是 MyClass.class
5  public static synchronized void method2() { }
6
7  // 锁的是 obj
8  synchronized (obj) { }
```

synchronized 的锁升级机制

Q: synchronized 的锁升级过程是怎样的？

面试官你好，JDK 1.6 对 synchronized 做了大量优化，引入了**锁升级机制**，从低到高依次是：

无锁 → 偏向锁 → 轻量级锁 → 重量级锁

这个过程是**单向的**，只能升级，不能降级（为了提高效率）。

我详细说一下这几种锁：

第一个是偏向锁（Biased Locking），它适用于锁总是被同一个线程获取，没有竞争的场景。原理是在对象头的 Mark Word 中记录获取锁的线程 ID。当这个线程再次请求锁时，无需做任何同步操作，直接获取锁。优势是几乎没有性能开销。但是当有其他线程尝试获取锁时，偏向锁会被撤销，升级为轻量级锁。

第二个是轻量级锁（Lightweight Locking），它适用于多个线程交替获取锁，但没有实际竞争（不同时获取）的场景。原理是通过 **CAS（Compare And Swap）** 操作尝试将对象头的 Mark Word 替换为指向锁记录的指针。优势是避免了重量级锁的线程阻塞和唤醒，性能较好。但是如果 CAS 失败（说明有竞争），轻量级锁会膨胀为重量级锁。

第三个是重量级锁（Heavyweight Locking），它适用于多个线程同时竞争锁的场景。原理是依赖操作系统的 **Mutex Lock（互斥锁）**，线程会被阻塞，进入等待队列。劣势是涉及用户态和内核态的切换，性能开销大。

我画个流程图可能更清楚：

追问1：为什么锁只能升级，不能降级？

这是 JVM 的设计选择。锁降级需要额外的开销来判断是否可以降级，而且降级后如果又发生竞争，还要再次升级，反而增加了复杂度。JVM 认为，一旦发生过竞争，后续大概率还会竞争，所以干脆不降级，保持在高级别的锁状态。

追问2：什么是自旋锁？为什么要自旋？

自旋锁是指线程在获取锁失败时，不立即阻塞，而是**循环尝试获取锁（自旋）**。

为什么要自旋呢？主要有两个原因：

第一个是线程的阻塞和唤醒需要从用户态切换到内核态，开销很大。

第二个是如果锁的持有时间很短，自旋等待一下，可能很快就能获取到锁，避免了线程切换的开销。

自旋的问题是：如果锁的持有时间很长，自旋会浪费 CPU 资源。所以 JVM 引入了**自适应自旋**：根据上次自旋的结果动态调整自旋次数。如果上次自旋成功了，这次可以多自旋几次；如果上次失败了，这次就少自旋或直接阻塞。

ReentrantLock的使用

Q: ReentrantLock 怎么使用？有什么注意事项？

面试官你好，ReentrantLock 的基本使用是这样的：

注意事项有这么几点：

第一个是**必须手动释放锁**，一定要在 `finally` 块中调用 `unlock()`，否则如果发生异常，锁永远不会被释放，导致死锁。

第二个是**lock 和 unlock 要成对出现**，每次 `lock()` 必须对应一次 `unlock()`。

第三个是**避免在 lock 之前写业务代码**，如果在 `lock()` 之前的代码抛异常，会导致 `finally` 中的 `unlock()` 被执行，但锁根本没获取到，会抛 `IllegalMonitorStateException`。

高级用法还有这些：

公平锁与非公平锁

Q: 什么是公平锁和非公平锁？有什么区别？

面试官你好，我先解释一下这两个概念：

公平锁是指多个线程按照**申请锁的顺序**来获取锁（先到先得），内部维护了一个 FIFO 队列。

非公平锁是指线程获取锁的顺序**不确定**，可能后来的线程先获取到锁。新来的线程会先尝试直接获取锁（插队），如果失败再进入队列。

区别主要体现在这几个方面：

为什么非公平锁性能更好呢？因为公平锁在获取锁时，必须先检查队列中是否有等待的线程，如果有，当前线程必须排队。而非公平锁可以直接尝试获取锁，如果成功，就省去了入队和出队的开销。

ReentrantLock 的公平性可以这样设置：

追问1: `synchronized` 是公平锁还是非公平锁?

`synchronized` 是非公平锁, 且无法改变。这也是为什么在需要公平性的场景下, 要使用 `ReentrantLock`。

可重入锁

Q: 什么是可重入锁? `synchronized` 和 `ReentrantLock` 是可重入的吗?

面试官你好, 可重入锁 (Reentrant Lock) 是指同一个线程可以多次获取同一把锁, 而不会被自己阻塞。

为什么需要可重入呢? 因为在递归调用或方法嵌套调用时, 如果锁不可重入, 线程会被自己持有的锁阻塞, 导致死锁。

`synchronized` 和 `ReentrantLock` 都是可重入锁。

我举个例子:

在这个例子中, `method1` 获取了锁, 然后调用 `method2`, `method2` 也需要获取同一把锁。因为 `synchronized` 是可重入的, 所以不会死锁。

实现原理是这样的: 锁内部维护了一个计数器和持有锁的线程 ID。当线程第一次获取锁时, 计数器等于 1。当同一个线程再次获取锁时, 计数器加 1。每次释放锁时, 计数器减 1, 当计数器为 0 时, 锁被完全释放。

追问1: 如果锁不可重入会怎样?

如果锁不可重入, 上面的例子中, 线程在 `method1` 中获取了锁, 然后调用 `method2` 时, 发现锁已经被占用 (被自己占用), 就会一直等待, 导致死锁。

并发经典题

并发经典题

并发经典题本身不难，但特别能区分“只会背概念”和“真能写出并发代码”的人。面试官经常会问交替打印、顺序控制这类问题。

多线程交替打印 0-100

Q：两个线程交替打印 0 到 100，你会怎么做？

这类题的关键不是语法，而是 线程之间怎么协调顺序。

常见做法有三种：

- `synchronized + wait/notify`
- `ReentrantLock + Condition`
- `CAS + 自旋`

如果是面试，我一般优先答 `synchronized + wait/notify`，因为它最能体现基础。

核心思路是：

- 共享一个计数器
- 打印完之后唤醒对方
- 自己不满足条件时进入等待

如果追求代码表达更清晰，也可以用 `ReentrantLock + Condition`。

三线程交替打印 ABC

Q：三个线程交替打印 ABC，你会怎么设计？

这个题和交替打印数字的本质一样，关键是引入一个 **状态变量**。

比如可以定义：

- 0 表示轮到 A
- 1 表示轮到 B
- 2 表示轮到 C

每个线程只在轮到自己时打印；打印后修改状态并唤醒其他线程。

实现手段同样可以是：

- `synchronized + wait/notifyAll`
- `ReentrantLock + 3 个 Condition`

如果线程数更多、规则更复杂，我更倾向于 `Lock + Condition`，可读性会更好。

如何回答这类题

Q：面试里遇到并发手撕题，怎么答更稳？

我一般会按这个顺序答：

第一步，先确认需求。

比如是两个线程还是三个线程，要不要严格交替，打印到多少结束。

第二步，说思路。

先说“需要共享状态 + 线程协作 + 顺序控制”。

第三步，选工具。

简单场景优先 `synchronized + wait/notify`，复杂一点再上 `Lock + Condition`。

第四步，说边界。

比如要注意唤醒后重新判断条件，所以通常写 `while` 而不是 `if`。

这样回答，面试官会觉得你不是在背模板，而是在真的设计并发逻辑。

线程安全与JMM

线程安全与JMM

线程安全和 JMM 是并发面试里非常高频的一组题。很多人会把它们拆开背，但更好的理解方式是：JMM 解释了为什么会出现线程安全问题，以及 Java 是怎么约束可见性、有序性和原子性的。

什么是线程安全

Q：你怎么理解线程安全？

如果一个类或一段逻辑在多线程并发访问下，仍然能够保持结果正确、状态一致，而且调用方不需要额外做同步控制，那我就认为它是线程安全的。

线程不安全的典型表现包括：

- 数据覆盖
- 脏读
- 结果不符合预期
- 对象状态被多个线程改乱

比如 `i++` 看起来只是一行代码，但实际上包含读取、计算、写回三个步骤，多线程并发执行就可能丢失更新。

JMM 是什么

Q：什么是 JMM？

JMM 指的是 **Java 内存模型**。

它不是说 JVM 真实内存怎么实现，而是一套并发访问内存时的规范。

它主要解决的是多线程环境下三个核心问题：

- 原子性
- 可见性
- 有序性

JMM 规定了线程和主内存之间的交互方式，也规定了什么场景下一个线程对共享变量的修改，必须让其他线程可见。

原子性、可见性、有序性

Q：原子性、可见性、有序性分别是什么意思？

原子性：一个操作要么全部执行完，要么完全不执行，中间不能被打断。

比如单次基本类型赋值通常具有原子性，但 `i++` 不是原子操作。

可见性：一个线程修改了共享变量，其他线程能够及时看到最新值。

如果没有可见性保证，一个线程可能一直读到旧值。

有序性：程序执行顺序不一定严格按照代码顺序，因为编译器和 CPU 可能会做指令重排序。

JMM 允许一定范围内的重排序，但不能破坏单线程语义和多线程同步语义。

happens-before

Q：你怎么理解 happens-before？

happens-before 可以理解成 JMM 里判断“一个操作的结果，对另一个操作是否一定可见”的规则。

只要满足 happens-before 关系，就意味着前一个操作的执行结果，对后一个操作是可见的，而且前一个操作的执行顺序也排在后一个之前。

常见规则包括：

- 程序次序规则
- 锁规则：解锁 happens-before 后续加锁
- `volatile` 变量规则：对一个 volatile 变量的写 happens-before 后续对它的读
- 线程启动规则： `start()` 先于线程内动作
- 线程终止规则：线程内动作先于其他线程感知它结束

面试里不用把规则全背全，只要知道它是 JMM 的可见性和有序性判断依据 就够了。

JMM 和 volatile 的关系

Q: volatile 为什么能保证可见性？

volatile 变量的写，会强制把工作内存中的值刷新到主内存；

volatile 变量的读，会强制从主内存重新读取最新值。

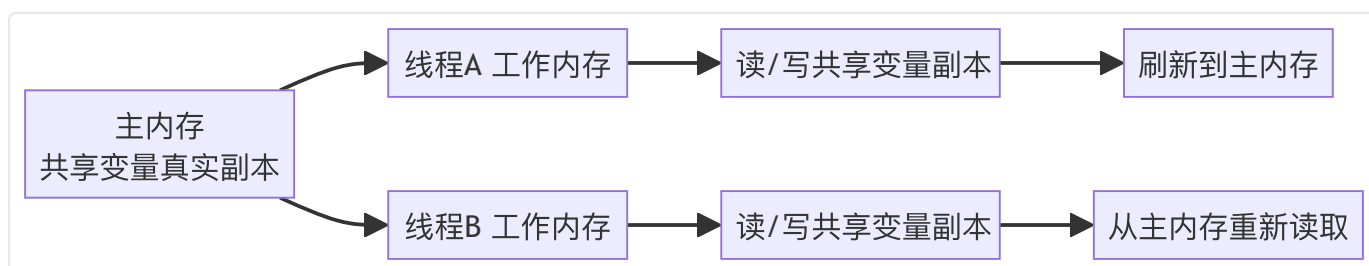
同时，volatile 在读写前后会插入内存屏障，禁止特定类型的指令重排序。

所以它能保证：

- 可见性
- 一定程度的有序性

但它不能保证复合操作的原子性，比如 `i++` 这种场景，还是需要加锁或 CAS。

图解补充：JMM 的主内存与工作内存



对比补充：happens-before 规则怎么记

规则	可以怎么理解	面试里的简短说法
程序次序规则	单线程里前面的操作先于后面的操作	单线程看代码顺序
锁规则	解锁先于后续加锁	同一把锁天然建立可见性
<code>volatile</code> 规则	写先于后续读	写完别人能看到
线程启动规则	<code>start()</code> 先于线程内动作	主线程先发车，子线程后执行
线程终止规则	线程内动作先于 <code>join()</code> 后的感知	等线程结束后看到它结果

高频追问补充

追问补充1：为什么 `i++` 不是线程安全的？

因为它不是一个原子动作，而是“读 -> 加一 -> 写回”三个步骤。即使变量是 `volatile`，也只能保证别的线程能看到最新值，不能阻止两个线程同时读到旧值后再覆盖写回。

追问补充2： `synchronized` 和 `volatile` 分别解决什么问题？

`volatile` 主要解决可见性和部分有序性； `synchronized` 同时能解决可见性、原子性和有序性。面试里如果只问“轻量同步”，可以先答 `volatile`；如果涉及复合操作，一般就要收口到锁或 CAS。

追问补充3：面试里怎么把 JMM 和线程安全串起来？

可以这么讲：线程安全问题的本质通常就落在三个维度上，原子性、可见性、有序性。JMM 定义了这三个问题的约束边界， `volatile`、 `synchronized`、CAS 这些手段，都是在这个模型上解决不同类型的并发风险。

速记版

JMM 建议按这条线记：

- 线程安全先看三个问题：原子性、可见性、有序性
- JMM 是规则，不是物理实现
- happens-before 是判断“可见且有序”的依据
- `volatile` 能见、不能包办原子性

一句话回答模板：

JMM 的核心作用是定义多线程下共享变量什么时候可见、什么时候允许重排序，以及通过什么同步手段建立可靠的可见性和有序性。

JVM

对象创建与内存布局

对象创建与内存布局

对象创建流程

Q：在 Java 代码中 new 一个对象，JVM 底层具体执行了哪些步骤？

A：

当虚拟机遇到一条 `new` 指令时，主要会经历以下 5 个步骤：

1. 类加载检查：

- 首先检查这个指令的参数（即类名）能不能在常量池中定位到一个类的符号引用。
- 并且检查这个类是不是已经被加载、解析和初始化过。如果没有，必须先执行类加载过程。

2. 分配内存：

- 类加载通过后，JVM 就在堆中划分一块内存给新对象。对象所需的内存大小在类加载完成后便可完全确定。
- 分配策略：如果内存规整，用**指针碰撞**；如果内存不规整，用**空闲列表**。

3. 初始化零值：

- 内存分配完后，JVM 需要将分配到的内存空间都初始化为零值（比如 `int` 设为 0，`boolean` 设为 false，引用设为 null）。
- 这就是为什么我们在 Java 代码中定义成员变量不需要赋初值就能直接用的原因。

4. 设置对象头：

- 初始化零值后，JVM 要对对象头进行设置。比如：这个对象是哪个类的实例？它的哈希码是多少？它的 GC 分代年龄是多少？

5. 执行 **** 方法****：

- 上面 4 步完成后，从 JVM 视角看对象已经产生了，但从 Java 程序视角看，对象创建才刚开始。
- 此时会执行构造函数（`<init>` 方法），按照程序员的意愿对对象进行真正的初始化赋值。

追问1：两个线程同时new对象，分配内存怎么保证线程安全

JVM 主要有两种方案来解决并发冲突：

1. **CAS + 失败重试**：这是最通用的方法。JVM 采用 CAS 机制配合失败重试的方式，保证更新操作的原子性。
2. **TLAB 【重点】**：
 - 这是 JVM 的默认优化方案。
 - **原理**：每个线程在 Java 堆中预先分配一小块私有内存，也就是 **本地线程分配缓冲（TLAB）**。
 - 当线程需要 `new` 对象时，优先在自己的 TLAB 上分配。只有当 TLAB 用完了，才需要通过 CAS 去锁住堆内存来分配。这样大大减少了并发冲突。

内存布局

Q：一个 Java 对象在内存中由哪几部分组成？对象头里都有什么？

A：

在 HotSpot 虚拟机中，对象在内存中的布局可以分为三块区域：

1. **对象头【核心】**：它包含两部分信息。
 - **Mark Word（标记字段）**：存储对象自身的运行时数据，如 **HashCode**、**GC 分代年龄**、**锁状态标志**（偏向锁/轻量级锁/重量级锁）、线程持有的锁等。
 - **Klass Pointer（类型指针）**：对象指向它的类元数据的指针，虚拟机通过这个指针来确定这个对象是哪个类的实例。
 - （如果是数组，还有一块用于记录数组长度）
2. **实例数据**：这是对象真正存储的有效信息，也就是我们在代码里定义的各种成员变量（字段）的内容。
3. **对齐填充**：这不是必须的，仅仅起占位符的作用。因为 HotSpot 规定对象的起始地址必须是 **8 字节的整数倍**。如果实例数据不够 8 字节，就需要填充对齐。

追问1：为什么对象的 GC 年龄最大是 15

这个限制是写死在对象头里的。在 **Mark Word** 中，用于存储 GC 分代年龄（GC Age）的区域只有 **4 个比特位（4 bit）**。二进制的 `1111` 换算成十进制就是 **15**。所以，一旦对象熬过了 15 次 GC，它的

年龄就会溢出这 4 位，JVM 规定它必须晋升到老年代。（注：CMS 默认为 6，但最大物理上限就是 15）

对象创建的完整过程

Q：new 一个对象时，JVM 里到底发生了什么？

A：

可以概括为下面几步：

1. 类加载检查

当执行 `new` 指令时，JVM 会先检查这个类是否已经被加载、链接和初始化。

如果还没有，就要先执行类加载。

2. 分配内存

类加载完成后，对象所需要的内存大小就确定了，JVM 会在堆中为对象分配一块连续的内存空间。

3. 处理并发安全问题

因为对象创建在高并发场景下可能有多个线程同时进行，所以 JVM 需要保证分配内存时线程安全。

常见做法是 **CAS + 失败重试**，或者通过 **TLAB** 让线程优先在自己的私有分配区中分配。

4. 初始化零值

内存分配完成后，JVM 会先把对象的实例字段都初始化为零值。

比如 `int` 为 `0`，引用类型为 `null`，这是为什么对象即使我们没手动赋值，也有默认值。

5. 设置对象头

JVM 会把对象头信息写进去，比如哈希码、GC分代年龄、锁状态、类型指针等。

6. 执行构造方法

最后再执行对象的构造方法，也就是 `init` 对应的逻辑，按程序员写的代码完成真正的初始化。

总结一下就是：**先检查类，再分配内存，再设默认值和对象头，最后执行构造方法。**

追问1：对象在堆里是怎么分配内存的？

答：

主要有两种方式：

1. 指针碰撞

如果堆内存是规整的，已用内存和空闲内存之间有明确边界，那么只需要把指针往空闲方向移动一段对象大小的距离即可。

2. 空闲列表

如果堆内存不是规整的，JVM 就需要维护一张空闲列表，记录哪些内存块可用，再从中找到合适空间分配给对象。

用哪种方式，取决于垃圾回收器是否会整理内存。

对象内存布局

Q：Java 对象在内存中的布局是怎样的？

A：

在 HotSpot 虚拟机里，一个普通对象在内存中一般分为三部分：

1. 对象头 (Header)

存储对象的运行时元数据，比如哈希码、GC分代年龄、锁状态、类型指针等。

2. 实例数据 (Instance Data)

也就是真正存储成员变量值的地方，包括当前类和父类继承下来的实例字段。

3. 对齐填充 (Padding)

HotSpot 要求对象大小通常按 **8 字节对齐**，所以如果前两部分加起来不是 8 的倍数，就需要补齐。

需要注意两点：

1. 静态变量不在对象实例里

静态变量属于类，不属于对象，它存放在方法区 / 元空间对应的类元数据中。

2. 成员变量的存储顺序不一定完全等于代码声明顺序

JVM 可能会为了对齐和节省空间，对字段顺序做一定调整。

追问1：对象头里都有什么？为什么 synchronized 会和对象头扯上关系？

答：

对象头里最核心的两部分是：

1. Mark Word

用来存对象运行时信息，比如哈希码、GC分代年龄、锁状态标志等。

2. Klass Pointer（类型指针）

指向这个对象所属类的元数据，JVM 通过它知道这个对象到底是哪个类的实例。

如果是**数组对象**，对象头里通常还会多一个**数组长度**字段。

之所以 `synchronized` 会和对象头扯上关系，是因为**锁状态信息**就记录在对象头的 **Mark Word** 里，所以我们常说 Java 里“锁是加在对象上的”，本质上和对象头是直接相关的。

追问2：一个空对象占多少字节？

答：

一般来说，在 64 位 HotSpot JVM 并开启**压缩指针** 的常见场景下，一个空的 `Object` 通常占 16 字节。

原因是：

1. 对象头通常占 12 字节
2. HotSpot 要求按 8 字节对齐
3. 所以最终会补齐到 16 字节

注意这是**常见情况**，不是绝对值，具体大小和 JVM 参数、是否开启指针压缩都有关系。

对象访问定位

Q：栈里的引用变量，是怎么定位到堆中的对象的？

A：

主要有两种方式：

1. 句柄访问

栈中的引用先指向句柄，句柄里再保存对象实例地址和类型数据地址。

2. 直接指针访问

栈中的引用直接保存对象地址，对象头里再通过类型指针找到类元数据。

HotSpot 虚拟机一般采用**的是直接指针访问**，因为这样少了一次间接寻址，访问速度更快。

对象分配的常见规则

Q：对象一般是怎么进入新生代、老年代的？

A：

一般有下面几种常见规则：

1. 大多数新对象优先分配在 Eden 区
这是最常见的情况。
2. 如果本线程的 TLAB 没满，会优先在 TLAB 中分配
这样可以减少多线程分配内存时的锁竞争。
3. 大对象有可能直接进入老年代
因为大对象如果频繁在新生代复制，成本会比较高。
4. 长期存活的对象会晋升到老年代
对象每熬过一次 Young GC，年龄会增加，到达阈值后就会晋升。
5. 如果 Survivor 放不下，也可能提前晋升到老年代
这本质上是空间分配担保的结果。

总结来说就是：新对象一般先进新生代，但大对象、长寿命对象或者 Survivor 放不下的对象，可能会更早进入老年代。

JVM内存区域

JVM内存区域

JVM运行时数据区

Q: 介绍一下 JVM 的运行时数据区（内存结构）？哪些是线程共享的，哪些是私有的？

A:

JVM 在运行 Java 程序时，会把内存划分为 5 个区域。

为了逻辑清晰，我习惯把它们按照生命周期和是否线程共享分为两类：

第一类是线程私有的，随线程生灭，**不需要垃圾回收（GC）**；**第二类是线程共享的**，也是 GC 重点关注的区域。

下面我具体展开说一下。

首先是**线程私有**的区域，包含三个部分：

1. **程序计数器**：它的作用是记录当前线程执行到哪一行字节码了。

- **为什么需要它**？因为 CPU 会在不同线程间切换，当切回来的时候，线程得知道自己刚才执行到哪儿了。
- **特点**：这是 JVM 中唯一一个不会发生内存溢出（OOM）的区域。

2. **虚拟机栈**：这是我们平时最常说的栈，它是 Java 方法执行的内存模型。

- 每个方法在执行时都会创建一个**栈帧**，里面存了局部变量表、操作数栈等信息。方法调用入栈，执行完出栈。
- **异常**：如果递归太深，会爆出 `StackOverflowError`；如果申请不到内存，会爆出 `OOM`。

3. **本地方法栈（Native Method Stack）**：这个和虚拟机栈逻辑一样，只不过它是服务于 `Native` 方法（比如 C++ 写的底层方法）的。

- **注**：在 HotSpot 虚拟机中，其实直接把这两个栈合二为一了。”

接下来是**线程共享**的区域，这也是出现问题最多的地方

1. **堆**：这是 JVM 内存中最大的一块。**几乎所有的对象实例和数组**都在这里分配内存。

- **关注点**：它是垃圾回收器（GC）的主要工作场所。我们常说的‘新生代’、‘老年代’划分，就是指在这里。
- **异常**：如果对象太多塞满了，就会报 `OutOfMemoryError: Java heap space`。

2. **方法区**：它存储了已被虚拟机加载的**类信息、常量、静态变量**等数据。

- **版本差异（亮点）**：在 **JDK 1.7 及以前**，它在实现上叫‘永久代’（PermGen），且位于堆内存中，容易导致 OOM。而在 **JDK 1.8 及以后**，JVM 移除了永久代，改用在本地内存（Native Memory）中实现的**元空间（Metaspace）**。这样做的好处是方法区不再受堆内存大小的限制，减少了 OOM 的风险。”

总结一下：**栈管运行，堆管存储**。栈是线程私有的，堆和方法区是线程共享的。

追问1： 这些区域里，哪些会发生 OOM（内存溢出）

除了程序计数器以外，其他 4 个区域都有可能发生 OOM。

1. **堆**：最常见。对象太多且无法回收时，报 `OutOfMemoryError: Java heap space`。
2. **方法区**：加载的类太多（比如 Spring 动态代理生成太多类），报 `OutOfMemoryError: Metaspace`。
3. **栈**：虽然平时更多报的是 `StackOverflowError`（递归太深），但如果线程请求的栈深度大于允许深度，或者创建太多线程导致内存耗尽，也会报 OOM。

栈与堆的区别

Q：简单说一下JVM里栈和堆的区别是什么？

A：

可以从三个维度来区分：

1. 存储内容：

- 栈主要存储局部变量（基本数据类型、对象的引用）和方法调用的上下文。
- 堆存储的是实际的对象实例（`new` 出来的东西）。

2. 内存管理：

- 栈是自动管理的，方法结束了自动出栈释放，不需要 GC。
- 堆是手动管理（C++）或 GC 管理（Java）的，是垃圾回收的主要场所。

3. 物理地址：

- 栈通常是连续的内存空间（速度快）。
- 堆是不连续的内存空间，通过链表等方式维护（灵活但稍慢）。

追问1： 什么是栈溢出（StackOverflow）？什么情况下会发生

`StackOverflowError` 是指线程请求的栈深度超过了虚拟机允许的深度。最典型的场景就是死递归（递归方法没有出口），或者递归层级过深。每一次方法调用都会压入一个栈帧，如果一直压栈不弹栈，内存很快就爆了。

JDK1.8的变化

Q：JDK 1.8 在内存区域上有什么重大变化？为什么要用元空间替代永久代

A：

最大的变化是：**移除了永久代，引入了元空间**。在 JDK 1.7 及之前，方法区是堆内存的一部分（永久代）。而在 JDK 1.8 中，元空间使用的是**本地内存**，不再占用 JVM 的堆内存。

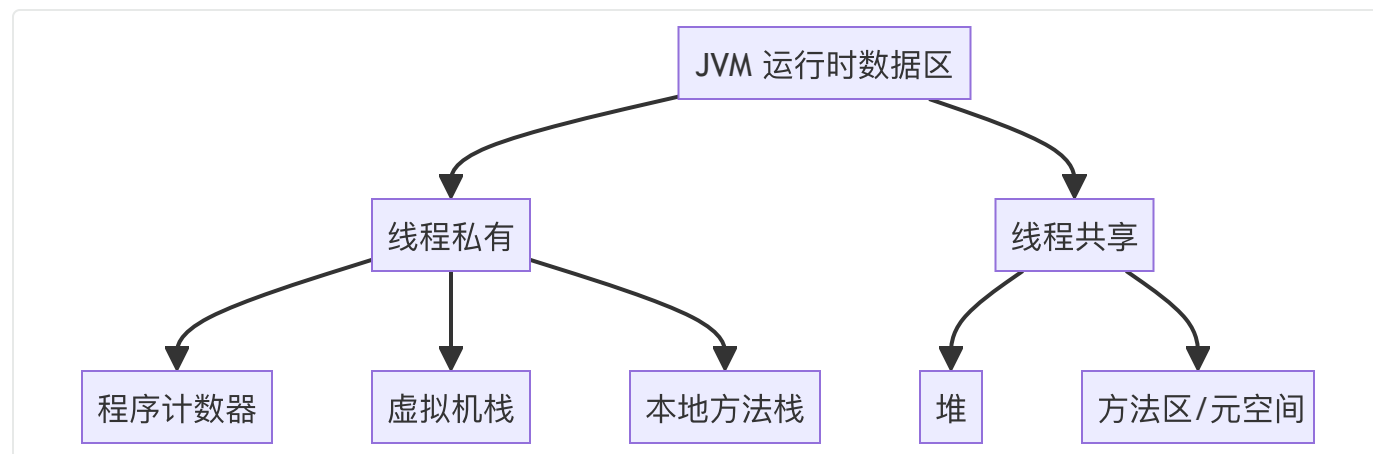
为什么要替换？主要有两个原因：

1. **避免 OOM**：永久代的大小是启动时固定的，很难调整。现在的应用（尤其是 Spring 这种大量使用动态代理的框架）会动态生成很多类，很容易把永久代撑爆。而元空间直接使用服务器的物理内存，上限大得多。
2. **合并 HotSpot 和 JRockit**：当年 Oracle 收购了 JRockit，JRockit 里面就没有永久代，为了代码统一，就把 HotSpot 的永久代也去掉了。

追问1：元空间既然用的是本地内存，是不是就永远不会 OOM 了

不是的，依然会 OOM。虽然本地内存很大，但不是无限的。如果在生产环境中，我们没有限制元空间大小（默认是无限），且程序有 Bug 导致动态疯狂生成类，可能会把服务器所有的物理内存吃光，导致整个服务器宕机（被系统 Kill 掉）。所以线上环境我们通常还是会配置 `-XX:MaxMetaspaceSize` 来兜底。

图解补充：运行时数据区总览



对比补充：哪些区域会 OOM

区域	是否线程共享	主要存什么	是否 GC 重点区域	典型异常
程序计数器	否	当前线程字节码位置	否	基本不会 OOM
虚拟机栈	否	栈帧、局部变量、操作数栈	否	StackOverflowError / OOM
本地方法栈	否	Native 方法调用信息	否	OOM
堆	是	对象实例、数组	是	Java heap space
方法区/元空间	是	类元数据、常量、静态变量	重点但不如堆常见	Metaspace

高频追问补充

追问补充1：为什么说“栈管运行，堆管存储”？

因为方法执行过程中的上下文信息主要在栈里，比如局部变量、操作数栈、方法调用链；而真正 new 出来的对象通常在堆里。这样回答能快速把栈和堆的职责区分开。

追问补充2：元空间移到本地内存之后，线上就不用管了吗？

不是。元空间虽然不再占用堆，但依然吃机器内存。如果动态生成类很多，比如代理类暴涨、类加载器泄漏，照样会把机器内存打满，所以线上一般还是会限制 MaxMetaspaceSize 。

速记版

内存区域建议按这条线记：

- 三私有：程序计数器、虚拟机栈、本地方法栈
- 两共享：堆、方法区/元空间
- 堆是 GC 主战场
- JDK8 之后永久代换成元空间

一句话面试版：

JVM 运行时内存区域可以先按线程私有和线程共享来分，私有区偏执行上下文，共享区偏对象和类元数据存储。

垃圾回收

垃圾回收

垃圾回收是 JVM 面试里非常核心的一条线，常见考法一般会从“对象什么时候会被回收”开始，再追问到垃圾回收算法、垃圾回收器、三色标记和根可达性。

如何判断对象是否可回收

Q: JVM 如何判断一个对象是否可以被回收?

现在主流 JVM 采用的是 **可达性分析算法**，而不是单纯的引用计数。

它的思路是：从一组称为 **GC Roots** 的根对象出发，沿着引用关系向下搜索。

如果一个对象从 GC Roots 出发不可达，那么这个对象就可以被判定为垃圾对象。

常见的 GC Roots 包括：

- 虚拟机栈中引用的对象
- 方法区中类静态属性引用的对象
- 方法区中常量引用的对象
- 本地方法栈 JNI 引用的对象

之所以不用引用计数，核心原因是 **它无法解决循环引用问题**。

常见的垃圾回收算法

Q: 常见的垃圾回收算法有哪些?

常见垃圾回收算法主要有四种：

第一种，标记-清除。

先标记存活对象，再清理未标记对象。优点是实现简单；缺点是会产生内存碎片。

第二种，复制算法。

把内存分成两块，每次只使用一块；回收时把存活对象复制到另一块。优点是不会有内存碎片；缺点是浪费一半空间。

第三种，标记-整理。

先标记存活对象，再把存活对象向一端移动，最后直接清理边界外内存。优点是不会有碎片；缺点是整理对象成本较高。

第四种，分代收集。

这不是一种具体算法，而是一种组合思想：年轻代和老年代的对象特点不同，所以采用不同算法更合适。

分代回收思想

Q：为什么 JVM 要做分代回收？

因为不同区域的对象生命周期分布非常不一样。

年轻代的对象特点是：**创建多、死亡快、存活率低。**

所以年轻代更适合复制算法，回收效率高。

老年代的对象特点是：**存活率高、对象体积更大。**

如果还用复制算法，成本就太高了，所以老年代通常更适合标记-清除或标记-整理。

所以分代回收的本质是：

针对不同对象生命周期，采用更匹配的回收策略。

三色标记法

Q：什么是三色标记法？

三色标记法是并发标记过程中常用的一种抽象模型，用来描述对象的扫描状态。

- **白色**：还没被标记，理论上可能是垃圾
- **灰色**：已经被标记，但它引用的对象还没扫描完
- **黑色**：已经被标记，而且它引用的对象也都扫描完成

并发标记时，GC 线程和用户线程可能同时修改对象引用关系。

如果处理不好，就可能出现“本来应该存活的对象被漏标”的问题。

因此现代垃圾回收器通常会配合：

- **增量更新**
- **SATB（原始快照）**

来修正并发标记带来的问题。

常见垃圾回收器

Q：常见的垃圾回收器有哪些？怎么选？

常见垃圾回收器可以按代际来看：

年轻代 常见有：

- Serial
- ParNew
- Parallel Scavenge

老年代 常见有：

- Serial Old
- CMS
- Parallel Old

整堆/区域化回收器 常见有：

- G1
- ZGC
- Shenandoah

如果是 JDK8 时代的常见答案，面试最常问的是 **CMS** 和 **G1**；

如果是更新一点的版本，也可能会问 **ZGC**。

追问1：G1 的特点是什么？

G1 的核心特点是把整个堆划分成很多 Region，不再机械区分连续的年轻代和老年代空间。

它会优先回收“垃圾最多、收益最高”的 Region，因此叫 Garbage First。

它的优势是：

- 更适合大堆
- 可预测停顿时间
- 整体吞吐和停顿之间比较均衡

CMS 和 G1 的区别

Q：CMS 和 G1 有什么区别？

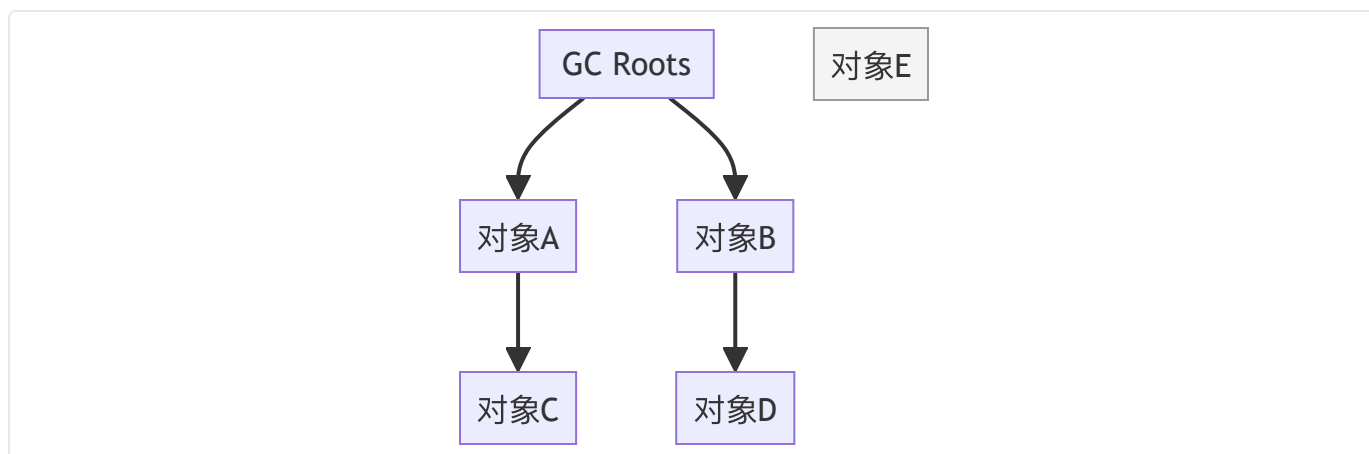
CMS 更强调低停顿，它的老年代回收流程里大量使用并发标记和并发清理。
缺点是会产生内存碎片，而且对 CPU 资源比较敏感。

G1 是区域化回收器，兼顾吞吐和停顿，更适合大堆和需要可控停顿的场景。
它可以通过目标停顿时间来做一定程度的回收调度。

所以工程里如果让我给面试官一个简短判断：

- JDK8 以前很多系统会选 CMS
- 现在更常见的默认方案是 G1

图解补充：GC Roots 与可达性分析



对比补充：回收算法怎么区分

算法	核心动作	优点	缺点	适合场景
标记-清除	标记后直接清理	实现简单	有碎片	老年代早期方案
复制	存活对象复制到新区域	无碎片、效率高	浪费空间	年轻代
标记-整理	标记后压缩存活对象	无碎片	移动对象成本高	老年代
分代收集	不同代组合不同算法	更贴合对象生命周期	实现更复杂	现代 JVM

对比补充：常见回收器怎么答

回收器	关键词	优势	典型短板
CMS	低停顿	老年代并发回收	碎片、多核依赖强
G1	Region、可预测停顿	大堆更友好	调优理解成本更高
ZGC	超低停顿、染色指针	超大堆场景优秀	版本要求更高

高频追问补充

追问补充1：为什么引用计数不适合作为主流回收方案？

因为它很难处理循环引用。两个对象互相引用时，即使业务上已经没人能访问它们了，引用计数也不为 0，无法回收。

追问补充2：三色标记为什么会漏标？

因为并发标记期间，用户线程可能同时改引用关系。如果一个黑对象新增了对白对象的引用，而灰对象到白对象的旧引用又被删掉，就可能让白对象被漏标。所以才需要 SATB 或增量更新这类补偿机制。

追问补充3：面试里 CMS 和 G1 怎么快速选型？

如果是老系统背景，可以说 CMS 曾经因为低停顿用得很多；如果是新系统或新版本，通常优先考虑 G1，因为它已经是更主流、更稳妥的默认方案。答题重点不是“背历史”，而是讲清楚停顿、碎片和大堆场景的权衡。

速记版

垃圾回收建议按这条线记：

- 先讲怎么判断垃圾：可达性分析
- 再讲算法：标记清除、复制、整理、分代
- 再讲并发标记：三色标记、SATB、增量更新
- 最后讲回收器：CMS、G1、ZGC

一句话面试版：

JVM 垃圾回收的本质，是先判断对象是否还可达，再根据对象生命周期和停顿目标选择合适的回收算法和回收器。

类加载机制

类加载机制

类加载的生命周期

Q：Java 类是从哪里来的？讲一下类加载的整个流程？

A：

Java 的类加载机制是指把 `.class` 文件加载到内存，并进行校验、解析和初始化，最终形成可以被虚拟机直接使用的 Java 类型。整个过程分为 5 个阶段：

1. 加载：通过类的全限定名（包名+类名）找到 `.class` 二进制流，把它读进内存，并在堆中生成一个 `java.lang.Class` 对象作为访问入口。
2. 验证：检查这个字节码是不是合法的（比如文件头是不是 `0xCAFEFABE`），防止恶意代码破坏 JVM。
3. 准备【重点】：为类的静态变量（static）分配内存，并设置默认初始值。
 - 注意：这里是设置默认值（比如 `int` 为 0，`Object` 为 null），而不是代码里写的那个值。
4. 解析：把常量池里的符号引用（比如仅仅是个字符串 "ArrayList"）替换为直接引用（内存中的实际地址指针）。
5. 初始化【重点】：这是类加载的最后一步。
 - 这时候才真正执行类中定义的 Java 代码（也就是 `static` 代码块和 `static` 变量的赋值操作）。

追问1：什么时候会触发类的初始化？

JVM 规定了只有 主动引用 时才会初始化，常见的场景有：

1. `new` 一个对象时。
2. 读取或设置一个类的静态字段（static）时。
3. 调用类的静态方法时。

4. 对类进行反射调用（`Class.forName`）时。
5. 初始化一个子类时，如果父类还没初始化，会先触发父类的初始化。

双亲委派模型（核中核，必须背熟）

Q：什么是类加载器？什么是双亲委派模型？为什么要这么设计？

A：

首先，类加载器的作用就是把编译好的 `.class` 字节码文件加载到 JVM 内存中。在 JDK 8 及之前的版本中，主要有三个核心的类加载器，它们呈层级关系：

1. **启动类加载器**：它是最顶层的。负责加载 JDK 的核心类库（即 `JAVA_HOME/lib` 目录下的类，比如 `String`、`System` 等）。
 - 注意点：它是由 C++ 实现的，所以我们在 Java 代码里拿不到它的引用（会返回 `null`）。
2. **扩展类加载器**：它负责加载 JDK 的扩展库（`JAVA_HOME/lib/ext` 目录）。
3. **应用类加载器**：这是我们平时接触最多的。它负责加载 `Classpath` 下的类，也就是我们自己的代码和引入的第三方 jar 包。”

所谓双亲委派模型，其实不是指有两个父母，而是指‘父辈委托’机制。它的核心逻辑可以概括为八个字：‘向上委托，向下加载’。

具体流程是这样的：当一个加载器（比如应用类加载器）收到类加载请求时，它不会自己先去加载，而是把这个请求原封不动地委托给父加载器去尝试。每一层都是如此，直到请求传送到最顶层的 **Bootstrap ClassLoader**。

- 如果父加载器在自己的搜索范围内找到了这个类，就成功返回；
- 只有当父加载器反馈‘我加载不了’（找不到）时，子加载器才会尝试自己去加载。

这么设计主要有两个目的：**安全和避免重复加载**。最核心的是**安全**。

举个最经典的例子：假设有人恶意写了一个 `java.lang.String` 类，里面植入了破坏系统的代码。

- **如果没有双亲委派**：应用类加载器可能直接就加载了这个恶意的 `String` 类，那整个系统的 `String` 就被替换了，JVM 也就崩溃了。
- **有了双亲委派**：当加载请求向上委托到 `Bootstrap ClassLoader` 时，它发现自己负责的 `rt.jar` 里已经有了 `java.lang.String`，它就会直接加载 JDK 自带的那个安全的 `String` 类。

这样，恶意的类就永远没有机会被加载进来。这就保证了 **Java 核心类库的安全性（沙箱安全机制）**

追问1: 父加载器是子加载器的父类吗?

不是继承关系, 而是组合关系。它们在代码层面并没有 `extends` 关系, 而是通过一个 `parent` 成员变量持有父加载器的引用。这是一种包含关系, 不是类继承

怎么打破双亲委派

Q: 有什么场景需要打破双亲委派, 怎么打破

A:

双亲委派不是强制约束, 而是一种建议

场景一: Tomcat

- Tomcat 上可能部署了两个 Web 应用。应用 A 依赖 Spring 4, 应用 B 依赖 Spring 5。
- 如果遵循双亲委派, 类加载器只会加载一份 Spring (假设是 4)。那应用 B 也是用的 Spring 4, 启动就报错了。
- **解决**: Tomcat 自定义了 `WebAppClassLoader`, 它重写了 `loadClass` 方法。优先自己加载 `WEB-INF/classes` 下的类, 加载不到再给父类。这样就实现了应用间的隔离。

场景二: JDBC

- JDBC 的接口 (`java.sql.Driver`) 是 Java 核心包, 由 Bootstrap 加载。
- 但具体的驱动实现 (如 MySQL Driver) 是厂商提供的第三方 Jar 包, 在 Classpath 下, 由 Application 加载器加载。
- **矛盾**: Bootstrap 加载的代码 (DriverManager) 需要去调用 Application 加载的代码 (MySQL Driver), 这违反了双亲委派 (父调子)。
- **解决**: 利用 线程上下文类加载器 (Thread Context ClassLoader)。核心类通过 `Thread.currentThread().getContextClassLoader()` 拿到应用类加载器, 去加载第三方的实现类。

JVM调优实战

JVM调优实战

JVM 调优这块，面试官更看重你是不是有“问题驱动”的思维，而不是背参数大全。回答时最好按“现象 -> 排查 -> 结论 -> 调整方案”来讲。

JVM 调优的基本思路

Q：你做过 JVM 调优吗？一般怎么做？

如果让我概括 JVM 调优的基本思路，我会分成四步：

第一步，先看现象。

- 接口 RT 抖动
- Full GC 频繁
- Young GC 特别频繁
- CPU 飙高
- 内存持续上涨

第二步，看监控。

- 堆使用率
- GC 次数和停顿时间
- 老年代增长趋势
- 晋升速率
- 对象分配速率

第三步，定位原因。

比如是年轻代太小、对象创建过快、存在内存泄漏、还是垃圾回收器不适配业务场景。

第四步，再做参数调整。

调优应该基于现象和监控，不是先拍脑袋改参数。

常见调优场景

Q: Young GC 很频繁，你会怎么排查？

Young GC 频繁通常说明年轻代压力过大，常见原因有：

- 年轻代空间偏小
- 短生命周期对象创建太多
- 批量操作导致瞬时分配过高

排查时我会重点看：

- Eden 区是否很快被打满
- 对象分配速率是否异常
- 接口是否存在大对象创建、重复对象创建

优化方向通常包括：

- 适当调大年轻代
- 减少无意义对象创建
- 优化批量逻辑，降低瞬时分配峰值

Q: Full GC 频繁怎么办？

Full GC 频繁比 Young GC 更危险，因为它往往意味着停顿更长，对线上 RT 影响更明显。

常见原因包括：

- 老年代空间不足
- 对象晋升过快
- 大对象直接进入老年代
- 存在内存泄漏

排查思路一般是：

- 看老年代占用是否持续上涨
- 看 Full GC 后内存是否能明显回落
- 如果回落不明显，就要怀疑内存泄漏
- 结合 dump 文件分析大对象和引用链

调优时常见的参数关注点

Q: JVM 调优时你会关注哪些参数?

我会优先关注这些参数:

- `-Xms`、`-Xmx` : 初始堆和最大堆
- `-Xmn` : 年轻代大小
- `-XX:SurvivorRatio`
- `-XX:MaxTenuringThreshold`
- `-XX:+UseG1GC` 等垃圾回收器选择参数
- `-XX:MaxGCPauseMillis` : 目标停顿时间

但面试里我不会把重点放在背参数上, 而是强调:
参数只是工具, 关键是它要和实际业务现象对应起来。

如何选择垃圾回收器

Q: 垃圾回收器怎么选?

一般我会结合三点来选:

- 机器资源情况
- 业务更关注吞吐还是停顿
- 堆大小和对象生命周期分布

如果是比较新的业务系统, 而且希望停顿更可控, 我会优先考虑 G1。

如果是对极低停顿要求非常高、JDK 版本也匹配, 才会进一步考虑 ZGC 这类更先进的回收器。

【了解】执行引擎与JIT

执行引擎与JIT

执行引擎和 JIT 一般不是最前面的基础题，但很适合拿来体现你对 JVM“不是只会背堆和栈”的理解深度。

Java 是编译型还是解释型

Q: Java 是编译型语言还是解释型语言？

更准确的说，Java 同时包含了 **编译** 和 **解释** 两个阶段。

源码先通过 `javac` 编译成字节码，这是编译阶段；

运行时，JVM 先解释执行字节码，在热点代码达到一定阈值后，再通过 JIT 编译成本地机器码执行。

所以如果面试官这样问，我不会简单回答“Java 是编译型”或者“Java 是解释型”，而是会说：
Java 是先编译成字节码，再由 JVM 解释执行并结合 JIT 动态优化。

JIT 是什么

Q: 什么是 JIT？

JIT 指的是 **Just In Time Compiler**，也就是即时编译器。

它的作用是把热点字节码编译成本地机器码，提高运行效率。

如果一直纯解释执行，灵活是灵活，但性能不够高；

JIT 的意义就在于：把最值得优化的那部分代码进一步编译优化。

为什么热点代码更适合 JIT

Q：为什么 JVM 不一开始就把所有代码都编译成机器码？

因为 JIT 编译本身也是有成本的。

如果一开始就把所有代码全部编译：

- 启动会变慢
- 很多根本不会高频执行的代码也会被白白编译
- 内存占用也会更高

所以 JVM 的策略更聪明：

先解释执行，等某段代码真的成为热点，再去编译优化。

这是一种典型的“按需优化”思路。

redis

数据类型

数据类型

基础中的基础，但又很喜欢考察

重点掌握String 的 SDS、ZSet 的跳表，以及 Bitmap/HyperLogLog 的应用

讲基础的东西要跟自己的项目结合，不要单讲

五大基础类型及应用场景

Q： Redis 有哪些常用的数据类型？你在项目中是怎么用的？

A：

redis 最常用的五大基础数据类型是：String、Hash、List、Set 和 ZSet (Sorted Set) 。

在我的项目中，我是这样用的【挑几个讲就行】：

1. **String**：最常用。主要用来做**常规缓存**（存 JSON 对象）、**计数器**（如视频播放量 `incr`）、以及**分布式锁**（`setnx`）。
2. **Hash**：用来存储**对象数据**，比如购物车（用户ID为 Key，商品ID为 field，数量为 value），或者用户信息。比起 String 存 JSON，Hash 可以单独修改某个字段，减少网络传输。
3. **List**：主要用作简单的**消息队列**（LPUSH + BRPOP 阻塞读取），或者用来做**最近 N 条记录**（比如微信公众号的文章列表）。
4. **Set**：利用它的‘去重’和‘交并差’特性。比如做**抽奖**（去重）、**共同好友**（交集）、**标签推荐**。
5. **ZSet（有序集合）**：主要用于做**排行榜**（Top N），或者做**延时队列**（Score 存过期时间戳，轮询取最小的）。

底层结构概览：

- **String**: SDS (简单动态字符串)。
- **List**: QuickList (双向链表 + 压缩列表)。
- **Hash**: Listpack (小数据) / Hashtable (大数据)。
- **Set**: IntSet (整数集合) / Hashtable。

- ZSet: Listpack (小数据) / SkipList (跳表) + Hashtable。

String的底层实现 (SDS)

Q: Redis 的 String 是直接用的 C 语言字符串吗? 什么是 SDS?

A:

Redis 没有直接使用 C 语言的字符串 (以 `\0` 结尾), 而是自己封装了一种叫 SDS (Simple Dynamic String, 简单动态字符串) 的结构。

为什么要这么做呢? 主要有三个原因:

1. 性能高: SDS 内部维护了一个 `len` 字段。获取字符串长度的时间复杂度是 $O(1)$, 而 C 语言需要遍历整个字符串是 $O(N)$ 。
2. 二进制安全: C 语言字符串遇到 `\0` 就结束了, 不能存图片或音频数据。SDS 是根据 `len` 长度来判断结束的, 所以可以存任意二进制数据。
3. 防止缓冲区溢出: SDS 在拼接字符串时, 会先检查空间够不够, 不够会自动扩容, 避免了 C 语言中常见的内存溢出问题。

SDS的结构 (供理解记忆):

```
1 struct sdshdr {  
2     int len;    // 已使用长度  
3     int free;   // 剩余可用空间  
4     char buf[]; // 字符数组  
5 };
```

- 预分配策略: 为了减少内存重分配次数, 扩容时会多分配空间 (空间预分配), 缩容时不会立刻回收空间 (惰性空间释放)。

追问1: SDS 的最大容量是多少

Redis 规定 String 类型的值最大能存储 512 MB

ZSet的底层实现 (重点!)

Q: ZSet底层是怎么实现的？为什么用跳表而不用红黑树？

A:

ZSet 的底层实现比较灵活。当元素较少且长度较短时，使用 Listpack（或 Ziplist）压缩列表来节省内存。当数据量变大时，会转换为 SkipList（跳表）+ Hashtable。

重点说一下跳表：它其实就是一个多层的链表。最底层是一个包含所有元素的普通链表。在这个链表之上，像建索引一样，每隔几个元素提取一个节点到上一层。查询时，先从顶层开始找，通过索引快速跳过大量元素，一层层往下找。它的查询复杂度是 $O(\log N)$ ，和二分查找类似。

至于为什么不用红黑树（或 B+ 树）：

1. **实现简单：**跳表的代码比红黑树简单得多，容易维护。
2. **范围查询更强：**ZSet 经常需要执行 ZRANGE（查排名 1-100）。跳表只需要找到起点，然后顺着底层链表遍历就行了；而红黑树通过中序遍历做范围查询稍微麻烦一点。
3. **并发调整优势：**在做插入操作时，跳表只需要修改局部的指针，锁的粒度可以更小（虽然 Redis 是单线程的，但这是跳表通用的优势）。

特殊数据类型（Bitmap/HyperLogLog）

Q: 如果我要统计一个网站 1 亿用户的日活（UV）和签到情况，你会怎么设计？

A:

这涉及到 Redis 的两个特殊数据类型：

1. **统计签到：**我会用 Bitmap。因为签到只有‘签了’和‘没签’两个状态，正好对应二进制的 1 和 0。我可以把用户 ID 映射为 Bitmap 的偏移量（Offset）。这样，一个用户一年的签到记录只需要 365 bit，大概 46 字节。1 亿用户也就几十兆内存，极其节省空间。
2. **统计日活 UV：**如果要求精确去重，可以用 Set，但 1 亿个 ID 太占内存了。如果允许极小的误差（Redis 标准误差 0.81%），我会选用 HyperLogLog。它利用概率统计原理（伯努利试验），不管存入多少数据，它占用的内存固定只需要 12KB，非常适合做大规模的 UV 统计。

追问1: HyperLogLog 可以取到具体的 userId 吗

不可以。HyperLogLog 是属于**基数统计**算法，它只记录‘大概有多少个不同的元素’，并不存储元素本身。如果业务需要把当天访问过的用户列表导出来，那就只能用 Set 或者数据库了。

高可用

高可用

关于 Redis 高可用，比较适合按“怎么存、怎么备、怎么切、怎么扩”来记忆，也就是：持久化 -> 主从 -> 哨兵 -> 集群。

持久化

Q: Redis 是内存数据库，宕机了数据怎么办？

Redis 主要有两种持久化方式：

RDB：定时把内存快照写到磁盘。

优点是恢复快、文件紧凑；缺点是两次快照之间如果宕机，可能丢失这段时间的数据。

AOF：把写命令按追加日志方式记录下来。

优点是数据更安全；缺点是文件更大，恢复速度通常比 RDB 慢。

生产里常见选择是 **混合持久化**，兼顾恢复速度和数据安全。

主从复制

Q: Redis 主从复制的意义是什么？

主从复制主要解决两个问题：

- 提升读能力，做读写分离
- 提高可用性，为故障切换提供基础

一般来说，主节点负责写，从节点负责读。

这样可以让写路径更聚焦，也能把大量读流量分摊到从节点上。

主从同步原理与优化

Q: Redis 主从是怎么同步数据的?

Redis 主从同步大致可以分成:

- 全量同步
- 增量同步

第一次建立复制关系时, 往往要做全量同步: 主节点生成 RDB, 发送给从节点, 从节点加载后再接收后续命令流。

如果连接短暂断开, 且复制积压缓冲区里还有断开期间的数据, 就可以做增量同步, 只补差量。

优化思路一般包括:

- 合理配置复制积压缓冲区
- 避免大事务和瞬时大量写入
- 控制从节点数量, 避免主节点同步压力过大

哨兵机制

Q: 哨兵的作用是什么?

哨兵本质上是 Redis 主从体系上的自动故障恢复方案, 主要做三件事:

- 监控主从节点状态
- 主节点故障时发起选主
- 把新主节点信息通知客户端

所以它解决的是:

主节点挂了之后, 不想再依赖人工半夜手动切换。

Redis Cluster

Q: 什么时候需要 Redis Cluster?

当单机 Redis 已经无法满足需求时，典型有两类情况：

- 单节点内存不够
- 单主写压力太大

这时就需要 Redis Cluster。

它通过 **16384 个哈希槽** 做数据分片，把不同 key 分散到多个主节点上，从而实现水平扩展。

追问1：为什么要把数据和槽位绑定，而不是直接和节点绑定？

因为这样在扩容、缩容、迁移时更灵活。

本质上是先定位到槽位，再由槽位映射到节点，能够降低节点变化时的数据迁移成本。

怎么回答 Redis 高可用

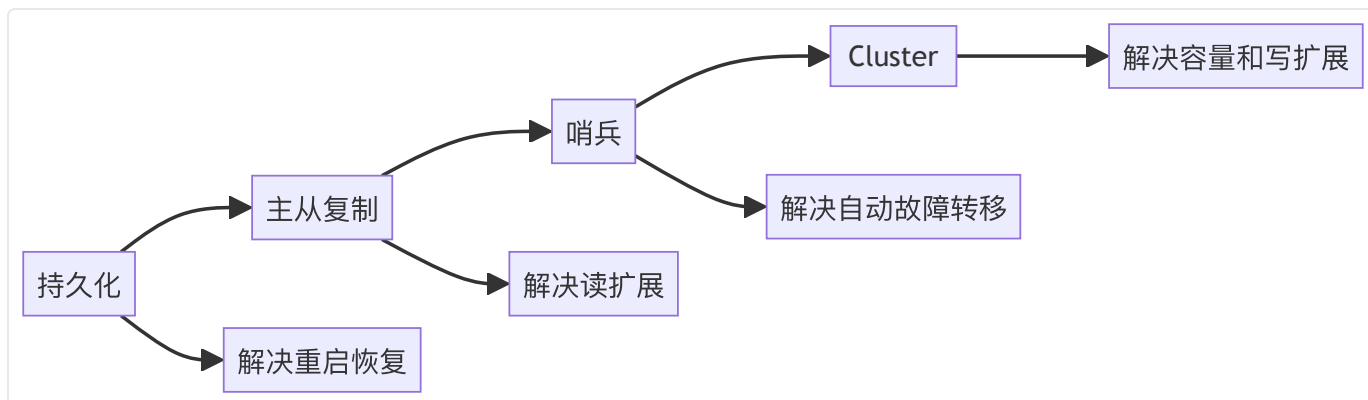
Q：如果面试官让你整体讲一遍 Redis 高可用，你怎么组织？

我会按这个顺序讲：

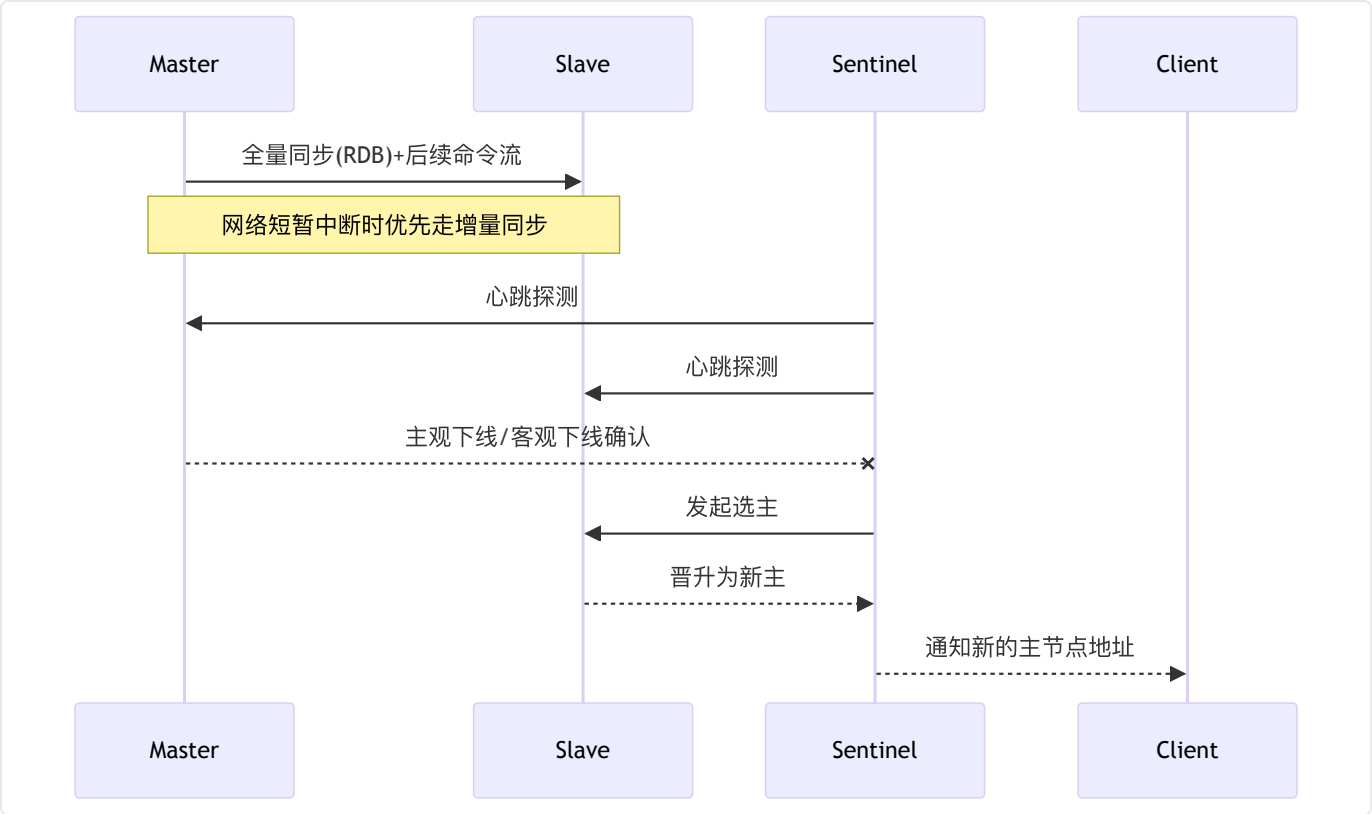
1. 持久化，解决重启后的数据恢复问题
2. 主从复制，解决读扩展和基础高可用问题
3. 哨兵，解决主节点故障自动切换问题
4. 集群，解决更大数据量和更高并发写入问题

这样回答比较有层次，也更像工程视角，而不是零散背点。

图解补充：Redis 高可用全景



图解补充：主从同步与故障切换



对比补充：怎么记高可用组件

组件	解决什么问题	代价/风险	面试关键词
RDB	重启恢复快	快照间隔内会丢数据	快照、恢复速度
AOF	数据更安全	文件更大、恢复更慢	追加写、重放
主从复制	读扩展、基础高可用	主从延迟	全量/增量同步
哨兵	自动故障转移	不解决容量问题	监控、选主、通知
Cluster	分片扩容	运维更复杂	16384 槽位

高频追问补充

追问补充1：为什么 Redis 高可用不能只靠主从复制？

因为主从复制只能“备份数据”，不能自动切换主节点。主挂了之后，业务还需要人工介入，所以还得靠哨兵做故障发现和自动选主。

追问补充2：哨兵一定能保证不丢数据吗？

不能。哨兵解决的是可用性，不是强一致性。主从切换过程中，如果主节点还没来得及把最新写入同步到从节点，就可能丢数据。所以面试里最好主动补一句：Redis 高可用偏 AP，不是 CP。

追问补充3：Cluster 为什么适合扩容？

因为数据不是直接绑定机器，而是先绑定槽位，再由槽位映射到节点。扩容时迁移的是槽位，不是全量重分布，成本更可控。

速记版

Redis 高可用按四层记：

- 持久化：解决“重启后还有没有数据”
- 主从：解决“读能不能扩”
- 哨兵：解决“主挂了能不能自动切”
- Cluster：解决“单机容量和写瓶颈”

一句话回答模板：

我通常按 持久化 -> 主从 -> 哨兵 -> 集群 来讲 Redis 高可用，因为这条线正好对应 恢复、备份、切换、扩容 四个工程问题。

过期策略

过期策略

Q: Redis 的 Key 过期了是立刻删除吗? 如果内存满了 Redis 会怎么做?

A:

这个问题其实包含两个层面，我们需要把它们区分开来看：一个是**时间维度**，也就是设置了 TTL（过期时间）的 Key 怎么处理，这叫**过期策略**；另一个是**空间维度**，也就是内存真的不够用了怎么办，这叫**内存淘汰策略**。

我先回答第一个问题：**Key 过期了并不会立刻删除**。

Redis 为了在‘节省内存’和‘不占用太多 CPU’之间找平衡，采用的是‘**惰性删除 + 定期删除**’的混合策略。

1. **首先是惰性删除**：这就好比是‘懒加载’。Redis 不会专门为了删除一个 Key 去中断当前的操作。而是等到客户端**主动查询**这个 Key 的时候，Redis 会先检查一下它有没有过期。如果过期了，就直接删掉并返回空。
 - **_好处_**是：非常节省 CPU，因为不做无用功。
 - **_坏处_**是：如果那些过期的 Key 一直没人查，它们就会一直占着内存，变成‘冷数据’垃圾。
2. **为了解决这个问题，Redis 还有定期删除**：Redis 默认每隔 100 毫秒，会**随机抽取**一部分设置了过期时间的 Key 进行检查，如果过期了就删除。注意这里是‘随机抽取’而不是‘全盘扫描’，因为如果 Key 太多，全盘扫描会直接把 CPU 跑满，导致 Redis 卡顿。

所以结论是：**通过惰性删除保证 CPU 性能，通过定期删除兜底清理内存垃圾。**”

接下来说第二个问题：**如果内存满了（达到了 maxmemory 上限）会怎么做？**

redis是提供了多种内存淘汰策略的，在工程实践中，我主要关注这三种：

1. **noeviction（默认策略）**：这是默认配置，内存满了直接报错，不淘汰任何数据。这在生产环境通常是不推荐的，因为会导致服务不可用。
2. **allkeys-lru（最常用/推荐）**：不管 Key 有没有设置过期时间，从**所有** Key 中，淘汰最近最少使用（LRU）的数据。
 - **适用场景**：这是最通用的选择。如果我们的应用也是‘热点数据’比较明显（比如新闻、电商），这个策略能保证热数据一直在内存里。
3. **volatile-lru**：它只从**设置了过期时间的** Key 里面去淘汰最近最少使用的。
 - **适用场景**：这通常用于那些‘既把 Redis 当缓存（设置了过期），又当持久化存储（没设置过期）’的混用场景。但我个人建议尽量把缓存和存储拆分开。”

总结一下：过期 Key 是靠‘惰性+定期’慢慢删的；而内存满了，通常会配置 **allkeys-lru** 来强制淘汰冷数据，保证新数据能写进去。

追问1：Redis 采用的是标准的 LRU 算法吗？为了维护 LRU 它是怎么做的

不是标准的。

Redis 采用的是一种近似 LRU 算法。之所以不使用标准的 LRU（基于双向链表），是因为为了省内存。

标准的 LRU 需要维护一个双向链表，把所有数据串起来。每当一个 Key 被访问，就得把它移动到链表头。这就带来两个问题：

1. **内存占用大**：每个 Key 都需要额外的指针（prev, next），这就增加了不必要的内存开销。
2. **性能损耗**：在大并发下，频繁地调整链表指针，会对 Redis 这种高性能组件造成压力。”

Redis 的做法非常聪明，它是一种**随机采样**的策略：

1. Redis 在每个 Key 的对象结构（`redisObject`）里，存了一个 24 bit 的字段，用来记录最后一次访问的时间戳（lru 字段）。
2. 当内存满了需要淘汰时，Redis 不会去遍历整个数据库，而是**随机抽取 N 个 Key**（默认是 5 个）。
3. 然后在这 5 个 Key 里面，把 `lru` 字段最旧的那个淘汰掉。

虽然是概率性的，但官方测试数据表明，当采样数（N）设置为 10 的时候，效果已经非常接近标准 LRU 了，但内存利用率却高得多。

追问2：那 LRU 策略有什么缺点吗？听说 Redis 后来引入了 LFU，是为了解决什么问题

LRU 的核心逻辑是‘最近被访问过，未来被访问的概率就大’。

但在一种场景下它会失效，那就是**全表扫描或突发流量**

举个例子：假设我的 Redis 里存满了热点数据。突然，有一个定时任务或者离线分析，对数据库做了一次**全量查询**。这些冷数据因为刚刚被查了一次，时间戳非常新，会被 LRU 误认为是热数据而留下来；反倒是原本真正的热点数据，因为被挤占了位置而被淘汰了。这就导致接下来的一段时间，缓存命中率急剧下降。

为了解决这个问题，Redis 4.0 引入了 **LFU（Least Frequently Used，最近最不常用）**。

和 LRU 关注‘时间’不同，LFU 关注的是访问频率。

如果使用 LFU 策略，哪怕刚才那批冷数据被扫描了一次，但因为它们的历史访问次数很低，依然会被优先淘汰。而真正的热点数据，因为长期积累了较高的访问频次，即便最近几秒没被查，也不会轻易被踢走。

所以，在存在因为‘扫描’导致缓存被冲刷的业务场景下，用 LFU 效果更好。

缓存三剑客

缓存三剑客

缓存三剑客（穿透、击穿、雪崩）是 Redis 面试中出场率 100% 的题目。

这三者名字很像，容易背混，记忆的时候着重记忆三者区别以及各自出现时候的解决方案

- **穿透**：直接穿过（Redis和DB都没有）。
- **击穿**：定点爆破（某一个热点 Key 突然挂了）。
- **雪崩**：全面崩盘（大面积 Key 挂了）。

缓存穿透

Q：什么是缓存穿透？怎么解决？

A：

缓存穿透是指查询一个**根本不存在的**数据。因为 Redis 中没有，请求会打到数据库；数据库查不到，也不会回写 Redis。如果这时有大量并发请求（比如恶意攻击者用脚本轮询不存在的 ID），数据库就会瞬间压力过大挂掉

解决方案通常有两种：

1. 缓存空对象：

- 如果数据库查不到，我也往 Redis 存一个 `null` 或特定空值，并设置一个较短的过期时间（比如 5 分钟）。
- **优点**：实现简单。
- **缺点**：消耗内存（存了一堆垃圾 Key），且存在数据不一致窗口。

2. 布隆过滤器【推荐】：

- 在请求到达 Redis 之前，先通过布隆过滤器判断一下‘这个 Key 是否可能存在’。如果布隆过滤器说不存在，直接返回，连 Redis 都不用查。
- **优点**：内存占用极小，性能极高。”

布隆过滤器原理：

利用一个很长的二进制位数组（BitMap）和多个 Hash 函数。

- 存入数据时：Hash 算出多个位置，把这些位置的 bit 置为 1。
- 查询数据时：Hash 算出位置，只要有一个位置是 0，说明绝对不存在。全是 1，说明可能存在（存在误判）

例子理解：



图片加载失败

追问1：你提到的布隆过滤器有误判率，怎么处理？怎么落地实现？

是的，有布隆过滤器的话，存在如果不一定真存在，但说不存在则一定不存在。少量的误判（比如 1%）通常是可以接受的，因为这 1% 打到数据库压力已经很小了。

至于落地实现：我们项目中一般不自己手写位运算，而是直接用 **Redisson** 客户端提供的 **RBloomFilter** 接口。它底层基于 Redis 的 BitMap 实现，可以在初始化时配置预估数据量和接受的误判率（比如 0.01），Redisson 会自动计算最优的 Hash 函数个数和数组长度。

缓存击穿

Q：什么是缓存击穿？和雪崩有什么区别？怎么解决？

A: **缓存击穿**是针对某一个特定 Key 的。这个 Key 是**热点数据**（比如微博热搜、秒杀商品），扛着极高的并发。当这个 Key **过期的一瞬间**，大量的并发请求同时发现 Redis 没数据，于是同时冲向数据库，瞬间把 DB 打垮。

解决方案主要有两套：

1. 互斥锁：

- 标准解法。当发现 Redis 没数据时，不是大家一起去查 DB，而是去抢一个锁（比如 `setnx`）。
- 抢到锁的线程去查 DB 并回写 Redis，没抢到的线程等待一下再重试。
- 特点：强一致性，但性能稍差（有等待）。

2. 逻辑过期：

- 把过期时间设置在 Value 的字段里，Redis Key 实际上**永不过期**。
- 查询时检查字段里的时间，如果过期了，异步开启一个线程去更新数据，当前线程直接返回旧数据。
- 特点：高可用，性能最好，但会得到短暂的旧数据（最终一致性）。

适用场景对比：

- **互斥锁**：适合对数据准确性要求极高的场景（如余额、库存）。
- **逻辑过期**：适合侧重吞吐量的场景（如文章点赞数、商品详情页）。

追问1：如果不使用逻辑过期，用 `setnx` 加锁时要注意什么

`setnx` 加锁最重要的是**避免死锁**。如果我抢到了锁，去查数据库的时候服务器断电了或者报错了，没能释放锁，那其他线程就永远等下去了。所以必须给锁设置一个**过期时间（TTL）**，最好把‘加锁’和‘设置过期时间’做成**原子操作**（Redis 命令 `SET key value EX 10 NX`），或者使用 Redisson 的看门狗机制自动续期。

缓存雪崩

Q： 什么是缓存雪崩？怎么预防？

A:

缓存雪崩是指在同一时段，大量的缓存 Key 同时失效，或者 Redis 服务宕机，导致所有请求瞬间全部打到数据库。

预防方案分为事前、事中、事后：

1. 针对大量过期：

- **随机 TTL**：在设置过期时间时，不要设成一样的（比如都是 1 小时），而是给它加一个随机值（比如 1-5 分钟的随机数），把过期时间打散。

2. 针对 Redis 宕机：

- **高可用架构**：搭建 Redis Sentinel（哨兵）或 Redis Cluster（集群），保证一个节点挂了，备用节点能顶上。
- **服务降级/熔断**：如果 Redis 真的全崩了，应用层通过 Hystrix 或 Sentinel 开启熔断，直接返回默认值或友好提示，暂缓访问数据库，保住底层的 DB 不死。

追问1： 缓存预热了解吗？什么时候做

了解。**缓存预热**就是在系统上线或活动开始前，提前把热点数据加载到 Redis 中，避免用户请求来触发查询。通常在**秒杀活动前**，我们会写一个定时任务或后台管理功能，把秒杀商品的库存、详情等信息预热进 Redis，并且也是要注意设置**随机过期时间**，防止活动中途集中失效引发雪崩。

缓存与数据库双写一致性

缓存与数据库双写一致性

Q：在使用 Redis 做缓存时，如果数据库更新了，是更新缓存还是删除缓存？怎么保证缓存和数据库的数据一致性？

A:

一般采用‘先更新数据库，然后删除缓存’的策略。”

这里我解释一下为什么这么做，主要基于两点考量：

第一就是为什么是删除而不是更新？ 这主要涉及性能和并发两个维度。

- **从性能上讲：**这是一种‘懒加载’的思想。试想一下，如果我的业务是‘写多读少’的，我改了 10 次数据库，如果每次都去更新缓存，但期间一次查询都没有，那这 10 次缓存计算就是纯粹的算力浪费。直接删除，等到下次真正有查询请求时，再去数据库读取最新数据回写，效率更高。
- **从并发安全上讲：**‘双写’很容易出问题。比如线程 A 和线程 B 先后修改数据库，但因为网络原因，B 先更新了缓存，A 后更新了缓存。这就导致数据库里是 B 的新值，但缓存里留下了 A 的旧值，造成数据不一致。直接删除缓存，就能规避这种复杂的并发时序问题。

第二点是为什么是先更 DB 后删缓存？ 如果我们反过来，‘先删缓存后更 DB’，在并发读写场景下问题更大：如果一个读请求进来，发现缓存没了，它去读数据库（此时数据库还是旧值），然后把旧值写回缓存。紧接着写请求才把数据库更新为新值。这样缓存里就会长时间停留脏数据。所以，**先更 DB 后删缓存**，是目前出问题概率最小的标准做法。

当然，先更 DB 后删缓存这个方案不是绝对完美的，它有一个风险点：**如果数据库更新成功了，但在删除缓存的那一步失败了（比如 Redis 挂了或网络抖动），怎么办？** 这会导致缓存里一直是旧数据。

为了解决这个问题，保证数据的**最终一致性**，我们在生产环境中通常会有**补偿机制**。目前比较成熟的方案有两种：

1. **简单的方案是延时双删：**就是在更新 DB 前删一次，更新后休眠一小会儿再删一次，但这需要精准把控休眠时间，代码侵入性较强。
2. **更可靠的工业级方案是利用消息队列（MQ）做异步重试：**利用中间件（比如 Canal）去监听数据库的 Binlog。一旦检测到数据变更，就发一条消息到 MQ。消费者收到消息后，去尝试删除 Redis。如果删除失败，就利用 MQ 的**自动重试机制**（Ack 机制），直到删除成功为止。

这样既解耦了业务逻辑，又彻底兜底了数据的一致性问题。

追问1：对于银行余额这样的强一致性业务场景怎么办？

如果是对一致性要求极高的场景（如余额、库存），**根本就不应该用缓存**，直接读写数据库。或者使用 Read-Write Lock 强行串行化，但那样就牺牲了性能，违背了引入 Redis 的初衷

看门狗机制

看门狗机制

看门狗算是面试里面的进阶考点，通常出现在分布式锁的问题中

redisson看门狗机制

Q：在使用 Redis 分布式锁时，如果业务逻辑执行时间超长，超过了锁的过期时间，导致锁自动释放了怎么办？怎么解决？

A：

如果锁过期了业务还没跑完，其他线程就会抢到锁，导致并发事故。

解决这个问题最好的办法是使用 Redisson 客户端提供的看门狗（Watchdog）机制。

它的核心原理是：当我们调用 `redisson.getLock("key").lock()` 时，如果我们不指定锁的过期时间（LeaseTime），Redisson 会默认给锁设置 30秒 的过期时间。同时，它会在后台启动一个定时任务（守护线程），每隔 10秒（也就是过期时间的 1/3）就会检查一下：‘嘿，持有锁的那个线程还在跑吗？’

- 如果还在跑，它就会把锁的过期时间重新重置为 30秒。
- 这就叫自动续期。

只要业务逻辑还在执行，看门狗就会一直续期，直到业务跑完释放锁，或者客户端宕机。

追问1：如果拿到锁的客户端宕机了（挂了），看门狗也没了，锁岂不是永远释放不掉了（死锁）

不会死锁。这也是看门狗机制设计的巧妙之处。看门狗是运行在客户端（业务服务）的后台线程中的，而不是在 Redis 服务端。如果客户端宕机了，那个负责续期的看门狗线程也就挂了，自然就不会再有人去续期了。Redis 里的这把锁，在等待完当前的 30秒（或者剩余时间）后，就会因为自然过期而被 Redis 自动删除。所以，其他客户端最多等待 30秒，就能重新抢到锁。

追问1: 如果我显式指定了过期时间, 比如 `lock.lock(10, TimeUnit.SECONDS)`, 看门狗还会生效吗?

不会生效。如果在 `lock()` 方法中手动传了过期时间 (LeaseTime), Redisson 就会认为你心里有数, 想要严格控制锁的时间, 所以它不会启动看门狗。到时候时间一到, 锁就自动释放, 不管你业务有没有跑完。所以, 如果想用看门狗, 一定不要传过期时间, 直接用无参的 `lock()`

事务与一致性

事务与一致性

Redis 事务相关题不算最难，但很容易暴露理解是否清楚。常见问题一般是：Redis 支持事务吗、能不能回滚、和数据库事务有什么区别。

Redis 支持事务吗

Q：Redis 支持事务吗？

支持，但 Redis 的事务语义和 MySQL 这种关系型数据库事务差别很大。

Redis 事务通常通过：

- MULTI
- EXEC
- DISCARD

来实现。

它能保证的是：

把一组命令打包，按顺序串行执行，中间不会被其他客户端命令插队。

但它不等于关系型数据库那种完整 ACID 事务。

Redis 事务能回滚吗

Q：Redis 事务支持回滚吗？

严格来说，Redis 不支持像 MySQL 那样的事务回滚。

如果在事务执行前，某条命令就有语法错误，Redis 会在 `EXEC` 时直接拒绝执行整个事务。

但如果事务执行过程中，某条命令因为业务原因失败，比如对错误类型的数据做操作，Redis 不会把前面已经成功执行的命令自动回滚。

所以 Redis 事务更接近：

- 保证顺序执行
- 不保证完整回滚

为什么 Redis 不做完整回滚

Q：为什么 Redis 不像数据库那样支持完整回滚？

核心原因还是设计取舍。

Redis 追求的是：

- 简单
- 高性能
- 命令执行路径短

如果像数据库一样做复杂的 undo/redo 和事务回滚机制，会明显增加系统复杂度和执行成本，这和 Redis 的定位并不一致。

所以 Redis 事务的定位不是“强事务”，而是“批量命令的原子提交入口”。

线程模型

线程模型

Redis 线程模型这块是非常高频的考点，面试官通常会连续问你：Redis 是单线程吗、为什么这么快、6.0 为什么引入多线程、Redis 是 AP 还是 CP。

Redis 是单线程吗

Q：Redis 是单线程吗？

更准确的说，Redis 的命令执行模型主要是单线程，但它不是整个 Redis 进程所有工作都只有一个线程。

比如：

- 网络读写在某些版本和场景下可以用多线程辅助
- 持久化会 fork 子进程
- 集群、后台任务也不完全等同于“只有一个线程”

所以标准答案最好不要说死成“Redis 完全单线程”，而是说：
核心命令执行是单线程，这是它设计上的关键点。

Redis 为什么这么快

Q：Redis 为什么这么快？

Redis 快的原因一般可以从四个角度来答：

第一，纯内存操作。

绝大多数请求直接在内存里完成，避免了磁盘 IO。

第二，数据结构高效。

比如 SDS、哈希表、跳表、压缩列表/quicklist 等，底层结构都很贴近高性能场景。

第三，核心命令执行模型简单。

单线程避免了大量锁竞争和线程切换成本。

第四，IO 多路复用。

Redis 使用 epoll 这类机制高效处理大量连接事件。

所以 Redis 快，不是只因为“单线程”，而是多种因素共同决定的。

为什么单线程还能高性能

Q：为什么 Redis 用单线程执行命令，性能还这么高？

因为在 Redis 的主要工作负载里，瓶颈往往不是 CPU 计算，而是：

- 网络 IO
- 内存访问
- 锁竞争

如果为了并行而引入复杂锁机制，反而可能得不偿失。

Redis 的设计思路是：把核心命令执行路径做到极短、极简单、极少锁。

所以在单核命令执行模型下，结合内存操作和高效数据结构，仍然能有非常高的吞吐。

Redis 6.0 为什么引入多线程

Q：Redis 6.0 为什么引入多线程？

Redis 6.0 引入多线程，并不是把核心命令执行彻底改成多线程，而是主要用于 **网络读写阶段**。

原因是：在更高并发场景下，网络收包、发包也会成为瓶颈。

所以 Redis 选择把网络 IO 的一部分工作并行化，而核心命令执行仍然尽量保持简单的单线程模型。

这也说明 Redis 的设计是非常克制的：

只把最值得并行化的地方拆出来，而不是盲目把所有逻辑改成多线程。

Redis 是 AP 还是 CP

Q：Redis 是 AP 还是 CP？

如果从经典主从复制和哨兵模式来看，Redis 更偏向 AP。

原因是：

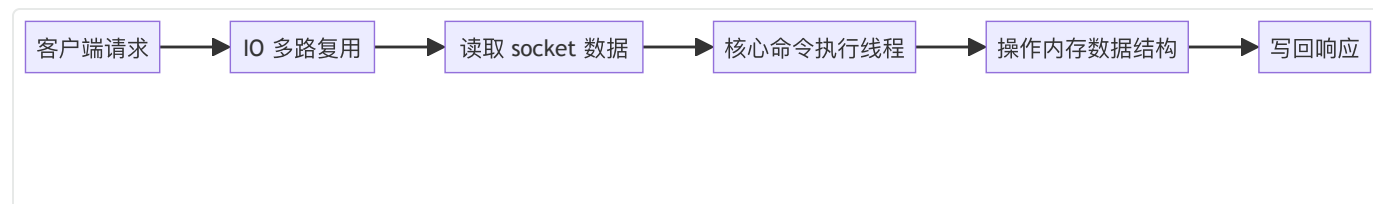
- 它更强调服务可用
- 主从复制是异步的
- 网络分区或主从切换期间，可能出现短暂不一致

但如果引入特定配置或某些强约束场景，也能在局部做得更偏 CP。

所以更准确的表述是：

Redis 默认更偏 AP，而不是天然强一致。

图解补充：Redis 请求处理链路



对比补充：Redis 为什么快

因素	本质	面试里的说法
纯内存	避开磁盘随机 IO	绝大多数请求都在内存完成

数据结构高效	操作路径短	SDS、跳表、哈希表都很贴近高性能场景
单线程命令执行	少锁、少上下文切换	核心执行路径很短
IO 多路复用	高效处理大量连接	一个线程能管很多连接事件

高频追问补充

追问补充1：Redis 6.0 的多线程为什么只放在网络 IO 阶段？

因为 Redis 想把多线程收益最大的部分拆出来，又不想把核心命令执行路径搞复杂。网络收发可以并行，但命令执行如果全面并行，就会引入锁竞争和共享数据结构同步成本，得不偿失。

追问补充2：为什么说 Redis 快，不能只回答“因为单线程”？

因为单线程只是其中一个因素。真正完整的答法应该包括：纯内存、高效数据结构、少锁、IO 多路复用。只答单线程会显得理解过浅，甚至会被追问“那单线程不是应该更慢吗”。

追问补充3：面试里 Redis 的 AP/CP 怎么答更稳？

最稳的答法是：默认主从复制和哨兵体系下，Redis 更偏 AP，因为优先保证可用性，复制又是异步的；但在某些特定约束和业务设计下，可以局部做得更偏强一致，不能机械地把 Redis 说成绝对 AP。

速记版

线程模型建议按这条线记：

- 核心命令执行主要是单线程
- Redis 快不是单点原因，而是“内存 + 数据结构 + 少锁 + 多路复用”
- 6.0 多线程主要补网络 IO 短板
- 一致性层面默认更偏 AP

一句话回答模板：

Redis 的设计重点不是把所有地方都并行化，而是把命令执行路径做得足够短，再在网络 IO 这些最值得并行化的地方引入有限多线程。

BigKey和HotKey问题

BigKey 和 HotKey 问题

Q: 什么是 Redis的BigKey? 它有什么危害? 在生产环境怎么发现和处理

A:

BigKey 并不是指 Key 本身的字符串变长了，而是指 Key 对应的 Value 很大。

在工程标准里，通常会有两个判断维度：

1. **String 类型**：如果 Value 超过 10KB（通常标准），就可以算 BigKey。
2. **集合类型 (Hash/Set/List)**：如果里面的元素个数超过 5000 个（或 1万个），我们就视为 BigKey。

虽然 Redis 支持更大的存储（比如 String 最大 512MB），但不建议这么做。

BigKey 的危害主要集中在两点，核心原因是 Redis 的单线程模型。

1. **阻塞主线程 (Block)**：Redis 处理命令是排队的。如果我操作一个 BigKey（比如删除它，或者做序列化/反序列化），由于数据量大，耗时会很久。这就导致后面的正常请求全部被堵住，表现为 Redis 响应延迟突增，甚至导致客户端超时。
2. **网络阻塞 (Network I/O)**：读取一个 BigKey（比如几 MB 的 String），会瞬间占用大量的网络带宽。这不仅会拖慢 Redis，还可能把物理机的网卡流量打满，影响同机器上的其他服务。”

发现 BigKey，有两种手段：

1. **在线扫描**：使用 `redis-cli --bigkeys` 命令。它会扫描整个 Key 空间，统计出每种数据类型里最大的 Key。
 - 注意：这个命令在生产环境执行要小心，虽然它用了 SCAN 游标，但还是会有一点点性能开销。
2. **离线分析 (推荐)**：利用工具（比如 RdbTools 或者云厂商提供的分析报告）直接分析 Redis 的 RDB 备份文件。
 - 优点：完全不影响线上服务，而且统计非常详尽。”

处理 BigKey，我有两种方案。

1. **拆分**：如果是大集合（比如一个 Hash 存了 100 万个用户），我们应该把它拆分成多个小 Hash。
 - 做法：比如 `UserHash_0`，`UserHash_1` ... 存取时通过 `id % 100` 来路由到对应的小 Hash 中。
2. **删除策略**：如果发现 BigKey 要删除，不能直接用 `DEL` 命令。因为直接 DEL 一个大 Key 会在主线程里执行内存回收，导致长时间阻塞。
 - **Redis 4.0 以后**：我们要用 `UNLINK` 命令。它只是把 Key 从元数据中解绑（逻辑删除），真正的内存释放动作会交给后台线程（Lazy Free）异步执行，不会阻塞主线程。
 - **Redis 4.0 以前**：我们需要写脚本，用 `HSCAN` 或 `SSCAN` 分批次读取元素，然后分批次删除，一点一点把这个大山移走。



setnx

setnx

这一块在面试中通常是作为分布式锁的入门题出现的

setnx的原子性问题

很经典的问题， 分开执行 SETNX 和 EXPIRE 会导致死锁

Q：了解 Redis 的 SETNX 吗？如果让你用它实现分布式锁，你会怎么写？

A：

了解。 **SETNX** 全称是 **Set If Not Exists**。它的规则是：如果 Key 不存在，则设置成功返回 1；如果 Key 已经存在，则设置失败返回 0。利用这个原子特性，我们可以实现简单的锁竞争。

但是，在实现分布式锁时，我绝不会直接使用 **SETNX** 命令。因为 **SETNX** 本身不带过期时间。如果我先执行 **SETNX** 抢到了锁，然后还没来得及执行 **EXPIRE** 设置过期时间，服务突然宕机了，那么这个锁就会变成死锁（永久有效），其他线程再也抢不到了。

正确的做法是：使用 Redis 2.6.12 之后提供的扩展命令：**SET key value NX EX 10** 这条命令把‘检查是否存在’、‘设置值’、‘设置过期时间’这三步合并成了一个原子操作。要么全成功，要么全失败，彻底规避了原子性问题。

追问1：这把分布式锁的 Value 你一般存什么？

有些人会存1，这是错误的

必须存一个唯一标识（比如 UUID + 线程 ID）。

这是为了防止误删别人的锁。举个例子：

1. 线程 A 抢到锁，但业务执行太慢，超过了 10秒，锁自动过期释放了。
2. 线程 B 顺势抢到了锁。
3. 此时线程 A 醒了，业务执行完，它去执行 **DEL** 删除锁。
4. 如果 Value 只是 "1"，线程 A 就会把线程 B 正在持有的锁给删掉了。

所以，Value 必须存线程唯一的 ID。在删除前，先 `GET` 出来判断一下是不是我加的锁，如果是才删，不是就别动。

怎么保证setnx的原子性

这个问题其实是考察Lua脚本的应用

接上个问题

Q：你刚才说删除前要先判断 Value 是不是自己的。那‘判断’和‘删除’是两步操作，这中间如果不原子怎么办？

如果我 Java 代码写成这样

```
1 if (redis.get("lock").equals(myUUID)) {  
2     redis.del("lock"); // 这一步之前，锁可能刚好过期被别人抢了  
3 }
```

在 `get` 判断通过后，到 `del` 执行前，如果刚好发生 GC 停顿或者网络卡顿，锁过期了，被线程 C 抢走了，那我还是会误删线程 C 的锁。

解决方案是使用 Lua 脚本。Redis 保证 Lua 脚本执行的原子性（整个脚本就像一个命令）。我会把‘比较 UUID’和‘删除 Key’写在一段 Lua 脚本里发给 Redis 执行：

Lua

```
1 if redis.call('get', KEYS[1]) == ARGV[1] then  
2     return redis.call('del', KEYS[1])  
3 else  
4     return 0  
5 end
```

这样就能保证 100% 安全了。

setnx的缺陷

Q：做到了 Lua 脚本删除，基于 SETNX 的锁就完美了吗？

A:

还不够完美，这种手写的分布式锁主要有**两个缺陷**：

1. **不可重入**：`SETNX` 是互斥的，同一个线程如果再次去抢（比如递归调用），会失败。需要我们在代码里自己维护计数器来实现可重入。
2. **续期问题**：也就是我们之前讨论过的，如果业务没跑完锁过期了怎么办？手写代码去搞守护线程续期太复杂了，容易出 Bug。

所以，在生产环境中直接使用 **Redisson** 框架。它把加锁、解锁、看门狗（续期）、可重入机制都封装好了，开箱即用。

mysql

事务与MVCC

事务与MVCC

事务和并发控制是数据库的核心能力。理解事务不仅要理解 ACID，更要理解隔离级别背后的实现机制，以及 MySQL 如何通过 MVCC 在并发性能和一致性之间做权衡。

事务隔离的实现机制

面试官：MySQL 的四种隔离级别分别解决了什么问题？

四种隔离级别，本质是在并发性能和数据一致性之间做取舍。

读未提交（Read Uncommitted）：

事务可以读到其他事务未提交的数据。实现简单，直接读最新版本，不加锁。但会产生**脏读**——读到其他事务可能回滚的数据，造成逻辑错误。

实际生产环境几乎不用，因为数据可靠性太低。

读已提交（Read Committed）：

事务只能读到其他事务已提交的数据。解决了脏读，但会产生**不可重复读**——在同一个事务内，两次读取同一行数据，结果可能不同（中间被其他事务修改并提交）。

Oracle、SQL Server 的默认隔离级别。

可重复读（Repeatable Read）：

MySQL 的默认隔离级别。在同一个事务内，多次读取同一行数据，结果始终相同。解决了不可重复读，但理论上会产生**幻读**——两次范围查询，结果集的行数不同（中间被其他事务插入或删除）。

InnoDB 通过 **MVCC + 间隙锁** 解决了幻读问题。

串行化（Serializable）：

完全串行执行，没有并发。解决了所有并发问题，但性能最差，吞吐量极低。

追问：InnoDB 如何解决幻读？

幻读的定义是：同一个事务内，两次范围查询，结果集的行数不同。

快照读（Snapshot Read）解决部分幻读：

普通的 `SELECT`（不加锁）是快照读，读取的是事务开始时的数据快照（通过 MVCC 实现）。其他事务插入的新数据，快照读看不到，避免了幻读。

当前读（Current Read）的幻读问题：

`SELECT ... FOR UPDATE`、`SELECT ... LOCK IN SHARE MODE`、`UPDATE`、`DELETE` 是当前读，读取的是最新数据，且要加锁。

如果只用行锁，只能锁住已存在的记录，其他事务可以插入新记录，导致当前读两次查询结果不同，出现幻读。

间隙锁（Gap Lock）解决当前读的幻读：

InnoDB 在事务中执行当前读时，不仅锁住符合条件的记录，还锁住这些记录之间的间隙（Gap），防止其他事务插入新记录。

例如：`SELECT * FROM user WHERE age > 20 FOR UPDATE`，InnoDB 会锁住 `age > 20` 的所有记录，以及这些记录之间的所有间隙。其他事务无法在 `age > 20` 的范围内插入新记录，直到当前事务提交。

注意：间隙锁只在 RR（可重复读）隔离级别生效，且只在当前读时加。如果是 RC（读已提交），InnoDB 不加间隙锁，只加行锁，会出现幻读，但并发性能更好。

MVCC 的多版本控制

面试官：MVCC 是怎么实现的？

MVCC (Multi-Version Concurrency Control) 的核心思想是读写不加锁，通过版本链实现并发控制。

隐藏字段：

InnoDB 的每行记录有两个隐藏字段：

- **DB_TRX_ID**：最后修改这行的事务 ID。
- **DB_ROLL_PTR**：回滚指针，指向这行数据的上一个版本（存储在 undo log 中）。

版本链：

每次更新数据，都会生成一个新的版本，旧版本通过回滚指针链接，形成版本链。

Read View（读视图）：

事务执行快照读时，会生成一个 Read View，记录当前活跃的事务 ID 列表。通过对比行记录的 `trx_id` 和 Read View，判断该行对当前事务是否可见：

- 如果 `trx_id` < 最小活跃事务 ID：说明这行数据在当前事务开始前已提交，可见。
- 如果 `trx_id` > 最大事务 ID：说明这行数据是当前事务开始后才创建的，不可见。
- 如果 `trx_id` 在活跃事务列表中：说明这行数据由未提交事务创建，不可见。
- 如果 `trx_id` 不在活跃事务列表中：说明这行数据由已提交事务创建，可见。

RC 和 RR 的区别：

- **Read Committed**：每次快照读都生成新的 Read View，能看到其他事务新提交的数据，所以会有不可重复读。
- **Repeatable Read**：事务第一次快照读时生成 Read View，后续快照读都复用这个 View，保证看到的数据始终一致，实现可重复读。

事务持久化的实现

面试官：redo log 和 binlog 有什么区别？

redo log 和 binlog 都是持久化机制，但设计目标完全不同。

redo log:

- 物理日志，记录的是"某个数据页做了什么修改"。
- 循环写，固定大小（如 4GB），写满后覆盖旧日志。
- 用于崩溃恢复。数据库崩溃后，通过 redo log 恢复未写入磁盘的数据。
- InnoDB 引擎特有。

binlog:

- 逻辑日志，记录的是 SQL 语句或行数据变更（ROW 格式）。
- 追加写，文件大小不固定，可以配置保留天数。
- 用于主从复制和数据恢复。主库的 binlog 同步到从库，从库重放 binlog 实现数据同步。
- MySQL Server 层，所有存储引擎共享。

为什么需要两个日志？

redo log 是 InnoDB 自己用的，其他存储引擎（如 MyISAM）没有 redo log，它们依赖 binlog 做恢复。

binlog 是 MySQL 层面统一的日志，用于跨引擎的复制和恢复。

两阶段提交：

为了保证 redo log 和 binlog 的一致性，InnoDB 使用两阶段提交：

1. **Prepare 阶段**：事务提交前，先写 redo log，状态为 prepare。
2. **Commit 阶段**：写 binlog，然后 redo log 状态改为 commit。

如果写 binlog 前崩溃，redo log 是 prepare 状态，没有 binlog，事务回滚。

如果写 binlog 后崩溃，redo log 可以通过 binlog 恢复，保证一致性。

事务的性能优化

面试官：大事务有什么问题？如何优化？

大事务是生产环境的常见问题，会带来多种风险。

第一，锁持有时间过长：

事务中如果修改了大量数据，会持有大量行锁甚至间隙锁，阻塞其他事务，导致并发性能下降。

第二，undo log 膨胀：

大事务修改的数据多，生成的 undo log 也多，占用大量磁盘空间，且长时间不释放。

第三，主从延迟：

主库执行大事务，binlog 量大，同步到从库需要时间，导致主从延迟。如果这时候主库故障，从库数据不完整。

第四，回滚代价高：

大事务如果失败回滚，需要撤销大量修改，耗时极长，期间数据库性能急剧下降。

优化策略：

分批处理：把大事务拆分成多个小事务，每批提交一次。

避免在事务中做耗时操作：

事务中不要调用外部接口、发送消息、复杂计算，这些操作会拉长事务时间，增加锁持有时间。

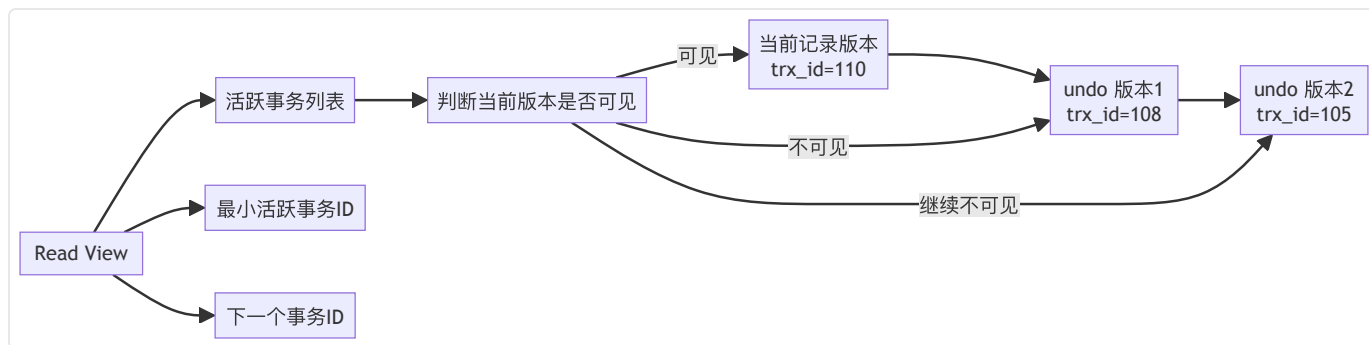
优化 SQL：

减少事务中的 SQL 数量，尽量用批量操作代替单条操作。

图解补充：隔离级别和并发问题

隔离级别	脏读	不可重复读	幻读	实现思路
RU	可能	可能	可能	直接读最新版本
RC	不会	可能	可能	每次快照读生成新 Read View
RR	不会	不会	理论上可能，InnoDB 基本规避	复用 Read View + 当前读加间隙锁
Serializable	不会	不会	不会	强串行化，性能最低

图解补充：版本链与 Read View



高频追问补充

追问补充1：快照读和当前读怎么区分？

普通 `SELECT` 是快照读，读的是历史可见版本，核心目标是提高并发；`SELECT ... FOR UPDATE`、`UPDATE`、`DELETE` 属于当前读，读的是最新版本，同时要加锁，核心目标是保证修改安全。

追问补充2：RC 和 RR 最核心的差别是什么？

最核心差别不只是“会不会不可重复读”，而是 **Read View 的生成时机不同**。RC 每次快照读都重新生成，所以更容易看到别的事务最新提交的数据；RR 通常在事务第一次快照读时生成并复用，所以同一事务里读到的数据更稳定。

追问补充3：为什么大事务会拖垮系统？

可以按这条线回答：锁持有时间变长、undo log 膨胀、主从复制变慢、失败回滚成本很高。这样就不是只背一个“影响性能”，而是能展开到存储、并发和高可用三个层面。

速记版

事务和 MVCC 建议按这条线记：

- ACID 是入口
- 隔离级别是并发问题的表象
- MVCC 是快照读的核心
- 间隙锁是当前读防幻读的关键
- redo/binlog 是提交和恢复的关键

一句话面试版：

InnoDB 通过 MVCC 解决读多写少场景下的并发读问题，通过 锁机制解决当前读和写冲突问题，两者配合实现了性能和一致性的平衡。

更新: 2026-01-31 01:20:52

原文: <https://www.yuque.com/u63312805/wnyutc/beq0z2dkbt1vsa0c>

日志与执行流程

日志与执行流程

这块内容通常会和事务、索引、SQL 优化一起问，常见题目包括 redo log、undo log、binlog、SQL 执行过程、执行计划怎么分析。

redo log、undo log、binlog 的区别

Q: redo log、undo log、binlog 有什么区别？

这三个日志的职责完全不同：

redo log：

是 InnoDB 的物理日志，记录数据页的修改，用于崩溃恢复，保证事务的持久性。

undo log：

记录数据修改前的旧版本，一方面用于事务回滚，另一方面为 MVCC 提供版本链支持。

binlog：

是 MySQL Server 层的逻辑日志，记录的是逻辑变更，主要用于主从复制和数据恢复。

如果让我一句话总结：

- redo log：向前恢复
- undo log：向后回滚
- binlog：复制和审计

两阶段提交

Q: 为什么需要两阶段提交？

因为 MySQL 事务提交时，既涉及 InnoDB 的 redo log，又涉及 Server 层的 binlog。如果这两个日志不一致，就会带来严重问题。

典型场景是：

- redo log 写成功了，但 binlog 没写成功
- 或者 binlog 写成功了，但 redo log 状态不一致

所以 MySQL 用两阶段提交来保证一致性：

第一阶段：redo log 先写成 prepare 状态

第二阶段：写 binlog，成功后再把 redo log 标记成 commit

这样崩溃恢复时，就可以根据日志状态判断事务是否真正提交成功。

SQL 查询语句的执行过程

Q：一条 SQL 查询语句的执行过程是什么？

如果是一条普通查询，大致流程可以概括为：

1. 客户端和 MySQL 建立连接
2. 经过连接器做权限认证
3. 由解析器做词法、语法分析
4. 由优化器选择执行计划
5. 执行器调用存储引擎接口取数据
6. 最后把结果返回给客户端

如果是更新语句，还会额外涉及：

- undo log
- redo log
- binlog

所以答这类题时，一般要先区分“查询语句”和“更新语句”。

执行计划怎么看

Q：你怎么分析 MySQL 执行计划？

最常见是用 `EXPLAIN` 看执行计划。

我会重点关注这些字段：

- `type`：访问类型，是否走了高效路径
- `key`：实际用到哪个索引
- `rows`：预估扫描行数
- `Extra`：是否出现 `Using filesort`、`Using temporary` 之类的额外开销

如果 `type` 很差，或者 `rows` 特别大，通常就说明 SQL 或索引设计存在优化空间。

SQL 优化的常见思路

Q：SQL 优化你一般怎么做？

我一般按这个顺序看：

- 先看是不是走了索引
- 再看索引是否合理，比如联合索引顺序是否匹配查询条件
- 再看是否存在回表、排序、临时表等额外开销
- 最后再看是不是需要改 SQL 写法或者分页方式

常见优化方向包括：

- 避免全表扫描
- 避免索引失效
- 减少回表次数
- 减少排序和临时表
- 控制返回字段，避免 `select *`

索引原理

索引原理

MySQL 索引的设计本质上是**如何通过空间换时间**，在海量数据中快速定位目标记录。理解索引的关键不在于背诵 B+ 树的结构，而在于理解为什么 MySQL 选择 B+ 树，以及在不同场景下索引如何工作、如何失效。

索引的数据结构选择

面试官：为什么 MySQL 用 B+ 树做索引，不用 Hash 或者红黑树？

索引的核心诉求有两个：**等值查询快**和**范围查询快**。不同的数据结构在这两点上有不同的 trade-off。

Hash 表：

等值查询是 $O(1)$ ，但 Hash 表不支持范围查询。如果要做 `WHERE id > 100 AND id < 200`，Hash 表无能为力，只能全表扫描。所以 Hash 只适合精准匹配的场所，比如内存缓存（Memcached、Redis 的核心结构）。Memory 存储引擎支持 Hash 索引，但 InnoDB 不支持（除了自适应 Hash 索引，那是内部优化）。

红黑树：

是平衡二叉搜索树，等值和范围查询都是 $O(\log n)$ 。但问题是，**树的高度太高**。数据量大的时候，树高可能达到 20–30 层，意味着一次查询要 20–30 次磁盘 IO。磁盘 IO 是毫秒级的，20 次 IO 就是几十毫秒，性能无法接受。

B+ 树：

多路平衡搜索树，每个节点可以存多个键值。InnoDB 默认一页 16KB，假设一个整型键 8 字节，指针 6 字节，一个节点可以存约 1000 个键。树高只有 3–4 层，查询只需 3–4 次磁盘 IO，性能提升一个数量级。

更重要的是，B+ 树的叶子节点是**有序链表**，范围查询时，找到起始位置后顺序遍历即可，效率极高。这是红黑树做不到的（红黑树只能中序遍历，无法顺序扫描）。

追问：B+ 树和 B 树有什么区别？

B 树（B-Tree）的每个节点都存储数据，非叶子节点也存数据。这导致：

第一，节点能存的键值变少。因为数据占用了空间，一页 16KB 可能只能存 100 个键，树高增加，IO 次数变多。

第二，范围查询效率低。B 树的叶子节点没有链表连接，范围查询需要中序遍历，涉及大量随机 IO。

B+ 树的改进：

非叶子节点只存键值，不存数据。数据都在叶子节点。这样非叶子节点能存更多键，树高更低。

叶子节点形成有序链表。范围查询时，找到起始叶子节点，顺序遍历链表即可，全是顺序 IO，性能极高。

查询性能稳定。B 树可能在非叶子节点就找到数据，返回很快；也可能要查到叶子节点，很慢。B+ 树无论查什么数据，都要到叶子节点，性能稳定可预期。

聚簇索引与二级索引

面试官：聚簇索引和非聚簇索引有什么区别？

这是 InnoDB 存储引擎的核心设计。

聚簇索引 (Clustered Index) :

InnoDB 的表数据本身就是按 B+ 树组织的，**聚簇索引的叶子节点存储的是完整的行数据**。一张表只能有一个聚簇索引，通常是主键索引。

如果没有主键，InnoDB 会选择第一个唯一索引作为聚簇索引；如果没有唯一索引，会隐式创建一个 6 字节的 row_id 作为聚簇索引。

二级索引 (Secondary Index) :

也叫辅助索引，叶子节点存储的是**主键值**，而不是行数据。查询时，先通过二级索引找到主键，再通过主键去聚簇索引查完整数据，这个过程叫**回表**。

为什么二级索引不存完整数据？

第一，节省空间。如果每个索引都存完整数据，数据量会爆炸式增长。

第二，保持数据一致性。如果一张表有多个索引，数据更新时要同步更新所有索引。如果二级索引存的是主键，只需要更新聚簇索引的数据；如果存完整数据，所有索引都要更新，代价极高。

回表的代价：

回表意味着两次索引查询。如果通过二级索引找到 100 条记录，就要回表 100 次，100 次随机 IO，性能很差。这时候**覆盖索引**就很重要。

追问：什么是覆盖索引？

覆盖索引是指查询的所有字段都在索引中，不需要回表。

比如：

`name` 和 `age` 都在联合索引中，查询直接走索引，拿到结果就返回，不需要回表查主键索引。

覆盖索引的优势：

减少 IO 次数。不需要回表，一次索引查询就搞定。

减少数据传输。索引通常比完整行数据小得多，网络传输量小。

利用索引的有序性。如果查询需要排序，而索引已经是有序的，可以直接用索引顺序，避免 filesort。

设计索引时的考虑：

高频查询的字段，可以考虑建成联合索引，让查询能覆盖。但要注意索引的维护成本和空间占用，不能为了覆盖索引把所有字段都加进索引，那是本末倒置。

联合索引的最左前缀

面试官：联合索引 (a, b, c) 什么情况下会生效？

联合索引的列顺序至关重要，InnoDB 按照最左前缀原则使用索引。

生效的情况：

失效的情况：

原理：

联合索引是先按 a 排序，a 相同再按 b 排序，b 相同再按 c 排序。如果没有 a，就不知道从哪里开始查。

范围查询的影响：

a 和 b 走索引，c 不走。因为 b 是范围查询，b 相同的记录有很多，c 在这些记录中是无序的，无法直接定位。

索引下推 (Index Condition Pushdown)：

MySQL 5.6 引入的优化。上面的例子中，虽然 c 不能直接用索引定位，但存储引擎可以先在索引中过滤 `c = 3` 的记录，减少回表次数。这叫 ICP，是 MySQL 内部的优化，不需要手动干预。

索引失效的场景

面试官：什么情况下索引会失效？

索引失效通常是因为查询条件无法利用索引的有序性。

第一，对索引列做函数运算：

对 `create_time` 做 `YEAR()` 函数运算后，MySQL 不知道运算结果和索引中的值是什么关系，只能全表扫描。

第二，隐式类型转换：

MySQL 会尝试把字符串转成数字，转换后值变了，无法匹配索引。

第三，LIKE 前导通配符：

`%` 在前，表示前面有任意字符，MySQL 不知道从哪里开始查。

第四，OR 条件：

如果 a 和 b 都有索引，MySQL 可能会用索引合并（Index Merge）；如果其中一个没索引，通常会导致全表扫描。

第五，`!=` 和 `<>`：

虽然技术上可以用索引，但通常 MySQL 会选择全表扫描，因为 `!=` 匹配的记录太多，回表成本太高。

第六，NULL 判断：

`IS NOT NULL` 匹配的记录通常很多，优化器认为全表扫描更快。

索引设计的权衡

面试官：设计索引时要考虑哪些因素？

索引不是越多越好，需要在**查询性能**和**维护成本**之间做权衡。

第一，区分度：

索引列的区分度要高。如果一列只有几个取值（如性别），索引意义不大。MySQL 优化器会发现走索引和全表扫描差不多，还不如直接全表扫描。

区分度可以用 `COUNT(DISTINCT col) / COUNT(*)` 计算，越高越好。

第二，最左前缀：

联合索引要把区分度高的列放左边，让索引在更多场景下生效。

第三，避免冗余索引：

这三个索引是冗余的，(a, b, c) 可以覆盖 (a, b) 和 (a)。冗余索引占用空间，增加维护成本。

第四，长字符串索引：

长字符串（如 URL、邮箱）建索引，可以用前缀索引：

只索引前 10 个字符，节省空间。但要注意前缀长度，太短区分度不够，太长索引太大。

第五，覆盖索引：

高频查询的字段，尽量让查询能覆盖索引，避免回表。

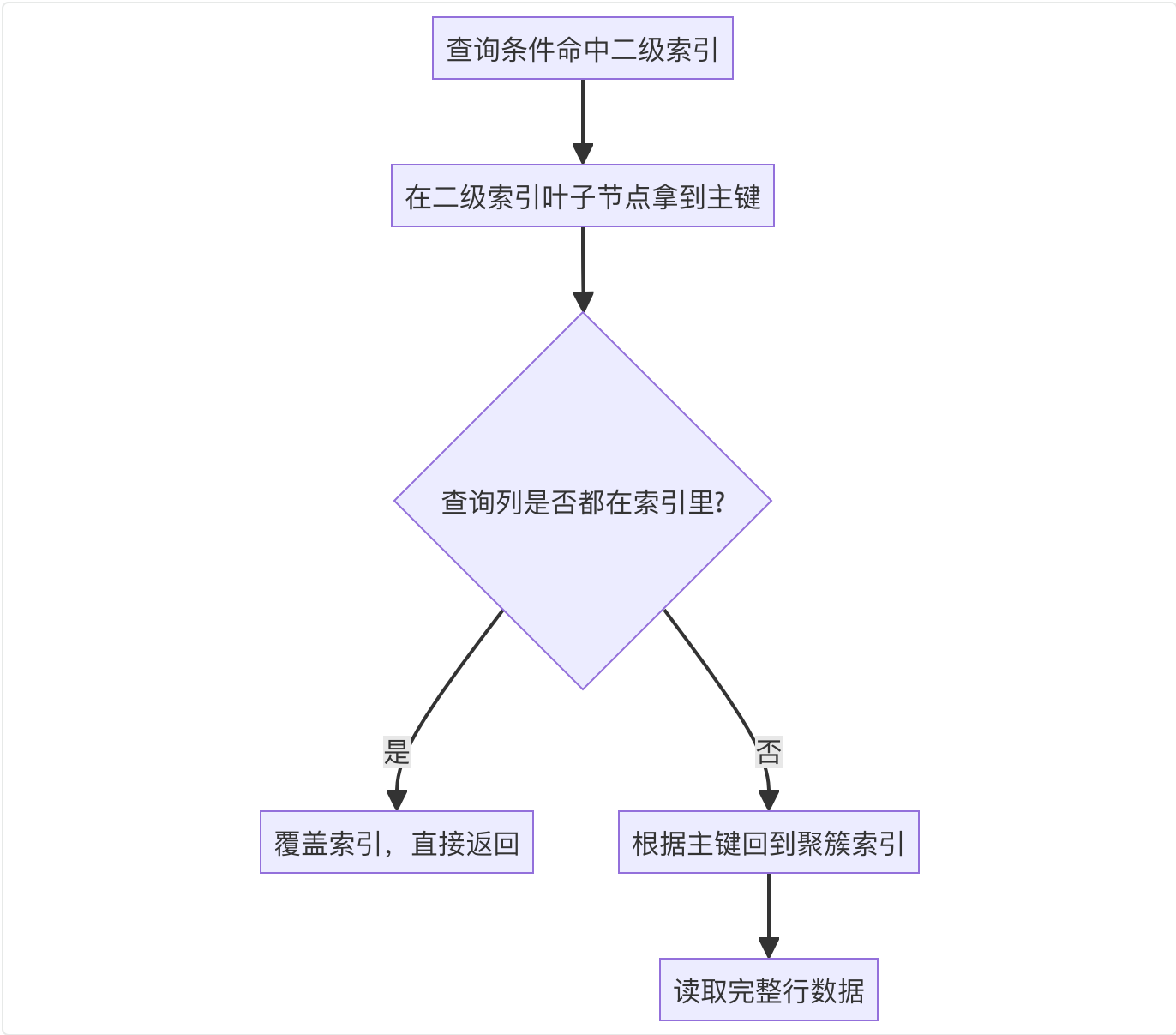
第六，写放大：

每次 INSERT/UPDATE/DELETE，都要维护所有索引。索引越多，写操作越慢。读多写少的表可以多建索引，写多读少的表要控制索引数量。

图解补充：为什么 MySQL 更偏爱 B+ 树

结构	等值查询	范围查询	磁盘 IO 层数	为什么不是它
Hash	快	差	低	不支持范围查询
红黑树	可以	可以	高	树太高，磁盘 IO 太多
B+ 树	快	快	低	叶子链表天然适合范围扫描

图解补充：聚簇索引、二级索引与回表



对比补充：联合索引 **(a, b, c)** 怎么记

查询条件	能否高效使用索引	原因
<code>a = 1</code>	能	命中最左列
<code>a = 1 and b = 2</code>	能	命中连续前缀
<code>a = 1 and b = 2 and c = 3</code>	能	命中完整联合索引
<code>b = 2</code>	不能	丢失最左列

<code>a = 1 and c = 3</code>	部分	用到 <code>a</code> ，但 <code>c</code> 无法直接连续定位
<code>a = 1 and b > 2 and c = 3</code>	部分	范围列后面的列通常不能继续做有序定位

高频追问补充

追问补充1：为什么回表会慢？

因为二级索引叶子节点只存主键，查到目标后还要再去聚簇索引找整行数据，本质上多了一次索引树访问。命中的记录越多，随机 IO 越多，成本就越高。

追问补充2：覆盖索引为什么重要？

因为它能直接在索引里拿到结果，避免回表。面试里可以主动补一句：覆盖索引不只是“更快”，还常常能减少磁盘 IO 和网络传输，尤其适合高频查询。

追问补充3：设计联合索引时第一列一定放区分度最高的吗？

不绝对。更准确的说法是：要综合查询条件的出现频率、等值匹配优先级、排序需求和区分度来排。面试里如果只答“区分度最高放最左边”，会显得过于机械。

速记版

索引建议按这条线记：

- 先讲 B+ 树为什么赢
- 再讲聚簇索引、二级索引和回表
- 再讲最左前缀和索引失效
- 最后讲覆盖索引和设计权衡

一句话面试版：

MySQL 索引的核心不是背结构，而是理解 B+ 树如何降低 IO、联合索引如何利用有序性、回表为什么昂贵。

锁机制

锁机制

锁机制是 MySQL 面试里非常容易和事务、隔离级别串起来的一块内容。答题时最好从“为什么要加锁”出发，再讲锁分类和具体场景。

MySQL 为什么需要锁

Q: MySQL 为什么需要锁?

数据库的锁，本质上是为了在并发环境下保证数据一致性。

如果多个事务同时修改同一份数据，而没有任何并发控制，就会出现：

- 丢失更新
- 脏读
- 不可重复读
- 幻读

所以锁的本质作用就是：

协调并发事务对共享数据的访问顺序。

表锁、行锁、间隙锁

Q: MySQL 常见锁有哪些?

如果从粒度上看，常见有：

表锁：锁住整张表，粒度大，并发能力弱，但实现简单。

行锁：锁住具体行，粒度小，并发能力更强。

间隙锁：锁住一个范围内的“间隙”，防止其他事务插入新记录。

InnoDB 最核心的是行锁体系，但它底层是通过 **锁索引记录** 来实现的，所以如果 SQL 没走索引，行锁就可能退化成更大范围的锁。

共享锁和排他锁

Q：共享锁和排他锁有什么区别？

共享锁（S 锁） 允许多个事务同时读，但不允许写。

排他锁（X 锁） 不允许其他事务再读写同一资源。

通常可以这样记：

- S 锁：读读可以并发，读写冲突
- X 锁：读写都冲突

InnoDB 的行锁

Q：InnoDB 的行锁是怎么实现的？

InnoDB 的行锁并不是直接锁“物理行”，而是 **锁索引记录**。

这意味着：

- 如果 SQL 命中了索引，锁的范围更精准
- 如果 SQL 没走索引，InnoDB 可能会扫描更多记录，锁范围也会变大

所以很多时候“锁冲突严重”并不只是事务问题，也可能是索引设计问题。

Next-Key Lock

Q: 什么是 Next-Key Lock?

Next-Key Lock 可以理解成：记录锁 + 间隙锁。

它的作用是：

不仅锁住当前命中的记录，还锁住相邻区间，防止其他事务在这个范围内插入新数据，从而避免幻读。

这也是为什么在 RR 隔离级别下，InnoDB 能比较好地解决当前读的幻读问题。

死锁

Q: MySQL 为什么会发生死锁? 怎么处理?

死锁的本质是多个事务互相等待对方释放资源，形成循环依赖。

典型场景是：

- 事务 A 先锁记录 1，再等记录 2
- 事务 B 先锁记录 2，再等记录 1

InnoDB 能检测死锁，一旦发现会主动回滚其中一个事务。

实际工程里避免死锁的常见做法有：

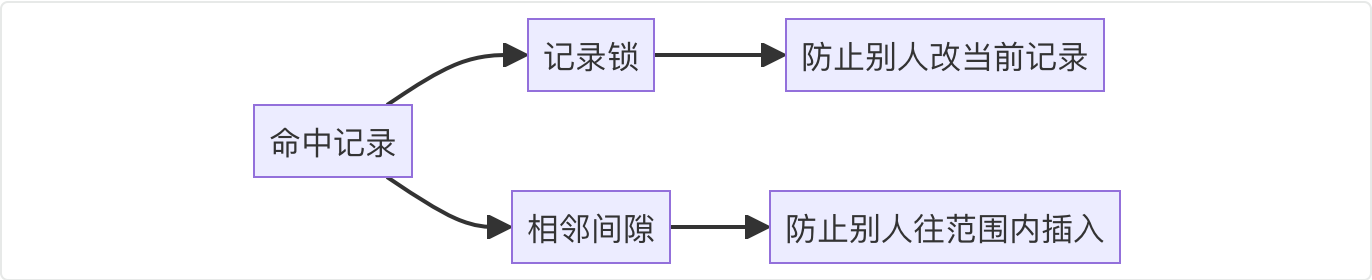
- 尽量按固定顺序访问资源
- 减少事务范围和持锁时间
- 保证 SQL 能走索引，减少锁范围

对比补充：锁类型怎么记

锁	粒度/语义	能解决什么	典型代价
表锁	整张表	实现简单	并发度低
行锁	单条或少量记录	精准控制冲突	依赖索引效果
间隙锁	记录间的空隙范围	防止插入幻读	容易扩大阻塞范围

S 锁	共享读锁	保证并发读	阻塞写
X 锁	排他写锁	保证写安全	读写都可能被拦

图解补充：Next-Key Lock 的作用



高频追问补充

追问补充1：为什么说 InnoDB 的行锁本质上是“锁索引”？

因为 InnoDB 是通过索引项来定位和加锁的，不是直接给物理行贴一个锁标记。也正因为如此，SQL 如果没走索引，锁的范围往往会变大，性能和并发都会受影响。

追问补充2：为什么锁问题经常和索引问题绑在一起？

因为索引决定了扫描范围，也就决定了加锁范围。索引设计差，可能不是查询慢一点那么简单，而是直接把锁冲突、阻塞和死锁概率一并拉高。

追问补充3：怎么从工程上降低死锁概率？

最实用的回答是三点：固定资源访问顺序、缩小事务和持锁时间、确保 SQL 走索引。这样回答比只说“注意并发”更像真实项目经验。

速记版

锁机制建议按这条线记：

- 先讲为什么要锁：解决并发一致性
- 再讲粒度：表锁、行锁、间隙锁
- 再讲语义：S 锁和 X 锁
- 最后讲 Next-Key Lock 和死锁

一句话面试版：

MySQL 锁的核心不是“有几种锁”，而是不同事务在不同隔离级别下，数据库如何用锁来约束读写顺序并控制锁范围。

主从复制与隔离级别

主从复制与隔离级别

这块内容经常被一起问，因为很多面试官会从“为什么线上会选择 RC”一路追问到主从延迟和读写分离。

InnoDB 和 MyISAM 的区别

Q: InnoDB 和 MyISAM 有什么区别？

如果只讲面试最核心的差异，我会抓这几个点：

- InnoDB 支持事务，MyISAM 不支持
- InnoDB 支持行锁，MyISAM 主要是表锁
- InnoDB 支持外键，MyISAM 不支持
- InnoDB 有 redo log、undo log，崩溃恢复能力更强

所以现在绝大多数线上核心业务表，基本都会优先选 InnoDB。

RR 和 RC 的区别

Q: RR 和 RC 的区别是什么？

RC（读已提交）：每次快照读都生成新的 Read View，所以能读到其他事务新提交的数据。它解决了脏读，但会有不可重复读。

RR（可重复读）：事务第一次快照读生成 Read View，后续复用同一个 View，所以同一事务内多次读取结果更稳定。

它解决了不可重复读，而且 InnoDB 在当前读场景下还会通过间隙锁减少幻读问题。

为什么很多互联网公司喜欢 RC

Q: MySQL 默认是 RR, 为什么很多互联网公司反而更常用 RC?

因为线上很多业务更关注 **并发性能和锁冲突控制**。

在 RR 下, InnoDB 会更积极地使用间隙锁和 Next-Key Lock, 这虽然能增强一致性, 但也更容易扩大锁范围, 带来阻塞和死锁风险。

而 RC 的锁范围通常更小, 并发能力更好。

对于很多互联网系统来说, 只要业务层能接受, 选择 RC 往往是更平衡的方案。

所以这里不是说 RR 不好, 而是:

默认值不一定等于线上最佳选择。

主从复制原理

Q: MySQL 主从复制的原理是什么?

主从复制的核心, 是主库把 binlog 变更传给从库, 从库再重放这些日志。

经典流程可以概括为:

1. 主库写入 binlog
2. 从库的 IO 线程拉取主库 binlog, 写入 relay log
3. 从库的 SQL 线程读取 relay log 并执行

所以主从复制本质上是 **基于 binlog 的异步复制机制**。

主从延迟怎么理解

Q: 为什么会出现主从延迟?

主从延迟一般有几类常见原因：

- 主库写入压力大，binlog 产生太快
- 从库执行能力不足
- 大事务导致日志量太大
- 从库上还有额外查询压力

主从延迟会直接影响：

- 读写分离的一致性体验
- 故障切换时的数据完整性

所以线上除了做主从复制本身，还要关注：

- 尽量避免大事务
- 合理做读写分离
- 对关键业务做主读或延迟感知

进程线程协程

进程、线程、协程的区别

Q：进程、线程、协程有什么区别？

进程 是资源分配的基本单位，拥有独立的地址空间。

线程 是 CPU 调度的基本单位，同一进程内线程共享进程资源。

协程 则更轻量，通常由用户态调度，不依赖操作系统直接调度。

简单理解：

- 进程更重，隔离性强
- 线程更轻，共享资源方便
- 协程更轻量，切换成本低

为什么需要协程

Q：为什么有了线程还需要协程？

因为线程虽然已经比进程轻，但它的创建和切换仍然有系统调用成本。

在高并发 IO 场景下，大量线程阻塞等待会浪费很多资源。

协程的意义在于：

用更低的上下文切换成本支撑更多并发任务。

进程和线程的核心差异

Q：面试里怎么快速区分进程和线程？

最稳妥的答法是从三个维度说：

- **资源归属**：进程有独立地址空间，线程共享进程的大部分资源
- **调度单位**：线程是 CPU 调度的基本单位，进程更偏资源容器
- **切换成本**：进程切换通常更重，因为涉及地址空间切换；线程切换相对更轻

如果只说“进程重、线程轻”太空，最好再补一句：

进程强调隔离，线程强调并发执行。

进程切换和线程切换为什么成本不同

Q：为什么进程切换比线程切换更重？

因为进程切换除了要保存和恢复寄存器、程序计数器、栈指针这些上下文之外，往往还要切换页表、地址空间，TLB 也可能失效。

线程切换通常只需要：

- 保存线程私有寄存器状态
- 切换线程栈
- 继续在同一个进程地址空间里执行

所以面试里可以概括成一句话：

线程切换主要是执行流切换，进程切换还叠加了资源环境切换。

进程间通信方式有哪些

Q：进程之间怎么通信？

常见的 IPC 方式有：

- 管道
- 消息队列
- 共享内存
- 信号量
- Socket

其中：

- **共享内存** 速度最快，但要自己处理同步问题
- **Socket** 最通用，既能本机通信，也能跨机器通信
- **管道** 适合父子进程或简单单向通信

如果面试官继续追问，通常会问：

为什么共享内存快？ 因为少了一层反复拷贝，多个进程直接映射到同一块物理内存。

线程之间怎么通信

Q：线程之间怎么通信？

线程因为共享进程内存，所以“通信”本质上往往就是共享变量、共享对象。

但共享带来的问题是竞态条件，所以通常要结合这些同步手段：

- `synchronized` / `Lock`
- 条件变量
- 信号量
- `wait/notify`
- 原子类和 CAS

所以线程通信可以一句话概括：

线程通信比进程容易，但同步比进程更重要。

并发和并行有什么区别

Q：并发和并行的区别是什么？

并发 是一段时间内看起来多个任务都在推进。

并行 是同一时刻多个任务真的同时执行。

比如：

- 单核 CPU 上多个线程交替执行，属于并发
- 多核 CPU 上多个线程各占一个核心同时跑，属于并行

所以一句话记忆就是：

并发强调“交替推进”，**并行**强调“同时执行”。

用户级线程和内核级线程怎么理解

Q：什么是用户级线程和内核级线程？

可以把线程实现分成三种常见模型：

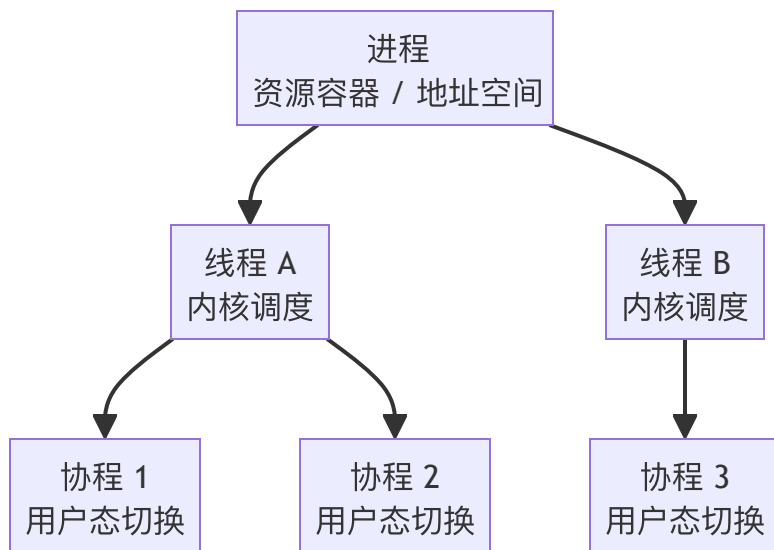
- **用户级线程**：线程调度由用户态运行时完成，内核感知不到每个线程
- **内核级线程**：线程由操作系统直接管理和调度
- **混合模型**：用户态和内核态共同参与映射和调度

面试里最重要的理解是：

- 用户级线程切换快，但一个线程阻塞可能拖住整个进程
- 内核级线程能更好利用多核，但创建和切换成本更高

Java 传统线程模型更接近 **1:1 的内核线程模型**，而虚拟线程更接近“用户态轻量调度”的思路。

图解补充：进程、线程、协程关系



高频追问

Q：协程一定比线程好吗？

不一定。协程主要适合 IO 密集型、高并发、等待时间长 的场景。

如果是 CPU 密集型任务，瓶颈在计算，不在切换，协程并不会天然更快。

这时候更关键的是：

- 核心数够不够
- 任务能不能并行拆分
- 调度和缓存命中是否合理

Q：为什么多线程程序容易出问题？

因为线程共享进程内存，带来三个经典问题：

- 竞态条件
- 可见性问题
- 有序性问题

所以线程的难点从来不只是“开几个线程”，而是：

共享资源时如何保证正确性。

面试回答模板

如果面试官问“进程、线程、协程有什么区别”，可以按这个顺序答：

第一，进程是资源分配的基本单位，线程是 CPU 调度的基本单位，协程是更轻量的用户态执行单元。

第二，进程之间地址空间隔离，线程共享同一进程资源，所以线程切换比进程切换轻，但同步问题更多。

第三，协程的优势在于用户态切换成本低，特别适合高并发 IO 场景，但 CPU 密集型任务不一定占优。

速记版

- 进程：资源隔离强，成本高
- 线程：共享资源，调度方便，同步复杂
- 协程：用户态切换，适合 IO 密集
- 并发：交替推进；并行：同时执行
- IPC 高频考点：管道、消息队列、共享内存、信号量、Socket

内存管理

内存管理

操作系统为什么需要内存管理

Q：操作系统为什么需要内存管理？

内存管理的目标主要有三个：

- 让多个进程安全共享内存资源
- 提高内存利用率
- 给程序提供统一、连续的地址空间抽象

如果没有内存管理，程序直接操作物理内存，安全性和可维护性都会非常差。

虚拟内存

Q：什么是虚拟内存？

虚拟内存可以理解成：

程序看到的是虚拟地址，操作系统再把虚拟地址映射到物理地址。

这样做的好处是：

- 进程之间互相隔离
- 程序不必关心真实物理内存布局
- 可以通过分页把部分不常用数据放到磁盘，缓解内存压力

分页和分段

Q：分页和分段有什么区别？

分页 是把内存划分成固定大小的页。

分段 是按逻辑模块划分，比如代码段、数据段、栈段。

分页更利于内存管理和物理映射，分段更符合程序逻辑语义。

现代操作系统里，分页通常更核心。

地址翻译过程是怎样的

Q: CPU 是怎么把虚拟地址翻译成物理地址的?

核心流程可以概括成四步：

- CPU 先拿到虚拟地址
- 根据页号去查页表
- 找到对应的物理页框
- 再拼上页内偏移，得到最终物理地址

这里的关键点在于：

高位通常用来定位页，低位偏移表示页内位置。

如果面试官追问性能问题，就继续说 TLB。

什么是页表和 TLB

Q: 页表和 TLB 分别是什么?

页表 是操作系统维护的映射表，用来记录“虚拟页 -> 物理页框”的对应关系。

TLB 可以理解成页表的高速缓存，它缓存在 CPU 里，保存最近访问过的地址映射。

如果每次访存都先去内存里查页表，成本太高，所以需要 TLB 来加速。

因此最标准的回答是：

- 页表解决的是“怎么映射”
- TLB 解决的是“映射查得太慢”

缺页中断是什么

Q：什么是缺页中断？

当进程访问的虚拟页当前不在物理内存中时，就会触发 **缺页中断**。

操作系统会做这些事：

- 暂停当前进程
- 去磁盘把目标页调入内存
- 必要时淘汰一个旧页
- 更新页表
- 恢复进程继续执行

所以缺页中断本质上说明：

程序访问的逻辑地址有效，但对应数据当前不在内存里。

页面置换算法有哪些

Q：常见页面置换算法有哪些？

高频算法主要有这几个：

- **FIFO**：先进先出，实现简单，但可能出现 Belady 异常
- **LRU**：淘汰最近最少使用页，更符合局部性原理
- **Clock**：对 LRU 的工程化近似实现

面试里一般这样答更稳：

- 理论上 LRU 更理想
- 实际系统里常常用 Clock 这类近似算法，因为真正精确维护 LRU 成本高

进程地址空间一般怎么分布

Q：一个进程的虚拟地址空间一般怎么划分？

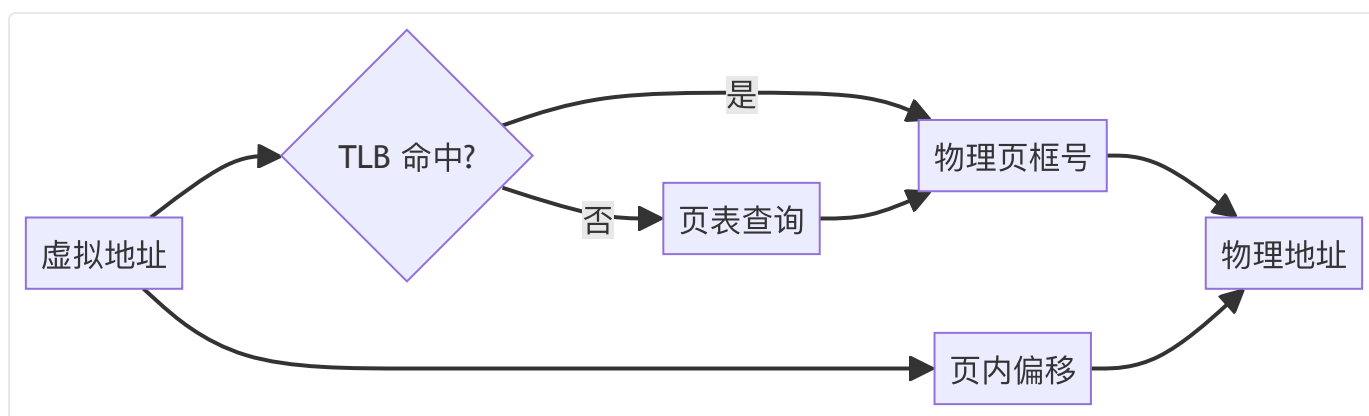
典型划分可以这样理解：

- 代码段
- 数据段
- 堆
- 栈
- 共享库映射区

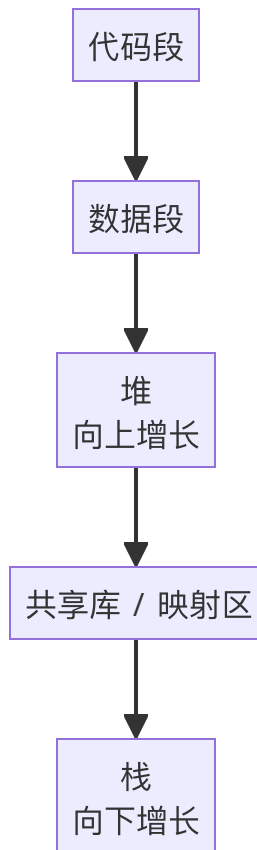
其中最容易考的是：

- **堆**：向上增长，主要用于动态分配
- **栈**：向下增长，主要保存函数调用现场和局部变量

图解补充：虚拟地址到物理地址



图解补充：进程地址空间



高频追问

Q：为什么有了物理内存还要搞虚拟内存？

因为虚拟内存同时解决了三件事：

- 进程隔离
- 地址抽象
- 内存扩展

如果程序直接操作物理内存，会带来安全问题，而且程序装载、重定位、内存复用都会变复杂很多。

Q：分页为什么比纯分段更主流？

因为分页是固定大小，映射和管理更标准，能更好地配合页表、TLB、缺页中断这一整套机制。纯分段更贴近逻辑语义，但容易产生外部碎片，工程上不如分页稳定。

面试回答模板

如果面试官问“什么是虚拟内存”，可以这样答：

虚拟内存本质上是操作系统给每个进程提供的一层地址抽象，程序访问的是虚拟地址，再通过页表映射到物理内存。

它的价值主要有三点：第一做进程隔离，第二简化程序对内存的使用，第三可以把不活跃页放到磁盘，提升整体内存利用率。

为了避免每次查页表太慢，CPU 还会用 TLB 缓存热点映射。

速记版

- 虚拟内存：隔离 + 抽象 + 扩展
- 页表：负责映射；TLB：负责加速
- 缺页中断：页合法，但当前不在内存
- 置换算法：FIFO、LRU、Clock
- 堆向上，栈向下

死锁

死锁

什么是死锁

Q: 什么是死锁?

死锁指的是多个进程或线程互相等待对方释放资源，最终谁都无法继续执行。

它的本质是：

形成了循环等待。

死锁的四个必要条件

Q: 死锁的四个必要条件是什么?

四个必要条件是：

- 互斥
- 请求并保持
- 不可剥夺
- 循环等待

只要破坏其中任意一个条件，就可以避免死锁。

如何避免死锁

Q: 怎么避免死锁?

常见做法包括：

- 固定资源获取顺序
- 一次性申请所需资源
- 缩短持锁时间
- 尽量减少资源嵌套依赖

在 Java 业务开发里，最常见也最实用的是：

按统一顺序加锁。

IO多路复用

IO 多路复用是什么

Q: 什么是 IO 多路复用?

IO 多路复用的核心思想是:

一个线程可以同时监听多个文件描述符的 IO 事件。

这样就不用像传统阻塞模型那样, 一个连接配一个线程, 从而显著提高并发能力。

select、poll、epoll 的区别

Q: select、poll、epoll 有什么区别?

select:

- 有监听数量上限
- 每次调用都要把所有 fd 从用户态拷贝到内核态
- 遍历成本较高

poll:

- 解决了 select 的数量上限问题
- 但本质上仍然需要轮询遍历

epoll:

- 基于事件驱动
- 不需要每次完整遍历所有 fd
- 在大量连接场景下性能更好

所以高并发网络编程里, 最常见的是 epoll。

阻塞、非阻塞、同步、异步怎么区分

Q: 阻塞/非阻塞、同步/异步有什么区别?

这四个概念最容易混。建议分成两个维度来答:

- **阻塞 / 非阻塞**: 调用线程会不会被挂起等待
- **同步 / 异步**: 结果是不是还要调用方主动参与处理

常见组合理解:

- BIO: 同步阻塞
- NIO 多路复用: 同步非阻塞
- AIO: 异步非阻塞

所以 IO 多路复用并不等于异步, 它更准确地说是:
同步非阻塞的事件驱动模型。

epoll 为什么更高效

Q: 为什么 epoll 比 select、poll 更高效?

关键是两个点:

- **监听集合管理更高效**
- **返回的是就绪事件, 不是所有 fd**

也就是说, select/poll 的问题不只是“API 老”, 而是:

- 每次都要把关注列表从用户态拷到内核态
- 每次都要遍历全量 fd

而 epoll 更接近“先注册、后通知”的模式, 所以在连接数大、活跃连接比例低的场景下更有优势。

epoll 的 LT 和 ET 模式

Q: 什么是 LT 和 ET?

`epoll` 常见有两种触发模式：

- LT (Level Trigger, 水平触发)
只要缓冲区里还有数据没处理完，就会持续通知
- ET (Edge Trigger, 边缘触发)
只有状态变化时通知一次，不会反复提醒

面试里最常问的是区别：

- LT 更稳，更不容易漏处理
- ET 性能潜力更高，但要求一次把数据尽量读完，否则容易漏事件

如果你答 Netty，也可以补一句：

ET 模式通常要配合非阻塞 IO 一起用。

Reactor 模型怎么理解

Q: IO 多路复用和 Reactor 有什么关系？

IO 多路复用提供的是“事件监听能力”，Reactor 提供的是“事件分发模型”。

可以这样理解：

- `epoll` 负责告诉你“哪个连接就绪了”
- Reactor 负责把这个就绪事件分发给对应处理器

所以像 Java NIO、Netty，本质上都是：

- 底层用 IO 多路复用监听事件
- 上层用 Reactor 组织事件处理流程

零拷贝是什么，和 IO 多路复用有什么关系

Q: 零拷贝是什么？

零拷贝关注的是 数据搬运路径优化，IO 多路复用关注的是 连接事件处理效率。

两者解决的问题不一样：

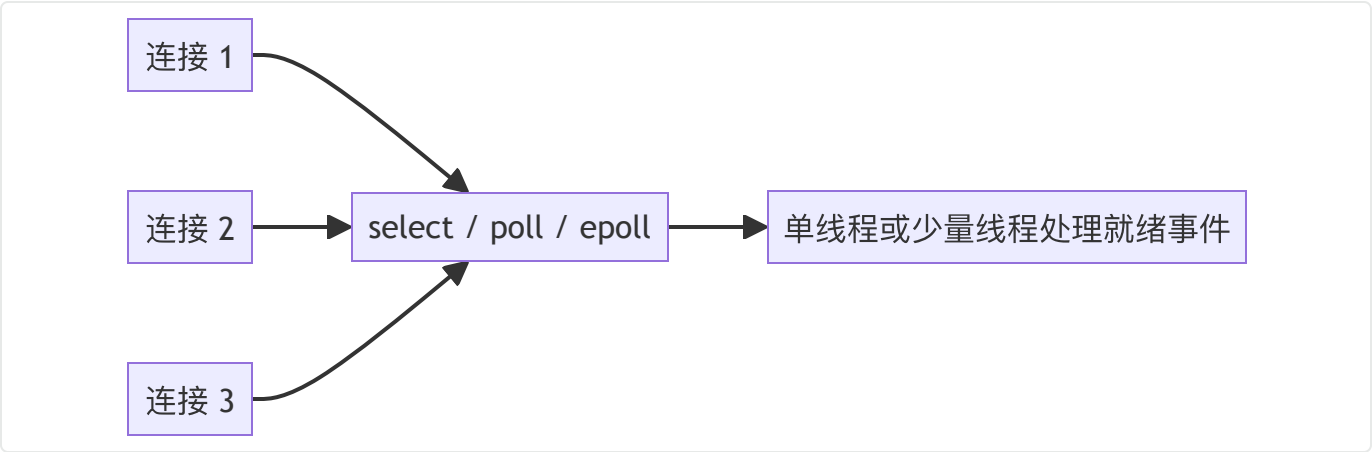
- IO 多路复用：减少线程数量，提升连接管理效率
- 零拷贝：减少用户态和内核态之间的数据复制、减少 CPU 参与

典型零拷贝手段有：

- `mmap`
- `sendfile`

面试时不要把它们混成一个概念，但可以说明它们经常一起出现在高性能网络框架里。

图解补充：IO 多路复用 workflow



图解补充：select、poll、epoll 对比

机制	数据结构/特征	性能瓶颈	适用结论
select	位图，数量有限制	全量拷贝 + 全量遍历	老方案，小规模可用
poll	链表/数组描述集合	无上限但仍然全量遍历	比 select 更灵活
epoll	事件驱动	实现复杂度更高	高并发场景首选

高频追问

Q：为什么 Redis、Nginx、Netty 都爱讲 IO 多路复用？

因为它们都要面对“大量连接 + 较少活跃事件”的场景。

如果每个连接一个线程，线程数会爆炸；

如果用 IO 多路复用，一个线程就能管很多连接，把真正活跃的事件挑出来处理。

Q：IO 多路复用是不是完全没有遍历成本？

不是。真正准确的说法是：

- 它避免了对所有连接做无意义的阻塞等待
- 尤其是 `epoll`，把“全量轮询”优化成了“基于就绪事件处理”

但事件分发、业务处理本身依然有成本。

面试回答模板

如果面试官问“什么是 IO 多路复用”，可以这样答：

IO 多路复用就是一个线程同时监听多个文件描述符，当某个连接可读可写时再去处理它。

它解决的是高并发场景下线程数量过多的问题，所以比传统 BIO 的“一连接一线程”模型更适合高并发。

常见实现有 `select`、`poll`、`epoll`，其中 `epoll` 的优势在于事件驱动和返回就绪 `fd`，所以在 Linux 下最常用。

速记版

- 一个线程监听多个 `fd`，就是 IO 多路复用
- BIO：同步阻塞；NIO：同步非阻塞；AIO：异步非阻塞
- `epoll` 关键优势：事件驱动、只处理就绪 `fd`
- LT 稳，ET 快但更容易写错
- IO 多路复用解决连接管理；零拷贝解决数据搬运

CPU缓存与一致性协议

为什么要有 CPU 缓存

Q：为什么主存前面还要加多级缓存？

因为 CPU 的计算速度远快于主存访问速度，如果每次取数据都直接访问内存，CPU 会经常空等。

所以现代计算机一般会设计成：

- L1 缓存
- L2 缓存
- L3 缓存
- 主存

这本质上是在解决 CPU 和内存速度不匹配 的问题。

什么是缓存局部性

Q：为什么有些代码缓存命中率高，有些很差？

这背后是局部性原理：

- **时间局部性**：刚访问过的数据，后面很可能还会访问
- **空间局部性**：访问了一个地址，它附近的数据也很可能很快被访问

所以连续遍历数组通常比跳跃访问更友好，因为更容易命中缓存。

什么是缓存行

Q：什么是 Cache Line？

CPU 和缓存之间不是按一个字节一个字节搬数据的，而是按 **缓存行** 为单位搬。

你可以理解成：

- 主存里相邻的一小块数据会一起被加载到缓存
- 常见缓存行大小是 64 字节

这也是为什么“相邻变量”有时候会互相影响，尤其是在多线程环境里。

什么是伪共享

Q：什么是伪共享？

伪共享指的是：

- 两个线程修改的是不同变量
- 但这两个变量恰好落在同一个缓存行里
- 导致缓存行频繁失效和同步，性能明显下降

所以伪共享不是数据语义冲突，而是：

硬件层面上共享了同一个缓存行。

什么是缓存一致性协议

Q：为什么多核 CPU 需要缓存一致性协议？

因为每个 CPU 核心可能都有自己的私有缓存。

如果核心 A 改了某个变量，而核心 B 还在用旧缓存，就会出现数据不一致问题。

缓存一致性协议的目标就是：

保证多个核心看到的数据最终一致。

MESI 协议怎么理解

Q: 什么是 MESI?

MESI 是经典缓存一致性协议，缓存行通常有四种状态：

- **M (Modified)**：已修改，只在当前缓存里，和主存不一致
- **E (Exclusive)**：独占，只在当前缓存里，和主存一致
- **S (Shared)**：共享，可能在多个缓存里，和主存一致
- **I (Invalid)**：失效，当前缓存副本不可用

面试里不用背每一条状态转移规则，但一定要理解：

- 一个核心写数据后，其他核心对应缓存行可能失效
- 这样才能避免不同核心长期看到不同值

缓存一致性和内存可见性是什么关系

Q: CPU 缓存一致性和 Java 可见性有什么关系？

它们不是同一个层次，但有强相关。

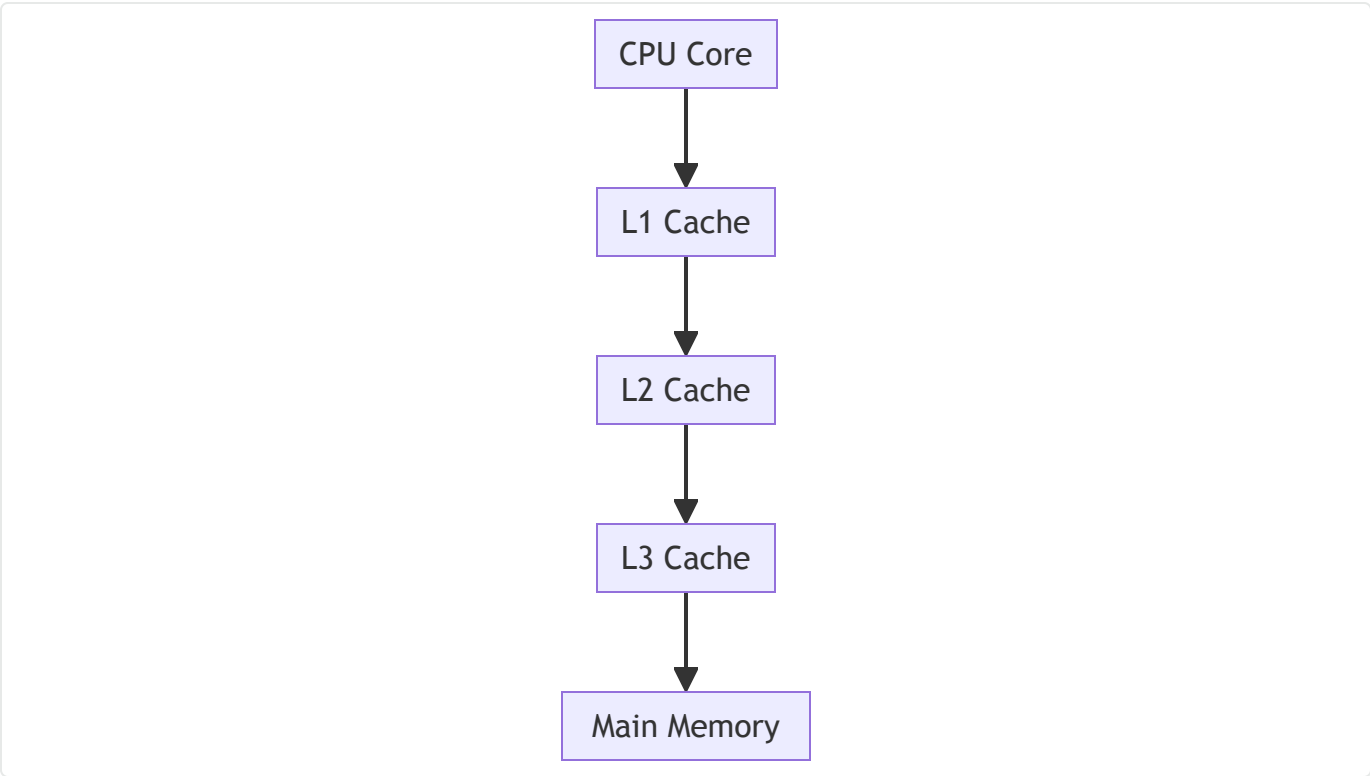
可以这样理解：

- CPU 缓存一致性是 **硬件层面**
- `volatile`、内存屏障、JMM 是 **语言/运行时层面**

语言层的可见性保证，最终还是要落到硬件执行上。

所以很多并发问题，底层都和缓存、重排序、写回时机有关。

图解补充：多级缓存结构



图解补充：MESI 核心含义

状态	含义	和主存是否一致	是否可能被其他核心共享
M	已修改	否	否
E	独占	是	否
S	共享	是	是
I	失效	无意义	无意义

高频追问

Q：为什么连续遍历数组通常更快？

因为它更符合空间局部性，CPU 一次把一整条缓存行加载进来后，后续访问很可能直接命中缓存，不用频繁去主存取数据。

Q：伪共享为什么会拖慢程序？

因为不同线程虽然在修改不同变量，但变量处于同一缓存行中，导致缓存行在多个核心之间频繁同步、失效和回写。

所以伪共享的重点不是锁，而是：
缓存行为单位的冲突。

面试回答模板

如果面试官问“为什么需要缓存一致性协议”，可以这样答：

现代多核 CPU 每个核心都有自己的缓存，如果一个核心修改了共享变量，而另一个核心还在读旧缓存，就会出现数据不一致。

所以硬件层需要缓存一致性协议来协调多个核心对同一份数据的读取和写入，MESI 就是最经典的协议之一。

它通过缓存行状态流转来保证多个核心最终看到一致的数据。

速记版

- CPU 快，内存慢，所以要有多级缓存
- 局部性分时间局部性和空间局部性
- 数据以缓存行为单位搬运
- 伪共享：变量不同，但缓存行相同
- MESI：M、E、S、I 四种状态

Linux常用命令

查看进程和资源

Q: 排查线上问题时，你常用哪些 Linux 命令？

我最常用的有：

- `top` / `htop` : 看 CPU、内存、进程资源占用
- `ps -ef` : 看进程信息
- `free -m` : 看内存
- `df -h` : 看磁盘空间
- `du -sh` : 看目录大小

查看日志和网络

Q: 常见日志和网络排查命令有哪些？

常见会用到：

- `tail -f` : 实时看日志
- `grep` : 过滤关键字
- `less` : 分页查看文件
- `netstat` / `ss` : 看端口和连接
- `lsof -i` : 看端口占用

如果是 Java 服务排障，还经常会配合 `jps`、`jstack`、`jmap` 一起用。

CPU 过高时看什么

Q: 线上 CPU 飙高时，你一般怎么查？

我的排查顺序通常是：

- `top` 看整体 CPU 占用
- `top -Hp <pid>` 看哪个线程最耗 CPU
- `ps -mp <pid> -o THREAD,tid,time` 辅助看线程
- 把线程 ID 转成 16 进制，再配合 `jstack` 找具体代码栈

如果是面试，建议把“命令 + 思路”一起说出来，不要只背命令名。

内存问题怎么查

Q：怀疑内存有问题时，你会用哪些命令？

常见组合是：

- `free -m`：看整体内存和 swap
- `top`：看哪个进程吃内存
- `vmstat`：看内存、换页、上下文切换
- `pmap -x <pid>`：看具体进程内存映射

如果是 Java，还会继续配合：

- `jmap -histo`
- `jcmd GC.heap_info`

磁盘和 IO 问题怎么查

Q：磁盘满了或者 IO 异常时怎么查？

我一般这么查：

- `df -h`：先看分区整体使用率
- `du -sh *`：看哪个目录占空间大
- `du -sh /path/* | sort -h`：逐层缩小范围
- `iostat -x`：看磁盘读写压力和 util

磁盘类问题最常见的坑是：

- 日志膨胀
- 临时文件没清理
- 大目录或大文件积压

网络问题怎么查

Q：端口占用、连接异常、超时问题怎么查？

常用命令有：

- `ss -lntp`：看监听端口和对应进程
- `lsof -i :端口号`：看某个端口被谁占了
- `netstat -anp`：看连接状态
- `ping`：测连通性
- `curl`：测接口可达性和返回

如果连接很多，还要关注：

- `ESTABLISHED`
- `TIME_WAIT`
- `CLOSE_WAIT`

这些 TCP 状态常常是面试追问点。

日志排查怎么组织命令

Q：查日志时你常怎么组合命令？

最常用的几种组合：

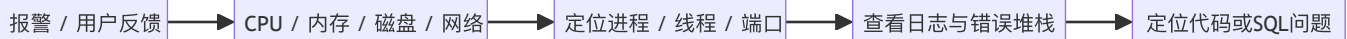
- `tail -f app.log`
- `grep "ERROR" app.log`
- `grep "用户ID" app.log | grep "关键字"`
- `cat app.log | grep "Exception" | tail -n 50`

如果日志很大，一般优先：

- `less`
- `tail`
- `grep`

不要一上来 `cat` 整个大文件。

图解补充：线上排障常见顺序



高频追问

Q：为什么现在更推荐 `ss`，而不是老用 `netstat`？

因为 `ss` 更快、更现代，查询 socket 状态的性能通常更好。

不过面试时不用踩 `netstat`，你可以说：

- 老环境里 `netstat` 也很常见
- 新环境里更推荐 `ss`

Q：排查问题时你是先看日志还是先看资源？

一般先看资源概况，再看日志细节。

因为如果是 CPU 打满、内存泄漏、磁盘写满、端口异常，先从资源视角能更快缩小范围。

日志更适合在定位到某个进程、某个接口、某个时间段之后精确分析。

面试回答模板

如果面试官问“线上问题常用哪些 Linux 命令”，可以这样答：

我通常按资源维度来排查。先用 `top`、`free -m`、`df -h` 看 CPU、内存、磁盘，再用 `ps`、`ss`、`lsof` 定位进程和端口，最后用 `tail`、`grep`、`less` 查日志。

如果是 Java 服务，我还会继续用 `jstack`、`jmap` 结合线程和堆信息定位问题。

速记版

- CPU: `top`、`top -Hp`、`ps`
- 内存: `free -m`、`vmstat`、`pmap`
- 磁盘: `df -h`、`du -sh`、`iostat`
- 网络: `ss`、`netstat`、`lsof -i`
- 日志: `tail`、`grep`、`less`

操作系统补充

用户态和内核态

Q: 什么是用户态和内核态?

用户态是应用程序运行的受限环境，权限较低；

内核态是操作系统内核运行的高权限环境，能直接操作硬件资源。

像网络 IO、磁盘 IO、进程调度等很多能力，都需要从用户态切到内核态完成。

中断

Q: 什么是中断?

中断可以理解成硬件或软件向 CPU 发出的“请先处理我”的信号。

操作系统借助中断机制来处理外设输入、时钟调度等事件。

孤儿进程和僵尸进程

Q: 什么是孤儿进程和僵尸进程?

孤儿进程 是父进程先退出，子进程还在运行。此时子进程会被系统托管。

僵尸进程 是子进程已经结束，但父进程还没回收它的退出状态。

僵尸进程如果大量堆积，会浪费系统资源。

进程调度算法

Q：常见进程调度算法有哪些？

常见的有：

- 先来先服务
- 短作业优先
- 时间片轮转
- 多级反馈队列

面试里最常被提到、也最有代表性的是 **时间片轮转** 和 **多级反馈队列**。

系统调用是什么

Q：什么是系统调用？

系统调用就是应用程序向操作系统请求服务的方式。

比如这些操作，本质上都离不开系统调用：

- 读写文件
- 创建进程
- 建立网络连接
- 申请内存

最常见的面试表达是：

用户程序自己没有直接操作硬件的权限，所以要通过系统调用进入内核态，由内核代为完成。

用户态切换到内核态有哪些场景

Q：什么时候会发生用户态到内核态切换？

常见有三类：

- **系统调用**：应用主动请求内核服务
- **中断**：外设或时钟中断打断当前执行流
- **异常**：比如缺页、除零、非法访问

所以面试里可以总结为：

主动切换常见是系统调用，被动切换常见是中断和异常。

上下文切换是什么

Q：什么是上下文切换？为什么它贵？

上下文切换就是 CPU 从当前执行单元切到另一个执行单元前，先保存当前状态，再恢复目标状态。

它贵的原因主要有：

- 要保存和恢复寄存器、程序计数器、栈指针
- 线程切换也可能导致缓存局部性变差
- 进程切换还可能引入地址空间切换和 TLB 失效

所以很多高性能问题，本质上不是“线程太少”，而是：
切换太频繁。

中断、异常、系统调用怎么区分

Q：中断、异常、系统调用有什么区别？

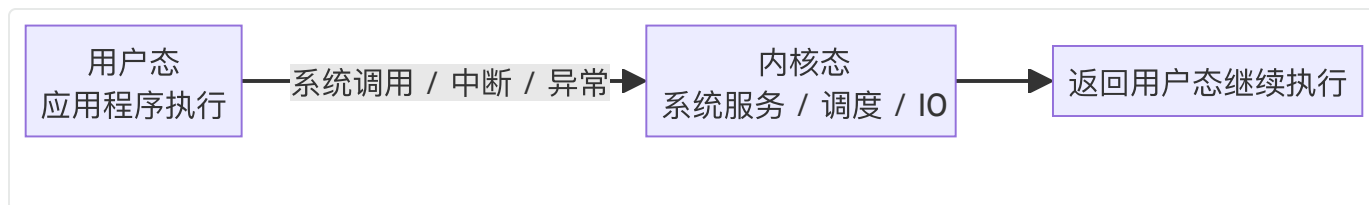
这三个概念常被放在一起考：

- **系统调用**：应用程序主动发起，请求内核服务
- **异常**：程序运行出错或特殊事件触发，比如缺页
- **中断**：来自外设或时钟的外部事件

所以区分它们最简单的角度是：

谁发起、为什么切换。

图解补充：用户态到内核态



高频追问

Q：孤儿进程和僵尸进程哪个更危险？

通常是 **僵尸进程** 更值得警惕。

因为孤儿进程会被系统接管，不一定是坏事；

但僵尸进程说明退出状态没有被回收，堆积多了会占用 PID 等系统资源。

Q：上下文切换一定发生在进程之间吗？

不是。线程之间也会发生上下文切换。

只是：

- 线程切换通常更轻
- 进程切换通常更重

不要把“上下文切换”只理解成进程切换。

面试回答模板

如果面试官问“用户态和内核态有什么区别”，可以这样答：

用户态是应用程序运行的受限环境，不能直接操作硬件；内核态拥有更高权限，可以管理 CPU、内存、磁盘、网络等系统资源。

当程序需要文件 IO、网络 IO、进程管理这类能力时，就要通过系统调用进入内核态。

此外，中断和异常也会触发从用户态切换到内核态。

速记版

- 用户态权限低，内核态权限高
- 切换来源：系统调用、中断、异常
- 系统调用是主动，中断和异常更偏被动
- 孤儿进程会被接管，僵尸进程要及时回收
- 上下文切换的成本主要来自状态保存和缓存失效

网络分层与基础

基础内容。后面的 `TCP`、`HTTP`、`DNS`、`URL` 到页面展示，都建立在这里。

为什么网络要分层

Q：为什么计算机网络要分层？

分层的核心目的，是把复杂问题拆开，让每一层只关心自己的职责。

如果不分层，应用开发者既要关心数据格式、连接管理、路由寻址，还要关心网卡怎么发电信号，系统会非常难维护。

分层之后：

- 应用层只关心“我要发什么数据”
- 传输层只关心“怎么可靠或高效地送到对端进程”
- 网络层只关心“怎么跨网络找到目标机器”
- 链路层只关心“怎么在当前链路上把帧发出去”

所以面试里可以一句话总结：

网络分层的本质，是通过职责隔离来降低复杂度，提高协议的可复用性和可演进性。

OSI 七层和 TCP/IP 四层

Q：OSI 七层模型和 TCP/IP 四层模型有什么区别？

OSI 七层更偏理论，用来帮助理解网络分工；TCP/IP 四层更偏实际，是今天互联网真正落地的协议栈。

常见对应关系可以这样记：

- OSI 的应用层、表示层、会话层，大体映射到 TCP/IP 的应用层
- OSI 的传输层，对应 TCP/IP 的传输层
- OSI 的网络层，对应 TCP/IP 的网络层
- OSI 的数据链路层、物理层，大体对应 TCP/IP 的网络接口层

面试里不要死背七层定义，重点要能说出每层的代表协议和职责。

各层分别做什么

层次	主要职责	典型协议/设备	面试表达
应用层	面向应用提供网络能力	HTTP、HTTPS、DNS、SMTP	规定数据语义，决定“发什么”
传输层	端到端传输	TCP、UDP	决定“怎么在进程之间传”
网络层	跨网络寻址和路由	IP、ICMP	决定“机器怎么找到机器”
网络接口层	在当前链路上传输数据	以太网、ARP、交换机	决定“这一跳怎么发出去”

数据封装和解封装

Q：应用数据在网络中是怎么一层层发出去的？

发送时是“层层封装”，接收时是“层层解封装”。

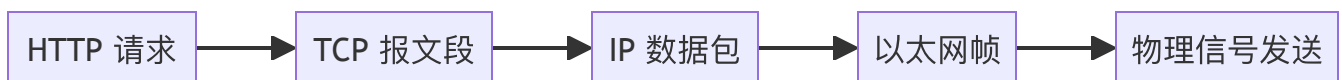
例如浏览器发一个 HTTP 请求时：

1. 应用层先组织 HTTP 报文
2. 传输层给它加上 TCP 首部，形成报文段
3. 网络层再加 IP 首部，形成数据包
4. 链路层再加 MAC 首部和尾部，形成帧
5. 最后由网卡把比特流发出去

接收方则反过来一层层拆开，最终把 HTTP 数据交给应用程序。

所以面试里常见的一句话是：

应用层看到的是消息，传输层看到的是段，网络层看到的是包，链路层看到的是帧。



MAC、IP、端口、Socket 的区别

Q：MAC 地址、IP 地址、端口号、Socket 分别是什么？

这几个概念经常被混着问，建议按“定位层级”来回答。

- **MAC 地址**：链路层地址，用来标识局域网里网卡的身份，解决“这一跳发给谁”
- **IP 地址**：网络层地址，用来标识网络中的主机，解决“跨网络最终发给哪台机器”
- **端口号**：传输层概念，用来区分一台机器上的不同进程
- **Socket**：通常可以理解为 **IP + 端口 + 协议** 的通信端点，是应用编程时真正接触的抽象

一句话串起来就是：

IP 找主机，端口找进程，MAC 负责当前链路这一跳，Socket 是程序视角下的通信入口。

追问1：既然已经有 IP 地址了，为什么还需要 MAC 地址？

因为 IP 解决的是跨网络寻址，MAC 解决的是局域网内逐跳转发。

数据包到了某一跳时，网卡并不会看全局路径，它只关心这个帧是不是发给自己，所以链路层还需要 MAC。

追问2：一个 TCP 连接能唯一确定吗？

通常用四元组唯一确定：

- 源 IP
- 源端口
- 目的 IP
- 目的端口

如果把协议也算上，可以扩展理解成五元组。

TCP 和 UDP 的区别

Q：TCP 和 UDP 有什么区别？怎么选？

面试里最稳的答法，不是背概念，而是按“连接、可靠性、顺序、开销、场景”来比较。

TCP：

- 面向连接
- 可靠传输
- 保证有序
- 有流量控制和拥塞控制
- 头部更大，开销更高

UDP：

- 无连接
- 不保证可靠和有序
- 没有重传、流控、拥塞控制这一整套复杂机制
- 头部更小，时延更低

所以选择标准通常是：

- 对正确性要求高，用 TCP
- 对实时性要求高、能容忍少量丢失，用 UDP

维度	TCP	UDP
是否连接导向	是	否
是否可靠	是	否
是否保证顺序	是	否
传输粒度	字节流	数据报
典型场景	HTTP、MySQL、RPC	DNS、音视频、实时游戏

追问1：DNS 为什么经常用 UDP？

因为大多数 DNS 查询报文很小，而且希望尽快返回结果，UDP 没有建连开销，效率更高。但当响应太大、需要可靠传输，或者做区域传输时，也会使用 TCP。

交换机和路由器

Q：交换机和路由器有什么区别？

最核心的区别是它们工作的层次不同、转发依据不同。

- 交换机主要工作在链路层，按照 **MAC 地址** 转发帧
- 路由器主要工作在网络层，按照 **IP 地址** 和路由表转发数据包

面试里一句话总结：

交换机更偏“同一局域网内怎么转”，路由器更偏“不同网络之间怎么走”。

设备	主要工作层次	转发依据	典型作用
交换机	数据链路层	MAC 地址	组建局域网
路由器	网络层	IP 地址	连接不同网络

速记版

这一篇建议按下面这条线记：

- 先讲为什么要分层
- 再讲 OSI 和 TCP/IP 的映射
- 再讲封装与解封装
- 再讲 **MAC / IP / 端口 / Socket**
- 最后讲 **TCP / UDP** 和 **交换机 / 路由器**

一句话面试版：

网络分层本质上是把“应用语义、端到端传输、跨网络路由、链路转发”拆成不同职责，每层做自己的事，最终协作完成通信。

从输入URL到页面展示

这道题本质上是网络、浏览器、系统、服务端、前端渲染的综合串讲题。回答得好不好，往往能直接看出知识链条是否完整。

面试里先给一个总流程

Q：从输入 URL 到页面展示，中间发生了什么？

如果面试官先问总流程，建议先给一版 10 步主线，不要一上来就把细节说炸。

可以这样答：

1. 浏览器解析 URL
2. 判断协议、域名、端口、路径等信息
3. 查询浏览器缓存、系统缓存、`hosts` 和 DNS 缓存
4. 如果没命中，就发起 DNS 解析拿到 IP
5. 与目标服务器建立 TCP 连接
6. 如果是 HTTPS，再进行 TLS 握手
7. 浏览器发送 HTTP 请求
8. 服务端处理请求并返回 HTTP 响应
9. 浏览器解析 HTML、CSS、JS，构建页面
10. 完成布局和绘制，把页面显示出来

这一版说清楚后，再看面试官往哪一层追问。

第一步：解析 URL

Q：浏览器拿到 URL 后，第一步做什么？

浏览器会先解析 URL，拆出这些信息：

- 协议：`http` 还是 `https`
- 域名
- 端口
- 路径
- 查询参数
- 锚点

这一步的意义是明确：

- 后面该走哪个协议
- 去连接谁
- 请求哪一个资源

第二步：缓存和 DNS

Q：DNS 解析前会先查哪些地方？

通常会先看：

1. 浏览器缓存
2. 操作系统缓存
3. `hosts`
4. 本地 DNS 解析器缓存

如果都没有，才真正去做 DNS 查询。

这也是为什么有时候你刚改了 DNS，但浏览器里访问结果还没变，因为旧缓存可能还在。

第三步：DNS 解析

Q：DNS 解析拿 IP 的过程怎么讲？

一条比较完整的链路是：

1. 客户端把查询交给本地 DNS
2. 本地 DNS 没有缓存，就去问根域名服务器
3. 根域名服务器告诉它去问顶级域名服务器
4. 顶级域名服务器告诉它去问权威域名服务器
5. 权威域名服务器返回最终 IP
6. 本地 DNS 把结果缓存后返回给客户端

如果业务前面接了 CDN，DNS 返回的往往不一定是源站 IP，而可能是一个更合适的 CDN 节点地址。

第四步：建立 TCP 连接

Q：拿到 IP 之后下一步是什么？

浏览器会根据目标 IP 和端口去建立 TCP 连接。

这一步本质上是为后续应用层通信先准备一条可靠的传输通道。

如果是 HTTP，通常就是对 **80** 端口建连；如果是 HTTPS，通常就是对 **443** 端口建连。

TCP 建连本身会走三次握手。

第五步：如果是 HTTPS，还要做 TLS 握手

Q：为什么 HTTPS 比 HTTP 多一步？

因为 HTTPS 不会一开始就直接发业务数据，而是要先协商安全参数，验证证书，建立加密通信所需的会话密钥。

所以链路上其实是：

- 先 TCP 建连
- 再 TLS 握手
- 然后才真正发 HTTP 请求

这也是为什么 HTTPS 的理解不能只停留在“端口变成 443”。

第六步：发送 HTTP 请求

Q：连接建立好后，浏览器会做什么？

浏览器会组织 HTTP 请求报文并发送出去。

其中通常会包含：

- 请求行
- 请求头
- 请求体

请求头里常见会有：

- Host
- Cookie
- User-Agent
- Accept
- 缓存相关头

如果本地有缓存副本，浏览器还可能带上协商缓存头，问服务端“这个资源我还能继续用吗”。

第七步：服务端处理请求

Q：请求到服务端以后一般怎么走？

比较常见的线上链路是：

1. 先到负载均衡或网关
2. 再到 Nginx 或应用网关
3. 转发到后端应用
4. 应用可能访问 Redis、MySQL、下游服务
5. 最终生成响应结果返回

所以从浏览器视角看是一次请求，但从服务端视角看，后面可能是一整条调用链。

第八步：浏览器接收响应

Q：浏览器收到响应后会立刻显示页面吗？

不一定。

浏览器先要看响应内容是什么：

- 如果是 HTML，进入页面解析流程
- 如果是图片、CSS、JS、字体等资源，就按资源类型分别处理
- 如果状态码是重定向，还可能继续发下一次请求

所以“收到响应”和“页面展示出来”之间，中间还隔着渲染流程。

第九步：解析 HTML、CSS、JS

Q：页面渲染的大体过程怎么说？

浏览器收到 HTML 后，通常会：

1. 解析 HTML，构建 DOM
2. 解析 CSS，构建 CSSOM
3. 把 DOM 和 CSSOM 合成渲染树
4. 进行布局，计算每个节点的位置和大小
5. 进行绘制，把页面画出来

如果 HTML 里还有外链 CSS、JS、图片、字体，浏览器还会继续发请求加载这些资源。

第十步：JS 执行和后续资源加载

Q：为什么有时候 HTML 回来了，页面还是没完全出来？

因为首个 HTML 文档往往只是开始。

后续还可能涉及：

- 外链 CSS 下载
- JS 下载和执行
- 图片和字体加载
- 异步接口请求
- 前端框架挂载和二次渲染

所以用户看到“白屏”或者“骨架屏”的时间，不一定只取决于网络建连快不快，还和资源体积、JS 执行、接口慢不慢有关。

一道题里怎么把网络和浏览器串起来

Q：这道题怎么回答才不显得零散？

建议你按下面这条链讲：

URL 解析 -> 缓存/DNS -> TCP/TLS -> HTTP 请求 -> 服务端处理 -> HTTP 响应 -> 浏览器解析渲染

这样结构清楚，面试官也更容易沿着某个点继续追问，比如：

- DNS 细节
- TCP 三次握手
- HTTPS 握手
- HTTP 缓存
- 页面渲染阻塞

高频追问补充

追问1：输入 URL 后，为什么不一定每次都要走完整 DNS？

因为浏览器、操作系统、本地 DNS 都可能已经缓存了解析结果，命中缓存时可以直接复用。

追问2：如果是 CDN 域名，浏览器最终访问的是谁？

通常先通过 DNS 被调度到某个 CDN 节点，如果节点有缓存就直接返回，没有再回源。

追问3：为什么有时候接口明明已经返回了，页面还是慢？

因为网络只是其中一环。

后面还可能有资源加载、JS 执行、前端渲染、主线程阻塞等问题。

项目里怎么把这道题说得更像真实经验

Q：如果面试官让你结合项目说，你会怎么组织？

可以用“一个页面首屏加载”的场景来讲。

比如：

- 用户访问首页，域名先走 DNS 和 CDN
- 建立 HTTPS 连接后，请求 HTML
- HTML 回来后继续请求 CSS、JS、图片
- 后端接口再查缓存和数据库
- 浏览器解析和渲染完成后，用户看到首屏

再顺带补一句优化思路：

- CDN 缓存静态资源
- 浏览器缓存减少重复请求
- 压缩和合并资源
- 接口做缓存和降耗

这样回答会更像做过线上性能优化。

速记版

这道题建议按下面这条线记：

- 解析 URL
- 查缓存和 DNS
- 建立 TCP
- HTTPS 再做 TLS
- 发 HTTP 请求
- 服务端处理并返回
- 浏览器解析 HTML/CSS/JS
- 布局绘制，页面展示

一句话面试版：

从输入 URL 到页面展示，本质上就是“先找到对端，再建立连接，再完成请求响应，最后由浏览器把响应内容解析渲染出来”。

TCP

TCP 是计算机网络面试里的绝对高频题，通常会从“三次握手”切入，然后一路追到 `TIME_WAIT`、可靠性、滑动窗口、拥塞控制、粘包拆包。

TCP 的核心特点

Q: TCP 有哪些核心特点?

TCP 最关键的标签有五个:

- 面向连接
- 可靠传输
- 面向字节流
- 保证有序
- 有流量控制和拥塞控制

所以 TCP 不是简单的“能发数据”，而是它会尽量保证:

- 数据不丢
- 顺序不错
- 对端能按顺序还原
- 不把接收方和网络整体打爆

三次握手

Q: TCP 为什么要三次握手?

三次握手的目的是让通信双方都确认两件事：

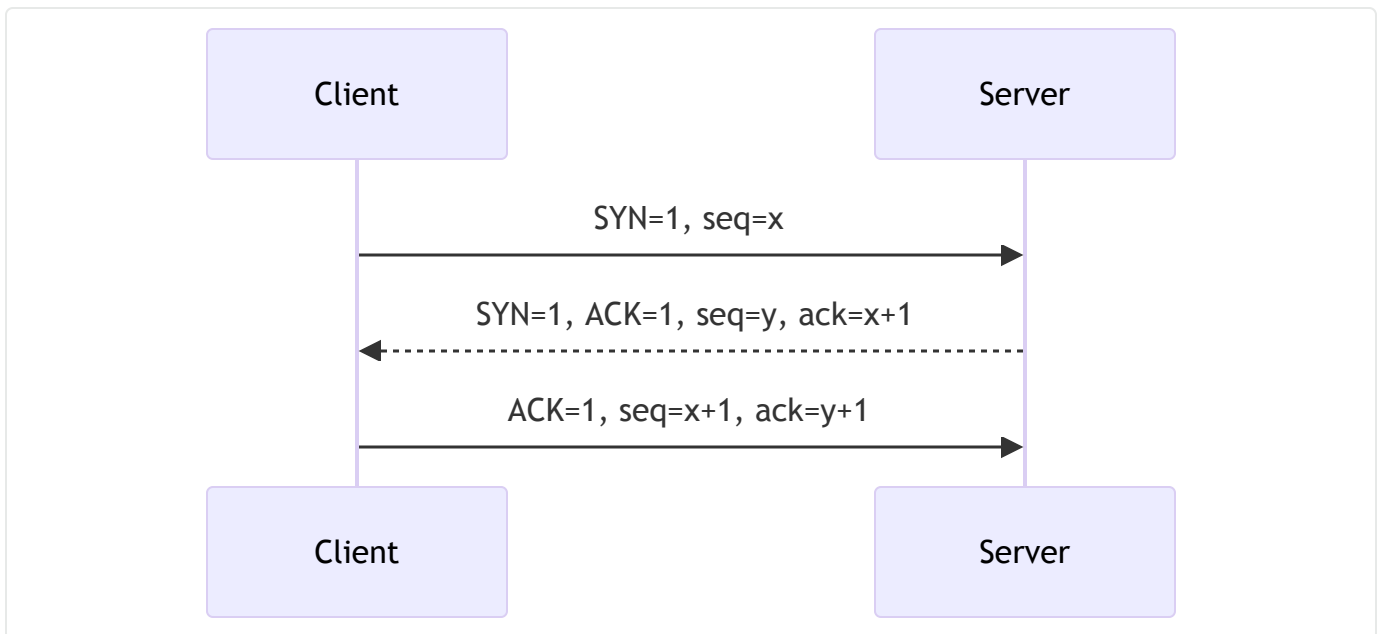
1. 对方能收也能发
2. 双方已经同步好初始序列号，可以开始可靠传输

典型过程是：

1. 客户端发 **SYN**，表示要建立连接
2. 服务端回 **SYN + ACK**，表示收到，也同意建立连接
3. 客户端再回 **ACK**，表示我确认你这边也准备好了

所以第三次握手本质上是：

客户端对服务端发起连接能力的一次最终确认。



追问1：为什么不能只用两次握手？

如果只有两次，服务端在发完 **SYN+ACK** 后就认为连接建立了，但它并不能确认客户端是否真的收到了这个响应。

第三次握手就是为了解决“服务端如何确认客户端接收能力正常”这个问题。

追问2：为什么不是四次握手？

因为服务端收到客户端的 **SYN** 后，完全可以把“确认客户端请求”和“自己也发起连接”合并成一个 **SYN+ACK**，没必要拆成两次。

四次挥手

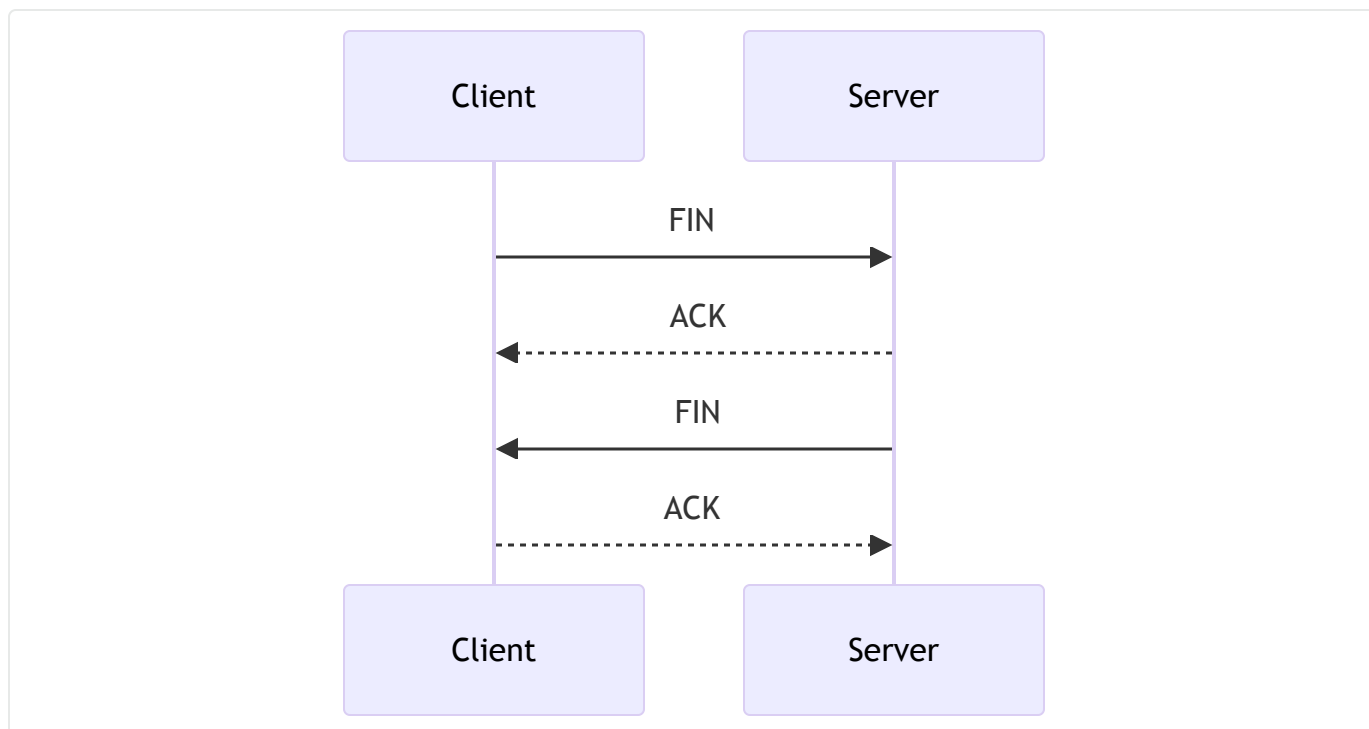
Q: TCP 为什么要四次挥手?

TCP 是全双工通信，双方都可以独立发送数据，所以关闭连接时，要分别关闭两个方向。

典型过程是：

1. 一方发送 **FIN**，表示我这边不再发数据了
2. 对方回 **ACK**，表示知道了
3. 对方把自己剩余的数据发完后，再发送 **FIN**
4. 最后一方回 **ACK**

所以关闭连接往往需要四次，而不是三次。



追问1: 为什么挥手时 ACK 和 FIN 往往不能合并?

因为收到 **FIN** 的一方，可能还有数据没发完。

它可以先确认“我知道你不发了”，但自己未必能立刻关闭发送通道，所以通常要等业务数据处理完成后，再发 **FIN**。

TIME_WAIT 和 CLOSE_WAIT

Q: `TIME_WAIT` 和 `CLOSE_WAIT` 分别是什么？

这是 TCP 状态题里的重点。

`TIME_WAIT` :

- 通常出现在主动关闭连接的一方
- 收到对方 `FIN` 后，回完最后一个 `ACK` 进入
- 会等待一段时间后再真正关闭

`CLOSE_WAIT` :

- 通常出现在被动关闭连接的一方
- 表示对方已经要求关闭连接了
- 但本端应用还没有真正执行关闭动作

一句话记忆：

`TIME_WAIT` 是“我已经收尾了，但还要再等等”；`CLOSE_WAIT` 是“对方都结束了，但我还没收尾”。

追问1：为什么要有 `TIME_WAIT` ？

主要有两个原因：

1. 确保最后一个 `ACK` 能让对方收到，如果对方没收到还会重发 `FIN`
2. 让旧连接中的残留报文在网络里自然消失，避免影响后续同四元组的新连接

追问2：线上 `CLOSE_WAIT` 很多通常意味着什么？

这通常不是网络抖动，而是应用没有及时 `close` 连接，比如代码分支没释放 `Socket`、连接池回收有问题、异常处理不完整。

TCP 如何保证可靠传输

Q: TCP 是怎么保证可靠传输的？

TCP 的可靠性不是“绝对不出错”，而是通过一整套机制尽量保证数据正确到达。

常见机制包括：

- 序列号：保证数据可排序、可重组
- 确认应答：告诉发送方哪些数据已经收到
- 超时重传：超时没收到确认就重发
- 校验和：检查数据是否损坏
- 去重和乱序重排：避免重复、恢复顺序
- 滑动窗口：控制未确认数据量

所以面试里最稳的结论是：

TCP 的可靠，靠的是“编号、确认、重传、排序、控速”这一整套机制。

序列号和确认应答

Q：序列号和 ACK 在 TCP 里起什么作用？

TCP 是面向字节流的，每个字节都可以认为有自己的编号。

发送方发数据时，会带上起始序列号；接收方收到后，会通过 ACK 告诉发送方“下一个我期望收到哪个序号的数据”。

这样带来的好处是：

- 能发现丢包
- 能发现重复包
- 能恢复乱序

超时重传和快速重传

Q：TCP 丢包了以后怎么处理？

常见有两种思路：

1. **超时重传**：发出去一段时间还没收到 ACK，就认为可能丢了，重发
2. **快速重传**：如果发送方连续收到多个重复 ACK，就能更早判断某个报文段丢了，不必硬等超时

这也是为什么 TCP 既能保证可靠性，又尽量不把恢复动作做得太慢。

滑动窗口和流量控制

Q：什么是滑动窗口？什么是流量控制？

滑动窗口可以理解成：

发送方在还没收到 ACK 之前，最多可以连续发送多少数据。

接收方会把自己的可用缓冲区大小告诉发送方，发送方再根据这个值决定发多快，这就是流量控制。

所以流量控制关注的是：

不要把接收方撑爆。

追问1：如果接收方窗口变成 0 会怎么样？

发送方会暂停继续发送大量数据，但不会永远不管，一般还会做零窗口探测，确认接收方是不是已经恢复可接收状态。

拥塞控制

Q：什么是拥塞控制？和流量控制有什么区别？

流量控制关注的是接收方顶不顶得住，拥塞控制关注的是整个网络顶不顶得住。

简单说：

- 流量控制：别把对端接收缓冲区撑爆
- 拥塞控制：别把网络链路打满，导致整体拥塞

TCP 常见拥塞控制关键字要能说出来：

- 慢启动
- 拥塞避免
- 快速重传
- 快速恢复

追问1：慢启动为什么叫“慢”，但看起来增长很快？

“慢”不是指增长速度慢，而是指它一开始很保守，从比较小的拥塞窗口开始探测网络容量。虽然前期常表现为指数增长，但它的出发点是“先小心试探”。

TCP 是面向字节流的

Q：为什么说 TCP 是面向字节流，而不是消息？

因为 TCP 只保证一串字节按顺序传到对端，但不会替你保留“你这次发的是一条消息还是两条消息”的边界。

发送端写两次数据，接收端未必就是两次读出来，可能被合并，也可能被拆开。

这就是后面“粘包和拆包”的根源。

粘包和拆包

Q：TCP 为什么会出现粘包和拆包？怎么解决？

因为 TCP 是字节流协议，没有天然消息边界。

所以：

- 发送方连续写入两段数据，接收方可能一次都读到了，这叫粘包
- 一段完整消息太长，接收方一次没读全，这叫拆包

常见解决方案有三种：

- 固定长度协议
- 分隔符协议
- 长度字段 + 消息体

工程里最常见的是第三种，也就是先读消息头，再按长度读消息体。

KeepAlive 和心跳

Q：TCP KeepAlive 和业务心跳有什么区别？

二者都和“连接活性检测”有关，但层次不同。

- **TCP KeepAlive**：操作系统 TCP 层提供的保活机制，周期通常较长
- **业务心跳**：应用层自己定义的探活包，控制更灵活，也更贴近业务

面试里建议不要把两者混成一回事。

很多长连接系统真正依赖的，往往是应用层心跳，而不是仅靠 TCP KeepAlive。

SYN 洪泛

Q：什么是 SYN Flood？

SYN Flood 是一种典型的半连接攻击。

攻击者大量发送 `SYN`，但不完成后续握手，让服务端长期停留在半连接状态，消耗连接资源。

常见防护思路有：

- SYN Cookie
- 增大 backlog
- 限流和防火墙策略
- 网关层清洗

项目里怎么回答更像真实经验

Q：如果面试官问你 TCP 在项目中的价值，你会怎么答？

可以结合接口调用、RPC、数据库连接来讲。

比如：

- HTTP、RPC、MySQL 通信一般都跑在 TCP 上
- 看重的是可靠、有序和成熟的拥塞控制
- 长连接还能减少频繁握手和挥手的成本
- 出现大量 `TIME_WAIT`、`CLOSE_WAIT` 时，可以从连接复用和连接释放角度排查

这样回答会比只背概念更像做过系统的人。

速记版

TCP 建议按这条线记：

- 先讲“面向连接、可靠、有序、字节流”
- 再讲三次握手、四次挥手
- 再讲 `TIME_WAIT / CLOSE_WAIT`
- 再讲可靠性：序列号、ACK、重传、滑动窗口
- 再讲流量控制和拥塞控制
- 最后讲粘包拆包、KeepAlive、SYN 洪泛

一句话面试版：

TCP 的本质，是在不可靠的网络之上，通过连接管理、确认重传、窗口和拥塞控制，尽量提供可靠有序的字节流传输。

HTTP与HTTPS

HTTP 和 HTTPS 是面试中非常高频的网络基础题，通常会从“区别”切入，再追问状态码、GET/POST、TLS 握手。

HTTP 和 HTTPS 的区别

Q: HTTP 和 HTTPS 有什么区别？

最核心的区别是：HTTPS = HTTP + TLS/SSL 加密层。

HTTP 明文传输，安全性差；

HTTPS 在传输层之上增加了加密、身份认证、完整性校验，因此更安全。

除此之外常见差异还有：

- 默认端口不同：HTTP 80，HTTPS 443
- HTTPS 需要证书
- HTTPS 握手阶段会有一定额外开销

HTTPS 握手的大致过程

Q: HTTPS 是怎么建立安全连接的？

简化管理可以分成几步：

1. 客户端发起 HTTPS 请求
2. 服务端返回证书和公钥信息
3. 客户端校证书是否合法
4. 客户端生成会话密钥，并用服务端公钥加密发送过去
5. 双方后续使用对称加密通信

所以 HTTPS 的关键思想是：

非对称加密负责安全交换密钥，对称加密负责后续高效通信。

GET 和 POST 的区别

Q: GET 和 POST 有什么区别?

如果从面试角度回答，我会抓这几个点：

- GET 更偏获取资源，POST 更偏提交资源
- GET 参数通常放在 URL，POST 更多在请求体
- GET 更容易被缓存和收藏
- 从规范语义上讲，GET 应该是幂等的，POST 通常不是

但要注意：

GET 并不是绝对不能带请求体，POST 也不是绝对更安全。
安全性更多取决于是不是 HTTPS。

常见状态码

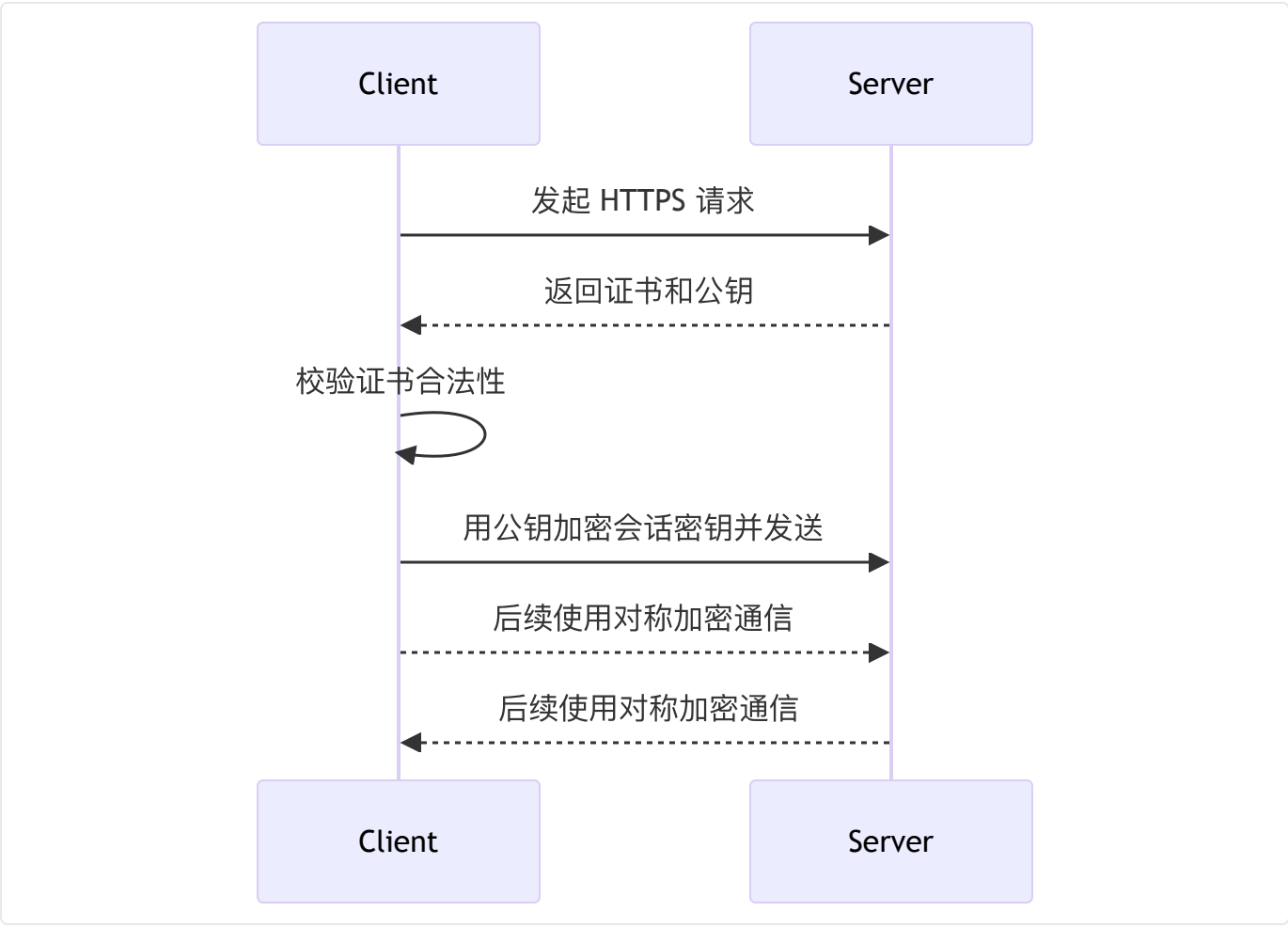
Q: 常见 HTTP 状态码有哪些?

高频要记住这些：

- 200 : 请求成功
- 301 / 302 : 重定向
- 400 : 请求参数错误
- 401 : 未认证
- 403 : 无权限
- 404 : 资源不存在
- 500 : 服务端内部错误
- 502 : 网关错误
- 503 : 服务不可用

面试里一般不是让你背全，而是看你是否能结合场景说出含义。

图解补充：HTTPS 握手怎么记



对比补充：HTTP 高频点速查

主题	核心结论	面试表达
HTTP vs HTTPS	HTTPS 多了 TLS 安全层	安全性来自加密、认证、完整性校验
GET vs POST	语义不同，不是“谁更安全”	安全看是否走 HTTPS
301 vs 302	都是重定向	一个更偏永久，一个更偏临时
401 vs 403	一个未认证，一个无权限	先登录再谈授权

高频追问补充

追问补充1：HTTPS 为什么既要非对称加密又要对称加密？

因为非对称加密适合安全交换密钥，但性能较差；对称加密性能高，适合后续大量数据传输。两者组合，兼顾安全和效率。

追问补充2：POST 比 GET 安全吗？

不绝对。POST 只是参数位置通常在请求体，不代表天然安全。只要是明文 HTTP，GET 和 POST 都可能被窃听；真正决定安全性的还是 HTTPS。

追问补充3：状态码怎么回答更像项目经验？

不要只背数字，可以带场景说，比如 **401** 常见于登录态失效，**403** 常见于权限校验失败，**502/503** 常见于网关或下游服务异常，这样更像实际排障经验。

速记版

HTTP/HTTPS 建议按这条线记：

- 先讲 HTTPS 比 HTTP 多了 TLS
- 再讲证书、公钥、会话密钥
- 再讲 GET/POST 的语义区别
- 最后讲高频状态码

一句话面试版：

HTTPS 的本质是在 HTTP 通信前先建立一条可信且加密的传输通道，后续再承载普通 HTTP 请求和响应。

DNS与CDN

DNS 和 CDN 是网络面试里经常和 `URL 到页面展示`、`HTTP 性能优化` 连着问的两块。

DNS 是做什么的

Q: DNS 是什么？为什么需要它？

DNS 的作用，是把人类更容易记忆的域名解析成机器真正通信需要的 IP 地址。

如果没有 DNS，用户访问网站时就只能记 IP，这既不方便，也不利于服务迁移、负载均衡和容灾。

所以 DNS 可以理解成互联网的“电话簿”，但它不只是做简单映射，还承担了：

- 域名解析
- 结果缓存
- 负载分流
- 容灾切换

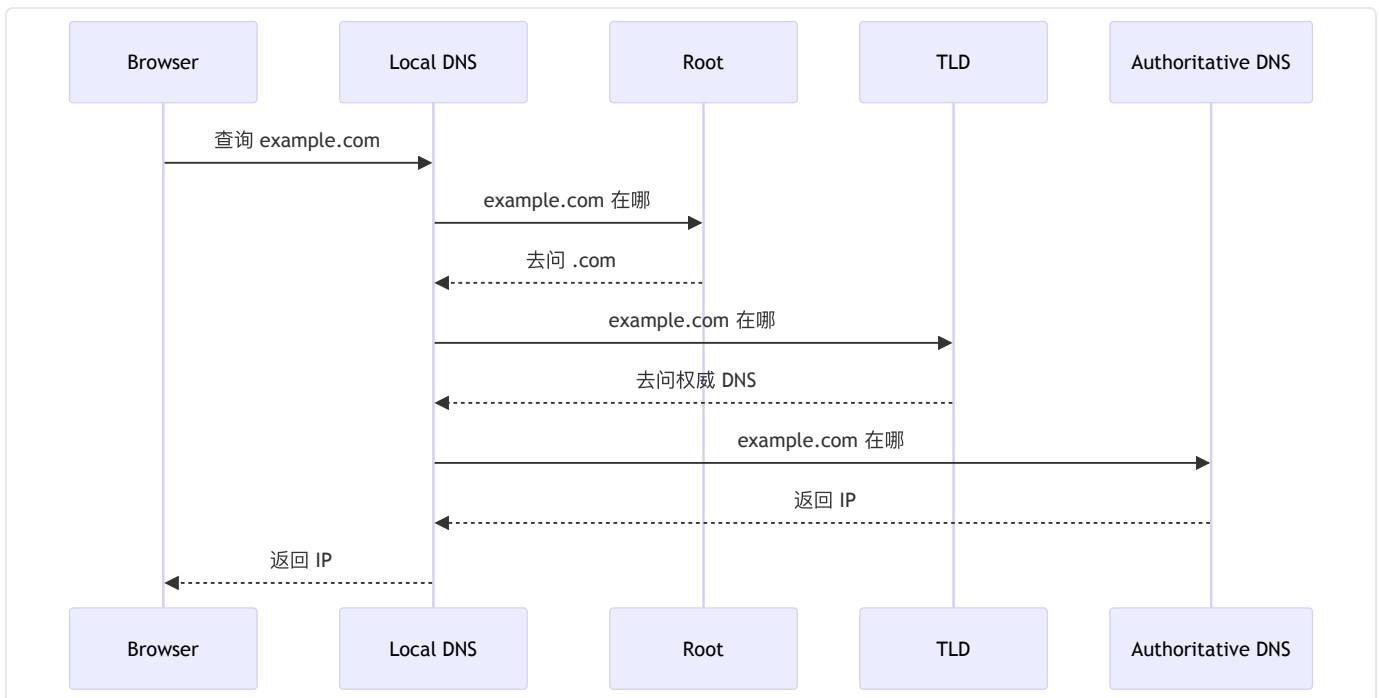
DNS 查询流程

Q: DNS 查询一般是怎么走的？

比较完整的链路通常是：

1. 浏览器先查自己的 DNS 缓存
2. 再查操作系统缓存和 `hosts`
3. 还没命中，就把请求发给本地 DNS 解析器
4. 本地解析器如果也没有缓存，就去问根域名服务器
5. 根域名服务器返回对应顶级域名服务器地址，比如 `.com`
6. 再去问顶级域名服务器
7. 顶级域名服务器返回权威域名服务器地址
8. 再去问权威域名服务器
9. 权威服务器返回最终 IP
10. 本地 DNS 缓存结果，再返回给客户端

面试时建议把“缓存 -> 递归解析器 -> 根 -> TLD -> 权威”这条主线讲清楚。



递归查询和迭代查询

Q：什么是 DNS 的递归查询和迭代查询？

这也是高频题，关键是区分“谁替你继续问”。

- **递归查询**：你问了一个 DNS 服务器之后，希望它帮你把最终答案查到底，再直接返回给你
- **迭代查询**：你问一个服务器，它如果不知道最终答案，就告诉你“下一步该去问谁”

通常：

- 客户端到本地 DNS，常常表现为递归查询
- 本地 DNS 再去问根、TLD、权威 DNS，通常是迭代查询

常见 DNS 记录类型

Q：常见 DNS 记录类型有哪些？

面试里最常见的这几个要能说出来：

- **A**：域名映射到 IPv4 地址
- **AAAA**：域名映射到 IPv6 地址
- **CNAME**：别名记录，一个域名指向另一个域名
- **NS**：指定某个域的权威 DNS 服务器
- **MX**：邮件服务器记录
- **TXT**：存文本信息，常用于校验、SPF、DKIM 等

如果面试官问 CDN、对象存储、自定义域名绑定，**CNAME** 通常特别高频。

TTL 和 DNS 缓存

Q：DNS 为什么会有缓存？TTL 是什么？

DNS 查询如果每次都从根域名服务器一路问到底，延迟和系统压力都会很大，所以一定要缓存。

TTL 是一条解析记录的生存时间。缓存没过期时，可以直接复用；过期后需要重新查询。

TTL 配得太长：

- 切换 IP、故障迁移会变慢

TTL 配得太短：

- 解析请求会变多，缓存命中率下降

所以它本质上是性能和灵活性之间的权衡。

CDN 是做什么的

Q：CDN 的核心作用是什么？

CDN 的核心目标有三个：

- 就近访问，降低时延
- 缓存静态资源，减少回源压力
- 提升可用性和抗突发流量能力

用户访问资源时，请求通常会先被调度到离自己更近的边缘节点。如果边缘节点已经缓存了资源，就直接返回；如果没有，就回源到源站拉取，再缓存一份。

所以面试里可以一句话总结：

CDN 本质上是把内容提前分发到离用户更近的地方，用空间换时间、用边缘节点抗流量。

CDN 的工作流程

Q：CDN 请求一般怎么走？

大致流程可以这样理解：

1. 用户访问业务域名
2. DNS 解析把请求引导到 CDN 节点
3. 边缘节点检查本地缓存
4. 如果命中，直接返回
5. 如果未命中，回源到源站拉取资源
6. 回源结果返回给用户，同时缓存在边缘节点

这也是为什么很多站点首次访问慢一点，后续访问会更快。

CDN 适合哪些内容

Q：CDN 只能加速图片、JS、CSS 吗？

最适合 CDN 的，确实是图片、静态资源、下载文件、视频切片这类“可缓存内容”。

但现在 CDN 不只做静态缓存，还会做：

- 动静分离
- 智能调度
- 边缘计算
- 动态加速
- WAF 和基础安全防护

只是从面试角度看，先把“边缘缓存 + 回源”讲清楚最重要。

DNS 和 CDN 的关系

Q：DNS 和 CDN 有什么关系？

CDN 经常借助 DNS 做流量调度。

也就是用户访问某个域名时，DNS 不一定直接返回源站 IP，而是根据地理位置、运营商、负载情况等因素，返回某个更合适的 CDN 节点地址。

所以可以理解为：

- DNS 负责“把用户引到合适的节点”
- CDN 节点负责“尽量在边缘直接把内容返回”

高频追问补充

追问1：为什么有时候改了 DNS 记录，结果不会立刻生效？

因为缓存还没过期。浏览器、本地系统、本地 DNS、运营商 DNS 都可能缓存旧结果，所以 TTL 没过之前，旧记录可能还在被使用。

追问2：CDN 缓存和浏览器缓存有什么区别？

浏览器缓存发生在客户端本地；CDN 缓存发生在网络边缘节点。
前者减少客户端重复请求，后者减少用户到源站的距离和源站压力。

追问3：CDN 回源会带来什么问题？

如果缓存命中率低、回源链路慢、源站抗压差，CDN 反而可能成为“放大器”，把大量未命中请求集中打到源站，所以命中率、缓存策略、回源带宽都很关键。

项目里怎么回答更像真实经验

Q：如果面试官问你 DNS/CDN 在项目里有什么价值，你会怎么答？

可以结合首页静态资源、图片资源、下载资源来讲。

比如：

- 域名通过 DNS 解析到 CDN
- 首页静态资源、商品图片先走 CDN
- 热资源命中边缘缓存，用户就近访问
- 对源站来说，静态流量被卸载掉，核心应用只处理真正需要回源的动态请求

这样回答比单纯说“CDN 提高速度”更像工程视角。

速记版

这一篇建议按下面这条线记：

- DNS 把域名解析成 IP
- 查询链路是：缓存 -> 本地 DNS -> 根 -> TLD -> 权威
- 客户端常见递归，本地 DNS 往外通常是迭代
- CDN 通过边缘缓存和智能调度加速访问
- DNS 常常负责把用户引流到合适的 CDN 节点

一句话面试版：

DNS 负责找到地址，CDN 负责把内容放到更近的地方，两者配合解决“去哪找”和“怎么更快拿到”这两个问题。

网络排查与高频场景

网络排查与高频场景

把常见网络问题按排查链路串起来。很多面试官会通过这类问题判断你是不是只会背八股。

排查网络问题的基本顺序

Q: 接口访问失败时，你一般怎么排查？

比较稳的思路是按链路自上而下或者自下而上查，不要一上来就猜。

常见顺序可以是：

1. 先确认域名是否能解析
2. 再确认目标 IP 是否可达
3. 再确认目标端口是否可连
4. 再看 TLS 握手是否成功
5. 再看 HTTP 请求和响应
6. 最后再看应用日志、网关、下游依赖

也可以简化成一句话：

先分清楚是 DNS 问题、连通性问题、端口问题、协议问题，还是应用本身的问题。

ping 能看什么

Q: ping 是干什么的？

`ping` 本质上是用 ICMP 回显报文测试目标主机是否可达，以及粗略观察网络时延和丢包情况。

它适合回答这些问题：

- 目标 IP 是否能通
- 延迟大概多少
- 是否有明显丢包

但要注意：

- `ping` 通，不代表应用端口一定通
- `ping` 不通，也不一定代表机器挂了，可能只是禁了 ICMP

traceroute 能看什么

Q: `traceroute` / `tracert` 有什么用？

它的作用是看请求经过了哪些路由节点，以及大致在哪一跳开始出现延迟或丢包。

如果用户说“某地访问很慢”，`traceroute` 很适合用来判断问题更像：

- 本地网络问题
- 中间路由问题
- 机房出口问题
- 目标区域网络问题

nslookup / dig 能看什么

Q: `nslookup` 或 `dig` 一般什么时候用？

当你怀疑问题出在 DNS 时，这两个工具非常常用。

可以用来确认：

- 域名当前解析到了哪个 IP
- TTL 是多少
- 权威 DNS 是谁
- `A`、`AAAA`、`CNAME` 等记录是否正常

如果线上刚切流量、刚改域名，`dig` 查出来的结果和预期不一致，通常就要先看 DNS 缓存和 TTL。

curl 最适合看什么

Q: `curl` 在网络排查里为什么很常用？

因为它很适合直接验证 HTTP/HTTPS 层到底有没有问题。

常见用途有：

- 测试接口是否能访问
- 看返回状态码、响应头、响应体
- 看是否跳转
- 看 TLS 握手和证书
- 带上 `Host` 头绕过部分域名问题，直接打 IP 验证

所以如果 `ping` 没问题但页面打不开，通常下一步就该用 `curl` 看协议层是不是已经异常了。

ss / netstat 能看什么

Q: `ss` 或 `netstat` 在排查中有什么价值？

它们非常适合看机器当前的连接状态。

常见用途：

- 看某个端口是不是在监听
- 看连接数是不是异常高
- 看 `TIME_WAIT`、`CLOSE_WAIT`、`ESTABLISHED` 是否异常堆积
- 看服务到底监听在 `127.0.0.1` 还是 `0.0.0.0`

如果服务明明启动了，但外部还是连不上，先看监听地址和端口状态，常常就能定位一半问题。

502、503、504 的区别

Q: `502`、`503`、`504` 经常怎么区分？

这也是很常见的排障题。

- `502 Bad Gateway`：网关拿到了一个无效响应，常见于上游服务异常、协议不匹配、连接被重置
- `503 Service Unavailable`：服务当前不可用，常见于过载、限流、维护、熔断
- `504 Gateway Timeout`：网关等待上游超时了，本质上是“请求发过去了，但上游太久没回”

面试里最好带一句：

`502` 更像上游回得不对，`504` 更像上游回得太慢，`503` 更像服务当前不接单。

TIME_WAIT 过多怎么办

Q: 如果机器上 `TIME_WAIT` 很多，说明什么？

`TIME_WAIT` 多，通常说明这台机器主动关闭了大量 TCP 连接。

常见原因：

- 短连接太多
- 压测期间请求量大
- 服务端频繁主动断开连接
- 长连接没有复用好

这不一定是 bug，但如果数量非常夸张，就可能带来端口占用和资源压力。

排查时重点看：

- 是否可以复用连接
- 是否误用了短连接
- 是否有异常重试导致大量瞬时连接

CLOSE_WAIT 过多怎么办

Q: `CLOSE_WAIT` 很多又意味着什么？

这通常说明对端已经把连接关了，但本端应用没有及时调用 `close` 把连接真正释放掉。

所以 `CLOSE_WAIT` 堆积更值得警惕，因为它往往不是网络抖动，而是应用侧连接释放有问题，比如：

- 代码忘记关闭 Socket
- 连接池回收不及时
- 异常分支没有执行清理逻辑

面试里一句话就够：

`TIME_WAIT` 更像关闭策略问题，`CLOSE_WAIT` 更像应用没有好好收尾。

连接超时和连接重置

Q: 连接超时和 `Connection reset` 有什么区别？

二者都常见，但含义不一样。

- **连接超时**：通常是请求没有在预期时间内建立连接或收到响应，更像“迟迟等不到”
- **Connection reset**：通常是对方或中间设备直接把连接重置了，更像“连接被粗暴打断”

前者多关注：

- 网络不通
- 防火墙
- 上游响应慢
- 超时配置过小

后者多关注：

- 对端进程异常退出
- 上游服务主动关闭
- 网关或 LB 中途断开
- 协议不匹配

项目化回答模板

Q：如果线上说“页面很慢”，你会怎么讲排查思路？

可以按这一套说：

1. 先确认是所有用户都慢，还是某个地区、某条链路慢
2. 看 DNS 是否正常，是否解析到了错误节点
3. 看 TCP/TLS 建连时间是否异常
4. 用 `curl` 或浏览器开发者工具看是首包慢、下载慢，还是前端渲染慢
5. 再看网关日志、应用日志、数据库或缓存是否变慢

这样回答会让面试官觉得你有“分层定位”的意识，而不是只会说“先看日志”。

速记版

排障题建议按这条线记：

- `nslookup/dig` 看 DNS
- `ping` 看可达性
- `tracert` 看路径
- `curl` 看 HTTP/HTTPS
- `ss/netstat` 看连接状态
- `502/503/504` 分清“回错了 / 不接单 / 回太慢”
- `TIME_WAIT` 看主动关闭多不多
- `CLOSE_WAIT` 看应用有没有把连接收干净

一句话面试版：

网络排查的核心不是背命令，而是顺着 DNS、连通性、端口、协议、应用这一条链路逐层缩小范围。

进阶内容

分布式理论

分布式理论是大厂面试的进阶面试题，面试官通常会问"CAP 理论"和"如何保证分布式事务"。

CAP理论

Q：什么是 CAP 理论？

面试官你好，CAP 理论是分布式系统的基础理论，由 Eric Brewer 提出。CAP 代表三个特性：

第一个是 C（Consistency，一致性），所有节点在同一时间看到的数据是一致的。比如我在节点 A 写入数据，立即在节点 B 读取，应该能读到最新的数据。

第二个是 A（Availability，可用性），系统在任何时候都能响应请求，不会出现请求超时或拒绝服务的情况。

第三个是 P（Partition Tolerance，分区容错性），系统在网络分区（部分节点无法通信）的情况下，仍然能够继续工作。

CAP 理论的核心：在分布式系统中，最多只能同时满足 CAP 中的两个特性，无法三者兼得。

为什么无法三者兼得：

在分布式系统中，网络分区是不可避免的（P 是必须满足的）。当网络分区发生时，我们只能在 C 和 A 之间做选择：

- **选择 CP（一致性 + 分区容错性）：**牺牲可用性。当网络分区发生时，为了保证一致性，系统会拒绝服务，直到网络恢复。比如 ZooKeeper、HBase。
- **选择 AP（可用性 + 分区容错性）：**牺牲一致性。当网络分区发生时，系统继续提供服务，但可能返回旧数据。比如 Cassandra、DynamoDB。

实际应用：

大多数分布式系统都是 AP 或 CP，没有完全的 CA 系统（因为 P 是必须的）。

- **金融系统：**通常选择 CP，保证数据一致性，宁可牺牲可用性。
- **社交网络：**通常选择 AP，保证可用性，允许短暂的数据不一致。

追问1：CAP 理论有什么局限性？

CAP 理论虽然经典，但也有一些局限性：

第一个是**过于绝对**。CAP 理论说只能选择两个，但实际上可以在 C 和 A 之间做权衡，不是非黑即白。

第二个是**没有考虑延迟**。CAP 理论只考虑了一致性、可用性、分区容错性，没有考虑延迟 (Latency) 。

第三个是**没有考虑一致性的程度**。一致性有强一致性、最终一致性等不同程度，CAP 理论没有区分。

所以后来提出了 **BASE 理论**，作为 CAP 理论的补充。

BASE理论

Q：什么是 BASE 理论？

面试官你好，BASE 理论是对 CAP 理论的补充，它是 **Basically Available（基本可用）、Soft State（软状态）、Eventually Consistent（最终一致性）** 的缩写。

第一个是 **BA（Basically Available，基本可用）**，系统在出现故障时，允许损失部分可用性，但核心功能仍然可用。比如系统响应时间变长、部分功能降级，但不会完全不可用。

第二个是 **S（Soft State，软状态）**，系统允许存在中间状态，数据可以在一段时间内不一致。比如订单状态从"待支付"到"已支付"，中间可能有一个"支付中"的状态。

第三个是 **E（Eventually Consistent，最终一致性）**，系统不要求数据实时一致，但保证在一段时间后，数据最终会达到一致状态。

BASE 理论的核心思想：

牺牲强一致性，换取更高的可用性和性能。这是对 CAP 理论中 AP 的具体实现。

实际应用：

大多数互联网应用都采用 BASE 理论，比如：

- **电商系统**：下单后，库存可能不会立即扣减，而是异步扣减，最终保证一致。
- **社交网络**：点赞数、评论数可能不是实时更新的，但最终会一致。
- **分布式缓存**：缓存和数据库的数据可能短暂不一致，但最终会一致。

分布式事务

Q: 什么是分布式事务? 有哪些解决方案?

面试官你好, 分布式事务是指事务的参与者、支持事务的服务器、资源服务器分布在不同的节点上, 需要保证多个节点的操作要么全部成功, 要么全部失败。

分布式事务的解决方案主要有以下几种:

第一种是 2PC (两阶段提交), 它分为准备阶段和提交阶段。协调者先询问所有参与者是否可以提交, 如果都同意, 再通知所有参与者提交; 如果有一个拒绝, 就通知所有参与者回滚。

优点: 实现简单, 保证强一致性。

缺点: 性能差, 同步阻塞, 协调者是单点故障。

第二种是 3PC (三阶段提交), 它是 2PC 的改进版, 增加了超时机制和 CanCommit 阶段, 减少了阻塞时间。

第三种是 TCC (Try-Confirm-Cancel), 它分为三个阶段: Try (尝试执行)、Confirm (确认执行)、Cancel (取消执行)。每个服务需要实现这三个接口。

优点: 性能好, 不需要长时间锁定资源。

缺点: 实现复杂, 需要业务代码配合。

第四种是 Saga, 它是一种长事务解决方案, 将一个分布式事务拆分成多个本地事务, 每个本地事务都有一个补偿事务。如果某个本地事务失败, 就执行前面所有事务的补偿事务, 回滚到初始状态。

优点: 适合长事务, 性能好。

缺点: 实现复杂, 需要设计补偿逻辑。

第五种是本地消息表, 它是一种基于消息的最终一致性方案。在本地事务中, 同时写入业务数据和消息表, 然后异步发送消息到消息队列, 其他服务消费消息, 执行本地事务。

优点: 实现简单, 性能好。

缺点: 只能保证最终一致性, 不能保证强一致性。

第六种是 MQ 事务消息, 比如 RocketMQ 的事务消息。它保证消息的发送和本地事务的执行是原子的。

追问1: 2PC 有什么问题?

2PC 主要有三个问题:

第一个是同步阻塞。在准备阶段，所有参与者都需要等待协调者的指令，期间资源被锁定，无法处理其他请求，性能很差。

第二个是单点故障。协调者是单点，如果协调者宕机，整个事务就无法继续，所有参与者都会一直等待。

第三个是数据不一致。在提交阶段，如果协调者发送提交指令后宕机，部分参与者收到指令提交了，部分参与者没收到指令，就会导致数据不一致。

所以 2PC 在实际应用中很少使用，通常使用 TCC、Saga 或消息队列来实现分布式事务。

分布式锁

Q：什么是分布式锁？如何实现？

面试官你好，分布式锁是指在分布式系统中，多个节点需要访问共享资源时，通过锁机制保证同一时间只有一个节点能访问资源。

分布式锁的实现方式主要有以下几种：

第一种是基于数据库，利用数据库的唯一索引或行锁来实现。

优点：实现简单，不需要额外的组件。

缺点：性能差，数据库压力大，不支持锁的自动过期。

第二种是基于 Redis，利用 Redis 的 SETNX 命令来实现。

优点：性能好，支持锁的自动过期，实现简单。

缺点：在 Redis 主从架构下，可能出现锁丢失的问题（主节点宕机，从节点还没同步锁数据）。

第三种是基于 ZooKeeper，利用 ZooKeeper 的临时顺序节点来实现。

优点：可靠性高，支持锁的自动释放（客户端断开连接，临时节点自动删除）。

缺点：性能比 Redis 差，实现复杂。

第四种是基于 Redisson，它是 Redis 的 Java 客户端，提供了分布式锁的实现，支持锁的自动续期（看门狗机制）。

推荐使用 Redisson，因为它实现了看门狗机制，可以自动续期，避免锁过期导致的问题。

追问1：Redis 分布式锁有什么问题？如何解决？

Redis 分布式锁主要有以下几个问题：

第一个是锁过期问题。如果业务执行时间超过锁的过期时间，锁会自动释放，其他线程可能获取到锁，导致并发问题。

解决方案：使用 Redisson 的看门狗机制，自动续期。

第二个是锁误删问题。线程 A 获取锁后，锁过期了，线程 B 获取到锁，然后线程 A 执行完业务，删除了线程 B 的锁。

解决方案：释放锁时，先判断锁是否是自己的（通过唯一标识），再删除。使用 Lua 脚本保证原子性。

第三个是主从同步问题。在 Redis 主从架构下，主节点获取锁后，还没同步到从节点就宕机了，从节点升级为主节点，其他线程可能获取到锁。

解决方案：使用 Redlock 算法，在多个独立的 Redis 节点上获取锁，只有在大多数节点都获取成功时，才认为获取锁成功。

分布式ID

Q：如何生成分布式 ID？

面试官你好，分布式 ID 需要满足以下几个特性：全局唯一、趋势递增、高性能、高可用。

常见的分布式 ID 生成方案：

第一种是 UUID，它是通用唯一识别码，由 128 位组成。

优点：实现简单，性能高，本地生成，不需要网络请求。

缺点：无序，不适合作为数据库主键（影响索引性能）；占用空间大（36 个字符）。

第二种是数据库自增 ID，利用数据库的 AUTO_INCREMENT 特性。

优点：实现简单，有序递增。

缺点：性能差，数据库压力大；单点故障；扩展性差。

第三种是 Snowflake（雪花算法），它是 Twitter 开源的分布式 ID 生成算法，生成 64 位的 Long 型 ID。

结构：

- 1 位：符号位，固定为 0。
- 41 位：时间戳（毫秒级），可以使用 69 年。
- 10 位：机器 ID（5 位数据中心 ID + 5 位机器 ID），支持 1024 台机器。
- 12 位：序列号，同一毫秒内可以生成 4096 个 ID。

优点：性能高（每秒可以生成 400 万个 ID），有序递增，占用空间小（8 字节）。

缺点：依赖系统时钟，如果时钟回拨，可能生成重复 ID。

第四种是 Redis，利用 Redis 的 INCR 命令生成自增 ID。

优点：性能高，实现简单。

缺点：依赖 Redis，单点故障；不支持趋势递增（只是递增，不是按时间递增）。

第五种是美团的 Leaf，它提供了两种模式：Leaf-segment（号段模式）和 Leaf-snowflake（雪花算法模式）。

推荐使用 Snowflake 算法，因为它性能高、有序递增、占用空间小，适合大多数场景。

追问1：Snowflake 的时钟回拨问题如何解决？

时钟回拨是指系统时间被调整到过去的时间，可能导致生成重复的 ID。

解决方案：

第一种是**拒绝生成 ID**。检测到时钟回拨时，拒绝生成 ID，抛出异常，等待时钟恢复。

第二种是**等待时钟追上**。检测到时钟回拨时，等待一段时间，直到时钟追上上次生成 ID 的时间。

第三种是**使用备用机器 ID**。检测到时钟回拨时，切换到备用的机器 ID，继续生成 ID。

第四种是**使用逻辑时钟**。不依赖系统时钟，使用逻辑时钟（如 Lamport 时钟），保证 ID 的单调递增。

分布式Session

Q：如何实现分布式 Session？

面试官你好，在分布式系统中，用户的请求可能被负载均衡到不同的服务器，如果 Session 存储在单个服务器的内存中，就会导致 Session 丢失。

分布式 Session 的解决方案：

第一种是 Session 复制，将 Session 复制到所有服务器。

优点：实现简单，不需要额外的组件。

缺点：占用大量内存和网络带宽，扩展性差。

第二种是 Session 粘滞（Sticky Session），通过负载均衡器，将同一个用户的请求始终路由到同一台服务器。

优点：实现简单，不需要修改代码。

缺点：如果服务器宕机，Session 会丢失；负载不均衡。

第三种是 Session 集中存储，将 Session 存储在 Redis、Memcached 等缓存中间件中。

优点：扩展性好，高可用，性能好。

缺点：需要额外的组件，增加了系统复杂度。

第四种是 Token（JWT），不使用 Session，而是使用 Token。用户登录后，服务器生成一个 Token，客户端每次请求都携带 Token，服务器验证 Token 的有效性。

优点：无状态，扩展性好，不需要存储 Session。

缺点：Token 无法主动失效（除非设置过期时间）；Token 泄露风险。

推荐使用 Redis 存储 Session 或 JWT Token，因为它们扩展性好，性能高。

追问1：JWT 有什么优缺点？

JWT（JSON Web Token）是一种无状态的认证方式。

优点：

第一个是**无状态**。服务器不需要存储 Session，减少了服务器的压力，扩展性好。

第二个是**跨域友好**。JWT 可以在不同的域之间传递，适合微服务架构。

第三个是**性能好**。不需要查询数据库或缓存，直接验证 Token 的签名即可。

缺点：

第一个是**无法主动失效**。JWT 一旦签发，在过期之前一直有效，无法主动撤销。如果用户退出登录或修改密码，Token 仍然有效。

解决方案：设置较短的过期时间，或者维护一个黑名单（但这样就失去了无状态的优势）。

第二个是**Token 泄露风险**。如果 Token 被窃取，攻击者可以冒充用户。

解决方案：使用 HTTPS 传输，设置较短的过期时间，使用 Refresh Token 机制。

第三个是**Token 体积大**。JWT 包含了用户信息和签名，体积比 Session ID 大，增加了网络传输量。

微服务调用链日志追踪分析

Q：微服务场景下，怎么做调用链追踪？

在微服务里，一个请求往往会经过网关、订单服务、库存服务、支付服务等多个节点。如果没有统一的 Traceld，就很难把一次请求完整串起来。

所以调用链追踪的核心做法是：

- 给每次请求分配一个全局 Traceld
- 在服务调用链路中透传这个 Traceld
- 让日志、监控、链路平台围绕这个 Traceld 聚合

常见方案包括：

- Sleuth + Zipkin
- SkyWalking
- OpenTelemetry

负载均衡

Q：什么是负载均衡？常见策略有哪些？

负载均衡的目的，是把请求尽量均匀地分发到多个服务节点上，避免某个节点被打爆。

常见策略包括：

- 轮询
- 加权轮询
- 随机
- 一致性哈希
- 最少连接

如果是通用业务流量，轮询和加权轮询很常见；

如果是缓存、分片这类需要“同一类请求尽量落同一节点”的场景，一致性哈希更合适。

微服务架构与拆分

【进阶】微服务架构与拆分

微服务是进阶面试里很容易被追问的一块。

这里最怕的不是“不知道”，而是把“拆服务”说成一句空话，所以一定要能讲出拆分依据、服务边界和落地代价。

什么是微服务

Q：什么是微服务？

微服务可以理解成：

把一个大系统拆成多个围绕业务能力组织的小服务，每个服务独立开发、部署、扩容。

它强调的不是“服务一定要很小”，而是：

- 边界清晰
- 职责单一
- 可以独立演进

所以面试里更稳的表述是：

微服务本质上是按业务能力做拆分，用分布式协作替代单体内部调用。

为什么要拆微服务

Q：什么时候会从单体演进到微服务？

常见触发点有四类：

- 团队变大，单体代码协作成本太高
- 某些模块流量特别大，需要独立扩容
- 不同业务节奏差异大，希望独立发布
- 单体模块耦合严重，改一个地方容易影响全局

但注意，不是所有系统都该拆。

如果业务还很早期、团队不大、复杂度不高，单体反而更快。

追问1：微服务一定比单体好吗？

不一定。

单体的优势是开发简单、调用链短、事务天然好处理。

微服务只有在系统规模、团队规模、复杂度都上来之后，收益才会显现。

微服务怎么拆

Q：微服务拆分一般按什么维度来做？

最常见的拆分维度有三个：

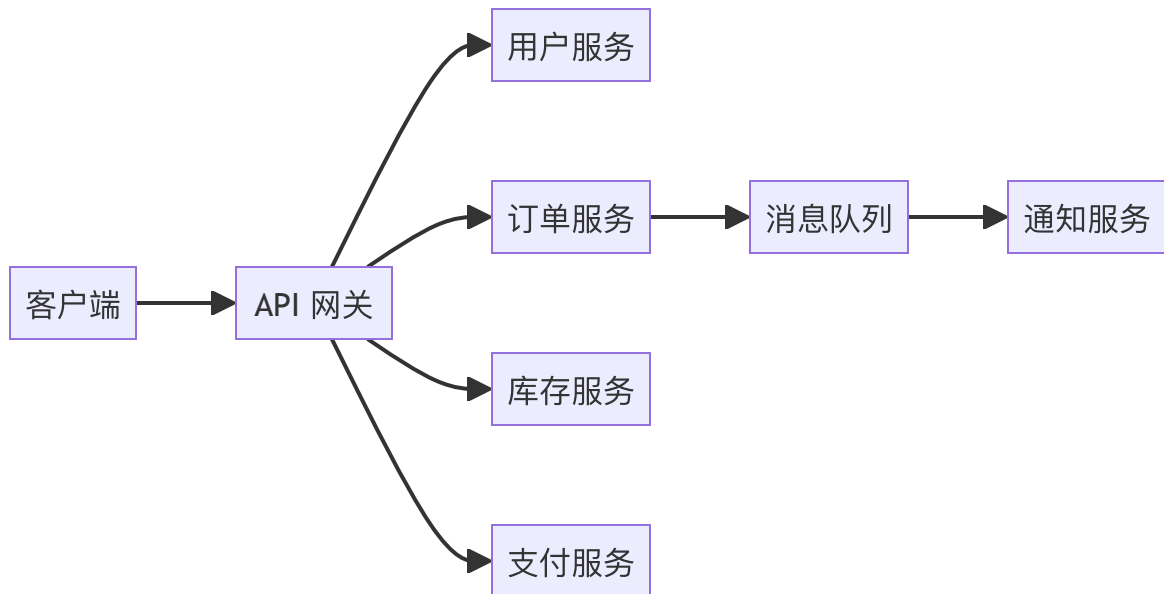
- 按业务领域拆，比如用户、订单、库存、支付
- 按核心链路拆，比如交易链路、营销链路、履约链路
- 按通用能力拆，比如搜索、推荐、消息通知

最不推荐的，是按技术层拆，比如“Controller 服务”“Service 服务”“DAO 服务”。

因为这种拆法没有业务闭环，只会制造大量跨服务调用。

面试时最好强调一句：

服务拆分不是按代码层次拆，而是按业务边界拆。



服务粒度怎么控制

Q：服务是不是越小越好？

不是。

服务太大，会退化成“小单体”；服务太小，会导致：

- 调用链变长
- 维护成本上升
- 事务变复杂
- 排查困难

判断粒度是否合适，可以看三个点：

- 服务是否有清晰业务边界
- 大多数高频调用是否还在服务内部
- 这个服务是否真的有独立扩容和独立发布价值

微服务之间怎么通信

Q：微服务之间常见的通信方式有哪些？

常见方式主要有两类：

- 同步调用：HTTP / RPC
- 异步调用：MQ / 事件驱动

选择思路通常是：

- 结果要立刻拿到，用同步
- 解耦、削峰、异步通知，用消息

很多系统不是二选一，而是混合使用：

- 下单时同步调用库存校验
- 下单成功后异步发消息做积分、通知、埋点

微服务为什么更强调无状态

Q：为什么微服务里经常强调服务无状态？

因为服务一旦无状态：

- 可以随时水平扩容
- 请求落到任意节点都能处理
- 节点挂掉时影响更小

如果会话状态放在应用本地内存，就会带来：

- 扩容困难
- Session 丢失
- 节点迁移麻烦

所以微服务里常见方案是：

- Session 集中存储到 Redis
- 直接改成 Token / JWT

微服务拆分后最常见的问题

Q：单体拆成微服务后，最容易踩什么坑？

最常见的问题有：

- 服务边界模糊，互相直连，调用网变蜘蛛网
- 一个请求串很多服务，链路太长
- 分布式事务处理不好
- 日志、监控、追踪没跟上，出了问题查不出来
- 组织结构没变，技术拆了但协作方式还是单体时代

所以微服务不是只拆代码，还要配套：

- 服务治理
- 链路追踪
- 配置中心
- 灰度发布
- 容错降级

项目里怎么说

Q：如果面试官问你项目里为什么拆微服务，你怎么答？

可以按下面的结构说：

1. 先讲原来的瓶颈，比如单体模块耦合、发布风险高、订单流量压力大
2. 再讲拆分依据，比如按用户、订单、库存、支付这些业务边界拆
3. 再讲拆分后的问题，比如事务、调用链、治理成本
4. 最后讲你的配套方案，比如注册发现、网关、MQ 解耦、链路追踪

这样会让面试官觉得你不是只会背“微服务优点”，而是真的知道拆分后会多什么麻烦。

高频追问补充

追问1：拆数据库和拆服务谁先做？

多数情况下先按服务边界拆业务能力，再逐步推动数据隔离。

如果一上来强行完全拆库，但边界没梳理好，反而容易把复杂度提前引爆。

追问2：微服务是不是一定要上 Spring Cloud 或 Kubernetes？

不一定。

这些只是治理工具，不是微服务本身。

即使不用这些框架，只要系统按业务能力拆分、独立部署和协作，本质上也是微服务。

DDD与领域建模

DDD 很容易被说得特别虚，所以这篇的目标不是讲“术语大全”，而是把它还原成一套更有工程感的建模方法。

什么是 DDD

Q: 什么是 DDD?

DDD 是 `Domain-Driven Design`，中文通常叫领域驱动设计。

它的核心不是某种框架，而是：

- 先理解业务领域
- 再围绕业务概念建模
- 最后让代码结构尽量贴合业务结构

所以 DDD 不是“让代码更复杂”，而是当业务足够复杂时，用更清晰的模型去承载复杂度。

什么是统一语言

Q: DDD 里为什么一直强调统一语言？

统一语言指的是：

产品、业务、开发、测试讨论同一个业务对象时，尽量使用同一套词汇。

比如：

- 是叫“订单关闭”还是“订单取消”
- 是叫“履约”还是“发货”
- 是叫“支付成功”还是“记账成功”

如果语言不统一，代码里的命名和业务语义就会越来越混乱。

DDD 很强调把这套语言直接沉淀到代码模型里。

什么是限界上下文

Q：什么是限界上下文？

同一个词，在不同业务边界里含义可能不同。

限界上下文就是给这些概念划边界。

比如“订单”这个词：

- 在交易域里，它强调下单和支付
- 在履约域里，它强调发货和签收
- 在售后域里，它强调退款和退货

所以 DDD 会强调：

不要试图用一个超级大对象表示全世界的“订单”，而要在不同上下文里定义自己的模型。

实体、值对象、聚合

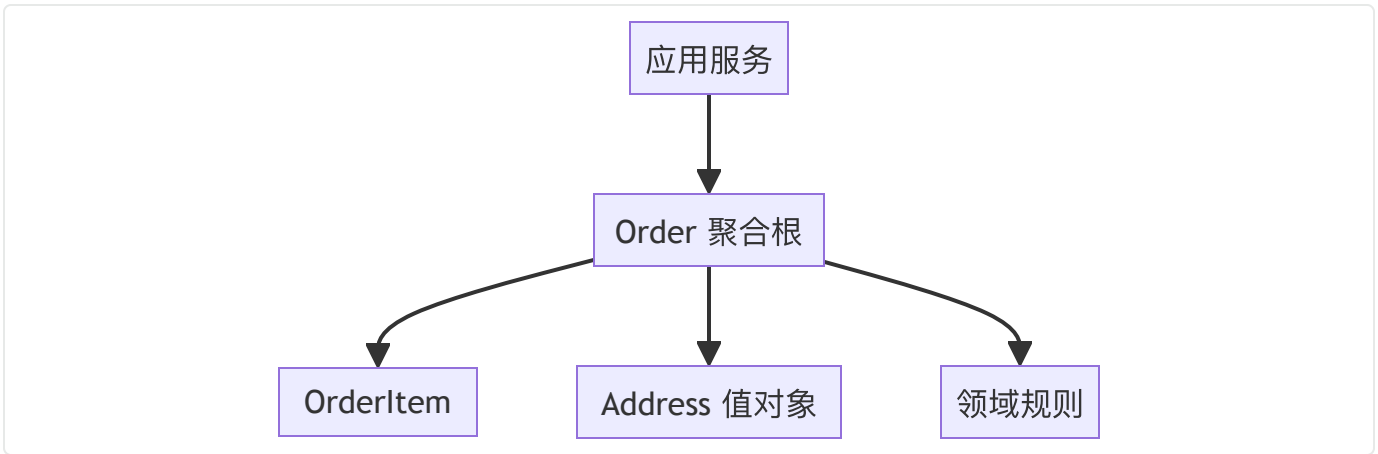
Q：实体、值对象、聚合分别是什么？

这几个是 DDD 高频基础概念。

- **实体 (Entity)**：有唯一标识，关注的是“它是谁”
- **值对象 (Value Object)**：没有唯一标识，关注的是“它是什么值”
- **聚合 (Aggregate)**：一组强关联对象的边界，保证内部一致性
- **聚合根 (Aggregate Root)**：聚合对外暴露的唯一入口

比如订单场景里：

- **Order** 是实体，也是聚合根
- 收货地址可以设计成值对象
- 订单项可能属于订单聚合内部成员



仓储、领域服务、应用服务怎么区分

Q: 仓储、领域服务、应用服务分别做什么？

这几个概念特别容易混。

- 仓储 (Repository) : 负责聚合的持久化和重建
- 领域服务 (Domain Service) : 放不适合放进实体/值对象的领域规则
- 应用服务 (Application Service) : 负责编排流程、事务边界、调用外部系统

简单理解：

- 领域层放“业务规则”
- 应用层放“流程编排”

DDD 适合什么场景

Q: 是不是所有项目都适合上 DDD？

不是。

DDD 更适合：

- 业务规则复杂
- 概念多且容易混
- 团队规模较大
- 系统生命周期长

如果只是 CRUD 后台、流程很简单的小系统，硬上 DDD 往往只会增加理解成本。

所以面试里一定不要说“DDD 适用于一切项目”。

贫血模型和充血模型

Q：什么是贫血模型，为什么 DDD 不太喜欢它？

贫血模型指的是：

- 实体里只有字段和 getter/setter
- 所有业务规则都堆在 Service 里

这样代码虽然看起来分层了，但业务规则并没有真正沉淀到模型中。

DDD 更希望模型本身能表达业务约束，也就是更偏“充血模型”。

但也不要绝对化。

有些团队会采用折中方案：关键聚合做充血，外围模块保持简单。

项目里怎么说

Q：如果面试官问你项目里有没有用 DDD，你怎么答？

最稳的答法不是说“我们全量 DDD”，而是说：

1. 哪些复杂业务模块用了领域建模思路
2. 怎么划分上下文和聚合边界
3. 哪些规则沉淀到了领域对象里
4. 哪些模块没用，为什么没必要

这种回答会比“我们项目用了 DDD 框架”更有说服力。

高频追问补充

追问1：DDD 和微服务是什么关系？

DDD 不是微服务，但它常常能为微服务拆分提供边界参考。

很多团队会先做领域划分，再决定服务边界。

追问2：DDD 一定要分很多层吗？

不一定。

核心是建模是否贴合业务，而不是目录一定要分成多少层。

补充

建议看下我项目笔记里面大营销里关于DDD的面试题。

trace与可观测

系统一旦拆成多个服务，排查问题就不再是“看一台机器的日志”这么简单了。这时候链路追踪和可观测性就会变成进阶面试的高频题。

什么是可观测性

Q：什么是可观测性？

可观测性可以理解成：

当系统出现异常时，我们能否从外部输出快速判断它内部到底发生了什么。

常见的三大支柱是：

- **Metrics**：指标
- **Logs**：日志
- **Traces**：链路追踪

它们分别回答不同问题：

- 指标告诉你“哪里不正常”
- 日志告诉你“发生了什么”
- 链路告诉你“问题是沿着哪条调用链传播的”

什么是链路追踪

Q：什么是调用链追踪？

调用链追踪的目标是：

把一次请求在多个服务之间的完整路径串起来。

比如一个请求可能经过：

- 网关
- 订单服务
- 库存服务
- 支付服务

如果没有统一的 `TraceId`，就很难把这些节点上的日志和耗时串到一起。

所以链路追踪的核心是：

- 为一次请求分配全局 `TraceId`
- 每个调用节点生成自己的 `Span`
- 在跨线程、跨服务调用时持续透传上下文

TraceId 和 SpanId 是什么

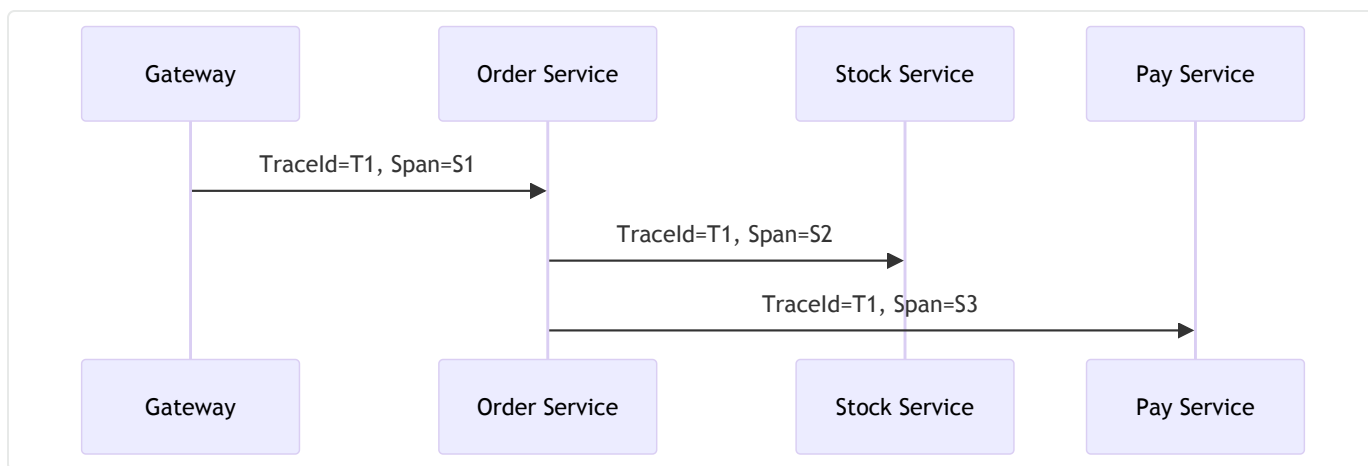
Q: `TraceId` 和 `SpanId` 分别是什么？

- `TraceId`：标识一整条调用链
- `SpanId`：标识链路上的某一个调用片段

可以把它理解成：

- `TraceId` 是“整条树”
- `SpanId` 是“树上的一个节点”

父子 `Span` 之间会形成调用层级，这样你就能看到在一个请求在各个服务、各个方法、各个中间件里的耗时分布。



上下文传播为什么重要

Q：为什么调用链系统一定要强调上下文传播？

因为链路追踪不是只在一个方法里打点，而是要把同一次请求在：

- HTTP 调用
- RPC 调用
- MQ 消费
- 线程池切换

这些场景里都串起来。

如果上下文没有正确透传，就会出现：

- Trace 断链
- 日志关联不上
- 一个请求被拆成多段孤立链路

常见方案有哪些

Q：Java 项目里常见的链路追踪方案有哪些？

常见方案包括：

- Zipkin
- SkyWalking
- OpenTelemetry

简单理解：

- **Zipkin**：经典链路方案，概念直观
- **SkyWalking**：Java 场景里很常见，APM 能力比较完整
- **OpenTelemetry**：现在越来越通用，强调标准化埋点和生态兼容

如果面试官继续追，Java 场景里还可以补一句：

OpenTelemetry 官方更推荐从 Java Agent 这类自动埋点方式入手，再按需补手工埋点。

日志、指标、链路怎么关联

Q：日志、指标、链路三者怎么配合？

实际排障时常见链路是：

1. 先看指标，发现错误率、RT、QPS 异常
2. 再点进 Trace，看慢在了哪一跳
3. 最后顺着 TraceId 去日志里找详细异常栈

所以很多团队会做：

- 日志自动打印 TraceId
- 监控面板支持从指标跳转 Trace
- Trace 再跳转到对应日志

这才叫完整闭环。

只打日志为什么不够

Q：为什么不能只靠日志排查？

因为分布式系统里，一个请求可能跨 5 到 10 个服务。

只看日志会有几个问题：

- 很难手工拼出完整调用链
- 很难快速知道到底哪一跳慢
- 线程池、异步任务、消息消费会让排查更碎

所以链路追踪不是替代日志，而是用来把日志组织起来。

项目里怎么说

Q：如果面试官问你项目里怎么做调用链追踪，你怎么答？

可以按下面的顺序回答：

1. 网关入口生成或接收 Traceld
2. 服务间通过 HTTP Header、RPC 附件、MQ Header 透传
3. 日志打印 Traceld，便于检索
4. 通过 SkyWalking 或 OpenTelemetry 统一汇总链路
5. 把指标、日志、链路打通，用于排障和性能分析

如果你能再补一句“线程池和异步场景里要特别注意上下文传递”，会更像真实项目经验。

高频追问补充

追问1：链路追踪会不会影响性能？

会有一定开销，所以通常要配合采样策略。

高流量场景下不可能所有请求都 100% 全量采集。

追问2：MQ 场景下怎么不断链？

关键是把 Trace 上下文放到消息头里，生产端写入，消费端继续解析并创建子 Span。

高并发系统设计

很多进阶面试最后都会落到方案设计题。

真正拉开差距的，不是你能不能说出“Redis、MQ、分库分表”这些词，而是你能不能把它们按问题顺序组织起来。

高并发设计先看什么

Q：拿到一个高并发设计题，第一步看什么？

不要一上来就说技术栈，先看问题本身：

- 读多还是写多
- 峰值 QPS 有多大
- 是否允许异步
- 数据一致性要求多高
- 热点是否集中
- 核心瓶颈在 CPU、IO、数据库还是下游依赖

面试里最加分的点是：

先抽象业务约束，再谈架构方案。

高并发系统常见手段

Q：高并发系统常见优化手段有哪些？

常见手段可以按层次理解：

- 流量层：CDN、网关、限流
- 计算层：无状态扩容、多实例部署
- 缓存层：本地缓存、Redis、热点隔离
- 存储层：读写分离、分库分表
- 异步层：MQ 削峰填谷
- 业务层：幂等、降级、兜底

不要把这些手段当“固定套餐”，而要看瓶颈在哪里。

秒杀系统怎么讲

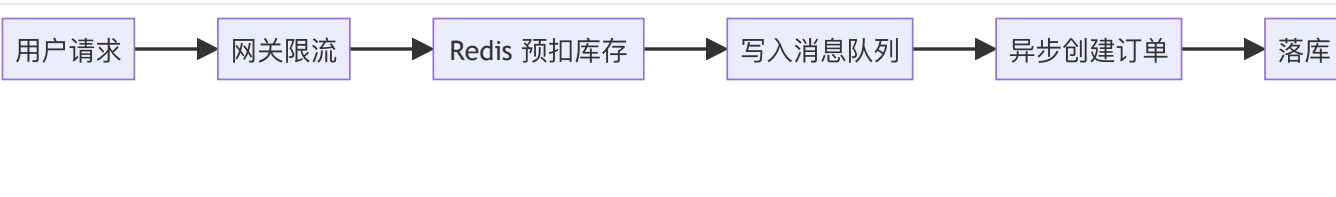
Q：秒杀系统设计题一般怎么答？

秒杀题最经典的思路是：

1. 流量拦截在前面：验证码、限流、排队、静态化
2. 库存判断前移：Redis 预扣减
3. 下单流程异步化：请求写入 MQ
4. 数据最终落库：异步创建订单
5. 防止超卖和重复下单：幂等 + 状态校验

核心目标通常有三个：

- 抗瞬时高峰
- 防止库存超卖
- 保证系统不被打穿



为什么要削峰填谷

Q：什么叫削峰填谷？

削峰填谷的本质是：

把瞬时大量请求先缓冲起来，再按系统能承受的速度慢慢处理。

消息队列就是非常典型的工具。

它的价值在于：

- 让数据库和核心服务不直接承受峰值洪峰
- 把同步高峰请求改成异步处理
- 为重试、失败转移、补偿提供空间

热点问题怎么处理

Q：高并发系统里的热点问题怎么处理？

热点通常表现为：

- 某个商品被疯狂访问
- 某个 key 被集中打爆
- 某张表或某行数据成为争抢点

常见处理方式：

- 多级缓存
- 热点 key 拆分
- 本地缓存 + Redis
- 请求合并
- 分桶、分片
- 降级和兜底

性能优化一般从哪里下手

Q：接口性能优化一般怎么做？

推荐按“先定位、再优化”的思路来讲：

1. 先看监控，定位是 CPU、内存、IO、锁竞争还是数据库慢
2. 再看调用链，判断慢在哪一层
3. 最后按瓶颈选择优化手段

常见方向有：

- 减少不必要的远程调用
- 缓存热点数据
- 异步化非核心流程
- SQL 和索引优化
- 批量化、池化、并行化

设计题回答框架

Q：面试里方案设计题怎么组织回答更稳？

可以固定用下面这个框架：

1. 明确业务目标和约束
2. 估算流量、数据量、峰值
3. 识别核心瓶颈
4. 给出整体架构
5. 解释关键模块和关键取舍
6. 补充一致性、可用性、容灾、监控、降级

这样回答的好处是，不容易一上来就散。

项目里怎么说

Q：如果面试官让你讲一次高并发优化经历，你怎么答？

建议按“问题 -> 定位 -> 方案 -> 结果”来讲：

- 问题是什么，比如大促时订单接口 RT 飙升
- 怎么定位，比如慢 SQL、线程池打满、下游依赖抖动
- 采取了什么方案，比如缓存、限流、异步化、SQL 优化
- 效果怎么样，比如 RT 从多少降到多少、错误率下降多少

尽量把故事讲成闭环，而不是罗列技术词。

高频追问补充

追问1：是不是所有高并发问题都要上分库分表？

不是。

很多时候问题根本不在存储层，而在缓存、调用链、线程模型、下游依赖。

追问2：异步化是不是万能解法？

不是。

异步会带来最终一致、排查复杂、重试补偿、顺序性等新问题。

服务治理与高可用

系统拆成微服务之后，真正难的部分不是“能调用”，而是“在流量波动、节点故障、依赖超时的时候还能稳住”。

这篇主要解决的就是服务治理和高可用问题。

什么是服务治理

Q：什么是服务治理？

服务治理可以理解成：

围绕服务注册、发现、路由、容错、限流、降级、监控等问题建立一整套运行机制。

简单说，微服务解决的是“怎么拆”，服务治理解决的是“拆完怎么稳”。

注册发现是做什么的

Q：为什么微服务里需要注册中心？

因为服务实例通常会动态扩缩容，IP 和端口经常变化。

如果服务之间把地址写死，维护成本会非常高。

注册中心的职责通常包括：

- 服务注册
- 服务发现
- 健康检查
- 实例上下线通知

常见实现有：

- Eureka
- Nacos
- Consul
- ZooKeeper

负载均衡策略

Q：常见负载均衡策略有哪些？

常见策略包括：

- 轮询
- 加权轮询
- 随机
- 最少连接
- 一致性哈希

选择思路：

- 普通无状态流量，多数场景用轮询或加权轮询
- 对会话、缓存路由、分片定位有要求时，一致性哈希更合适

超时、重试、熔断、降级是什么关系

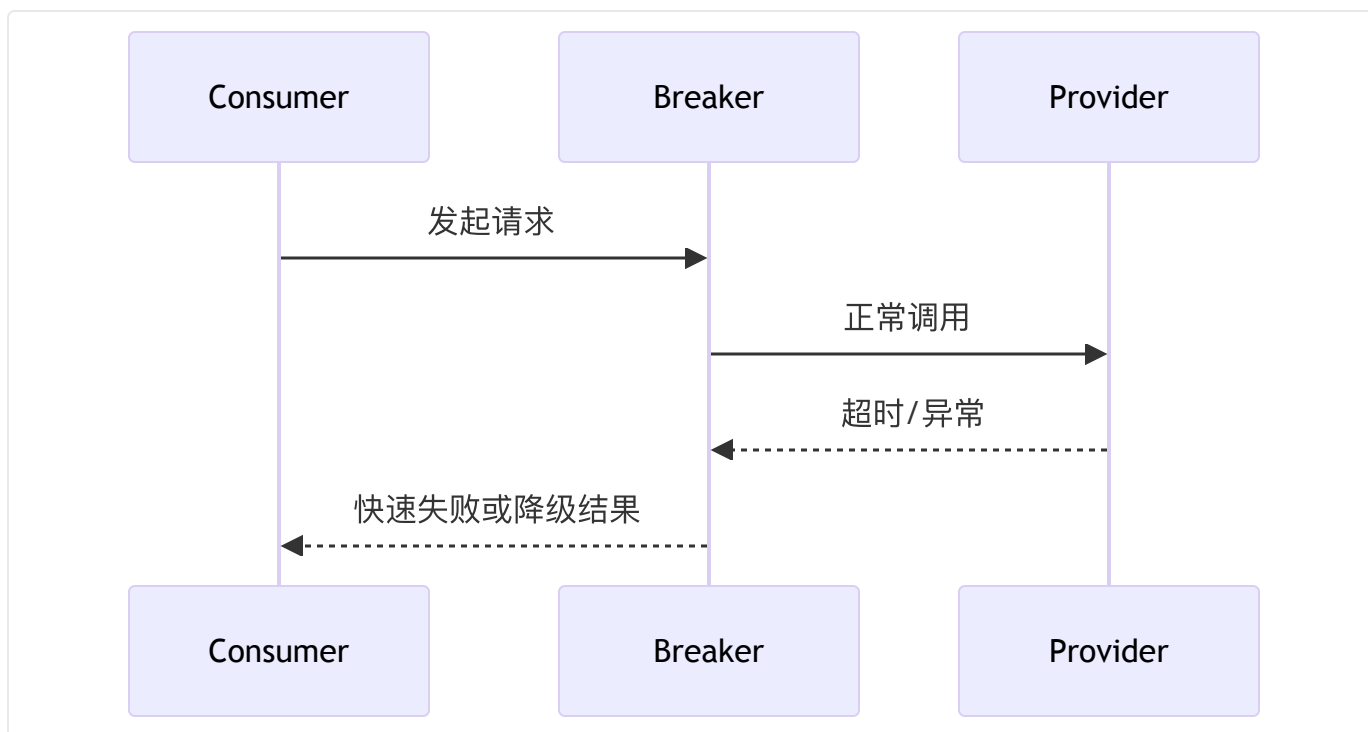
Q：超时、重试、熔断、降级分别是什么？

这几个概念经常一起问，建议按“控制风险传播”的逻辑来回答。

- **超时**：调用不能无限等，超过阈值就结束
- **重试**：请求失败后再尝试几次
- **熔断**：检测到失败率过高时，临时拒绝继续调用故障服务
- **降级**：核心链路优先，非核心能力返回兜底结果

它们的关系是：

超时控制等待成本，重试提高成功率，熔断防止雪崩，降级保证核心功能。



为什么重试有风险

Q：请求失败重试不是更稳吗，为什么还有风险？

重试不是越多越好。

如果下游本来就已经很慢或快挂了，盲目重试会把压力再放大一轮。

所以重试一定要配合：

- 明确的超时
- 幂等保障
- 限制重试次数
- 指数退避

面试时最好顺带补一句：

对写请求做重试时，要特别注意幂等问题。

什么是限流和隔离

Q：限流和隔离分别解决什么问题？

限流 解决的是“流量太大，系统扛不住”的问题。

隔离 解决的是“某类请求把整个系统资源耗尽”的问题。

常见限流算法：

- 计数器
- 滑动窗口
- 漏桶
- 令牌桶

常见隔离思路：

- 线程池隔离
- 信号量隔离
- 资源池隔离
- 核心和非核心链路隔离

什么是高可用

Q：高可用一般怎么理解？

高可用不是一句“有主有备”就够了，它至少包含：

- 节点故障能自动摘除
- 单点故障不会导致全站不可用
- 流量高峰时系统有保护机制
- 发布、扩容、故障恢复过程中业务可连续运行

常见手段包括：

- 多实例部署
- 主从/主备
- 多机房或多可用区部署
- 健康检查和自动摘流
- 限流、降级、熔断

灰度发布和健康检查

Q：为什么灰度发布也是服务治理的一部分？

因为很多事故不是机器坏了，而是“新版本上线把自己搞挂了”。

灰度发布的核心价值是：

- 小流量先验证
- 快速回滚
- 降低全量发布风险

健康检查则用于判断一个实例是否还能继续接流量，常见有：

- 存活检查
- 就绪检查
- 依赖检查

项目里怎么说

Q：如果面试官问你怎么做服务治理，你怎么答？

建议按“调用稳定性”来回答：

1. 用注册中心做服务发现和实例上下线
2. 用网关做统一路由、鉴权、限流
3. 服务间调用配置超时、重试、熔断、降级
4. 核心链路和非核心链路做隔离
5. 用监控和链路追踪持续观察错误率、RT、QPS

如果你能再补一句“重试必须建立在幂等前提上”，基本就不是背稿水平了。

高频追问补充

追问1：为什么限流和降级通常一起出现？

因为限流拦的是入口流量，降级保的是核心功能。

单独做其中一个，通常都不够。

追问2：注册中心自己挂了怎么办？

正常情况下会通过集群部署、持久化、客户端本地缓存、保护阈值等机制降低影响。

而且服务注册与发现通常要和业务调用解耦，不能让注册中心短暂抖动直接打爆业务链路。

分库分表

概念与分片键

概念与分片键

为什么要分库分表

Q: 什么时候需要分库分表?

当单库单表已经无法承载数据量或并发压力时，就要考虑分库分表。

典型触发点包括：

- 单表数据量太大
- 索引太大，查询性能下降
- 写入压力太高
- 单机资源达到瓶颈

什么是分片键

Q: 分片键应该怎么选?

分片键就是决定一条数据落到哪个库、哪张表的关键字段。

选择分片键时，通常希望它满足：

- 分布尽量均匀
- 查询场景高频使用
- 尽量避免热点

很多业务里常见会选：

- 用户 ID
- 订单 ID
- 商户 ID

查询与常见问题

查询与常见问题

非分片键查询怎么办

Q: 如果按用户 ID 分片，但现在要按订单状态查怎么办？

这就是典型的非分片键查询问题。

常见解决思路包括：

- 先查索引表再回源
- 建冗余字段或宽表
- 借助搜索引擎
- 业务上尽量避免跨片查询

所以分库分表不是银弹，它会把很多简单查询变复杂。

分库分表的常见问题

Q: 分库分表后会带来哪些问题？

常见问题包括：

- 分布式事务更复杂
- 跨库 join 变难
- 聚合查询更麻烦
- 分页和排序更复杂
- 扩容迁移成本高

所以分库分表一定是“业务量逼出来”的方案，不是项目一开始就该默认上。

分片算法

分片算法

常见分片算法

Q: 常见分片算法有哪些?

常见的有:

- 范围分片
- 取模分片
- 一致性哈希

范围分片 直观, 但容易热点集中。

取模分片 简单高效, 但扩容时数据迁移成本高。

一致性哈希 扩容更友好, 但实现会复杂一些。

一致性哈希适合什么场景

Q: 一致性哈希为什么适合扩容场景?

因为它能在节点变更时, 把需要迁移的数据量控制得更小。

这比普通取模分片“扩容几乎全量重算”的代价要低得多。

SpringMVC

SpringMVC

Spring MVC 最常见的考法就是“执行流程”和“核心组件”。

Spring MVC 的核心组件

Q: Spring MVC 有哪些核心组件?

最核心的组件有:

- `DispatcherServlet`
- `HandlerMapping`
- `HandlerAdapter`
- `HandlerInterceptor`
- `ViewResolver`

其中 `DispatcherServlet` 是前端控制器，负责统一接收请求，再协调后续组件完成处理。

Spring MVC 的执行流程

Q: Spring MVC 的执行流程是什么?

大致流程可以概括为：

1. 请求到达 `DispatcherServlet`
2. `DispatcherServlet` 通过 `HandlerMapping` 找到目标处理器
3. `HandlerAdapter` 调用具体 Controller 方法
4. 过程中完成参数绑定、类型转换、校验等逻辑
5. Controller 返回结果
6. 如果是前后端分离项目，通常通过 `HttpMessageConverter` 转成 JSON 返回

所以 Spring MVC 的核心思想就是：

前端控制器统一调度 + 可插拔组件链。

`@RequestBody` 的底层依赖

Q: `@RequestBody` 为什么能把 JSON 变成 Java 对象？

因为 Spring MVC 在处理请求时，会结合参数解析器和 `HttpMessageConverter`。

当请求体是 JSON 时，常见会交给 Jackson 对应的消息转换器去完成反序列化，把 JSON 转成 Java 对象。

SpringIOC

Spring IOC

Spring IOC 是 Spring 框架的核心，也是面试的必考题，面试官通常会问"什么是 IOC"，然后深挖 Bean 生命周期和循环依赖。

什么是IOC

Q: 什么是 IOC? 有什么好处?

IOC 全称是 **Inversion of Control** (控制反转)，它是一种设计思想。

传统的程序设计中，我们在对象内部直接创建依赖对象，对象的控制权在程序员手里。而 IOC 是把对象的创建和依赖关系的管理交给容器来做，对象只需要声明自己需要什么依赖，容器会自动注入进来。这就是"控制反转"的含义——对象的控制权从程序员手里反转到了容器手里。

IOC 的好处主要有三个：

第一个是**降低耦合度**。对象不需要自己创建依赖，只需要声明依赖，这样对象之间的耦合度就降低了。

第二个是**提高可维护性**。如果要替换一个依赖的实现，只需要修改配置，不需要修改代码。

第三个是**方便测试**。在单元测试时，可以很容易地注入 Mock 对象，不需要依赖真实的实现。

****DI (Dependency Injection, 依赖注入) ****是 IOC 的一种实现方式。Spring 通过 DI 来实现 IOC，把依赖的对象注入到需要的地方。

追问1: IOC 和 DI 有什么区别?

IOC 是一种设计思想，DI 是 IOC 的一种实现方式。

IOC 强调的是"控制权的反转"，把对象的创建和管理交给容器。

DI 强调的是"依赖的注入"，容器把依赖的对象注入到需要的地方。

可以说，DI 是实现 IOC 的手段，IOC 是 DI 的目的。

Spring IOC容器的实现

Q: Spring IOC 容器是如何实现的?

Spring IOC 容器的核心是 **BeanFactory** 和 **ApplicationContext**。

BeanFactory 是 Spring IOC 容器的顶层接口，它提供了最基本的 IOC 功能，比如获取 Bean、判断 Bean 是否存在等。**BeanFactory** 是**懒加载**的，只有在获取 Bean 时才会创建 Bean。

ApplicationContext 是 **BeanFactory** 的子接口，它在 **BeanFactory** 的基础上增加了很多企业级功能，比如国际化、事件发布、资源加载等。**ApplicationContext** 是**立即加载**的，容器启动时就会创建所有的单例 Bean。

常用的 **ApplicationContext** 实现类：

- **ClassPathXmlApplicationContext**：从类路径加载 XML 配置文件。
- **FileSystemXmlApplicationContext**：从文件系统加载 XML 配置文件。
- **AnnotationConfigApplicationContext**：基于注解的配置。

IOC 容器的工作流程：

1. **加载配置**：读取 XML 配置文件或扫描注解。
2. **解析配置**：把配置信息解析成 **BeanDefinition**（Bean 的定义信息）。
3. **注册 BeanDefinition**：把 **BeanDefinition** 注册到 **BeanFactory** 中。
4. **实例化 Bean**：根据 **BeanDefinition** 创建 Bean 实例。
5. **依赖注入**：把依赖的 Bean 注入到目标 Bean 中。
6. **初始化 Bean**：调用 Bean 的初始化方法。
7. **使用 Bean**：应用程序可以从容器中获取 Bean 使用。
8. **销毁 Bean**：容器关闭时，调用 Bean 的销毁方法。

追问1: **BeanFactory** 和 **ApplicationContext** 有什么区别?

主要有三个区别：

第一个是**加载时机不同**。**BeanFactory** 是懒加载，**ApplicationContext** 是立即加载。

第二个是**功能不同**。**BeanFactory** 只提供基本的 IOC 功能，**ApplicationContext** 提供了更多企业级功能（国际化、事件发布、AOP 等）。

第三个是**使用场景不同**。BeanFactory 适合资源受限的环境（如移动设备），ApplicationContext 适合企业级应用。

实际开发中，我们基本都用 ApplicationContext。

Bean的生命周期

Q：Spring Bean 的生命周期是怎样的？

Spring Bean 的生命周期可以分为四个阶段：

第一个阶段是实例化（Instantiation），Spring 通过反射调用 Bean 的构造方法创建 Bean 实例。

第二个阶段是属性赋值（Populate），Spring 把配置文件或注解中定义的属性值注入到 Bean 中，这就是依赖注入。

第三个阶段是初始化（Initialization），这个阶段会执行一系列的初始化操作：

1. 如果 Bean 实现了 `Aware` 接口（如 `BeanNameAware`、`ApplicationContextAware`），会调用相应的 `setXxx()` 方法。
2. 如果有 `BeanPostProcessor`，会调用它的 `postProcessBeforeInitialization()` 方法。
3. 如果 Bean 实现了 `InitializingBean` 接口，会调用 `afterPropertiesSet()` 方法。
4. 如果配置了 `init-method`，会调用自定义的初始化方法。
5. 如果有 `BeanPostProcessor`，会调用它的 `postProcessAfterInitialization()` 方法。

第四个阶段是销毁（Destruction），容器关闭时会销毁 Bean：

1. 如果 Bean 实现了 `DisposableBean` 接口，会调用 `destroy()` 方法。
2. 如果配置了 `destroy-method`，会调用自定义的销毁方法。

简化记忆：实例化 → 属性赋值 → 初始化 → 销毁。

追问1：BeanPostProcessor 有什么作用？

BeanPostProcessor（Bean 后置处理器）是 Spring 提供的一个扩展点，可以在 Bean 初始化前后插入自定义逻辑。

它有两个方法：

- `postProcessBeforeInitialization()`：在 Bean 初始化之前调用。
- `postProcessAfterInitialization()`：在 Bean 初始化之后调用。

典型应用：

第一个是 AOP。Spring AOP 就是通过 `BeanPostProcessor` 实现的，在 `postProcessAfterInitialization()` 中创建代理对象。

第二个是 `@Autowired` 注解。`AutowiredAnnotationBeanPostProcessor` 会在 `postProcessBeforeInitialization()` 中处理 `@Autowired` 注解，完成依赖注入。

第三个是 `@PostConstruct` 注解。`CommonAnnotationBeanPostProcessor` 会在初始化阶段调用 `@PostConstruct` 标注的方法。

循环依赖问题

Q：什么是循环依赖？Spring 如何解决循环依赖？

循环依赖是指两个或多个 Bean 相互依赖，形成一个闭环。比如 A 依赖 B，B 又依赖 A。

Spring 解决循环依赖的核心是三级缓存：

第一级缓存（`singletonObjects`）：存放完全初始化好的单例 Bean，也叫"单例池"。

第二级缓存（`earlySingletonObjects`）：存放早期暴露的 Bean，这些 Bean 已经实例化，但还没有完成属性赋值和初始化。

第三级缓存（`singletonFactories`）：存放 Bean 的工厂对象，用于创建早期 Bean。

解决循环依赖的流程（以 A 依赖 B，B 依赖 A 为例）：

1. 创建 A，先实例化 A，然后把 A 的工厂对象放入三级缓存。
2. 给 A 注入属性，发现依赖 B，去创建 B。
3. 创建 B，先实例化 B，然后把 B 的工厂对象放入三级缓存。
4. 给 B 注入属性，发现依赖 A，从三级缓存中获取 A 的工厂对象，创建 A 的早期引用，放入二级缓存，然后注入给 B。
5. B 完成初始化，放入一级缓存。
6. A 拿到完整的 B，完成初始化，放入一级缓存。

注意：Spring 只能解决单例 Bean 的 setter 注入的循环依赖，无法解决构造器注入和 prototype 类型的循环依赖。

追问1：为什么需要三级缓存？二级缓存不够吗？

这是一个很经典的问题。理论上，如果只是解决循环依赖，二级缓存就够了。

但 Spring 需要考虑 AOP 的情况。如果 Bean 需要被代理（比如有 @Transactional 注解），那么注入给其他 Bean 的应该是代理对象，而不是原始对象。

三级缓存的作用：

三级缓存存放的是 Bean 的工厂对象（ObjectFactory），这个工厂对象可以创建 Bean 的早期引用。如果 Bean 需要被代理，工厂对象会返回代理对象；如果不需要代理，工厂对象会返回原始对象。

这样，即使在循环依赖的情况下，注入的也是正确的对象（代理对象或原始对象）。

如果只有二级缓存，就无法处理 AOP 的情况，可能导致注入的是原始对象，而不是代理对象。

追问2：构造器注入为什么无法解决循环依赖？

因为构造器注入是在实例化阶段就需要依赖对象，而此时依赖对象还没有创建。

比如 A 的构造器依赖 B，B 的构造器依赖 A：

1. 创建 A，需要先创建 B（构造器参数）。
2. 创建 B，需要先创建 A（构造器参数）。
3. 陷入死循环。

而 setter 注入是在属性赋值阶段才需要依赖对象，此时对象已经实例化了，可以先暴露一个早期引用，打破循环。

解决方案：

- 改用 setter 注入。
- 使用 @Lazy 注解，延迟加载依赖。

依赖注入的方式

Q：Spring 有哪些依赖注入的方式？推荐哪种？

Spring 主要有三种依赖注入方式：

第一种是构造器注入，通过构造方法注入依赖。

优点：依赖是不可变的（final），保证了线程安全。而且依赖是必需的，如果缺少依赖，编译时就会报错。

缺点：如果依赖太多，构造方法会很长。而且无法解决循环依赖。

第二种是 setter 注入，通过 setter 方法注入依赖。

优点：可以解决循环依赖。而且依赖是可选的，可以有默认值。

缺点：依赖是可变的，不够安全。而且依赖可能为 null，需要做空指针检查。

第三种是字段注入，直接在字段上使用 @Autowired 注解。

优点：代码简洁，使用方便。

缺点：依赖不明显，不利于测试。而且无法注入 final 字段。

推荐使用构造器注入，因为它能保证依赖的不可变性和必需性，更加安全。Spring 官方也推荐使用构造器注入。

追问1: @Autowired 和 @Resource 有什么区别?

这两个注解都可以用来注入依赖，但有一些区别：

@Autowired：

- 是 Spring 提供的注解。
- 默认按**类型（byType）**注入。
- 如果有多个同类型的 Bean，可以配合 @Qualifier 指定名称。
- 可以标注在构造器、setter 方法、字段上。

@Resource：

- 是 JDK 提供的注解（JSR-250）。
- 默认按**名称（byName）**注入。
- 如果按名称找不到，再按类型注入。
- 只能标注在字段和 setter 方法上，不能标注在构造器上。

推荐使用 @Autowired，因为它是 Spring 的标准注解，功能更强大。

Bean的作用域

Q: Spring Bean 有哪些作用域?

Spring Bean 主要有五种作用域:

第一种是 singleton (单例)，这是默认的作用域。整个 Spring 容器中只有一个 Bean 实例，所有对该 Bean 的请求都返回同一个实例。

第二种是 prototype (原型)，每次请求都会创建一个新的 Bean 实例。

第三种是 request，每个 HTTP 请求都会创建一个新的 Bean 实例，只在 Web 应用中有效。

第四种是 session，每个 HTTP Session 都会创建一个新的 Bean 实例，只在 Web 应用中有效。

第五种是 application，整个 ServletContext 中只有一个 Bean 实例，只在 Web 应用中有效。

实际开发中，我们主要用 singleton 和 prototype。singleton 适合无状态的 Bean (如 Service、DAO)，prototype 适合有状态的 Bean (如每次请求都需要新实例的对象)。

追问1: singleton 的 Bean 是线程安全的吗?

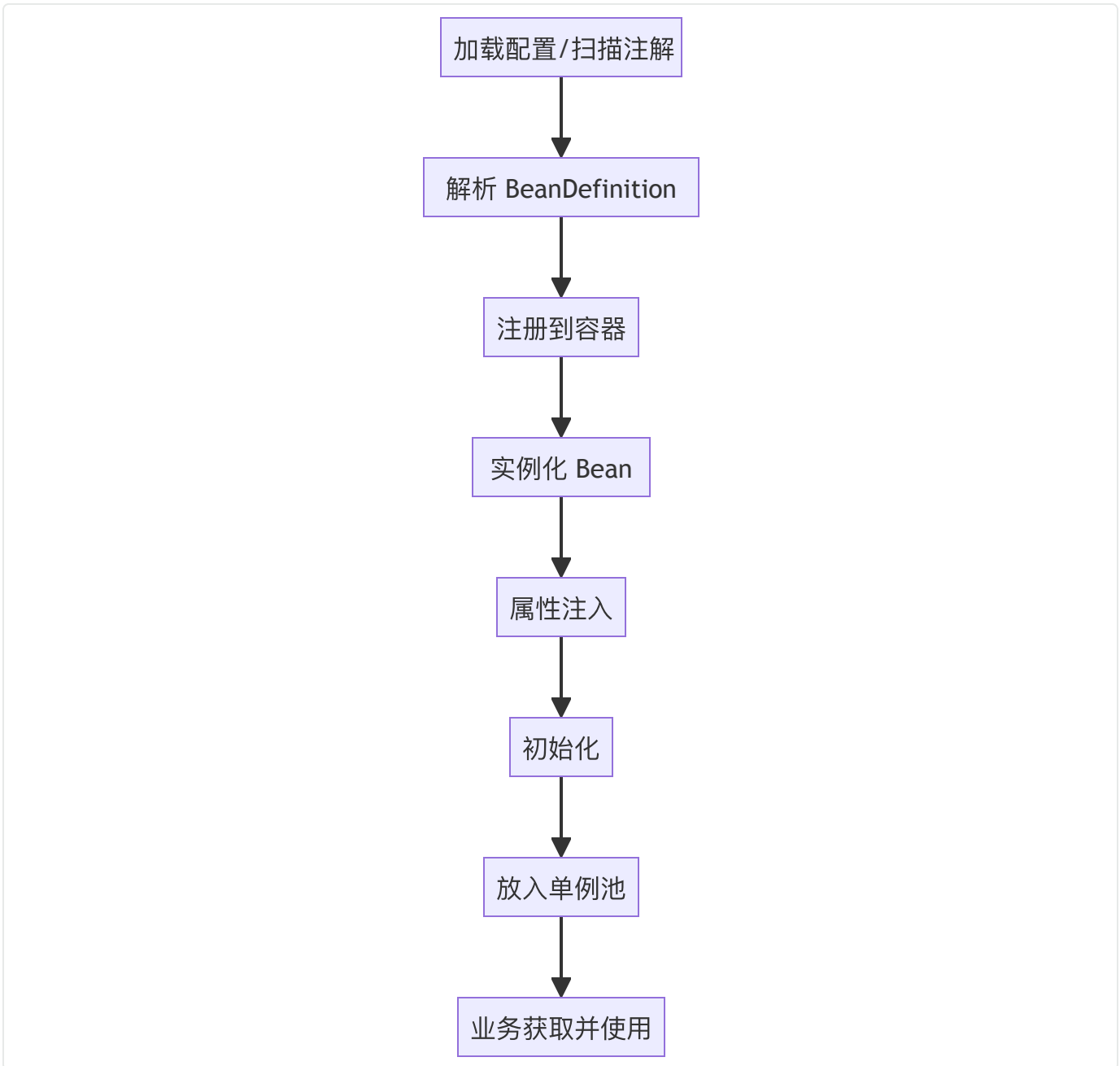
不一定。singleton 的 Bean 在整个容器中只有一个实例，如果 Bean 是**无状态的** (没有成员变量，或者成员变量是不可变的)，那就是线程安全的。

如果 Bean 是**有状态的** (有可变的成员变量)，那就不是线程安全的。多个线程同时访问这个 Bean，可能会出现线程安全问题。

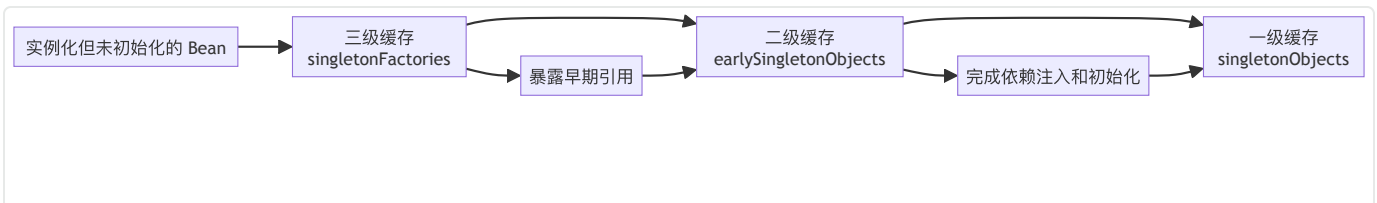
解决方案:

- 尽量让 Bean 无状态。
- 如果必须有状态，使用 ThreadLocal 存储状态。
- 或者改用 prototype 作用域，每次请求都创建新实例。

图解补充: IOC 容器工作链路



图解补充：三级缓存怎么打破循环依赖



对比补充：BeanFactory 和 ApplicationContext

维度	BeanFactory	ApplicationContext
定位	基础容器接口	企业级容器
Bean 加载	偏懒加载	默认更偏预加载单例
能力	基础 IOC	事件、国际化、AOP、资源加载
实际使用	很少直接用	业务开发主流选择

高频追问补充

追问补充1：为什么 Spring 只擅长解决单例 setter 循环依赖？

因为 setter 注入发生在实例化之后，这时对象已经创建出来了，可以先暴露早期引用；而构造器注入在实例化时就要拿到完整依赖，根本来不及把对象先放进缓存。

追问补充2：三级缓存为什么不能简单降成一级或二级？

一级缓存放的是完整 Bean，显然来不及解决循环依赖；二级缓存虽然能放早期对象，但 Spring 还要兼顾 AOP 代理，所以需要三级缓存里的 `ObjectFactory` 来决定暴露原始对象还是代理对象。

追问补充3：面试里 IOC 和 DI 怎么说更自然？

可以说：IOC 是思想，强调控制权交给容器；DI 是落地方式，强调容器把依赖注入进来。这样既区分了抽象层次，也不会只背定义。

速记版

Spring IOC 建议按这条线记：

- IOC 是控制反转，DI 是实现手段
- 容器核心是 BeanDefinition + BeanFactory/ApplicationContext
- Bean 生命周期四段：实例化、注入、初始化、销毁
- 循环依赖重点盯住三级缓存

一句话面试版：

Spring IOC 的本质，是由容器统一管理对象的创建、依赖注入和生命周期，并通过缓存和扩展点把对象装配过程组织起来。

更新: 2026-01-31 00:49:31

原文: <https://www.yuque.com/u63312805/wnyutc/txg29ig37ly43vvg>

SpringAOP

Spring AOP

Spring AOP 是 Spring 框架的另一个核心，面试中经常会问"什么是 AOP"，然后深挖动态代理的实现原理。

什么是AOP

Q: 什么是 AOP? 有什么作用?

AOP 全称是 **Aspect Oriented Programming**（面向切面编程），它是一种编程思想，是对 OOP（面向对象编程）的补充。

AOP 的核心思想是：把系统中的**横切关注点**（Cross-cutting Concerns）从业务逻辑中分离出来，通过**动态代理**的方式，在不修改源代码的情况下，给程序动态地添加功能。

横切关注点是指那些与业务逻辑无关，但又在多个模块中重复出现的功能，比如日志记录、性能统计、事务管理、权限控制等。

AOP 的好处：

第一个是**降低耦合度**。把横切关注点从业务逻辑中分离出来，业务代码更加纯粹，只关注核心业务。

第二个是**提高复用性**。横切关注点可以在多个模块中复用，不需要在每个模块中重复编写。

第三个是**提高可维护性**。如果要修改横切关注点的逻辑，只需要修改一个地方，不需要修改所有使用的地方。

追问1: AOP 的应用场景有哪些?

AOP 的典型应用场景有这么几个：

第一个是**日志记录**。在方法执行前后记录日志，不需要在每个方法里写日志代码。

第二个是**性能统计**。统计方法的执行时间，用于性能分析。

第三个是**事务管理**。Spring 的 @Transactional 注解就是基于 AOP 实现的，在方法执行前开启事务，执行后提交或回滚事务。

第四个是**权限控制**。在方法执行前检查用户是否有权限，没有权限就抛出异常。

第五个是**异常处理**。统一处理方法抛出的异常，记录日志或返回友好的错误信息。

AOP的核心概念

Q: AOP 有哪些核心概念?

AOP 的核心概念主要有这么几个:

第一个是切面 (Aspect)，它是横切关注点的模块化，一个切面可以包含多个通知和切点。在 Spring 中，用 `@Aspect` 注解标注的类就是一个切面。

第二个是连接点 (Join Point)，它是程序执行过程中的一个点，比如方法的调用、方法的执行、异常的抛出等。在 Spring AOP 中，连接点总是方法的执行。

第三个是切点 (Pointcut)，它是连接点的集合，定义了在哪些连接点上应用通知。在 Spring 中，用切点表达式来定义切点，比如 `execution(* com.example.service.*.*(..))`。

第四个是通知 (Advice)，它是在切点上执行的动作，定义了什么时候做什么事。Spring AOP 有五种通知:

- **前置通知 (Before)**：在方法执行之前执行。
- **后置通知 (After)**：在方法执行之后执行，无论方法是否抛出异常。
- **返回通知 (AfterReturning)**：在方法正常返回之后执行。
- **异常通知 (AfterThrowing)**：在方法抛出异常之后执行。
- **环绕通知 (Around)**：在方法执行前后都可以执行，最强大的通知类型。

第五个是目标对象 (Target Object)，它是被代理的对象，也就是包含业务逻辑的对象。

第六个是代理对象 (Proxy Object)，它是 AOP 创建的对象，包含了目标对象和通知的逻辑。

第七个是织入 (Weaving)，它是把切面应用到目标对象，创建代理对象的过程。织入可以在编译期、类加载期、运行期进行。Spring AOP 是在运行期织入。

动态代理的实现

Q: Spring AOP 是如何实现的? JDK 动态代理和 CGLIB 代理有什么区别?

Spring AOP 是基于**动态代理**实现的，主要有两种代理方式：**JDK 动态代理**和 **CGLIB 代理**。

JDK 动态代理：

它是 JDK 原生支持的代理方式，基于**接口**实现。如果目标对象实现了接口，Spring AOP 默认使用 JDK 动态代理。

实现原理：JDK 动态代理通过 `java.lang.reflect.Proxy` 类和 `InvocationHandler` 接口来实现。Proxy 类会动态生成一个代理类，这个代理类实现了目标对象的接口，并在方法调用时委托给 `InvocationHandler` 处理。

CGLIB 代理：

它是一个第三方库，基于**继承**实现。如果目标对象没有实现接口，Spring AOP 会使用 CGLIB 代理。

实现原理：CGLIB 通过字节码技术，动态生成目标类的子类，并重写目标类的方法，在方法中插入增强逻辑。

两者的区别：

Spring 的选择策略：

- 如果目标对象实现了接口，默认使用 JDK 动态代理。
- 如果目标对象没有实现接口，使用 CGLIB 代理。
- 可以通过配置强制使用 CGLIB 代理（`@EnableAspectJAutoProxy(proxyTargetClass = true)`）。

追问1：为什么 CGLIB 创建代理对象慢，但调用方法快？

CGLIB 创建代理对象慢：因为 CGLIB 需要通过字节码技术动态生成目标类的子类，这个过程涉及到字节码的生成和类的加载，比较耗时。

CGLIB 调用方法快：因为 CGLIB 生成的代理类是目标类的子类，方法调用是直接调用，不需要通过反射。而 JDK 动态代理每次方法调用都需要通过反射（`InvocationHandler.invoke`），反射的性能比直接调用差。

不过，在现代 JVM 中（JDK 8 及以后），JDK 动态代理的性能已经优化得很好了，两者的性能差距不大。

切点表达式

Q: Spring AOP 的切点表达式怎么写?

Spring AOP 的切点表达式主要使用 **execution** 表达式，语法是：

简化一下就是：

常用的通配符：

- *****：匹配任意字符，但只能匹配一个元素（一个包名、一个类名、一个方法名、一个参数）。
- **..**：匹配任意字符，可以匹配多个元素。在包名中表示当前包及其子包，在参数中表示任意个参数。
- **+**：匹配指定类及其子类。

常用的切点表达式示例：

其他切点表达式：

除了 **execution**，还有一些其他的切点表达式：

- **@annotation**：匹配标注了指定注解的方法。
- **@within**：匹配标注了指定注解的类中的所有方法。
- **within**：匹配指定包或类中的所有方法。
- **bean**：匹配指定名称的 Bean 中的所有方法。

AOP的使用示例

Q: 如何使用 Spring AOP?

使用 Spring AOP 主要有三个步骤：

第一步是添加依赖，在 `pom.xml` 中添加 `spring-boot-starter-aop` 依赖。

第二步是定义切面，创建一个类，用 `@Aspect` 注解标注，并定义通知方法。

第三步是启用 AOP，在 Spring Boot 中，默认已经启用了 AOP。如果是传统的 Spring 项目，需要在配置类上添加 `@EnableAspectJAutoProxy` 注解。

追问1: 多个通知的执行顺序是怎样的?

如果一个方法被多个通知匹配，执行顺序是：

正常情况（方法正常返回）：

```
1  @Around 前半部分
2    ↓
3  @Before
4    ↓
5  目标方法执行
6    ↓
7  @AfterReturning
8    ↓
9  @After
10   ↓
11 @Around 后半部分
```

异常情况（方法抛出异常）：

```
1  @Around 前半部分
2    ↓
3  @Before
4    ↓
5  目标方法执行（抛出异常）
6    ↓
7  @AfterThrowing
8    ↓
9  @After
10   ↓
11 @Around 后半部分（不执行，因为异常被抛出）
```

多个切面的执行顺序：

如果有多个切面，可以通过 @Order 注解指定顺序，数字越小，优先级越高。

```
1  @Aspect
2  @Component
3  @Order(1)
4  public class LogAspect {
5  }
6
7  @Aspect
8  @Component
9  @Order(2)
10 public class PerformanceAspect {
11 }
```

执行顺序是：LogAspect 的 @Around 前半部分 → PerformanceAspect 的 @Around 前半部分 → 目标方法 → PerformanceAspect 的 @Around 后半部分 → LogAspect 的 @Around 后半部分。

AOP的局限性

Q：Spring AOP 有什么局限性？

Spring AOP 主要有以下几个局限性：

第一个是只能拦截 public 方法。因为 Spring AOP 是基于代理实现的，代理对象只能拦截 public 方法。如果方法是 private、protected 或 default，AOP 无法拦截。

第二个是同类内部方法调用无法拦截。如果在同一个类中，方法 A 调用方法 B，即使方法 B 有切面，也不会被拦截。因为内部调用是通过 this 调用的，不是通过代理对象调用的。

解决方案：

- 把方法 B 移到另一个类中。
- 通过 ApplicationContext 获取代理对象，然后调用方法 B。
- 使用 AspectJ 的编译期织入（不是 Spring AOP）。

第三个是性能开销。虽然动态代理的性能已经很好了，但相比直接调用，还是有一定的开销。如果对性能要求极高，需要谨慎使用 AOP。

第四个是调试困难。因为 AOP 是在运行期动态生成代理对象，调试时可能看不到切面的逻辑，增加了调试难度。

追问1：如何解决同类内部方法调用无法拦截的问题？

有几种解决方案：

第一种是通过 ApplicationContext 获取代理对象：

```

1  @Service
2  public class UserService {
3      @Autowired
4      private ApplicationContext context;
5
6      public void methodA() {
7          // 获取代理对象，然后调用 methodB
8          UserService proxy = context.getBean(UserService.class);
9          proxy.methodB();
10     }
11
12     @Transactional
13     public void methodB() {
14         // 业务逻辑
15     }
16 }

```

第二种是使用 `AopContext.currentProxy()`:

```

1  @Service
2  @EnableAspectJAutoProxy(exposeProxy = true) // 暴露代理对象
3  public class UserService {
4
5      public void methodA() {
6          // 获取当前代理对象
7          UserService proxy = (UserService) AopContext.currentProxy();
8          proxy.methodB();
9      }
10
11     @Transactional
12     public void methodB() {
13         // 业务逻辑
14     }
15 }

```

第三种是把方法移到另一个类:

```
1  @Service
2  public class UserService {
3      @Autowired
4      private UserHelper userHelper;
5
6      public void methodA() {
7          userHelper.methodB(); // 通过代理对象调用
8      }
9  }
10
11 @Service
12 public class UserHelper {
13     @Transactional
14     public void methodB() {
15         // 业务逻辑
16     }
17 }
```

推荐使用第三种方案，因为它更符合单一职责原则，代码更清晰。

更新: 2026-01-31 00:49:38

原文: <https://www.yuque.com/u63312805/wnyutc/ia9yt81icp1lq54m>

Spring常见注解

Spring常见注解

Spring 注解一般不是让你背名字，而是看你是否知道“这个注解在容器里起什么作用”。

组件注册类注解

Q: Spring 中常见的组件注解有哪些？

最常见的是：

- `@Component`
- `@Service`
- `@Controller`
- `@Repository`

它们本质上都能把类注册进 Spring 容器，只是语义分层不同。

`@Autowired` 和 `@Resource`

Q: `@Autowired` 和 `@Resource` 有什么区别？

`@Autowired` 是 Spring 提供的，默认按类型注入。

`@Resource` 是 JSR-250 规范里的注解，默认语义更偏按名称注入。

如果同类型 Bean 太多，`@Autowired` 往往需要配合 `@Qualifier` 使用。

`@Configuration` 和 `@Bean`

Q: `@Configuration` 和 `@Bean` 是做什么的？

`@Configuration` 用来声明配置类。

`@Bean` 用来把方法返回值注册成 Spring Bean。

这套方式非常适合：

- 接入第三方组件
- 初始化复杂对象
- 替代早期 XML 配置

Web 层常见注解

Q: Web 层常见注解有哪些?

最常见的是：

- `@RestController`
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- `@RequestParam`
- `@PathVariable`
- `@RequestBody`

其中 `@RestController` 本质上等于 `@Controller + @ResponseBody`。

Spring事务

Spring事务

Spring 事务是面试的高频考点，面试官通常会问"事务的传播机制"和"@Transactional 什么时候会失效"。

Spring事务的基本概念

Q：什么是事务？Spring 如何管理事务？

事务是数据库操作的一个执行单元，它要么全部成功，要么全部失败，保证数据的一致性。事务有四个特性（ACID）：

原子性（Atomicity）：事务中的所有操作要么全部成功，要么全部失败回滚。

一致性（Consistency）：事务执行前后，数据库的完整性约束没有被破坏。

隔离性（Isolation）：多个事务并发执行时，一个事务的执行不应影响其他事务。

持久性（Durability）：事务一旦提交，对数据库的改变是永久的。

Spring 管理事务的方式有两种：

第一种是**编程式事务**，通过 TransactionTemplate 或 PlatformTransactionManager 手动管理事务。这种方式灵活，但代码侵入性强。

第二种是**声明式事务**，通过 @Transactional 注解或 XML 配置来管理事务。这种方式简单，代码侵入性低，是推荐的方式。Spring 通过 AOP 实现声明式事务。

追问1：Spring 事务的实现原理是什么？

Spring 事务是基于 AOP 实现的。

当我们在方法上加 @Transactional 注解时，Spring 会通过 AOP 创建一个代理对象。在方法执行前，代理对象会开启事务；方法执行成功后，提交事务；方法抛出异常时，回滚事务。

具体来说，Spring 使用 TransactionInterceptor 拦截器来实现事务管理。这个拦截器在方法执行前后插入事务管理的逻辑。

事务的传播机制

Q: Spring 事务的传播机制有哪些?

Spring 事务的传播机制定义了当一个事务方法被另一个事务方法调用时，事务应该如何传播。

Spring 定义了七种传播机制：

第一种是 REQUIRED（默认），如果当前存在事务，就加入该事务；如果当前没有事务，就创建一个新事务。这是最常用的传播机制。

第二种是 SUPPORTS，如果当前存在事务，就加入该事务；如果当前没有事务，就以非事务方式执行。

第三种是 MANDATORY，如果当前存在事务，就加入该事务；如果当前没有事务，就抛出异常。

第四种是 REQUIRES_NEW，无论当前是否存在事务，都创建一个新事务。如果当前存在事务，就把当前事务挂起。

第五种是 NOT_SUPPORTED，以非事务方式执行。如果当前存在事务，就把当前事务挂起。

第六种是 NEVER，以非事务方式执行。如果当前存在事务，就抛出异常。

第七种是 NESTED，如果当前存在事务，就在嵌套事务中执行；如果当前没有事务，就创建一个新事务。嵌套事务是外部事务的一部分，外部事务回滚，嵌套事务也会回滚；但嵌套事务回滚，不会影响外部事务。

常用的传播机制：

实际开发中，我们主要用 **REQUIRED（默认）** 和 **REQUIRES_NEW**。

- **REQUIRED**：适合大多数场景，多个方法共享一个事务。
- **REQUIRES_NEW**：适合需要独立事务的场景，比如记录日志，即使主业务失败，日志也要保存。

追问1: REQUIRED 和 REQUIRES_NEW 有什么区别?

REQUIRED：

- 如果当前有事务，就加入当前事务。
- 多个方法共享一个事务，任何一个方法抛出异常，整个事务都会回滚。

REQUIRES_NEW：

- 无论当前是否有事务，都创建一个新事务。

- 新事务和外部事务是独立的，新事务的提交或回滚不会影响外部事务。

示例：

```
1  @Service
2  public class OrderService {
3      @Autowired
4      private LogService logService;
5
6      @Transactional
7      public void createOrder() {
8          // 创建订单
9          saveOrder();
10         // 记录日志 (使用 REQUIRES_NEW)
11         logService.saveLog();
12         // 如果这里抛出异常，订单会回滚，但日志不会回滚
13         throw new RuntimeException("订单创建失败");
14     }
15 }
16
17 @Service
18 public class LogService {
19     @Transactional(propagation = Propagation.REQUIRES_NEW)
20     public void saveLog() {
21         // 保存日志
22     }
23 }
```

在这个例子中，即使 createOrder 方法抛出异常，订单会回滚，但日志已经在独立的事务中提交了，不会回滚。

追问2: NESTED 和 REQUIRES_NEW 有什么区别？

NESTED（嵌套事务）：

- 嵌套事务是外部事务的一部分。
- 外部事务回滚，嵌套事务也会回滚。
- 嵌套事务回滚，不会影响外部事务（可以捕获异常继续执行）。
- 底层使用数据库的 Savepoint 机制实现。

REQUIRES_NEW（独立事务）：

- 新事务和外部事务完全独立。
- 外部事务回滚，不会影响新事务。

- 新事务回滚，也不会影响外部事务。

选择建议：

- 如果需要部分回滚（子事务失败，主事务继续），用 `NESTED`。
- 如果需要完全独立的事务（如记录日志），用 `REQUIRES_NEW`。

事务的隔离级别

Q：Spring 事务的隔离级别有哪些？

Spring 事务的隔离级别定义了一个事务可能受其他并发事务影响的程度。Spring 定义了**五种隔离级别**：

第一种是 `DEFAULT`（默认），使用数据库默认的隔离级别。MySQL 默认是 `REPEATABLE_READ`，Oracle 默认是 `READ_COMMITTED`。

第二种是 `READ_UNCOMMITTED`（读未提交），最低的隔离级别，允许读取未提交的数据。会出现脏读、不可重复读、幻读问题。

第三种是 `READ_COMMITTED`（读已提交），只能读取已提交的数据。可以避免脏读，但会出现不可重复读、幻读问题。

第四种是 `REPEATABLE_READ`（可重复读），保证在同一个事务中多次读取同一数据的结果是一致的。可以避免脏读、不可重复读，但会出现幻读问题。MySQL 的 InnoDB 引擎通过 MVCC 和 Next-Key Lock 解决了幻读问题。

第五种是 `SERIALIZABLE`（串行化），最高的隔离级别，完全串行化执行，避免了所有并发问题，但性能最差。

并发问题：

- **脏读**：读取到其他事务未提交的数据。
- **不可重复读**：在同一个事务中，多次读取同一数据，结果不一致（其他事务修改并提交了数据）。
- **幻读**：在同一个事务中，多次查询，结果集的数量不一致（其他事务插入或删除了数据）。

实际开发中，我们通常使用 `DEFAULT`（数据库默认）或 `READ_COMMITTED`。

追问1：如何选择隔离级别？

这要看业务需求和性能要求：

READ_UNCOMMITTED：几乎不用，因为会出现脏读，数据不可靠。

READ_COMMITTED：适合大多数场景，Oracle 的默认隔离级别。可以避免脏读，性能较好。

REPEATABLE_READ：MySQL 的默认隔离级别。适合对数据一致性要求较高的场景。

SERIALIZABLE：适合对数据一致性要求极高的场景（如金融系统），但性能很差，一般不用。

我在项目中的选择：通常使用数据库的默认隔离级别（DEFAULT），不需要特别指定。如果有特殊需求，再根据业务场景调整。

@Transactional的失效场景

Q：@Transactional 注解什么时候会失效？

@Transactional 注解失效的场景主要有以下几种：

第一种是方法不是 public。Spring AOP 只能拦截 public 方法，如果方法是 private、protected 或 default，@Transactional 会失效。

第二种是同类内部方法调用。如果在同一个类中，方法 A 调用方法 B，即使方法 B 有 @Transactional 注解，事务也不会生效。因为内部调用是通过 this 调用的，不是通过代理对象调用的。

第三种是异常被捕获。@Transactional 默认只回滚 RuntimeException 和 Error，如果方法抛出的异常被 catch 住了，事务不会回滚。

第四种是异常类型不匹配。@Transactional 默认只回滚 RuntimeException 和 Error。如果方法抛出的是 checked exception（如 IOException），事务不会回滚。可以通过 `rollbackFor` 属性指定回滚的异常类型。

第五种是数据库引擎不支持事务。如果使用的是 MyISAM 引擎（不支持事务），@Transactional 会失效。应该使用 InnoDB 引擎。

第六种是没有被 Spring 管理。如果类没有被 Spring 管理（没有 @Service、@Component 等注解），@Transactional 会失效。

第七种是事务传播机制设置不当。如果传播机制设置为 NOT_SUPPORTED 或 NEVER，事务会失效。

追问1: 如何解决同类内部方法调用导致的事务失效?

有几种解决方案:

第一种是把方法移到另一个类 (推荐):

```
1  @Service
2  public class OrderService {
3      @Autowired
4      private OrderHelper orderHelper;
5
6      public void createOrder() {
7          // 调用另一个类的方法, 通过代理对象调用
8          orderHelper.saveOrder();
9      }
10 }
11
12 @Service
13 public class OrderHelper {
14     @Transactional
15     public void saveOrder() {
16         // 保存订单
17     }
18 }
```

第二种是通过 ApplicationContext 获取代理对象:

```
1  @Service
2  public class OrderService {
3      @Autowired
4      private ApplicationContext context;
5
6      public void createOrder() {
7          OrderService proxy = context.getBean(OrderService.class);
8          proxy.saveOrder();
9      }
10
11     @Transactional
12     public void saveOrder() {
13         // 保存订单
14     }
15 }
```

第三种是使用 AopContext.currentProxy():

```

1  @Service
2  @EnableAspectJAutoProxy(exposeProxy = true)
3  public class OrderService {
4
5      public void createOrder() {
6          OrderService proxy = (OrderService) AopContext.currentProxy();
7          proxy.saveOrder();
8      }
9
10     @Transactional
11     public void saveOrder() {
12         // 保存订单
13     }
14 }

```

推荐使用第一种方案，代码更清晰，符合单一职责原则。

追问2：如何让 checked exception 也回滚事务？

默认情况下，@Transactional 只回滚 RuntimeException 和 Error。如果要让 checked exception 也回滚，可以使用 `rollbackFor` 属性：

```

1  @Transactional(rollbackFor = Exception.class)
2  public void createOrder() throws Exception {
3      // 业务逻辑
4      throw new IOException("IO 异常"); // 会回滚
5  }

```

或者，把 checked exception 包装成 RuntimeException 抛出：

```

1  @Transactional
2  public void createOrder() {
3      try {
4          // 业务逻辑
5      } catch (IOException e) {
6          throw new RuntimeException("IO 异常", e); // 会回滚
7      }
8  }

```

事务的最佳实践

Q：使用 Spring 事务有哪些最佳实践？

使用 Spring 事务的最佳实践主要有这么几点：

第一个是事务范围尽量小。事务持有数据库连接，事务越大，连接占用时间越长，影响并发性能。应该只在需要事务的地方加 `@Transactional`，不要在整个 Service 类上加。

第二个是避免在事务中执行耗时操作。比如调用外部接口、发送邮件、文件 IO 等，这些操作应该放在事务外执行，或者使用异步处理。

第三个是合理设置超时时间。可以通过 `timeout` 属性设置事务超时时间，避免事务长时间占用连接。

第四个是明确指定回滚的异常类型。如果业务中可能抛出 checked exception，应该使用 `rollbackFor` 属性明确指定。

第五个是避免大事务。如果一个事务中有大量的数据库操作，可以考虑拆分成多个小事务，或者使用批处理。

第六个是注意事务的传播机制。如果需要独立的事务（如记录日志），使用 `REQUIRES_NEW`；如果需要嵌套事务，使用 `NESTED`。

第七个是只读事务优化。如果方法只是查询数据，不修改数据，可以设置 `readOnly = true`，数据库可以做一些优化。

追问1：为什么要避免在事务中执行耗时操作？

因为事务会持有数据库连接，如果在事务中执行耗时操作（如调用外部接口），会导致：

第一个是连接占用时间长，数据库连接池的连接被长时间占用，其他请求可能拿不到连接，影响并发性能。

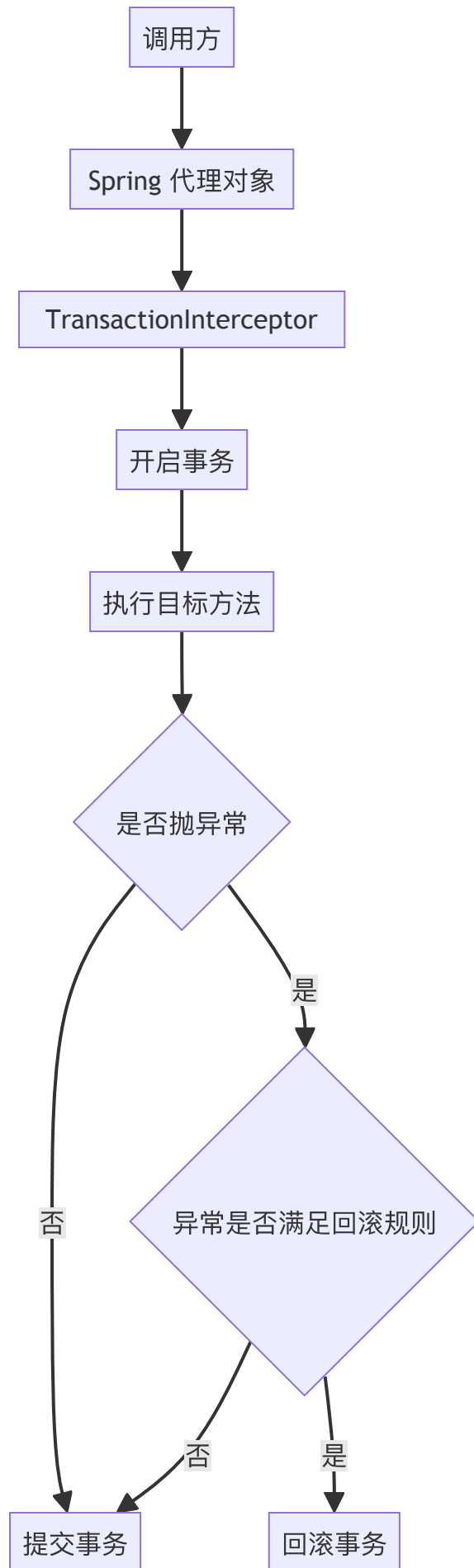
第二个是锁持有时间长，如果事务中有更新操作，会持有行锁或表锁，其他事务可能被阻塞，导致性能下降甚至死锁。

第三个是事务超时，如果耗时操作太久，可能导致事务超时，事务被回滚。

解决方案：

- 把耗时操作放在事务外执行。
- 使用异步处理（如 `@Async`）。
- 使用消息队列（如 RabbitMQ）解耦。

图解补充：@Transactional 生效链路



对比补充：传播行为速记

传播行为	当前有事务时	当前没事务时	适用场景
REQUIRED	加入当前事务	新建事务	默认主业务
REQUIRES_NEW	挂起当前事务，新建事务	新建事务	日志、审计、补偿
SUPPORTS	加入当前事务	非事务执行	可有可无的事务读操作
MANDATORY	加入当前事务	抛异常	强依赖上游事务
NOT_SUPPORTED	挂起当前事务	非事务执行	明确不想进事务
NEVER	抛异常	非事务执行	禁止事务环境
NESTED	建立保存点	新建事务	局部回滚

高频追问补充

追问补充1：为什么同类内部调用会让事务失效？

因为事务是靠 Spring AOP 代理织进去的。同类内部 `this.xxx()` 调用绕开了代理对象，没有经过 `TransactionInterceptor`，自然也就没有事务增强。

追问补充2：为什么异常被吃掉之后事务不回滚？

因为对 Spring 来说，方法已经“正常返回”了。事务拦截器只有在感知到异常向外抛出，或者显式标记 `setRollbackOnly()` 时，才会回滚。

追问补充3：面试里怎么回答事务失效更完整？

可以按这四类讲：

- 1. 没走代理，例如同类调用、对象不是 Spring 管理。
- 2. 回滚规则没命中，例如 checked exception。
- 3. 方法签名不符合代理拦截条件，例如非 public。

4. 底层资源不支持事务，例如引擎不支持或传播行为主动关闭事务。

速记版

Spring 事务面试建议按这条线背：

- 先讲 `@Transactional` 基于 AOP 代理
- 再讲传播行为，重点盯住 `REQUIRED` 和 `REQUIRES_NEW`
- 再讲隔离级别和数据库默认值
- 最后讲事务失效场景

一句话记忆：

Spring 事务的本质不是注解本身，而是 代理 + 拦截器 + 底层事务管理器 的组合。

Spring自动装配

Spring自动装配

SpringBoot 自动装配是面试的高频考点，面试官通常会问"SpringBoot 是如何实现自动配置的"。

SpringBoot的核心注解

Q: @SpringBootApplication 注解的作用是什么？

@SpringBootApplication 是 SpringBoot 的核心注解，它是一个组合注解，包含了三个重要的注解：

第一个是 @SpringBootConfiguration，它本质上就是 @Configuration，表示这是一个配置类，可以在这个类中定义 Bean。

第二个是 @EnableAutoConfiguration，这是自动配置的核心注解，它会根据类路径中的 jar 包和配置，自动配置 Spring 应用。

第三个是 @ComponentScan，它会扫描当前包及其子包下的所有组件（@Component、@Service、@Controller 等），并注册到 Spring 容器中。

所以，@SpringBootApplication 相当于：

自动配置的原理

Q: SpringBoot 的自动配置是如何实现的？

SpringBoot 的自动配置主要依赖 `@EnableAutoConfiguration` 注解和 `Spring Boot Starter` 机制。

`@EnableAutoConfiguration` 的工作流程：

1. `@EnableAutoConfiguration` 注解上有一个 `@Import(AutoConfigurationImportSelector.class)` 注解。
2. `AutoConfigurationImportSelector` 会读取 `META-INF/spring.factories` 文件（在 SpringBoot 2.7 之后改为 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`）。
3. 这个文件中列出了所有的自动配置类，比如 `RedisAutoConfiguration`、`DataSourceAutoConfiguration` 等。
4. SpringBoot 会加载这些自动配置类，但不是所有的配置类都会生效，只有满足条件的配置类才会生效。
5. 每个自动配置类上都有 **条件注解**（如 `@ConditionalOnClass`、`@ConditionalOnMissingBean`），只有满足条件，配置类才会生效。
6. 如果条件满足，配置类中定义的 Bean 就会被注册到 Spring 容器中。

举个例子（Redis 自动配置）：

这个配置类的意思是：

- 如果类路径中有 `RedisOperations` 类（说明引入了 Redis 依赖）。
- 并且容器中没有 `redisTemplate` 这个 Bean。
- 那么就自动创建一个 `RedisTemplate` Bean。

追问1：如何自定义一个 Starter？

自定义 Starter 主要有四个步骤：

第一步是创建一个 Maven 项目，命名规范是 `xxx-spring-boot-starter`。

第二步是添加依赖，引入 `spring-boot-autoconfigure` 依赖。

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3
4   <artifactId>spring-boot-autoconfigure</artifactId>
5
6 </dependency>
7
```

第三步是创建自动配置类，定义需要自动配置的 Bean。

```

1  @Configuration
2  @ConditionalOnClass(MyService.class)
3  @EnableConfigurationProperties(MyProperties.class)
4  public class MyAutoConfiguration {
5
6      @Bean
7      @ConditionalOnMissingBean
8      public MyService myService(MyProperties properties) {
9          return new MyService(properties);
10     }
11 }

```

第四步是创建 `spring.factories` 文件，在 `src/main/resources/META-INF/spring.factories` 中注册自动配置类。

```

1  org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
2  com.example.MyAutoConfiguration

```

或者在 SpringBoot 2.7 之后，创建 `src/main/resources/META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 文件：

```

1  com.example.MyAutoConfiguration

```

这样，其他项目引入这个 Starter 后，就会自动配置 `MyService`。

条件注解

Q: SpringBoot 有哪些常用的条件注解？

SpringBoot 提供了很多条件注解，用于控制配置类或 Bean 是否生效。常用的条件注解有这么几个：

第一个是 `@ConditionalOnClass`，当类路径中存在指定的类时，配置才生效。比如 `@ConditionalOnClass(RedisOperations.class)` 表示只有引入了 Redis 依赖，配置才生效。

第二个是 `@ConditionalOnMissingClass`，当类路径中不存在指定的类时，配置才生效。

第三个是 `@ConditionalOnBean`，当容器中存在指定的 Bean 时，配置才生效。

第四个是 `@ConditionalOnMissingBean`，当容器中不存在指定的 Bean 时，配置才生效。这个注解很常用，用于避免覆盖用户自定义的 Bean。

第五个是 `@ConditionalOnProperty`，当配置文件中存在指定的属性时，配置才生效。比如 `@ConditionalOnProperty(name = "spring.redis.host")` 表示只有配置了 redis 的 host，配置才生效。

第六个是 `@ConditionalOnWebApplication`，当应用是 Web 应用时，配置才生效。

第七个是 `@ConditionalOnNotWebApplication`，当应用不是 Web 应用时，配置才生效。

示例：

追问1：如果我想自定义一个 Bean，如何覆盖自动配置的 Bean？

SpringBoot 的自动配置类通常会使用 `@ConditionalOnMissingBean` 注解，这样如果用户自定义了 Bean，自动配置就不会生效。

比如，如果我想自定义 `RedisTemplate`：

```
1  @Configuration
2  public class MyRedisConfig {
3
4      @Bean
5      public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory factory) {
6          RedisTemplate<String, Object> template = new RedisTemplate<>();
7          template.setConnectionFactory(factory);
8          // 自定义序列化方式
9          template.setKeySerializer(new StringRedisSerializer());
10         template.setValueSerializer(new GenericJackson2JsonRedisSerializer
11         ());
12         return template;
13     }
14 }
```


因为 RedisAutoConfiguration 中的 redisTemplate 有 @ConditionalOnMissingBean 注解，所以我自定义的 Bean 会生效，自动配置的 Bean 不会生效。

配置文件的加载

Q: SpringBoot 如何加载配置文件？配置文件的优先级是怎样的？

SpringBoot 支持多种配置文件格式，主要有 application.properties 和 application.yml。

配置文件的加载位置（优先级从高到低）：

1. 命令行参数： `java -jar app.jar --server.port=8081`
2. 操作系统环境变量
3. Java 系统属性： `System.getProperties()`
4. JNDI 属性
5. RandomValuePropertySource: `${random.int}`
6. jar 包外的 application-{profile}.properties 或 .yml（带 profile）
7. jar 包内的 application-{profile}.properties 或 .yml（带 profile）
8. jar 包外的 application.properties 或 .yml（不带 profile）
9. jar 包内的 application.properties 或 .yml（不带 profile）

配置文件的搜索位置（优先级从高到低）：

1. 项目根目录的 /config 子目录
2. 项目根目录
3. classpath 的 /config 包
4. classpath 根路径

多环境配置：

SpringBoot 支持多环境配置，通过 `spring.profiles.active` 属性指定激活的环境。

然后创建 `application-dev.properties`、`application-prod.properties` 等文件，SpringBoot 会根据激活的环境加载对应的配置文件。

追问1: 如何在代码中获取配置文件的值？

有三种方式：

第一种是使用 @Value 注解：

```
1  @Component
2  public class MyService {
3      @Value("${server.port}")
4      private int port;
5  }
```

第二种是使用 @ConfigurationProperties 注解（推荐）：

```
1  @Component
2  @ConfigurationProperties(prefix = "my.config")
3  public class MyConfig {
4      private String name;
5      private int age;
6      // getter 和 setter
7  }
```

```
1  # application.properties
2  my.config.name=张三
3  my.config.age=18
```

第三种是使用 Environment 对象：

```
1  @Component
2  public class MyService {
3      @Autowired
4      private Environment env;
5
6      public void test() {
7          String port = env.getProperty("server.port");
8      }
9  }
```

推荐使用 @ConfigurationProperties，因为它支持类型安全、批量注入、JSR-303 校验等功能。

SpringBoot的启动流程

Q：SpringBoot 的启动流程是怎样的？

面试官你好，SpringBoot 的启动流程主要分为以下几个步骤：

第一步是创建 `SpringApplication` 对象，在 `SpringApplication.run()` 方法中，会先创建一个 `SpringApplication` 对象。这个对象会做一些初始化工作：

- 推断应用类型（Web 应用还是非 Web 应用）。
- 加载 `ApplicationContextInitializer` 和 `ApplicationListener`。
- 推断主类（包含 `main` 方法的类）。

第二步是运行 `SpringApplication`，调用 `run()` 方法启动应用。

第三步是准备环境（Environment），加载配置文件，设置环境变量。

第四步是创建 `ApplicationContext`，根据应用类型创建对应的上下文（如 `AnnotationConfigServletWebServerApplicationContext`）。

第五步是准备 `ApplicationContext`，设置环境、加载 `BeanDefinition`、注册单例 `Bean` 等。

第六步是刷新 `ApplicationContext`，这是最核心的步骤，会执行以下操作：

- 调用 `BeanFactoryPostProcessor`。
- 注册 `BeanPostProcessor`。
- 初始化 `MessageSource`（国际化）。
- 初始化事件广播器。
- 创建 Web 服务器（如 Tomcat）。
- 注册监听器。
- 实例化所有单例 `Bean`。
- 完成刷新，发布事件。

第七步是启动完成，调用 `ApplicationRunner` 和 `CommandLineRunner`，执行自定义的启动逻辑。

简化流程：

追问1：SpringBoot 是如何启动内嵌的 Tomcat 的？

SpringBoot 通过 `ServletWebServerFactory` 来创建和启动内嵌的 Web 服务器。

在刷新 `ApplicationContext` 的过程中，会调用 `onRefresh()` 方法，这个方法会创建 Web 服务器：

1. 从容器中获取 `ServletWebServerFactory`（如 `TomcatServletWebServerFactory`）。
2. 调用 `getWebServer()` 方法创建 Web 服务器（如 Tomcat）。
3. 启动 Web 服务器。

为什么 SpringBoot 可以打成 jar 包运行：

因为 SpringBoot 使用了 `spring-boot-maven-plugin` 插件，这个插件会把应用打成一个 **可执行的 jar 包** (Fat Jar)，jar 包中包含了所有的依赖和内嵌的 Tomcat。

运行 `java -jar app.jar` 时，会先执行 jar 包中的 `MANIFEST.MF` 文件指定的主类 (JarLauncher)，JarLauncher 会加载 jar 包中的所有依赖，然后启动 SpringBoot 应用。

SpringBoot的优势

Q: SpringBoot 相比传统的 Spring 有什么优势？

面试官你好，SpringBoot 相比传统的 Spring 主要有以下几个优势：

第一个是简化配置。传统的 Spring 需要大量的 XML 配置，而 SpringBoot 通过自动配置和约定优于配置的理念，大大简化了配置。

第二个是快速开发。SpringBoot 提供了大量的 Starter，引入一个 Starter 就可以快速集成一个功能（如 Redis、MyBatis），不需要手动配置。

第三个是内嵌服务器。SpringBoot 内嵌了 Tomcat、Jetty 等服务器，可以直接打成 jar 包运行，不需要部署到外部服务器。

第四个是生产就绪。SpringBoot 提供了 Actuator 模块，可以监控应用的健康状况、性能指标等，方便运维。

第五个是微服务友好。SpringBoot 是 Spring Cloud 的基础，非常适合构建微服务应用。

总结：SpringBoot 让开发者可以专注于业务逻辑，而不是配置和基础设施，大大提高了开发效率。

追问1: SpringBoot 和 Spring Cloud 有什么关系？

SpringBoot 是一个快速开发框架，用于简化 Spring 应用的开发。

Spring Cloud 是一个微服务框架，基于 SpringBoot 构建，提供了服务注册与发现、配置中心、负载均衡、熔断器、网关等微服务组件。

关系：Spring Cloud 依赖于 SpringBoot，SpringBoot 是 Spring Cloud 的基础。可以说，SpringBoot 是单体应用的开发框架，Spring Cloud 是微服务架构的解决方案。

常用的 Spring Cloud 组件：

- **Nacos：**服务注册与发现、配置中心。

- **Gateway**: API 网关。
- **OpenFeign**: 声明式 HTTP 客户端。
- **Sentinel**: 熔断降级、限流。
- **Seata**: 分布式事务。

SpringBoot 如何优雅停机

Q: SpringBoot 如何优雅停机?

优雅停机的核心目标是：不再接收新请求，但尽量处理完已经在执行中的请求，再关闭应用。

常见做法包括：

- 开启 SpringBoot 的优雅停机配置
- 配合容器的关闭超时时间
- 让线程池、连接池、消息消费者都有机会释放资源

所以优雅停机不只是停 Web 服务，还要把后台资源一起纳入关闭流程。

Spring6.0 新特性

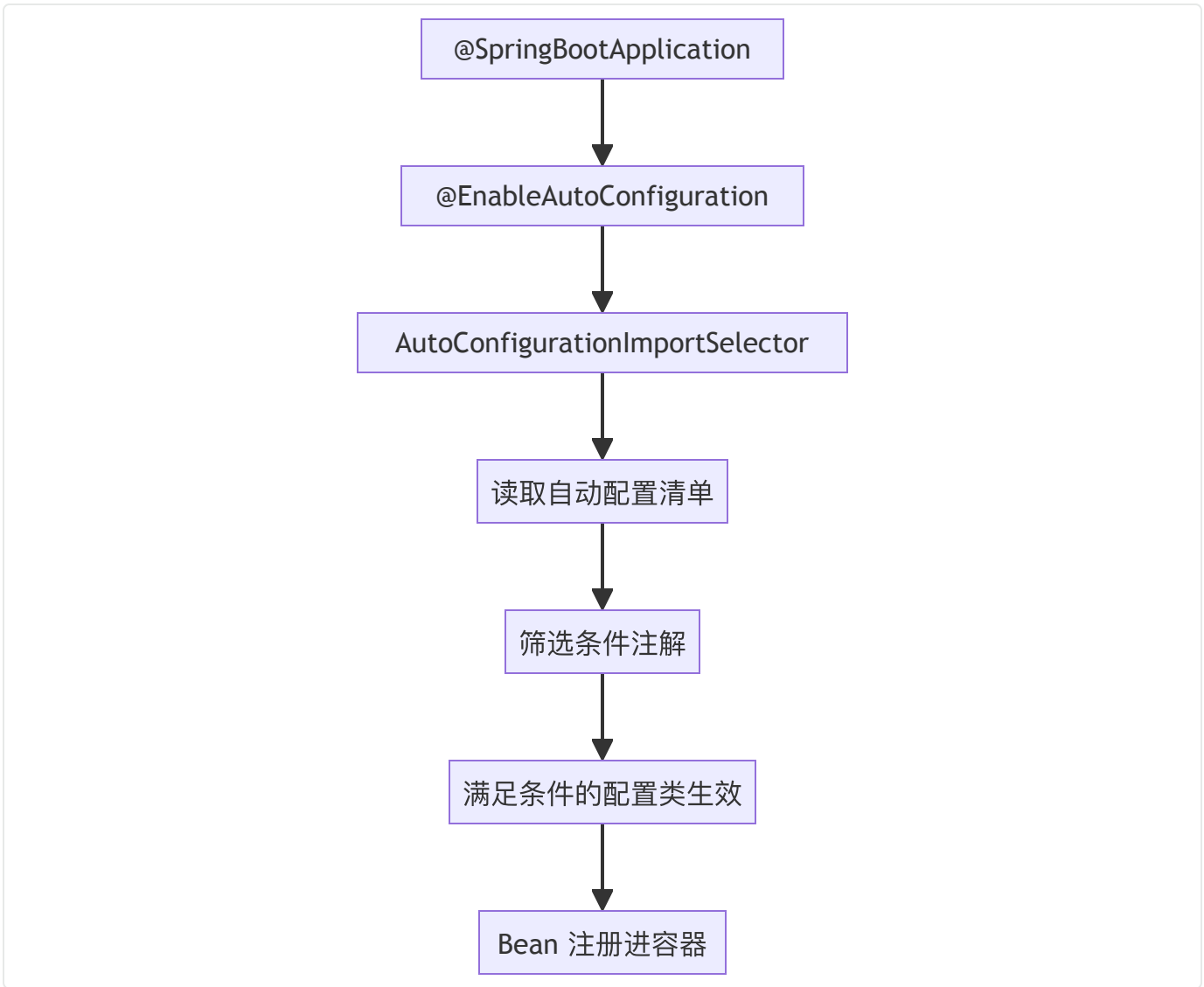
Q: 你了解 Spring6.0 有哪些重要变化吗?

面试里一般不用背得很细，但知道几个方向会比较加分：

- 全面拥抱 JDK17+
- 全面迁移到 Jakarta EE
- 对 AOT 和原生镜像支持更好
- 更贴近云原生部署场景

所以 Spring6 和 SpringBoot3 可以理解成一次比较明显的现代化升级。

图解补充：自动装配主链路



对比补充：常见条件注解怎么记

注解	典型含义	常见用途
<code>@ConditionalOnClass</code>	类路径里有某个类才生效	依赖存在才自动配
<code>@ConditionalOnMissingBean</code>	容器里没有 Bean 才生效	给用户留覆盖口子
<code>@ConditionalOnBean</code>	容器里已有 Bean 才生效	依赖已有配置
<code>@ConditionalOnProperty</code>	配置项满足才生效	开关控制
<code>@ConditionalOnWebApplication</code>	Web 场景才生效	MVC、WebFlux 自动配置

高频追问补充

追问补充1：为什么自动配置不会随便把用户 Bean 覆盖掉？

因为很多自动配置都会配合 `@ConditionalOnMissingBean`。只要你自己定义了同类型或同名 Bean，自动配置就会让位，这也是 Spring Boot“约定优于配置，但允许覆盖”的关键设计。

追问补充2： `spring.factories` 和 `AutoConfiguration.imports` 有什么区别？

本质上都是自动配置类清单，只是新版本把自动配置入口拆得更清晰了。面试里只要知道旧版本常见 `spring.factories`，新版本更多看 `AutoConfiguration.imports`，就够用了。

追问补充3：面试里怎么回答自动装配更像源码理解？

可以按这条线讲：启动类上的 `@EnableAutoConfiguration` 导入选择器，选择器去读自动配置清单，再根据条件注解筛选，最后把满足条件的配置类注册进容器。这个链路比只背“Spring Boot 会自动配置”更有说服力。

速记版

自动装配建议按这条线记：

- 入口在 `@EnableAutoConfiguration`
- 核心类是 `AutoConfigurationImportSelector`
- 清单里列自动配置类
- 条件注解决定是否真正生效
- `@ConditionalOnMissingBean` 给业务留覆盖空间

一句话面试版：

Spring Boot 自动装配的本质，是在启动时扫描一批候选配置类，再按类路径、配置项和现有 Bean 条件做筛选，最后把满足条件的 Bean 注册进容器。

中间件

kafka核心原理

kafka核心原理

Kafka 是大厂最常用的消息队列，面试中经常会问"Kafka 为什么这么快"和"如何保证消息不丢失"。

Kafka的基本概念

Q: 什么是 Kafka? 有什么特点?

Kafka 是一个分布式的流式处理平台，最初由 LinkedIn 开发，现在是 Apache 的顶级项目。

Kafka 的特点：

第一个是**高吞吐量**。Kafka 可以处理每秒数百万条消息，吞吐量非常高。

第二个是**低延迟**。Kafka 的消息延迟通常在毫秒级别。

第三个是**可扩展性**。Kafka 可以水平扩展，通过增加 Broker 节点来提高性能。

第四个是**持久化**。Kafka 的消息会持久化到磁盘，即使系统崩溃，消息也不会丢失。

第五个是**高可用**。Kafka 支持副本机制，即使某个节点宕机，消息也不会丢失。

Kafka 的核心概念：

- **Producer (生产者)**：发送消息到 Kafka。
- **Consumer (消费者)**：从 Kafka 读取消息。
- **Broker**：Kafka 的服务器节点，负责存储和转发消息。
- **Topic (主题)**：消息的分类，类似于数据库的表。
- **Partition (分区)**：Topic 的物理分组，每个 Partition 是一个有序的消息队列。
- **Consumer Group (消费者组)**：多个 Consumer 组成一个组，共同消费一个 Topic。

Kafka的架构

Q: Kafka 的架构是怎样的?

Kafka 的架构主要包括以下几个组件：

第一个是 Producer（生产者），负责发送消息到 Kafka。Producer 可以选择将消息发送到哪个 Partition，通常通过 Key 的 Hash 值来决定。

第二个是 Broker，Kafka 的服务器节点，负责存储和转发消息。一个 Kafka 集群通常有多个 Broker，每个 Broker 可以存储多个 Partition。

第三个是 Topic，消息的分类。一个 Topic 可以有多个 Partition，每个 Partition 是一个有序的消息队列。

第四个是 Partition，Topic 的物理分组。每个 Partition 是一个有序的消息队列，消息在 Partition 内是有序的，但不同 Partition 之间的消息是无序的。每个 Partition 可以有多个副本（Replica），分布在不同的 Broker 上，提高可用性。

第五个是 Consumer（消费者），负责从 Kafka 读取消息。Consumer 可以订阅一个或多个 Topic，从 Partition 中读取消息。

第六个是 Consumer Group（消费者组），多个 Consumer 组成一个组，共同消费一个 Topic。同一个 Consumer Group 中的 Consumer 不会重复消费同一条消息，每个 Partition 只会被一个 Consumer 消费。

第七个是 ZooKeeper（Kafka 2.8 之前）或 KRaft（Kafka 2.8 之后），负责管理 Kafka 集群的元数据，比如 Broker 的注册、Topic 的配置、Partition 的分配等。

架构图：

追问1: Kafka 为什么要用 Partition?

Partition 的主要作用有三个：

第一个是**提高并发性能**。一个 Topic 可以有多个 Partition，多个 Producer 可以并发地向不同的 Partition 写入消息，多个 Consumer 可以并发地从不同的 Partition 读取消息，提高了并发性能。

第二个是**实现负载均衡**。Partition 可以分布在不同的 Broker 上，实现负载均衡。

第三个是**实现水平扩展**。通过增加 Partition 的数量，可以提高 Kafka 的吞吐量。

Kafka的高吞吐原理

Q: Kafka 为什么这么快？高吞吐量是如何实现的？

Kafka 的高吞吐量主要依赖以下几个技术：

第一个是顺序写入磁盘。Kafka 的消息是顺序写入磁盘的，顺序写入的速度比随机写入快很多（可以达到几百 MB/s）。而且 Kafka 利用了操作系统的 Page Cache，写入时先写到内存，由操作系统异步刷新到磁盘，提高了写入速度。

第二个是零拷贝（Zero Copy）。Kafka 使用了零拷贝技术，减少了数据在内存和磁盘之间的拷贝次数。传统的数据传输需要经过 4 次拷贝（磁盘 → 内核缓冲区 → 用户缓冲区 → Socket 缓冲区 → 网卡），而零拷贝只需要 2 次拷贝（磁盘 → 内核缓冲区 → 网卡），大大提高了传输速度。

第三个是批量发送和压缩。Producer 可以批量发送消息，减少网络请求的次数。而且 Kafka 支持消息压缩（如 Gzip、Snappy、LZ4），减少了网络传输的数据量。

第四个是分区并行。一个 Topic 可以有多个 Partition，多个 Producer 可以并发地向不同的 Partition 写入消息，多个 Consumer 可以并发地从不同的 Partition 读取消息，提高了并发性能。

第五个是高效的数据结构。Kafka 使用了高效的数据结构，比如稀疏索引，可以快速定位消息的位置。

追问1：什么是零拷贝？

零拷贝是一种减少数据拷贝次数的技术。

传统的数据传输：

1. 数据从磁盘读取到内核缓冲区（DMA 拷贝）。
2. 数据从内核缓冲区拷贝到用户缓冲区（CPU 拷贝）。
3. 数据从用户缓冲区拷贝到 Socket 缓冲区（CPU 拷贝）。
4. 数据从 Socket 缓冲区发送到网卡（DMA 拷贝）。

总共 4 次拷贝，其中 2 次是 CPU 拷贝，占用 CPU 资源。

零拷贝：

使用 `sendfile()` 系统调用，数据直接从内核缓冲区发送到网卡，不需要经过用户缓冲区。

1. 数据从磁盘读取到内核缓冲区（DMA 拷贝）。
2. 数据从内核缓冲区发送到网卡（DMA 拷贝）。

总共 2 次拷贝，都是 DMA 拷贝，不占用 CPU 资源。

Kafka 使用了 Java 的 `FileChannel.transferTo()` 方法，底层就是 `sendfile()` 系统调用，实现了零拷贝。

消费者组

Q: 什么是消费者组? 有什么作用?

消费者组 (Consumer Group) 是 Kafka 的一个重要概念, 它是多个 Consumer 的集合。

消费者组的特点:

第一个是同一个 Consumer Group 中的 Consumer 不会重复消费同一条消息。每个 Partition 只会被一个 Consumer 消费, 实现了负载均衡。

第二个是不同的 Consumer Group 可以重复消费同一条消息。每个 Consumer Group 都有自己的消费进度 (Offset), 互不影响。

消费者组的作用:

第一个是实现负载均衡。一个 Topic 有多个 Partition, 多个 Consumer 可以并发地消费不同的 Partition, 提高消费速度。

第二个是实现广播和点对点。如果多个 Consumer 属于同一个 Consumer Group, 就是点对点模式 (每条消息只被消费一次)。如果多个 Consumer 属于不同的 Consumer Group, 就是广播模式 (每条消息被多次消费)。

消费者组的分配策略:

Kafka 有三种分配策略, 决定哪个 Consumer 消费哪个 Partition:

- Range (范围分配): 按 Partition 的范围分配给 Consumer。
- RoundRobin (轮询分配): 按轮询的方式分配给 Consumer。
- Sticky (粘性分配): 尽量保持原有的分配关系, 减少 Rebalance 的开销。

追问1: 什么是 Rebalance? 有什么问题?

****Rebalance (重平衡) ****是指 Consumer Group 中的 Consumer 数量发生变化时, 重新分配 Partition 的过程。

触发 Rebalance 的情况:

1. Consumer 加入或退出 Consumer Group。
2. Consumer 崩溃或超时。
3. Partition 数量发生变化。

Rebalance 的问题:

第一个是**停止消费**。Rebalance 期间，所有 Consumer 都会停止消费，等待 Rebalance 完成，影响消费速度。

第二个是**重复消费**。Rebalance 后，Consumer 可能会重新消费已经消费过的消息（如果 Offset 没有及时提交）。

如何减少 Rebalance：

第一个是**增加 Consumer 的超时时间**，避免因为网络抖动导致 Consumer 被踢出 Consumer Group。

第二个是**减少 Consumer 的处理时间**，避免因为处理时间过长导致 Consumer 超时。

第三个是**使用 Sticky 分配策略**，减少 Rebalance 的开销。

消息的可靠性

Q: Kafka 如何保证消息不丢失？

Kafka 保证消息不丢失主要依赖以下几个机制：

第一个是副本机制（Replication）。每个 Partition 可以有多个副本（Replica），分布在不同的 Broker 上。其中一个副本是 Leader，负责处理读写请求，其他副本是 Follower，负责同步 Leader 的数据。如果 Leader 宕机，会从 Follower 中选举一个新的 Leader，保证消息不丢失。

第二个是 ACK 机制。Producer 发送消息后，可以设置 ACK 级别，决定什么时候认为消息发送成功：

- **acks = 0**：Producer 不等待 Broker 的确认，直接认为发送成功。性能最高，但可能丢失消息。
- **acks = 1**：Producer 等待 Leader 确认，Leader 写入成功后返回确认。如果 Leader 宕机，Follower 还没同步，消息可能丢失。
- **acks = -1 (all)**：Producer 等待 Leader 和所有 ISR（In-Sync Replicas，同步副本）确认，所有副本都写入成功后返回确认。可靠性最高，但性能最低。

第三个是 ISR 机制。ISR 是指与 Leader 保持同步的 Follower 列表。只有 ISR 中的 Follower 才能被选举为 Leader。如果 Follower 落后太多，会被踢出 ISR。

第四个是 HW 和 LEO。HW（High Watermark，高水位）是指所有 ISR 都已经同步的最新消息的 Offset。LEO（Log End Offset，日志末端偏移量）是指 Partition 的最新消息的 Offset。Consumer 只能消费 HW 之前的消息，保证消息的一致性。

第五个是 Consumer 的 Offset 管理。Consumer 消费消息后，需要提交 Offset，记录消费进度。如果 Consumer 崩溃，重启后可以从上次提交的 Offset 继续消费，避免消息丢失。

如何保证消息不丢失：

- **Producer 端**：设置 `acks = -1`，等待所有 ISR 确认。设置 `retries > 0`，失败后自动重试。
- **Broker 端**：设置 `min.insync.replicas > 1`，保证至少有 2 个副本同步成功。设置 `unclean.leader.election.enable = false`，禁止非 ISR 的 Follower 被选举为 Leader。
- **Consumer 端**：手动提交 Offset，消费成功后再提交，避免消息丢失。

追问1: Kafka 如何保证消息不重复？

Kafka 本身无法完全保证消息不重复，需要在业务层面实现幂等性。

消息重复的原因：

第一个是 **Producer 重试**。如果 Producer 发送消息后，Broker 已经写入成功，但网络抖动导致 Producer 没有收到确认，Producer 会重试，导致消息重复。

第二个是 **Consumer 重复消费**。如果 Consumer 消费消息后，还没来得及提交 Offset 就崩溃了，重启后会重新消费，导致消息重复。

如何保证消息不重复：

Producer 端：

- 开启幂等性（`enable.idempotence = true`）。Kafka 会为每条消息分配一个唯一的 ID，Broker 会去重，避免重复写入。
- 开启事务（`transactional.id`）。Kafka 支持事务，可以保证一批消息要么全部成功，要么全部失败。

Consumer 端：

- 在业务层面实现幂等性。比如在数据库中记录消息的唯一 ID，消费前先检查是否已经消费过。
- 使用唯一键约束，避免重复插入。

Kafka的消息顺序

Q：Kafka 如何保证消息的顺序？

Kafka 的消息顺序保证分为两个层面：

第一个是 **Partition 内有序**。同一个 Partition 内的消息是有序的，按照发送的顺序存储和消费。

第二个是 **Topic 内无序**。不同 Partition 之间的消息是无序的，因为多个 Partition 是并行处理的。

如何保证消息的顺序：

第一个是**将需要保证顺序的消息发送到同一个 Partition**。可以通过指定 Key，Kafka 会根据 Key 的 Hash 值决定发送到哪个 Partition。相同 Key 的消息会发送到同一个 Partition，保证顺序。

第二个是**设置 Partition 数量为 1**。如果 Topic 只有一个 Partition，所有消息都是有序的。但这样会降低并发性能，不推荐。

第三个是**Consumer 端单线程消费**。如果 Consumer 使用多线程消费，可能会导致消息乱序。应该使用单线程消费，或者在业务层面保证顺序。

追问1：如果要保证全局有序，怎么办？

如果要保证全局有序，有两种方案：

第一种是设置 Topic 只有一个 Partition。这样所有消息都在一个 Partition 中，自然是有序的。但这样会降低并发性能，吞吐量很低。

第二种是在业务层面保证顺序。比如给每条消息加一个序号，Consumer 消费时按序号排序，保证顺序。但这样会增加业务复杂度。

实际场景：

大多数场景下，不需要保证全局有序，只需要保证局部有序（比如同一个用户的消息有序）。这时候可以用 Key 来保证，相同 Key 的消息发送到同一个 Partition，保证顺序。

Kafka的存储机制

Q：Kafka 的消息是如何存储的？

Kafka 的消息存储机制主要有以下几个特点：

第一个是分段存储（Segment）。每个 Partition 的消息不是存储在一个大文件中，而是分成多个 Segment（段）。每个 Segment 包含一个数据文件（.log）和两个索引文件（.index 和 .timeindex）。

第二个是稀疏索引。Kafka 使用稀疏索引，不是每条消息都有索引，而是每隔一定数量的消息才有一个索引。这样可以减少索引的大小，提高查询速度。

第三个是顺序写入。Kafka 的消息是顺序写入磁盘的，顺序写入的速度比随机写入快很多。

第四个是定期清理。Kafka 会定期清理过期的消息，有两种清理策略：

- **按时间清理：**删除超过一定时间的消息（如 7 天）。
- **按大小清理：**删除超过一定大小的消息（如 1GB）。

Segment 的结构：

每个 Segment 的文件名是该 Segment 的起始 Offset。

查询消息的流程：

1. 根据 Offset 找到对应的 Segment。
2. 在 Segment 的索引文件中查找 Offset 的位置。
3. 在数据文件中读取消息。

追问1：Kafka 为什么不用数据库存储消息？

Kafka 不用数据库存储消息主要有两个原因：

第一个是**性能**。数据库的随机写入速度很慢，而 Kafka 使用顺序写入磁盘，速度快很多。而且数据库需要维护索引、事务等，开销很大。

第二个是**扩展性**。数据库的扩展性有限，而 Kafka 可以通过增加 Broker 和 Partition 来水平扩展，吞吐量可以达到每秒数百万条消息。

Kafka 的设计理念是"消息队列"，不是"数据库"，它的目标是高吞吐、低延迟，而不是复杂的查询和事务。

Rebalance 机制

Q：什么是 Rebalance？它有什么影响？

Rebalance 指的是消费者组内分区分配关系的重新调整。

常见触发条件包括：

- 消费者数量变化
- Topic 分区数变化
- Consumer 宕机或重启

它的好处是可以动态平衡消费负载，但坏处是：**Rebalance 期间消费会短暂停顿**。

如果 Rebalance 太频繁，就会导致吞吐抖动，甚至进一步引发消息堆积。

Offset 提交

Q：自动提交 Offset 和手动提交 Offset 有什么区别？

自动提交的优点是简单，但风险是业务还没真正处理成功，Offset 就已经提交了。

手动提交更安全，一般会在业务处理成功后再提交 Offset。

所以如果业务对可靠性要求高，我更倾向于使用 **手动提交 Offset**。

ISR 机制

Q: 什么是 ISR?

ISR 指的是 **In-Sync Replicas**，也就是和 Leader 保持同步的副本集合。

当 Producer 配置 `acks=-1` 时，本质上就是要求消息至少被 ISR 中的副本确认后才返回成功。这样可以提升可靠性，但写入延迟会稍微高一些。

消息堆积

Q: Kafka 发生消息堆积时，你怎么排查?

消息堆积通常说明：生产速度大于消费速度。

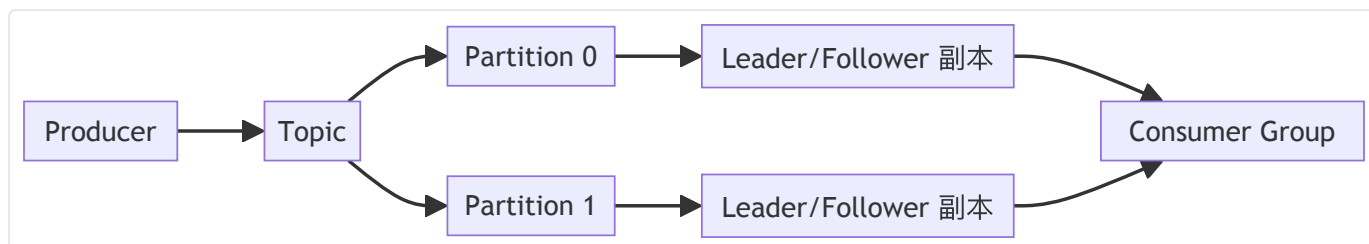
我会优先看：

- Consumer 是否异常或频繁 Rebalance
- 单条消息处理是否太慢
- 下游依赖是否卡住
- Partition 数量是否不够，导致并行度受限

常见优化方向包括：

- 增加 Consumer 并发
- 增加 Partition 数量
- 优化业务处理逻辑
- 临时扩容做削峰

图解补充：Kafka 核心拓扑



对比补充：可靠性怎么记

机制	作用	面试里的关键词
副本机制	Broker 宕机后仍可恢复	Leader、Follower
ACK	控制发送成功的确认时机	<code>acks=0/1/all</code>
ISR	保证只有同步足够的副本参与选主	同步副本集合
HW/LEO	控制消费者可见范围	高水位
Offset 提交	记录消费进度	至少一次/重复消费

高频追问补充

追问补充1：为什么 Kafka 的顺序性只能保证分区内有序？

因为每个 Partition 本质上就是一个有序日志文件，顺序是建立在单个分区内部的。多个分区并行写入和并行消费时，天然就没有全局顺序。

追问补充2：Rebalance 为什么会影响消费体验？

因为 Rebalance 期间消费者组要重新分配分区，消费者会短暂停止拉取消息。如果业务处理时间长、心跳不稳或频繁扩缩容，就容易反复触发 Rebalance，造成抖动。

追问补充3：Kafka 怎么回答“如何保证消息不丢”更完整？

最好按三端回答：Producer 端设 `acks=all` 和重试，Broker 端设副本和 `min.insync.replicas`，Consumer 端消费成功后再提交 Offset。这样更像完整链路，不是只背一端配置。

速记版

Kafka 建议按这条线记：

- Topic 用 Partition 拆并发
- Broker 用副本保高可用
- Producer 端靠 ACK 和重试保发送
- Consumer 端靠 Offset 管理进度
- Rebalance 是消费者组的常见风险点

一句话面试版：

Kafka 的核心设计就是 分区做并行，副本做可靠性，顺序写和零拷贝做高吞吐。

xxl-job

常问问题：

为什么不用 Spring 定时任务、调度中心和执行器怎么配合、失败重试和分片广播怎么做。

XXL-JOB 是什么

Q：什么是 XXL-JOB？它主要解决什么问题？

XXL-JOB 是一个分布式任务调度平台。

它要解决的核心问题是：

- 定时任务怎么统一管理
- 多台机器上任务怎么调度
- 任务失败了怎么重试
- 任务执行日志怎么查看
- 同一任务怎么避免多实例重复执行

如果一句话概括：

XXL-JOB 解决的是“任务怎么稳定地、可视化地、分布式地执行”。

为什么不用 Spring 自带定时任务

Q：为什么很多项目不用 `@Scheduled`，而是上 XXL-JOB？

Spring 自带定时任务适合简单场景，但一旦到了分布式环境，问题就来了：

- 多实例部署后可能重复执行
- 没有统一调度中心
- 任务日志、执行历史、告警能力较弱
- 动态修改任务配置不方便

而 XXL-JOB 提供了：

- 调度中心
- 执行器管理
- 任务路由
- 日志查看
- 失败重试
- 分片广播

所以它更适合中后台任务平台化管理。

XXL-JOB 的核心架构

Q：XXL-JOB 的核心架构是怎样的？

XXL-JOB 的典型架构里有两个核心角色：

- **Admin 调度中心**：负责任务配置、调度、监控、日志管理
- **Executor 执行器**：真正执行任务的应用节点

大致流程是：

1. 执行器启动后向调度中心注册
2. 调度中心根据 cron 或其他触发规则生成调度
3. 调度中心按路由策略选择一个或多个执行器
4. 执行器执行任务并回传结果
5. 调度中心记录日志、状态、失败重试信息

调度中心和执行器怎么配合

Q: XXL-JOB 里调度中心和执行器分别负责什么？

调度中心

主要负责：

- 维护任务配置
- 计算触发时间
- 选择目标执行器
- 记录任务执行日志
- 失败重试和报警

执行器

主要负责：

- 注册自己
- 暴露任务处理器
- 真正执行任务逻辑
- 把执行结果回传给调度中心

所以面试里可以直接说：

调度中心负责“什么时候、让谁执行”，执行器负责“具体怎么执行”。

路由策略怎么理解

Q: XXL-JOB 的路由策略是什么？为什么需要路由策略？

因为一个任务往往会部署在多个执行器实例上，所以调度中心要决定：

这次调度，到底交给哪台机器执行。

常见路由策略可以理解成：

- 轮询
- 随机
- 故障转移
- 忙碌转移
- 分片广播

其中最典型的是：

故障转移

某台执行器不可用时，切换到其他节点。

分片广播

把一个任务拆成多个分片，让多台执行器分别处理不同分片。

这对批处理、大数据同步、分片扫描任务很常见。

分片广播是什么

Q：XXL-JOB 的分片广播是干什么的？

分片广播适合：

同一个任务逻辑，需要多台机器并行处理不同数据片段。

比如：

- 多库分片数据扫描
- 大量用户数据批处理
- 大表分段修复任务

调度中心会把总分片数和当前分片号传给执行器，执行器按分片参数只处理自己负责的一段。

所以分片广播不是简单“广播执行”，而是：

同一任务的并行切片执行。

失败重试怎么做

Q：XXL-JOB 任务执行失败后，一般怎么处理？

XXL-JOB 常见失败处理思路有：

- 记录失败日志
- 配置失败重试次数
- 报警通知
- 人工补偿或手工重跑

但面试里要注意：

任务平台只能帮你做调度和重试，业务任务本身最好具备幂等性。

比如发券任务、同步任务、补偿任务，如果不幂等，重试可能反而放大问题。

XXL-JOB 和 Quartz 怎么比

Q：XXL-JOB 和 Quartz 有什么区别？

一个常见答法是：

- Quartz 更像一个任务调度组件库
- XXL-JOB 更像一个完整的分布式任务调度平台

XXL-JOB 提供了：

- Web 管理界面
- 执行器注册
- 路由策略
- 日志与告警
- 分片广播

所以很多团队不是因为 Quartz 不行，而是因为：

当任务数量、节点数量、运维复杂度上来后，更需要平台化的调度能力。

项目里怎么说

Q：如果项目里用了 XXL-JOB，面试时怎么讲？

可以这样组织：

1. 场景

比如定时同步、对账、补偿、批处理、数据修复。

2. 为什么选 XXL-JOB

比如多实例环境下需要统一调度和可视化管理。

3. 怎么保证稳定

比如执行器分组、失败重试、报警、幂等、分片广播。

4. 踩过什么坑

比如任务执行时间过长、重试导致重复处理、分片设计不均匀。

高频追问补充

追问1：XXL-JOB 能保证任务绝对只执行一次吗？

平台会尽量做调度控制，但在重试、网络抖动、执行器异常等情况下，业务仍然需要靠幂等兜底。

追问2：XXL-JOB 和 MQ 有什么区别？

XXL-JOB 主要做定时和调度；

MQ 主要做异步解耦和消息传递。

追问3：为什么任务平台也要监控和告警？

因为任务很多时候承担的是对账、补偿、数据同步这类关键后台链路，一旦失败，影响未必比主链路小

RocketMQ

RocketMQ 在 Java 面试里经常被放在“业务消息队列”语境下讨论。

面试官通常会追问的不是“RocketMQ 是什么”，而是：为什么很多交易系统会选它、顺序消息怎么做、事务消息怎么回事、消费失败怎么处理。

RocketMQ 是什么，适合什么场景

Q：什么是 RocketMQ？它更适合什么场景？

RocketMQ 是 Apache 顶级项目之一，本质上也是一个分布式消息中间件。

它在 Java 业务场景里比较常见，原因通常有三点：

- 对顺序消息支持比较友好
- 对延迟消息、事务消息这类业务型能力支持较完整
- 很多业务系统更关注“交易、订单、库存”这种消息链路，而不是极致日志吞吐

所以如果一句话概括：

RocketMQ 更适合核心业务消息场景，而 Kafka 更偏数据流和日志流。

RocketMQ 的核心概念

Q：RocketMQ 里最重要的几个概念是什么？

RocketMQ 里常见的核心概念有：

- **Producer**：生产者
- **Consumer**：消费者
- **NameServer**：路由中心，负责记录 Topic 和 Broker 的路由信息
- **Broker**：真正存储消息、提供读写服务的节点
- **Topic**：消息主题
- **Message Queue**：一个 Topic 下的逻辑队列
- **Consumer Group**：消费组
- **Tag**：更细粒度的消息分类

你可以把它理解成：

- NameServer 解决“我该把消息发到哪”
- Broker 解决“消息真正存在哪、怎么投递”
- Queue 解决“并行度和局部有序”

RocketMQ 的架构和 workflows

Q：RocketMQ 的架构是怎样的？

RocketMQ 架构里最典型的链路是：

1. Producer 启动时先从 NameServer 拉取 Topic 路由信息
2. Producer 根据路由把消息发给某个 Broker
3. Broker 把消息写入 CommitLog
4. Consumer 从 Broker 拉消息并消费
5. 消费成功后返回确认，失败则重试

RocketMQ 的一个特点是 **NameServer** 本身不做强一致协调，它更像轻量级路由发现组件。真正的消息存储和转发，还是在 Broker 上完成。

为什么很多业务系统会选 RocketMQ

Q: RocketMQ 相比 Kafka、RabbitMQ，优势通常体现在哪里？

RocketMQ 在业务系统里的优势通常会从下面几个点来讲：

第一，业务特性比较完整。

顺序消息、延迟消息、事务消息、重试、死信，这些业务里很常见。

第二，Java 生态结合比较顺手。

很多 Java 团队更熟悉它的使用方式、监控方式和工程表达。

第三，适合订单、支付、库存这类链路。

这些场景更看重消息可靠性、顺序控制、事务协同，而不只是极致吞吐。

所以如果面试问“为什么没选 Kafka”，一个合理回答是：

因为当前系统的重点不是日志流吞吐，而是核心业务消息的可靠投递、顺序控制和事务协同。

RocketMQ 的消息存储模型

Q: RocketMQ 的消息是怎么存储的？

RocketMQ 最核心的存储文件是 **CommitLog**。

它的大致思路是：

- 所有消息先顺序写入 **CommitLog**
- 再通过 **ConsumeQueue** 建立逻辑队列索引
- 通过索引帮助消费者快速定位消息

这样设计的好处是：

- 写入路径比较统一
- 顺序写磁盘性能较好
- 逻辑队列和物理存储分离，便于消费

这个思路和 Kafka 有点像，都是在利用**顺序写磁盘**来换高吞吐。

但 RocketMQ 在业务能力层面做了更多增强。

RocketMQ 有哪些常见消息类型

Q: RocketMQ 常见的消息类型有哪些？

面试里最常见的通常有这几类：

普通消息

最基础的异步解耦消息。

顺序消息

要求同一类业务消息按顺序消费。

常见于订单状态流转、步骤编排。

延迟消息

消息先发出去，但不是立刻消费，而是在指定延迟后投递。

常见于超时关单、超时取消、补偿检查。

事务消息

用于把“本地事务”和“消息最终可见”关联起来。

常见于下单成功后异步通知库存、积分、营销系统。

重试消息 / 死信消息

消费失败会重试，多次失败后进入死信队列，方便后续排查和补偿。

RocketMQ 怎么保证消息不丢

Q: RocketMQ 如何保证消息不丢失？

RocketMQ 的可靠性也要从三端来看：

Producer 端

- 发送失败重试
- 关键消息做好发送结果确认
- 必要时使用同步发送

Broker 端

- 主从复制
- 刷盘策略合理配置
- 持久化成功后再返回

Consumer 端

- 消费成功后再确认
- 失败时按重试策略处理
- 业务层面做幂等

所以不要只回答“RocketMQ 有主从”。

更完整的表达是：

生产端保证送达，Broker 保证持久化和复制，消费端保证正确确认和幂等处理。

顺序消息怎么实现

Q：RocketMQ 的顺序消息是怎么做的？

顺序消息的核心思路是：

让同一业务主键的消息落到同一个 Queue，再由消费端顺序处理。

比如一个订单从“创建 -> 支付 -> 发货 -> 完成”，如果要保证状态顺序：

- 可以按 `orderId` 选择同一个 Queue
- 消费端对这个 Queue 顺序消费

要注意：

- 顺序通常是局部顺序，不是全局顺序
- 一旦消费端并发打散，顺序也可能被破坏

所以顺序消息不只是 Broker 的事，生产端路由和消费端模型也要一起配合。

事务消息是什么

Q：RocketMQ 的事务消息解决什么问题？

RocketMQ 的事务消息主要解决的是：

本地事务和消息发送之间的一致性问题。

经典场景是：

1. 先发送半消息
2. 执行业务本地事务
3. 本地事务成功，则提交消息
4. 本地事务失败，则回滚消息
5. 如果状态不明确，Broker 会回查生产者

这个机制适合：

- 下单成功后通知库存
- 支付成功后通知营销系统
- 核心写库成功后再异步通知下游

但也要说明边界：

事务消息不是万能分布式事务，它更像一种“消息最终一致”的工程方案。

RocketMQ 消费失败怎么处理

Q: RocketMQ 消费失败后会怎样?

RocketMQ 对消费失败一般会做这几层处理:

1. 先重试

业务抖动、网络抖动类问题先靠重试兜住。

2. 达到重试上限后进死信队列

便于人工排查和补偿。

3. 业务幂等保护

防止因为重试把下游写坏。

所以一个成熟答案不能只说“RocketMQ 会重试”，还要补:

重试只是第一层，真正稳定的系统还要有死信、补偿、监控、人工介入通道。

广播消费和集群消费怎么理解

Q: RocketMQ 的广播消费和集群消费有什么区别?

集群消费

同一个消费组里的多个消费者共同分摊消息。

一条消息只会被组内一个实例处理。

广播消费

消息会被组内所有消费者各消费一次。

适合配置下发、缓存刷新、广播通知这类场景。

所以面试里可以直接说:

集群消费强调分摊吞吐，广播消费强调每个节点都收到。

项目里怎么说

Q: 如果项目里用了 RocketMQ, 面试时应该怎么讲?

推荐答法可以这样组织:

1. 场景

比如下单后异步发券、发通知、同步库存。

2. 为什么选 RocketMQ

比如看重顺序消息、延迟消息、事务消息等能力。

3. 可靠性设计

比如主从复制、重试、死信、消费幂等。

4. 遇到的问题

比如消息积压、消费重试过多、死信处理、顺序消费吞吐受限。

这样回答会比只背概念更像真实做过。

高频追问补充

追问1: RocketMQ 和 Kafka 最核心的区别怎么说?

一句话回答:

Kafka 更偏高吞吐数据流平台, RocketMQ 更偏业务型消息平台。

追问2: RocketMQ 的顺序消息是全局顺序吗?

通常不是。

大多数业务里做的是“同一业务主键的局部顺序”。

追问3: 用了事务消息, 是不是就不需要幂等了?

不是。

事务消息解决的是消息可见性和本地事务之间的一致性, 消费端仍然可能重试, 业务还是要做幂等。

更新: 2026-04-01

RabbitMQ

一般都考察Exchange 有哪些类型、消息怎么保证可靠、为什么会积压、死信和 TTL 怎么用。

RabbitMQ 是什么，适合什么场景

Q：什么是 RabbitMQ？它更适合哪些场景？

RabbitMQ 是一个实现了 AMQP 协议的消息中间件。

它最突出的特点不是极致吞吐，而是：

- 模型成熟
- 路由灵活
- 工程使用门槛相对低
- 很适合通知分发、解耦、工作队列、规则路由

所以 RabbitMQ 往往更适合：

- 中后台业务系统
- 规则路由较复杂的场景
- 通知、审批、异步任务分发
- 吞吐不是第一目标，但稳定性和路由灵活性很重要的系统

RabbitMQ 的核心概念

Q：RabbitMQ 的核心概念有哪些？

RabbitMQ 里最重要的概念是：

- **Producer**：生产者
- **Consumer**：消费者
- **Exchange**：交换机，负责路由消息
- **Queue**：队列，真正存消息
- **Binding**：绑定关系，决定 Exchange 怎么把消息投到 Queue
- **Routing Key**：路由键
- **Channel**：信道，AMQP 的轻量级通信通道
- **Virtual Host**：虚拟主机，用于隔离资源

RabbitMQ 最大的特点就是：

生产者通常不是把消息直接发到队列，而是先发给 Exchange，再由 Exchange 路由到 Queue。

Exchange 有哪些类型

Q: RabbitMQ 的 Exchange 有哪些类型?

最常见的有四种：

Direct Exchange

按精确的 `routing key` 路由。
适合一对一、定向投递。

Fanout Exchange

忽略 `routing key`，直接广播到所有绑定队列。
适合广播通知。

Topic Exchange

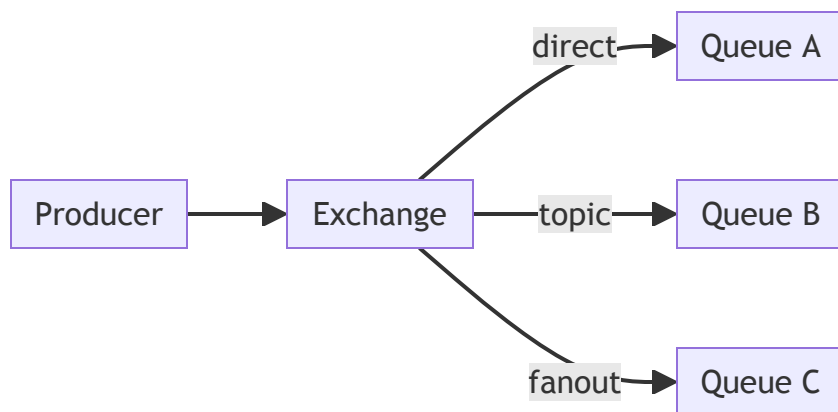
支持通配符匹配。
适合按主题、规则进行灵活路由。

Headers Exchange

按消息头匹配。
实际项目里没有前三种常见。

面试里最常用的答法是：

RabbitMQ 的强项就在于 Exchange 模型，它让消息路由比很多 MQ 更灵活。



RabbitMQ 的工作流程

Q: Producer 发消息到 RabbitMQ, 大致流程是什么?

RabbitMQ 的典型流程是:

1. Producer 建立连接和 Channel
2. Producer 把消息发到某个 Exchange
3. Exchange 根据 Binding 和 Routing Key 路由到一个或多个 Queue
4. Queue 持久化消息
5. Consumer 订阅 Queue 并消费
6. Consumer 消费成功后返回 ack

这个流程和 Kafka、RocketMQ 最大不同点在于:

RabbitMQ 先强调“路由”，再强调“存储和消费”。

RabbitMQ 如何保证消息不丢

Q: RabbitMQ 如何保证消息不丢失?

RabbitMQ 的可靠性一般要分三段看：

1. 生产者到 Broker

- 开启 Publisher Confirm
- 必要时配合 Return 机制 处理无法路由的消息

2. Broker 内部存储

- Exchange、Queue、Message 设置为持久化
- 必要时使用高可用队列

3. Broker 到消费者

- 使用手动 `ack`
- 消费成功后再确认
- 失败时重试或进入死信

所以 RabbitMQ 可靠性的标准答法通常是：

生产端 Confirm，Broker 端持久化，消费端手动 `ack`。

ack、nack、reject 是什么

Q：RabbitMQ 里的 `ack`、`nack`、`reject` 是什么？

这些都是消费者对消息处理结果的反馈：

- `ack`：消费成功，Broker 可以删消息
- `nack`：消费失败，可以选择重回队列
- `reject`：拒绝这条消息，也可以决定是否重回队列

面试里要说明：

如果不开手动 `ack`，或者业务还没成功就 `ack`，就可能导致消息丢失。

prefetch 有什么作用

Q: RabbitMQ 的 prefetch 是干什么的?

`prefetch` 可以理解成:

消费者一次最多预取多少条尚未确认的消息。

它的作用主要有两个:

- 防止某个消费者一次拿太多消息, 处理不过来
- 帮助做消费端流控和公平分发

如果 `prefetch` 设得太大, 可能导致某个慢消费者手里压很多消息;

如果设得太小, 吞吐又可能起不来。

所以它本质上是 RabbitMQ 消费端的重要流控参数。

TTL、死信队列、延迟消息怎么理解

Q: RabbitMQ 的 TTL、死信队列、延迟消息分别是什么?

TTL

给消息或队列设置生存时间，过期后不再正常消费。

死信队列

消息因为以下原因无法正常消费时，会进入死信：

- 被拒绝且不重新入队
- 超过 TTL
- 队列达到长度限制

延迟消息

RabbitMQ 原生更多依赖 **TTL + 死信交换机** 组合，或者使用延迟消息插件来实现。

这几个能力在业务里很常见：

- 超时取消订单
- 重试后仍失败的消息转人工处理
- 定时提醒、延时通知

RabbitMQ 为什么容易积压

Q：RabbitMQ 出现消息积压，一般从哪些方向排查？

消息积压通常从三个方向看：

1. 生产太快

上游突发流量，把队列瞬间打满。

2. 消费太慢

- 消费逻辑太重
- 下游数据库或 RPC 慢
- 消费者实例数不够

3. 配置不合理

- `prefetch` 设置不合理
- `ack` 太慢
- 单队列成瓶颈

RabbitMQ 更适合灵活路由，但如果拿它扛超大吞吐，又没有足够并行和治理，就容易积压。

RabbitMQ 的高可用怎么讲

Q：RabbitMQ 的高可用一般怎么实现？

RabbitMQ 的高可用通常会提到：

- 集群部署
- 队列镜像或更现代的 `Quorum Queue`
- 持久化
- 节点故障转移

面试里别只停留在“做集群”，还可以补一句：

RabbitMQ 的普通集群不等于消息天然高可用，关键还要看队列副本机制怎么配。

项目里怎么说

Q：如果项目里用了 RabbitMQ，面试时怎么讲更像真实做过？

推荐讲法：

1. 场景

比如消息通知、审批流、订单状态广播、异步任务解耦。

2. 为什么选 RabbitMQ

比如更看重路由灵活、协议成熟、团队使用成本低。

3. 可靠性怎么做

比如 confirm、持久化、手动 ack、死信队列、幂等。

4. 遇到的问题

比如消息积压、消费不均衡、延迟消息实现、死信补偿。

高频追问补充

追问1：RabbitMQ 和 Kafka 最大区别怎么说？

一句话回答：

RabbitMQ 更强调灵活路由和业务队列模型，Kafka 更强调高吞吐和流式处理。

追问2：RabbitMQ 的消息是直接发到队列吗？

通常不是。

Producer 是先到 Exchange，再由 Exchange 路由到 Queue。

追问3：有了手动 ack，为什么还要幂等？

因为消费失败重试、网络抖动、消费者重启都可能带来重复投递，业务层幂等仍然需要做。

Netty

Netty 在面试里经常被放在“高性能网络框架”语境下问。

它常常和 NIO、Reactor、Dubbo、网关、IM、长连接一起出现。

Netty 是什么

Q: 什么是 Netty? 为什么很多框架都喜欢基于 Netty?

Netty 是一个基于 Java NIO 的高性能网络通信框架。

它解决的核心问题是:

- 原生 NIO 编程太复杂
- 连接管理、事件分发、粘包拆包、心跳检测这些基础能力很麻烦
- 业务框架不想每次都从 Socket / Selector 重新造轮子

所以很多 RPC 框架、网关、IM 系统、消息中间件客户端都会基于 Netty。

一句话概括:

Netty 把高性能网络编程里那些复杂但重复的基础设施抽象好了。

Netty 和原生 NIO 的区别

Q: Netty 相比原生 NIO, 主要提升在哪里?

原生 NIO 的核心问题不是能力不够，而是编码复杂度高。

Netty 主要帮你封装了这些东西：

- 连接和线程模型
- 事件分发
- 编解码器
- 粘包拆包处理
- 心跳和空闲检测
- 内存管理和缓冲区封装

所以如果面试官问“为什么不用原生 NIO”，你可以答：

原生 NIO 能做，但工程复杂度高；Netty 更适合构建可维护的高性能网络服务。

Reactor 模型是什么

Q：Netty 为什么总会和 Reactor 模型一起说？

因为 Netty 的事件驱动模型，本质上就是 Reactor 思想的工程实现。

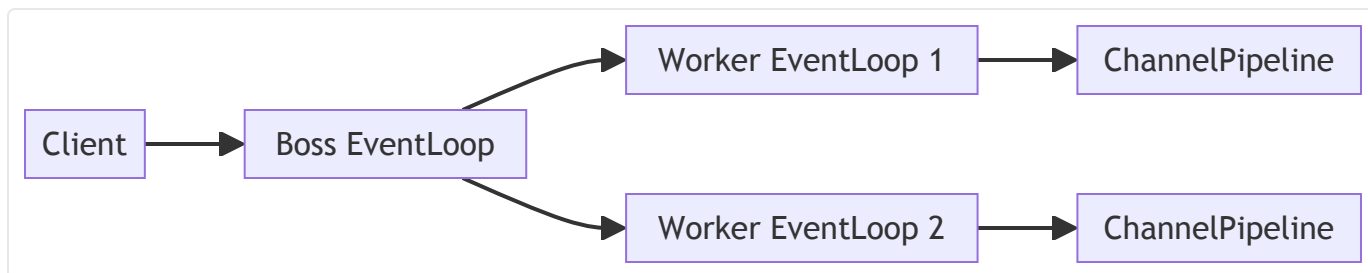
Reactor 的核心是：

把连接建立、事件监听、事件分发、业务处理拆开，由事件循环统一调度。

Netty 里最常见的是主从 Reactor：

- 一个 Boss 线程组负责接收连接
- 一个 Worker 线程组负责处理读写事件

这比“一个连接一个线程”的 BIO 模式更适合高并发连接场景。



EventLoop 是什么

Q: Netty 里的 EventLoop 是什么？

EventLoop 可以理解成：

一个不断执行事件循环的线程执行单元。

它通常负责：

- 监听 IO 事件
- 执行 Channel 上的任务
- 调度定时任务

一个 Channel 通常会绑定到一个固定的 EventLoop。

这样做好处是：

- 避免并发竞争
- 简化线程安全问题
- 保持同一连接上的事件顺序

Channel 和 Pipeline 是什么

Q: Netty 里的 Channel 和 Pipeline 分别是什么？

Channel

表示一个网络连接或 IO 通道。

它可以理解成对底层 Socket 的高级抽象。

ChannelPipeline

表示这条连接上的处理链。

消息进来后会沿着 Pipeline 经过多个 Handler。

所以 Netty 很像一个责任链模式：

- 入站事件一层层往下走
- 出站事件按另一方向处理

这让编解码、鉴权、日志、心跳、业务处理都可以拆分成独立 Handler。

ByteBuf 为什么重要

Q: Netty 为什么不用原生 ByteBuffer, 而是自己搞了 ByteBuf?

因为原生 `ByteBuffer` 在使用上有不少不便, 比如:

- 读写模式切换麻烦
- 扩容不友好
- API 体验一般

Netty 的 `ByteBuf` 做了更好的抽象:

- 读写指针分离
- 更灵活的内存操作
- 支持池化和零拷贝相关优化

所以 `ByteBuf` 是 Netty 提升易用性和性能的核心组件之一。

粘包拆包问题怎么理解

Q：为什么基于 TCP 的 Netty 通信会有粘包拆包问题？

因为 TCP 是字节流协议，它不关心应用层消息边界。

所以应用层看到的数据可能出现：

- 多条消息粘在一起
- 一条消息被拆成多段

Netty 常见解决方式有：

- 固定长度协议
- 分隔符协议
- 长度字段协议

面试里要说明：

粘包拆包不是 Netty 的问题，而是 TCP 字节流模型天然带来的问题，Netty 只是帮你更方便地解决。

心跳和空闲检测怎么做

Q：Netty 里为什么经常会做心跳检测？

长连接场景里，如果一条连接长时间没有数据，不代表它一定有效。

所以很多系统会做：

- 心跳包
- 空闲检测
- 超时关闭

Netty 常通过空闲检测 Handler 来感知读空闲、写空闲、全空闲，然后触发心跳或断连处理。

这在 IM、网关、RPC 长连接场景里都非常常见。

为什么说 Netty 性能高

Q: Netty 性能高主要体现在哪些地方?

Netty 的高性能通常来自这些点:

- 基于 NIO 的事件驱动模型
- Reactor 线程模型
- 减少锁竞争
- 高效的内存管理和对象复用
- 丰富的编解码和连接管理能力

但面试里不要说成“用了 Netty 就一定快”。

如果业务 Handler 里写了阻塞操作，再好的 EventLoop 也会被拖慢。

Netty 使用时常见坑

Q: 使用 Netty 时，常见的坑有哪些?

常见坑通常有:

- 在 IO 线程里做耗时阻塞逻辑
- 忽略粘包拆包
- 连接数太多但心跳和空闲处理不合理
- 直接内存泄漏
- Handler 线程安全问题

其中最经典的一句是:

不要在 EventLoop 线程里做重业务逻辑，IO 线程应该专注处理网络事件。

项目里怎么说

Q: 如果项目里用过 Netty，面试时怎么讲?

可以这样组织：

1. 用在什么场景

比如 RPC 通信、长连接网关、IM、内部二进制协议通信。

2. 为什么用 Netty

比如原生 NIO 开发复杂，Netty 提供更成熟的网络编程模型。

3. 你关注的点

比如线程模型、编解码、粘包拆包、心跳、连接管理。

4. 踩过的坑

比如阻塞 IO 线程、内存泄漏、拆包协议设计不合理。

高频追问补充

追问1：Netty 和 Tomcat 是一类东西吗？

不完全一样。

Netty 是底层网络通信框架，Tomcat 是 Servlet 容器和 Web 服务器。

追问2：Netty 和 Dubbo 有什么关系？

Dubbo 这类 RPC 框架经常会基于 Netty 做底层通信，但 Dubbo 还会再往上封装服务发现、序列化、容错治理等能力。

追问3：Netty 和 IO模型 怎么关联？

Netty 可以看成是对 NIO 多路复用模型的一层工程化封装，底层思想和 `Selector + Channel + 非阻塞` 是一脉相承的。

Elasticsearch

Elasticsearch 是面试里很典型的“搜索中间件”题。

它通常不会只考“ES 是什么”，而是会连着问：为什么搜得快、倒排索引怎么回事、为什么叫近实时、分片副本怎么设计、为什么不直接用 MySQL。

Elasticsearch 是什么

Q：什么是 Elasticsearch？它擅长解决什么问题？

Elasticsearch 本质上是一个分布式全文检索和分析引擎。

它最擅长做的是：

- 全文搜索
- 复杂条件检索
- 聚合分析
- 海量文档检索

它的典型场景包括：

- 商品搜索
- 日志检索
- 文章检索
- 运营分析和报表

如果一句话概括：

ES 的核心价值，是把“模糊搜索 + 海量数据 + 复杂查询”这件事做得比关系型数据库更合适。

为什么不直接用 MySQL 做搜索

Q：为什么很多搜索场景不用 MySQL，而要上 Elasticsearch？

MySQL 当然能做简单查询，但它在全文搜索场景下通常不是最佳选择。

主要原因有：

- 模糊查询性能差
- 多字段全文检索不擅长
- 排序、相关性评分能力弱
- 海量搜索扩展性有限

而 Elasticsearch 之所以更适合，是因为它底层依赖：

- 倒排索引
- 分词器
- 分片并行搜索

所以不是 MySQL 不能搜，而是：

搜索系统的核心诉求和关系型数据库的设计目标不一样。

什么是倒排索引

Q：什么是倒排索引？为什么 Elasticsearch 查得快？

倒排索引可以简单理解成：

不是“文档里有哪些词”，而是“每个词出现在哪些文档里”。

比如：

- “Java” -> 文档 1、文档 3、文档 8
- “中间件” -> 文档 2、文档 3

这样查关键词时，不需要全表扫描，而是直接通过词项找到命中的文档集合。

所以 ES 搜索快的核心不是“存在内存里”，而是：

它预先把检索结构组织好了。

分词器是什么

Q：为什么 Elasticsearch 需要分词器？

因为全文搜索不是简单拿整段文本做等值匹配，而是要把文本拆成适合检索的词条。

分词器大致会做三件事：

1. 字符过滤
2. 分词
3. 词项归一化

比如中英文、大小写、停用词、同义词，都会影响搜索效果。

面试里常见的表达是：

倒排索引决定“怎么存”，分词器决定“拆成什么词去存”。

Elasticsearch 的核心概念

Q：ES 里有哪些核心概念？

面试里最常见的概念有：

- **Cluster**：集群
- **Node**：节点
- **Index**：索引，类似数据库里的逻辑集合
- **Document**：文档，一条 JSON 数据
- **Shard**：分片
- **Replica**：副本
- **Segment**：Lucene 底层的索引段

可以粗暴理解成：

- Index 类似库表级逻辑对象
- Document 类似一条记录
- Shard 决定怎么水平拆分
- Replica 决定怎么高可用和读扩展

分片和副本怎么理解

Q: ES 的分片和副本分别解决什么问题?

分片

主要解决:

- 单机放不下
- 单机查不过来
- 数据和查询的水平扩展

副本

主要解决:

- 高可用
- 读请求分担

所以面试里可以这样答:

分片解决扩展性, 副本解决可用性和读能力。

Elasticsearch 为什么叫近实时

Q: 为什么 Elasticsearch 叫“近实时”, 而不是强实时?

因为一条数据写入 ES 后, 并不是立刻就对搜索可见。

典型流程是:

1. 文档先写入内存缓冲和 translog
2. 定期执行 `refresh`
3. 刷新后生成新的可搜索 segment

所以新写入的数据往往会在一个很短的时间窗口后才可被搜索到。

这个“不是立即可见, 但很快可见”的特性, 就叫近实时。

Elasticsearch 的写入流程

Q：一条文档写入 Elasticsearch，大致经历什么流程？

可以简单理解为：

1. 请求先到某个协调节点
2. 路由到目标主分片
3. 主分片先写本地
4. 再同步到副本分片
5. 写入 translog，等待 refresh 后对搜索可见

所以 ES 写入的关键点通常是：

- 主分片负责接收写请求
- 副本分片负责同步
- refresh 决定搜索可见性

Elasticsearch 的查询流程

Q：ES 查询时是怎么工作的？

查询大致分两步：

Query Phase

协调节点把查询分发到各个分片，各分片先返回本地 TopN 结果。

Fetch Phase

协调节点汇总排序后，再去对应分片拿完整文档。

这样做的好处是：

- 各分片并行搜索
- 协调节点统一汇总结果

所以 ES 查询快，很大一部分也来自分片并行。

Mapping 为什么重要

Q：为什么 ES 的 Mapping 设计很重要？

因为 Mapping 决定了：

- 字段是什么类型
- 是精确匹配还是全文检索
- 是否参与索引
- 是否参与聚合和排序

最典型的高频点是：

- `text`：适合全文检索
- `keyword`：适合精确匹配、聚合、排序

如果 Mapping 设计错了，后面搜索效果、聚合能力、性能表现都会受影响。

ES 常见问题怎么说

Q: 面试里关于 Elasticsearch 常见会追问哪些问题？

常见追问通常有这些：

- 为什么不用 MySQL 做全文检索
- 倒排索引和 B+ 树索引有什么差别
- 为什么叫近实时
- 分片数怎么设计
- 深分页为什么慢
- refresh、flush、merge 分别是什么

其中深分页常见答法是：

页数越深，协调节点和分片要跳过的数据越多，排序和汇总成本也越高。

项目里怎么说

Q: 如果项目里用了 Elasticsearch，面试时怎么讲？

可以按这四步来讲：

1. 场景

比如商品搜索、日志检索、文章检索。

2. 为什么选 ES

比如需要全文搜索、分词、复杂条件过滤和聚合分析。

3. 怎么设计索引

比如分片、副本、Mapping、分词器。

4. 踩过什么坑

比如深分页慢、Mapping 配错、写入延迟、数据同步一致性问题。

高频追问补充

追问1: ES 查询快的核心原因是什么？

核心是：

- 倒排索引
- 分词器
- 分片并行搜索

追问2：ES 能完全替代 MySQL 吗？

不能。

ES 更擅长搜索和分析，不适合直接承担强事务型主存储。

追问3：为什么新写入的数据搜不到？

因为 ES 是近实时，数据要经过 refresh 后才会对搜索可见。

更新: 2026-04-01

Dubbo

Dubbo核心原理

Dubbo 是 Java 面试里非常典型的 RPC 框架题。

它经常会和 ZooKeeper、注册中心、负载均衡、超时重试、服务治理一起问。

Dubbo 是什么

Q: 什么是 Dubbo? 它解决的核心问题是什么?

Dubbo 本质上是一个 RPC 服务框架。

它主要解决的是:

- 服务怎么暴露
- 服务怎么发现
- 远程调用怎么像本地调用一样方便
- 调用失败时怎么做超时、重试、容错、负载均衡

所以如果一句话概括:

Dubbo 解决的是分布式服务调用问题。

什么是 RPC

Q: 什么是 RPC? 为什么不直接用 HTTP 接口互调?

RPC 的全称是 Remote Procedure Call，远程过程调用。

它的目标是：

让你像调用本地方法一样去调用远程服务。

相比直接写 HTTP 接口，RPC 框架通常会帮你处理这些事：

- 服务发现
- 序列化 / 反序列化
- 连接管理
- 超时控制
- 负载均衡
- 失败重试和容错

所以 Dubbo 的价值不只是“发一个请求”，而是把整个服务调用链路的工程细节封装起来。

Dubbo 的核心架构

Q：Dubbo 的核心架构是怎样的？

Dubbo 经典架构里通常有几个角色：

- **Provider**：服务提供者
- **Consumer**：服务消费者
- **Registry**：注册中心
- **Monitor**：监控中心
- **Config Center**：配置中心

典型流程是：

1. Provider 启动时把服务地址注册到注册中心
2. Consumer 启动时从注册中心订阅服务列表
3. Consumer 本地生成代理对象
4. 业务调用代理对象时，底层发起远程调用
5. 返回结果后像本地方法一样继续执行

Dubbo 调用链路是怎样的

Q：一个 Dubbo 调用从发起到返回，大致经历哪些步骤？

可以按这条链路来答：

1. 业务代码调用本地代理对象
2. 代理对象把方法名、参数、接口信息封装成请求
3. 请求经过序列化后通过网络发送出去
4. Provider 端反序列化请求并定位到目标服务实现
5. 服务执行后把结果再序列化回传
6. Consumer 反序列化结果并返回给业务线程

这里最核心的几个关键词是：

- 动态代理
- 序列化
- 网络传输
- 服务发现
- 容错治理

Dubbo 为什么常和 ZooKeeper 一起出现

Q：为什么早期 Dubbo 常常配合 ZooKeeper 使用？

因为 Dubbo 需要一个注册中心来解决：

- Provider 地址存在哪
- Consumer 怎么动态感知服务上下线

ZooKeeper 的临时节点和 Watch 正好很适合这个场景：

- Provider 注册临时节点
- Provider 宕机时节点自动删除
- Consumer 监听节点变化，更新本地服务列表

所以早期经典组合就是：

Dubbo 做 RPC，ZooKeeper 做注册中心。

Dubbo 的负载均衡怎么讲

Q：Dubbo 有哪些常见负载均衡策略？

面试里常见的通常有：

- **随机**：默认比较常见，简单均匀
- **轮询**：按顺序分发
- **最少活跃数**：优先给当前处理压力小的节点
- **一致性 Hash**：同一类请求尽量打到同一节点

不同策略适合不同场景：

- 普通业务接口常用随机
- 有局部缓存、会话粘性诉求时可能会用一致性 Hash
- 节点性能差异明显时，最少活跃数更有意义

Dubbo 的超时、重试、容错

Q：Dubbo 调用失败时通常怎么处理？

Dubbo 常见治理能力包括：

超时

防止一个慢服务无限拖住上游线程。

重试

对幂等读请求或者部分可重试请求，失败后自动再试。

容错策略

比如：

- Failover：失败自动切换其他节点重试
- Failfast：失败立即报错
- Failsafe：失败时忽略异常
- Forking：并发调用多个节点，谁先返回用谁

面试里要补一句：

重试不是默认越多越好，如果接口非幂等，重试反而会把问题放大。

Dubbo 的序列化和协议

Q：Dubbo 调用为什么需要序列化？协议有什么作用？

远程调用本质上要把方法参数和返回值在网络上传输，所以必须：

- 先把对象变成字节流，也就是序列化
- 再按照协议格式发到对端

协议解决的是：

- 怎么组织请求头和请求体
- 怎么标识接口、方法、参数
- 怎么区分请求和响应

序列化则影响：

- 性能
- 网络开销
- 兼容性
- 安全性

所以 RPC 框架性能不只是“网络快不快”，还和序列化效率强相关。

Dubbo 和 Spring Cloud / OpenFeign 怎么比较

Q：Dubbo 和 Spring Cloud / OpenFeign 怎么比较？

一个常见答法是：

- Dubbo 更偏 RPC 框架，调用链路更轻、更贴近高性能服务调用
- Spring Cloud / OpenFeign 更偏微服务生态组合，治理、配置、网关、熔断等体系更完整

如果你的重点是：

- Java 内部服务高性能 RPC 调用
- 接口调用量大
- 追求更细的 RPC 治理能力

那很多团队会偏向 Dubbo。

如果更重视完整微服务体系、HTTP 协议兼容性和云原生集成，Spring Cloud 生态会更常见。

项目里怎么说

Q: 如果项目里用了 Dubbo，面试时怎么讲？

可以这样组织：

1. 场景

比如订单服务要调用库存服务、账户服务、营销服务。

2. 为什么选 Dubbo

比如内部调用量大、对性能和 RPC 治理有要求。

3. 怎么治理

比如注册发现、超时、重试、负载均衡、限流、熔断。

4. 踩过什么坑

比如超时设置不合理、重试把下游放大、注册中心抖动导致调用异常。

高频追问补充

追问1: Dubbo 只负责网络通信吗？

不是。

它还负责服务发现、动态代理、序列化、负载均衡、容错治理等一整套 RPC 能力。

追问2: Dubbo 重试为什么可能有风险？

因为非幂等接口一旦重试，可能导致重复扣款、重复下单这类问题。

追问3: Dubbo 和 MQ 的区别是什么？

Dubbo 是同步调用框架，强调请求-响应；

MQ 更偏异步解耦，强调削峰和事件通知。

消息队列选型

消息队列选型

消息队列选型题本质上不是让你背产品参数，而是看你能不能根据业务场景做取舍。

为什么要用消息队列

Q：项目里为什么要引入消息队列？

消息队列最核心的价值一般有三类：

第一，解耦。

上游系统只负责发消息，下游系统独立消费，不需要直接强依赖。

第二，削峰填谷。

高峰流量先进入 MQ，下游系统按自己的处理能力慢慢消费，避免被瞬时流量打垮。

第三，异步化。

一些不需要主链路同步完成的动作，可以异步执行，从而降低接口 RT。

Kafka、RabbitMQ、RocketMQ 怎么选

Q：常见消息队列怎么选？

如果从面试回答角度，我一般会这样概括：

Kafka：

更适合高吞吐日志流、埋点流、大数据链路。

RabbitMQ：

协议成熟、路由灵活，适合传统业务消息场景。

RocketMQ：

兼顾业务消息和高吞吐，顺序消息、事务消息等能力比较强。

所以如果只讲一句话：

- 大吞吐日志链路优先 Kafka
- 传统业务消息和复杂路由可考虑 RabbitMQ
- 电商类核心业务消息很多团队会选 RocketMQ

选型时看哪些维度

Q：做 MQ 选型时你会关注哪些维度？

我通常会看这些维度：

- 吞吐量要求
- 可靠性要求
- 是否需要顺序消息、延迟消息、事务消息
- 团队熟悉度
- 运维复杂度和生态成熟度

所以真正的选型不是“谁功能最多”，而是：
哪个最适合当前业务和团队。

策略模式

策略模式

什么是策略模式

Q：什么是策略模式？

策略模式的核心思想是：

把一组可互相替换的算法或行为抽象出来，在运行时按需选择。

这样做的好处是避免大量 `if-else`，让新增策略时更容易扩展。

策略模式的典型场景

Q：项目里哪些场景适合策略模式？

典型场景有：

- 支付方式选择
- 订单优惠计算
- 不同路由规则
- 不同校验规则

只要你发现“同一件事有多种可替换实现”，就可以考虑策略模式。

单例模式

单例模式

什么是单例模式

Q: 什么是单例模式?

单例模式的目标是:

保证一个类在系统中只有一个实例, 并提供全局访问点。

常见适用场景包括配置对象、线程池管理器、缓存管理等。

常见实现方式

Q: 单例模式有哪些实现方式?

常见实现包括:

- 饿汉式
- 懒汉式
- 双重检查锁
- 静态内部类
- 枚举单例

如果面试官问推荐哪种, 我通常会说:

- 工程里更推荐 **静态内部类** 或 **枚举单例**

因为它们在线程安全和实现简洁之间比较平衡。

观察者模式

观察者模式

什么是观察者模式

Q: 什么是观察者模式?

观察者模式的核心是:

一个对象状态变化后, 自动通知一组依赖它的对象。

它特别适合做事件发布、消息通知、解耦扩展逻辑。

项目里的典型应用

Q: 项目里哪里会用到观察者模式?

常见场景包括:

- 订单创建成功后通知库存、积分、消息中心
- Spring 的事件机制
- GUI 或前端框架中的事件监听

所以观察者模式本质上就是:

主流程只做核心动作, 附加动作交给监听者扩展。

数组链表与复杂度

以下这些都是面试里比较经常问的

- 时间复杂度和空间复杂度怎么理解
- 数组和链表到底差在哪
- `ArrayList` 和 `LinkedList` 怎么答
- 二分查找为什么总容易写错
- 链表题常见套路有哪些

时间复杂度和空间复杂度是什么

Q：什么是时间复杂度和空间复杂度？

时间复杂度表示：

随着输入规模 `n` 增长，算法执行次数大致怎么变化。

空间复杂度表示：

随着输入规模 `n` 增长，算法额外占用的存储空间怎么变化。

这里关注的不是某一次运行具体花了多少毫秒，而是增长趋势。

所以我们才会用 `O(1)`、`O(logn)`、`O(n)`、`O(nlogn)`、`O(n^2)` 这类记法来描述。

常见复杂度怎么理解

复杂度	典型含义	常见例子
<code>O(1)</code>	基本不随数据量变化	数组按下标访问、栈顶操作
<code>O(logn)</code>	每次规模减半	二分查找、堆调整
<code>O(n)</code>	扫一遍数据	遍历数组、链表查找
<code>O(nlogn)</code>	高质量排序常见级别	快排、归并、堆排
<code>O(n^2)</code>	双重嵌套扫描	冒泡、选择、插入最坏情况

追问1：为什么分析复杂度时通常忽略常数和低阶项？

因为复杂度关注的是规模变大后的主导趋势。

当 n 很大时， $3n + 10$ 和 n 的增长级别本质一样，所以都记作 $O(n)$ 。

数组和链表的本质区别

Q：数组和链表的本质区别是什么？

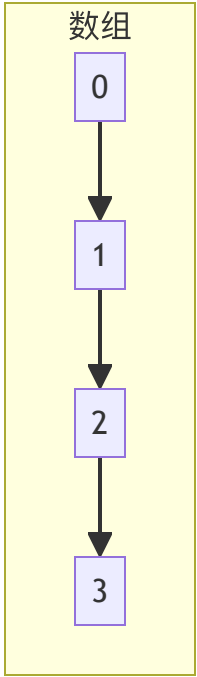
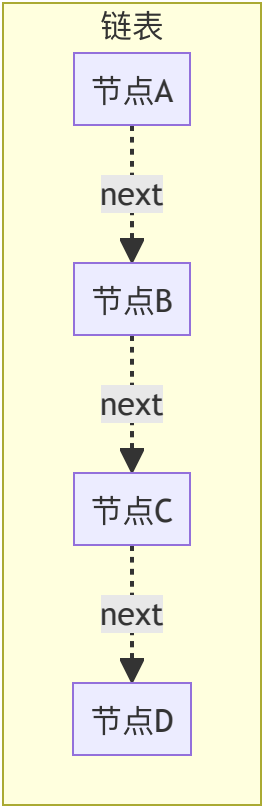
数组 的核心特征是连续内存。

链表 的核心特征是节点通过指针连接，内存不要求连续。

这会直接带来一组差异：

- 数组支持按下标随机访问，链表不行
- 数组插入删除可能要搬移元素，链表主要改指针
- 数组缓存友好，链表指针跳转多
- 数组扩容有成本，链表按需分配更灵活

所以二者不是谁绝对更强，而是适用场景不同。



数组和链表怎么对比

对比维度	数组	链表
------	----	----

内存布局	连续	非连续
随机访问	$O(1)$	$O(n)$
头部插入删除	$O(n)$	$O(1)$
中间插入删除	$O(n)$	找到节点后 $O(1)$
扩容	需要申请新空间	按需分配
缓存局部性	好	差

追问1：为什么链表说插入删除快，但很多题还是不一定比数组快？

因为链表“改指针”本身确实快，但前提是 **你已经拿到了待操作位置**。
如果还得先从头遍历去找那个位置，那么查找成本仍然是 $O(n)$ 。

动态数组怎么理解

Q：动态数组是什么？为什么 `ArrayList` 能自动扩容？

动态数组本质上还是数组，只是它会在容量不够时：

1. 申请一块更大的连续内存
2. 把旧数据拷贝过去
3. 替换底层引用

所以动态数组“看起来大小可变”，但底层仍然依赖连续内存。

这也是为什么：

- 普通访问很快
- 但扩容那次成本会比较高

不过因为扩容不是每次都发生，所以均摊下来，尾部追加通常仍可认为是 $O(1)$ 。

追问1：为什么 Java 里的 `ArrayList` 不每次只扩 1 个？

如果每次只扩 1 个，会导致频繁申请新内存和复制旧数据，整体成本太高。
按比例扩容能在空间利用率和扩容次数之间做折中。

ArrayList 和 LinkedList 怎么回答

Q: ArrayList 和 LinkedList 有什么区别?

这是 Java 面试里特别经典的桥接题。

比较稳的回答是：

- ArrayList 底层是动态数组，查询快，尾插均摊快，随机访问强
- LinkedList 底层是双向链表，理论上插入删除更灵活
- 但实际业务和算法题里，ArrayList 的使用频率通常更高

原因在于：

- 很多场景访问多于修改
- ArrayList 的缓存局部性更好
- LinkedList 有额外节点和指针开销

所以不要机械背“链表增删快”，更稳的说法是：

链表适合已知位置的插删，数组适合高频访问和遍历。

单链表、双向链表、循环链表怎么区分

Q: 单链表、双向链表、循环链表分别是什么?

单链表：

- 每个节点只有一个 `next`
- 结构简单

双向链表：

- 节点既有 `next`，也有 `prev`
- 更方便从两边移动和删除当前节点

循环链表：

- 尾节点重新指向头节点
- 没有天然的 `null` 终点

面试里最常见的是单链表和双向链表。

如果追问 `LinkedList`，通常可以顺带提一句：Java 中 `LinkedList` 是双向链表。

链表题为什么经常用双指针

Q：链表题为什么常用快慢指针？

因为链表不支持随机访问，下标法不好用，所以很多问题会转成“两个指针以不同方式走”。

常见套路有：

- 快慢指针找中点
- 快慢指针判断环
- 前后指针删除倒数第 `k` 个节点
- 两个指针判断链表相交

所以链表题的核心往往不是“链表结构本身”，而是你能不能想到合适的指针关系。

反转链表怎么理解

Q：反转链表的核心是什么？

反转链表最核心的动作只有一个：

把当前节点的 `next` 指向前一个节点。

但在修改之前，一定要先保存原来的下一个节点，不然链表就断了。

所以经典迭代写法通常会维护三个变量：

- `prev`
- `cur`
- `next`

这题几乎是链表面试的入门题。

二分查找什么时候能用

Q：二分查找的前提是什么？

二分查找最关键的前提是：

数据具有单调性。

最常见的形式是：

- 有序数组中查某个值
- 有序数组中查左边界或右边界
- 答案空间具有单调性时做“二分答案”

如果没有单调性，直接用二分通常就不成立。

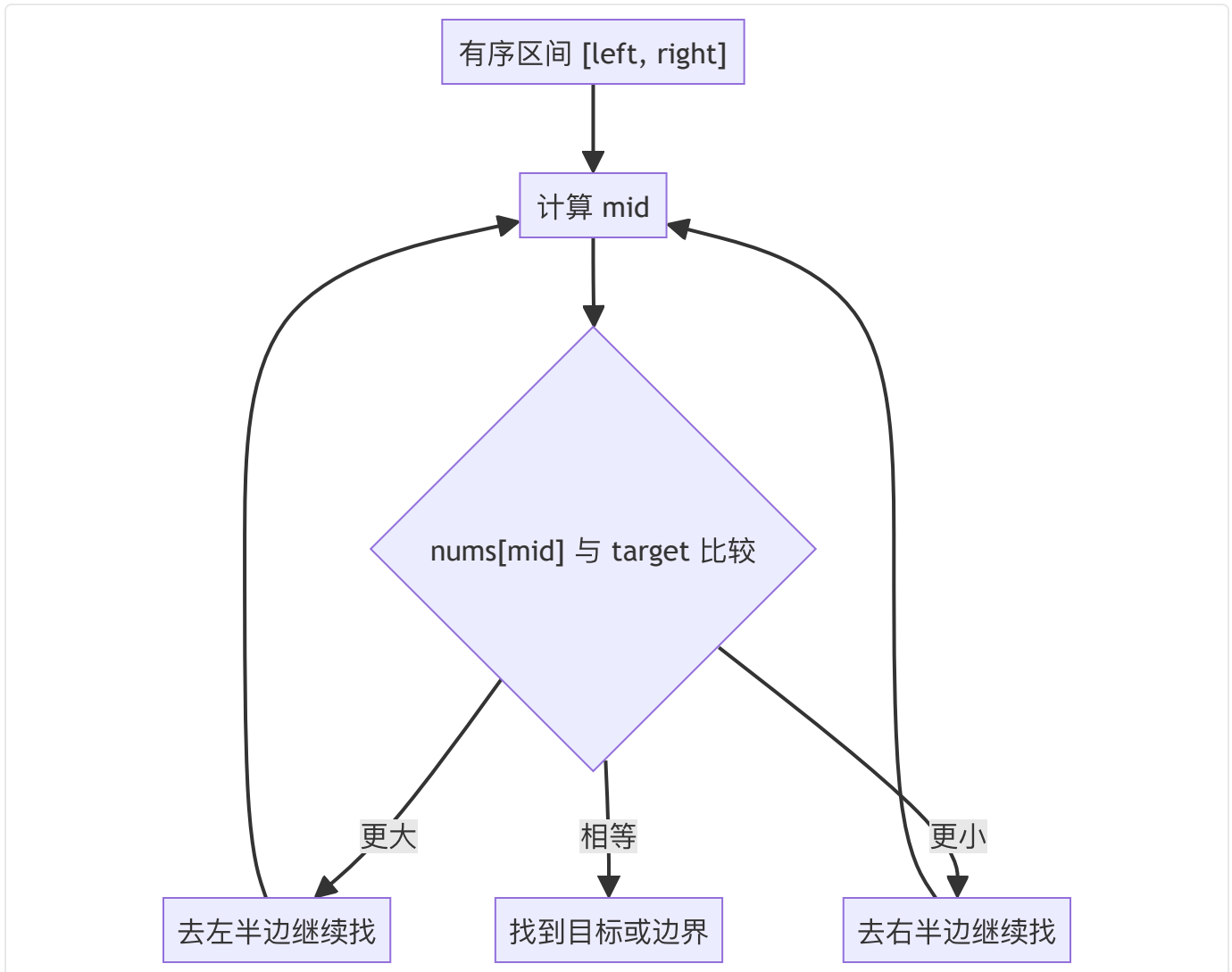
二分查找为什么总容易写错

Q：二分查找为什么是经典易错题？

因为二分真正难的不是“想到”，而是边界细节特别多：

- 区间到底是左闭右闭，还是左闭右开
- `mid` 怎么取
- 查到目标后继续往哪边收缩
- 最后返回 `left` 还是 `right`

所以面试里二分最稳的做法不是临场瞎写，而是固定一套自己熟悉的模板。



追问1: 为什么 `mid` 常写成 `left + (right - left) / 2` ?

为了避免 `left + right` 直接相加时发生整数溢出。

虽然现代很多场景里不一定真会溢出，但这是经典写法，面试里最好养成习惯。

二分查找有哪些常见变种

Q：二分查找除了“找某个值”，还有哪些变种？

最常见的变种有三类：

1. 找目标值是否存在
2. 找左边界
3. 找右边界

再往上一个层级，就是：

4. 二分答案

比如：

- 最小满足条件的值
- 最大可行值
- 某个容量、速度、时间的最优边界

这类题的本质不是查数组，而是查 满足条件的分界点。

面试里基础结构题怎么答

Q：面试官问数组、链表、复杂度，怎么答得更完整？

比较稳的结构是：

1. 先说本质特征
2. 再说复杂度差异
3. 再说适用场景
4. 最后补一个 Java 或题型视角

比如问“数组和链表区别”，可以答：

这样会比单独背复杂度表更像真的理解。

速记版

- 时间复杂度看增长趋势，不看一次运行的具体毫秒数
- 数组：连续内存，随机访问快，插删慢
- 链表：不连续存储，访问慢，已知位置插删灵活
- `ArrayList` 常用，`LinkedList` 不要只会背“增删快”
- 二分查找前提是单调性，难点在边界模板
- 链表题高频套路：反转、快慢指针、删除倒数第 `k` 个、找中点、找环

基础题真正拉开差距的地方不是“背定义”，而是：

你能不能把复杂度、结构特征、场景和题型一起讲出来。

树堆与哈希

树堆与哈希

树、堆、哈希是数据结构面试里的第二层内容。很多同学基础只停留在数组链表，一到树和哈希就开始碎。实际上这几类结构各自解决的问题非常明确：

- 树：表达层级关系
- 堆：快速拿最值
- 哈希：快速定位

树是什么

Q：树结构的核心特点是什么？

树是一种层级结构。

和线性结构相比，树最大的特点是：

- 一个节点可以有多个子节点
- 数据之间不是单一前后关系，而是父子关系

面试里最常见的是二叉树。

很多题只要一出现“层级”“左右子树”“祖先”“路径”，基本就在树的语境里了。

二叉树的常见概念

Q：二叉树有哪些基础概念？

常见基础词汇有：

- 根节点
- 叶子节点
- 父节点、子节点、兄弟节点
- 节点深度
- 树高度
- 左子树、右子树

还有几类常考树形态：

- 满二叉树：每层都满
- 完全二叉树：除了最后一层，前面都满，最后一层尽量靠左
- 平衡二叉树：左右子树高度差不大

堆这种结构，一般就建立在完全二叉树之上。

二叉树遍历怎么理解

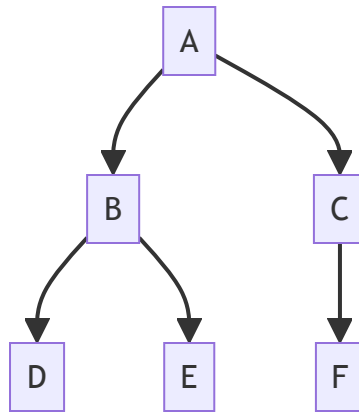
Q：前序、中序、后序、层序遍历分别是什么？

这是树题的绝对基础。

- 前序：根 -> 左 -> 右
- 中序：左 -> 根 -> 右
- 后序：左 -> 右 -> 根
- 层序：按层从上到下、从左到右

最值得记住的点是：

- 前序适合先处理当前节点
- 中序在 BST 中会得到有序序列
- 后序适合先处理子树再处理根
- 层序通常配合队列使用



追问1: 为什么很多树题最后都能转成“遍历”?

因为树的问题常常需要你在每个节点上做事。
而遍历，本质上就是“按照某种顺序访问所有节点”。
所以很多树题的核心，不是死记模板，而是想清楚：
我是在进入节点时做事，还是在处理完子树后做事。

二叉搜索树是什么

Q: 什么是二叉搜索树，为什么它查找快?

二叉搜索树，也就是 **BST**，满足：

- 左子树所有节点值都小于根
- 右子树所有节点值都大于根

这意味着查找时可以像二分一样不断缩小范围。

如果树比较平衡，查找、插入、删除的时间复杂度通常是 $O(\log n)$ 。

如果树严重退化成链表，复杂度就会退化成 $O(n)$ 。

追问1: BST 的中序遍历为什么有序?

因为中序遍历顺序是“左、根、右”，而 BST 恰好又满足左边都小、右边都大。
两者一叠加，结果天然就是有序序列。

平衡树、AVL、红黑树怎么答

Q：为什么需要平衡树？红黑树和 AVL 怎么区分？

BST 的问题在于，如果插入顺序很差，比如一直递增，树就可能退化成链表。

平衡树的目标就是尽量让树高保持在 $\log n$ 级别。

AVL 树：

- 平衡要求更严格
- 查询性能更稳定
- 插入删除时旋转更频繁

红黑树：

- 平衡要求没那么严格
- 插入删除调整成本相对更低
- 工程里更常见

所以很多语言或框架里的有序映射结构，更偏向选择红黑树。

追问1：Java 里哪些地方会用到红黑树？

常见回答可以带上：

- TreeMap
- TreeSet
- HashMap 在链表过长时也会树化成红黑树

堆是什么

Q：什么是堆？

堆本质上是一棵完全二叉树，同时满足堆序性质。

常见有两种：

- 大顶堆：父节点大于等于子节点
- 小顶堆：父节点小于等于子节点

堆的核心价值不是“有序”，而是：

可以在 $O(1)$ 时间拿到堆顶，在 $O(\log n)$ 时间插入和删除堆顶。

所以堆非常适合：

- 优先队列
- TopK
- 动态维护最值

堆和二叉搜索树有什么区别

Q：堆和二叉搜索树有什么区别？

这是一个高频对比题。

堆：

- 只保证父子之间有大小关系
- 不保证整棵树局部有序
- 优点是堆顶最值特别好拿

二叉搜索树：

- 保证左小右大
- 支持更高效的范围查找和有序遍历
- 但不一定像堆一样方便拿全局最值后继续维护

一句话说：

- 要最值，优先想到堆
- 要有序查找，优先想到搜索树

优先队列和普通队列有什么区别

Q：优先队列和普通队列有什么区别？

普通队列按进入顺序出队。

优先队列按优先级出队。

所以优先队列底层通常不是普通链表队列，而是堆。

常见应用包括：

- 定时任务
- 任务调度
- TopK
- Dijkstra 最短路
- 合并多个有序链表

哈希表是什么

Q：哈希表为什么查找这么快？

哈希表的核心是：

通过哈希函数，把 key 映射到数组下标附近。

这样查找时就不需要从头遍历，而是直接定位到目标桶，再在桶里继续确认。

理想情况下，哈希表的插入、删除、查找平均复杂度都可以做到 $O(1)$ 。

但这不是绝对的。

一旦冲突严重，性能就会下降。

哈希冲突怎么处理

Q：什么是哈希冲突？常见怎么处理？

哈希冲突就是：

不同 key 经过哈希计算后，落到了同一个位置。

常见处理方式有两类：

1. 拉链法
 - 每个桶后面挂链表或其他结构
2. 开放寻址法
 - 冲突后继续探测其他空位

Java 面试里最常答的是拉链法，因为 `HashMap` 就是这一路思路。

追问1：为什么 `HashMap` 里链表太长会树化？

因为如果某个桶里的链表太长，查找效率就会越来越接近 $O(n)$ 。
树化之后可以把这个桶内的查找复杂度优化到 $O(\log n)$ 。

负载因子和扩容怎么理解

Q：哈希表为什么需要负载因子和扩容？

桶太少，元素太多，冲突就会明显增加。
所以哈希表一般会设置一个负载因子阈值。

当：

元素数量 / 桶数量 超过阈值时，就会扩容并重新分布元素。

这也是为什么哈希表平均查找很快，但扩容那一下成本不低。
不过扩容不会每次都发生，所以均摊下来整体仍然高效。

哈希表和树怎么选

Q：哈希表和树结构在查找问题里怎么选？

如果只是追求平均快速查找，通常优先想到哈希表。

如果还需要：

- 保持有序
- 范围查询
- 找前驱后继

那树结构通常更合适。

所以不能机械地说“哈希表比树更强”，而是要看需求是不是只需要精确定位。

Trie 和并查集要不要会

Q：面试里 Trie 和并查集算高频吗？要怎么理解？

这两个结构没有数组、链表、树、哈希那么高频，但也值得有基本概念。

Trie：

- 又叫前缀树
- 适合处理字符串前缀匹配、词典检索

并查集：

- 适合处理连通性问题
- 核心操作是“查所属集合”和“合并集合”

如果题目出现：

- 字典前缀、搜索推荐
- 社交关系连通、岛屿合并、连通块

就可以往这两个结构上想。

面试里怎么把树、堆、哈希串起来

Q：面试里如果被连续追问树、堆、哈希，怎么回答不散？

可以按“它们分别解决什么问题”来组织：

- 树：表达层级和结构化关系
- BST / 红黑树：解决有序查找
- 堆：解决快速拿最值
- 哈希：解决快速定位

这样答的好处是，不会陷入一堆定义里出不来，而是把结构和用途绑定起来。

速记版

- 二叉树：层级结构，遍历是基础
- BST：左小右大，中序有序
- 平衡树：避免 BST 退化
- 红黑树：工程里更常见
- 堆：完全二叉树，适合最值和 TopK
- 哈希表：平均 $O(1)$ 查找，核心问题是冲突处理和扩容
- 树适合有序和范围，哈希适合快速精确定位

面试里不要只背定义，最好能回答：

为什么需要这个结构、解决什么问题、和相邻结构怎么区分。

排序算法

排序算法

排序是算法面试里的基础题库。很多面试官并不要求你把所有排序都手写一遍，但一定会问你：

- 常见排序怎么分类
- 时间复杂度和空间复杂度怎么比较
- 哪些稳定、哪些不稳定
- 快排、归并、堆排分别适合什么场景
- TopK、海量数据排序、链表排序该想到什么

排序算法整体怎么分类

Q：常见排序算法怎么分类？

常见分类方式有两种。

第一种，按是否基于比较：

- 比较排序：冒泡、选择、插入、希尔、快排、归并、堆排
- 非比较排序：计数排序、桶排序、基数排序

第二种，按是否稳定：

- 稳定排序：冒泡、插入、归并、计数、桶、基数
- 不稳定排序：选择、快排、堆排、希尔

面试里最常问的重点通常是：

- 插入排序
- 快速排序
- 归并排序
- 堆排序

如果题目带有“数据范围固定”“整数范围不大”“按位处理”这类条件，再考虑计数、桶、基数排序。

常见排序复杂度对比

排序算法	平均时间复杂度	最坏时间复杂度	空间复杂度	稳定性	典型特点
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	思想直观，基本不在工程里用
选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	交换次数少，但整体慢
插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	小规模、近乎有序时表现好
快速排序	$O(n\log n)$	$O(n^2)$	$O(\log n)$	不稳定	平均性能最好，面试最常考
归并排序	$O(n\log n)$	$O(n\log n)$	$O(n)$	稳定	时间复杂度稳定，适合链表和外排
堆排序	$O(n\log n)$	$O(n\log n)$	$O(1)$	不稳定	适合 TopK 、优先级问题
计数排序	$O(n+k)$	$O(n+k)$	$O(k)$	稳定	适合范围可控的整数

快排、归并、堆排怎么区分

Q：快排、归并、堆排分别有什么特点？

这是排序题里最值得背熟的一组对比。

快速排序：

- 平均时间复杂度最好，通常是工程里最常用的比较排序之一
- 原地排序，额外空间相对少
- 最坏会退化成 $O(n^2)$
- 不稳定

归并排序：

- 时间复杂度稳定，最坏也是 $O(n \log n)$
- 稳定排序
- 需要额外辅助空间
- 特别适合链表排序、外部排序、统计逆序对

堆排序：

- 时间复杂度稳定，空间复杂度低
- 不稳定
- 单次排序不一定比快排快，但非常适合维护“当前最值”或 TopK

一句话记忆：

- 追求平均性能，优先想到快排
- 追求稳定和确定性，优先想到归并
- 追求堆顶最值和 TopK，优先想到堆

冒泡排序

Q：冒泡排序的核心思想是什么？

冒泡排序的本质是：

每一轮通过相邻比较，把当前最大值“冒”到末尾。

如果数组长度是 n ：

- 第一轮结束，最大值到最后
- 第二轮结束，次大值到倒数第二
- 依次类推

优点是思路非常直观，代码也好写。

缺点是时间复杂度高，工程价值不大。

追问1：冒泡排序为什么稳定？

因为它只交换逆序的相邻元素。

如果两个元素值相等，通常不会主动交换它们的先后顺序，所以是稳定排序。

选择排序

Q：选择排序的核心思想是什么？

选择排序每一轮都会在未排序区间里找到最小值，然后放到当前起始位置。

它和冒泡排序的区别是：

- 冒泡是不断交换，把大值一点点推到后面
- 选择是先找到最小值，再一次性交换

选择排序的交换次数相对更少，但整体复杂度仍然是 $O(n^2)$ ，而且不稳定。

追问1：为什么选择排序不稳定？

因为它会把后面找到的最小值直接和前面元素交换，这个操作可能打乱相同元素的相对顺序。

插入排序

Q：插入排序的核心思想是什么？

插入排序可以理解成“打扑克牌整理手牌”。

它默认前面的区间已经有序，然后不断把新的元素插入到正确位置。

它的特点是：

- 实现简单
- 稳定
- 原地排序
- 当数组本身接近有序时，性能很好

所以插入排序虽然理论复杂度不高，但在“小规模 + 近乎有序”的场景下非常有价值。

追问1：为什么很多库排序在小区间会退化用插入排序？

因为当数据量很小时，快排和归并的递归、分治、额外控制逻辑反而会带来额外常数开销。这时插入排序虽然复杂度看起来更差，但常数更小，实际更快。

快速排序

Q：快速排序的核心思想是什么？

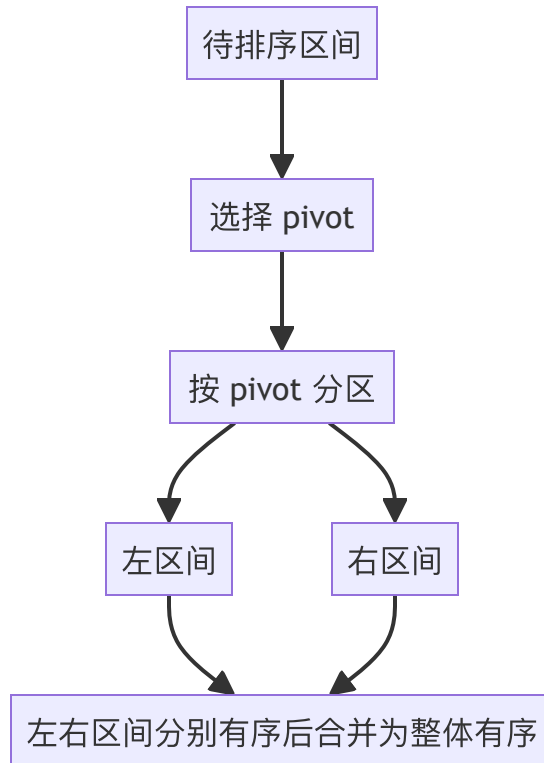
快速排序的核心是 分治 + 分区。

步骤通常是：

1. 选一个基准值 `pivot`
2. 把数组分成两部分
 - 左边都小于等于 `pivot`
 - 右边都大于等于 `pivot`
3. 对左右两边递归继续排序

它的关键不在“递归”，而在 分区质量。

如果每次都能把数组分得比较均匀，复杂度就是 $O(n \log n)$ ；如果每次都极度不均匀，就会退化成 $O(n^2)$ 。



追问1：快排为什么快？

因为它通常具备三个优势：

- 平均情况下分区比较均匀
- 原地排序，额外空间少
- 顺序扫描数组，缓存友好

所以很多场景下，快排的综合表现非常好。

追问2：快排最坏情况是什么？

如果每次选到的 `pivot` 都是当前区间最大值或最小值，那么划分会严重失衡，比如一边 `n-1`，另一边 `0`。

这时递归深度会退化到 `n`，时间复杂度也会退化成 `O(n^2)`。

追问3：怎么优化快排退化问题？

常见手段有：

- 随机选 `pivot`
- 三数取中

- 对重复元素很多的场景用三路快排
- 小区间切换插入排序

归并排序

Q：归并排序的核心思想是什么？

归并排序的核心是 **分治 + 合并两个有序区间**。

基本过程是：

1. 把数组不断二分，直到每个子区间长度为 1
2. 再把两个有序子区间不断合并
3. 最终得到整体有序数组

它最大的特点是：

- 时间复杂度稳定
- 是稳定排序
- 适合链表排序
- 很适合外部排序和多路归并

追问1：归并排序为什么稳定？

因为在合并两个有序区间时，如果两个元素相等，先取左边元素，就能保证相同元素的原始相对顺序不变。

追问2：为什么链表排序经常想到归并排序？

因为链表：

- 不支持高效随机访问，不适合快排那种频繁按下标操作
- 但很适合从中间断开，再把两个有序链表合并

所以“链表排序”几乎是归并排序的经典应用题。

堆排序

Q：堆排序的核心思想是什么？

堆排序依赖堆这种结构。

如果是升序排序，通常会构建大顶堆：

1. 先把数组建成大顶堆
2. 取出堆顶最大值，放到数组末尾
3. 调整剩余区间，重新维护堆
4. 重复直到排序完成

它的特点是：

- 时间复杂度稳定为 $O(n\log n)$
- 原地排序
- 不稳定

但比起“完整排序”，面试里更常问堆的地方是：

- TopK
- 优先级队列
- 海量数据找最值

追问1：堆排序为什么不稳定？

因为在建堆和交换堆顶的过程中，相同元素的相对顺序可能改变。

计数排序、桶排序、基数排序怎么理解

Q：非比较排序什么时候用？

这类排序的关键前提不是“更高级”，而是 数据本身有额外结构信息。

计数排序：

- 适合整数范围不大
- 先统计每个数出现次数，再按顺序回填

桶排序：

- 适合数据分布比较均匀
- 先分桶，桶内再排序

基数排序：

- 适合按位或按关键字排序
- 从低位到高位或从高位到低位逐步排序

这几类排序在特定场景下可以做到比 $O(n\log n)$ 更好，但适用范围明显更窄。

二分、快排、归并、堆排在工程里的典型场景

Q：工程里什么时候会想到这些排序或查找方式？

常见回答方式可以这样组织：

- 数据量一般、追求平均性能：想到快排
- 需要稳定性，或者要处理链表、外部文件：想到归并
- 需要随时拿最值或做 TopK：想到堆
- 有序数组里做查找：想到二分

面试里不要只背定义，最好把“题型和场景”带上。

这样会显得你知道这类结构是拿来解决什么问题的。

TopK 问题怎么回答

Q：海量数据找前 K 大，通常怎么做？

TopK 是排序板块特别高频的追问。

最经典的做法是 **维护一个大小为 K 的小顶堆**：

1. 先把前 **K** 个元素放进小顶堆
2. 后面每来一个元素，就和堆顶比较
3. 如果比堆顶大，就替换堆顶并重新调整堆
4. 遍历结束后，堆里剩下的就是前 **K** 大

这样时间复杂度是 $O(n\log k)$ ，比完整排序的 $O(n\log n)$ 更适合 **K** 远小于 **n** 的场景。

追问1：什么时候直接排序更划算？

如果 **K** 接近 **n**，或者你本来就需要完整有序结果，那么直接排序可能更简单。

速记版

- 快排：平均最快，最坏退化，不稳定
- 归并：稳定、复杂度稳定、适合链表和外排
- 堆排：复杂度稳定，适合 **TopK** 和最值维护
- 插入：小规模、近乎有序时很好用
- 计数/桶/基数：依赖数据范围和分布，不是通用排序

排序题不要只背定义，要能说出：

复杂度、稳定性、适用场景、典型追问。

栈与队列

栈和队列本身并不复杂，但它们在面试里出现频率极高。很多题目表面上问的是括号匹配、滑动窗口、层序遍历，底层其实都在考你对栈和队列的理解。

栈和队列的区别

Q：栈和队列有什么区别？

栈 是后进先出，也就是 **LIFO** 。

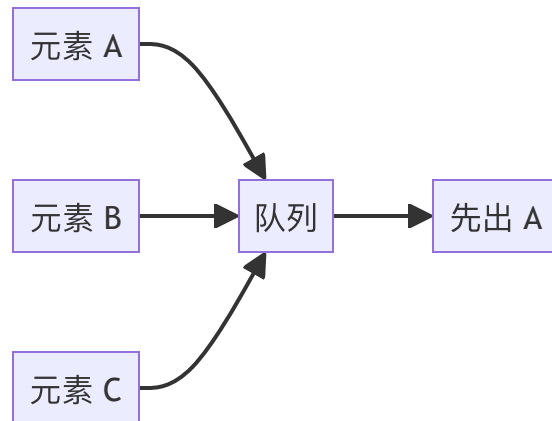
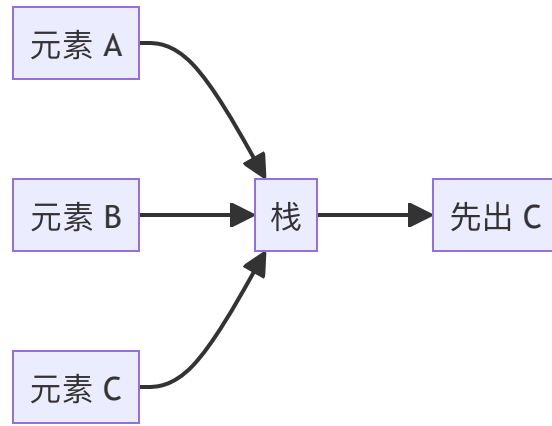
队列 是先进先出，也就是 **FIFO** 。

可以把它们理解成两种完全不同的“取数据规则”：

- 栈：最后放进去的元素先出来
- 队列：最早放进去的元素先出来

它们的核心不是“存数据”，而是 **控制顺序**。

这也是为什么很多算法题明明不叫“栈题”或“队列题”，却依然会用到它们。



栈常见应用场景

Q: 栈常见应用场景有哪些?

栈常见在以下几类问题里：

- 函数调用栈、递归
- 括号匹配
- 表达式求值
- 浏览器前进后退
- 单调栈
- DFS 的非递归写法

面试里如果只说“后进先出”还不够，最好顺手举一个算法场景。

比如：

- 校验括号合法性
- 求下一个更大元素
- 维护递增或递减序列

队列常见应用场景

Q：队列常见应用场景有哪些？

队列常见在以下几类问题里：

- BFS 层序遍历
- 任务调度
- 生产者消费者模型
- 消息缓冲
- 单调队列
- 线程池任务排队

算法题里最经典的应用是：

- 二叉树层序遍历
- 图的广度优先搜索
- 滑动窗口最大值

栈和队列各自的基本操作复杂度

结构	常见操作	时间复杂度
栈	push / pop / peek	O(1)
队列	offer / poll / peek	O(1)
双端队列	头尾插入删除	O(1)

栈和队列底层一般怎么实现

Q：栈和队列底层通常怎么实现？

它们都可以用数组或链表实现。

数组实现：

- 优点：连续内存，访问局部性好
- 缺点：容量可能受限，队列如果设计不好会涉及搬移

链表实现：

- 优点：扩容灵活
- 缺点：每个节点有额外指针开销，缓存局部性差

在 Java 里，面试更推荐你回答：

- 栈通常用 Deque
- 队列通常也用 Deque 或专门的 Queue
- 一般不推荐老的 Stack 类

追问1：为什么 Java 里经常推荐 ArrayDeque ？

因为它比 Stack 和 LinkedList 更现代，也更符合栈和队列的常见使用方式：

- 头尾操作高效
- API 清晰
- 没有 Stack 的历史包袱

双端队列是什么

Q：什么是双端队列？

双端队列，也就是 `Deque`，可以在队头和队尾都做插入、删除操作。

它比普通队列更灵活，所以很多题目都会用它：

- 当栈用：只在一端操作
- 当队列用：一端入队，另一端出队
- 当单调队列用：两端都可能调整

所以你可以把 `Deque` 看成是“栈和队列的通用版工具”。

递归和栈是什么关系

Q：为什么递归本质上也和栈有关？

因为递归调用本质上依赖函数调用栈。

每进入一层递归，系统就会把当前方法的局部变量、返回地址等信息压栈；递归返回时，再一层层弹栈。

所以很多递归题都可以改写成“显式栈”版本。

这也是面试里经常会追问：

- 递归和迭代能不能互相改写
- DFS 为什么既能递归写，也能用栈写

单调栈怎么理解

Q：什么是单调栈？

单调栈不是新的数据结构，而是 **带约束的栈**。

它要求栈内元素保持单调递增或单调递减。

常见用途是解决：

- 下一个更大元素
- 下一个更小元素
- 柱状图最大矩形
- 每日温度

本质原因是：

当我们维护单调性后，就能在元素出栈时，快速确定它和左右边界的关系。

追问1：什么时候会想到单调栈？

如果题目在问：

- 某个元素左边第一个比它大/小的元素
- 右边第一个比它大/小的元素
- 区间最值边界

就很容易联想到单调栈。

单调队列怎么理解

Q：什么是单调队列？

单调队列和单调栈类似，也是“带约束的队列”。

它最常见的用途是：

在滑动窗口移动过程中，维护窗口最值。

为什么不用普通队列？

因为普通队列只能保证“先入先出”，但不能保证队头永远是当前窗口最大值或最小值。

单调队列通过在尾部剔除不可能成为答案的元素，让队头始终保留候选最值。

BFS 为什么天然适合用队列

Q：为什么层序遍历、BFS 一般用队列？

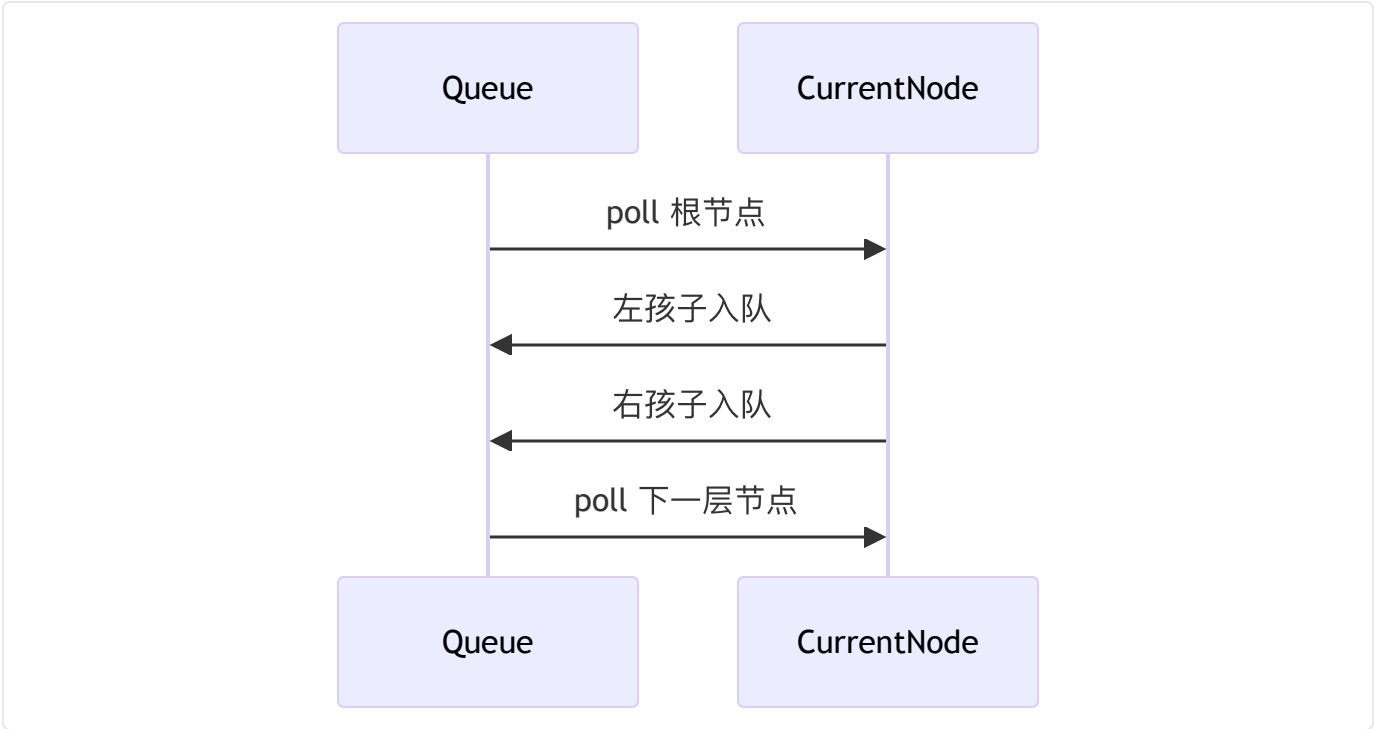
BFS 的目标是 按层推进。

队列的先进先出特性正好保证：

- 先进入当前层的节点，先被处理
- 当前层处理完后，自然轮到下一层

所以：

- 树的层序遍历用队列
- 图的最短步数问题也常用队列
- 多源 BFS 也还是队列



常见高频题

用两个栈实现队列

Q: 怎么用两个栈实现队列?

核心思路是:

- 一个栈负责入队
- 一个栈负责出队

当出队栈为空时, 把入队栈中的元素依次弹出并压入出队栈。
这样顺序就被反转了, 最终实现先进先出。

这个题本质上考的是:

两次 LIFO 叠加后, 可以得到 FIFO。

用两个队列实现栈

Q: 怎么用两个队列实现栈?

常见思路是:

- 新元素先进空队列
- 再把原队列中的旧元素依次追加到后面
- 然后交换两个队列的角色

这样队头永远是最近插入的元素, 就实现了后进先出。

最小栈

Q: 最小栈怎么设计?

最经典的做法是维护两个栈:

- 一个正常存数据
- 一个同步维护当前最小值

每次入栈时, 把“当前元素”和“当前最小值”做比较, 再把较小值压入辅助栈。
这样取最小值时就是 $O(1)$ 。

有效括号

Q：为什么括号匹配问题通常用栈？

因为括号匹配天然符合“后出现的左括号，要先被最近的右括号匹配”。

这就是典型的后进先出。

常见做法是：

- 遇到左括号就入栈
- 遇到右括号就检查栈顶能否匹配
- 最终栈为空才表示完全合法

滑动窗口最大值

Q：滑动窗口最大值为什么常用单调队列？

因为窗口每次只移动一步，但我们不希望每次都把整个窗口重新扫描一遍。

单调队列能做到：

- 新元素进入时，剔除尾部所有更小元素
- 旧元素滑出时，如果它正好在队头，就把它弹出
- 队头始终是当前窗口最大值

这样整个过程是线性的 $O(n)$ 。

面试里怎么比较栈、队列、堆

Q：栈、队列、堆有什么本质区别？

这三个结构的核心区别在于“取元素规则”不同。

- 栈：按最近插入顺序取
- 队列：按最早插入顺序取
- 堆：按当前优先级最高或最低取

所以：

- 栈适合处理嵌套、回退、最近关系
- 队列适合处理排队、分层、调度
- 堆适合处理最值、优先级、TopK

速记版

- 栈：后进先出，适合括号匹配、递归、单调栈
- 队列：先进先出，适合 BFS、任务调度、单调队列
- Deque：栈和队列的通用形态
- 单调栈：求左右第一个更大/更小元素
- 单调队列：维护滑动窗口最值

面试里如果被追问，一定要带上：

场景、题型、复杂度、为什么要这么设计。